

The data routing unit/engine called DRU, which enables user to perform high speed data transfers between memory mapped slave endpoints, processor caches [L2] and shared caches[L3] DRU behaves like an enhanced EDMA Transfer controller, moving data @CLK1 speed on up to 512 bit interface. It also can be configured to act like a Universal Transfer Controller (UTC) in the UDMA architecture defined in Keystone 3.

1.1 Primary Usage of DRU

- Services software driven page transfer between memory mapped spaces
- Allows for a constant fill into MSMC SRAM, CorePac L2, ARM L2 or Long Distance Port
- Allows transfer on event driven basis using both external PSI-L Events or Local Events
- Third Party DMA through PSI-L interface into compute cluster
- Assists in cache warming of DSPs, ARM L2 caches, L3 cache
 - Preload C7x L2 cache through pull model
 - Pre-warm L3 cache through Read Only Commands

1.2 Feature List

1.2.1 Features Revisions

1.2.1.1 Revision 0.7

This is the list of features that have been added or modified in Revision 0.7 of the architecture spec and are part of the MSMC Re-commissioning in July 2015.

Table 1-1 Delta Features List for Revision 0.7

Feature Set	Feature	Resolution	Summary
OpenCL	DTR Submission	Accepted	An additional configuration option for the number of cores that will be able to submit a Direct TR. The write to the highest aligned 64 bit register will load the entire 512 bit TR into the selected queue.
Vision	Transpose	Accepted for J7	The DRU will support the transpose fields in the TR and swap the two innermost counts on the read and write
Vision	Interleaved read mode	Accepted for J7	The DRU will add a mode that select an interleaved read mode where the DRU will switch between different offsets for each line read.
Vision	Reverse Data	Rejected	DRU will reverse the data on an element size boundary.
Vision	Change packing size of data	Accepted with Limitations for J7	DRU will change the data packing from a 12 bit element to a 16 bit element or from a 16 bit element to a 12 bit element. The start address and dimensions must be byte aligned and no other data manipulation is allowed at the same time as the element size
Safety	RAM Address Protection	Accepted	The RAM address will also have a parity check to ensure that the correct address is being accessed
Security	Firewall Integration	Accepted	The DRU will integrate a 16 region firewall for both the read and the write and block transactions if they violate the firewall.
Catalog	PSI-Link Transmit Data	Rejected for Area	A single read TR for a channel can be submitted and the data will be sent out on a specified thread of PSI_Link. This is to support the UDMA architecture.
Catalog	PSI-Link Receive Data	Rejected for Area	A single write TR for a channel can be submitted and the data will be received on a specified thread of PSI_Link. This is to support the UDMA architecture.
Catalog	Channelized TR Submission	Rejected for Area	A TR can now be submitted that is either a read or write and does not have to have both. This is to support the UDMA architecture.
Catalog	Channelized Event Interface	Accepted	An additional interface and client can be specified that will take a channel event and activate a partial transfer of a previously submitted channel TR. This is to support the UDMA architecture.
End of Table 1-1			

1.2.2 Features Supported

The Data Router Unit is a high bandwidth, flexible routing engine with programmable DMA transfer requests. Can support the following features:

- Programmable memory mapped registers for direct transfer request submission
 - A dedicated atomic MMR that require entire TR sent in a single burst.
 - Configurable number of non-atomic MMR that allow TR to be written in separate burst and a write to the submit 0 register will trigger the TR.
 - › Software controls that a single core writes to the register group.
- Read and Write command queue
 - Support for up to 8 different queues
 - › Queues arbitration is split between a fixed priority and round robin arbitration
 - Each channel can be configured to go to a single queue
 - RTL Configurable number of in-flight requests across all queues and all TRs
- Programmable CBA priority for each command queues
 - Allows setting priority for commands relative to other masters in the system
- Allow core MDMA/SCR slave interface to access transfer progress monitoring at real time
- One dedicated read port and one dedicated write port to generate independent read and write commands
 - Generate one read command every cycle
 - Generate one write data phase every cycle
 - Each interface is configurable and up to 512 bits wide
 - Moves the data at up to CLK1 speed
 - Optimizes transfers up to 128 byte boundaries
- Support for triggers to gate levels of the DMA loop
 - Supports 1 dedicated local trigger
 - Supports 2 dedicated external global triggers from the PSI-L
 - All triggers can be overwritten to be controlled by software instead of hardware based event
- Support for Region and Channelized Firewall
 - Each Realtime Channel specific register is controlled by a slot in a channelized firewall
 - The Global configuration and Non-realtime channel control is controlled by a unique Firewall
- Configurable Storage buffer holding up to 8KB of data organized as follows
 - Configurable entries, command ID tagged
 - Max burst size bytes/slot
 - Slot tracking mechanism for read return data

- Data formatting networks
 - Support non-aligned access for Read and Write commands
 - Transpose Data between source and destination memory to memory transfers
 - Interleave multiple source to a single destination
 - Circular Addressing Support for one side of the transfer
- IO Memory Management Unit and Micro-TLB
 - Features integrated uTLB (Translation look aside buffer)
 - Supports Virtual addresses and Physical addresses in TR requests
 - A read and write uTLB with 2 entries per uTLB
 - Communicates through PSI-L to IOMMU
- Independent 48-bit address fields for Source and destinations
- Up to Four dimensional data transfers
 - Has independent source and destination index for up to four dimensional transfers
- Error Handling
 - Debug support through Memory Mapped registers
 - Data transfer error reports
- Error detection and Correction
 - Full SECDED support for the incoming data
 - Full SECDED protection each entry in the TR queues
 - Single Bit Parity support for each 32-bit word in the data buffer if transpose is supported
 - Full SECDED protection for Response Buffer if transpose not supported
 - Parity Protection for the Address
- PSI_L Interface
 - Receive requests for Pre-warm of L2 or L3 Cache
 - Receive TR requests from the UDMA
 - Send Response TRs
 - Sends uTLB requests to IOMMU
 - Receive Data from a remote UTC
 - Send Data to a remote UTC
 - Maintains 2D formatting across PSI-L if requested in TR

1.2.3 Features Not Supported

- Does not optimize transfers across dimensions for optimal use of wide bus bandwidth provided by DRU
- Convert 12-bit to 16 bit and 16 bit to 12 bit data while doing other data formatting
- Repacking Type (9 or 11) Requires that the trigger or event size have the same counts. 1D Trigger or Event require ICNT0 == DICNT0 all other sizes require ICNT0 == DICNT0 && ICNT1 == DICNT1.

1.2.4 Features Descoped and Planned for Future Versions

- Reverse data on single dimension with a configured width

1.3 RTL Configuration Options

The DRU is written as configurable code so that unique versions can be generated based on the system requirements. Below are tables that show the options that can be selected. A list of current developed DRUs can be found at [DRU1 Generations](#).

Additionally, all versions of the DRU has a capabilities register which specifies exactly what optional features are supported by that instance of the DRU. This can be used by software to determine which type of TRs can be sent successfully to the DRU.

The options in ***bold italics*** are planned for future support but have are not in the initial verification scope. They are given here so that architecture in RTL and design can comprehend that these feature could be needed in the future and should be planned for in future efforts.

Table 1-2 DRU Configurations

Configuration Option	Description	Range
Bus Type	Select the bus type for the primary read and write DMA ports. VBUSMC_4.0, VBUSM_4.0 or VBUSP_4.0. If the bus type is VBUSM or VBUSP then an additional VBUSP Configuration bus will also be added. The VBUSM.C version maps the configuration bus onto a VBUSM.C thread.	C,M, <i>P</i>
Data Width	This is the width of the external DMA data bus.	512,256, <i>128, 64, 32</i>
PSI-L Data Width	This is the width of the PSI-L Bus. Must be zero for Ring support	<i>512,256,128,64,32,0</i>
PSI-L Async Interface	Add async FIFO on PSI-L to allow for independent clock	Y,N
MMR Data Width	This is the width of the VBUSP configuration bus if the Bus Type is not type C. If it is type C then this parameter is ignored.	64, 32
Address Width	The physical address width to use for the DMA buses. The TR address space is always 64 bits	48,36
Number of Channels	This is the number of channels that are being supplied by the DRU.	1-512
Number of TRs per Channel	This is the number of TRs that can be sent to a channel. Each TR will finish before the next TR starts for the same channel.	1-3
Number of Round Robin Queues	This is the number of round robin queues or Vitruel TCs that the DRU will have that arbitrate amongst themselves in a round robin manner. It will always have at least an urgent queue. Total number of queues cannot exceed 8	0-7 (Total Queues cannot exceed 8)
Number of Priority Queues	This is the number of queues or Virtual TCs that use a fixed priority for arbitration for memory to memory transactions. The lowest numbered queue will have highest priority and these queues always have priority over the round robin queues.	1-8 (Total Queues cannot exceed 8)
Dedicated DMA Port	This determines if each queue has a dedicated CBA port or if they arbitrate for a single queue.	Y,N
Cache Warm Support	A Cache Warm Type can be used for any of the channels in the design	Y,N
Cache Warm Unique Opcode Support	Will the DRU send Cache Warm Unique as specified in the TR or always send Cache Warm Shared and ignore the TR. This was added to allow for Eaglenest that did not support cache warm unique	Y,N
Number of In-flight Requests	This is the number of outstanding read requests that the DRU can have at any one time. This number will also determine the size of the response/reordering buffer.	8,16,32,64,128 Buffer Size of (1 kB,2kB,4kB, 8kB, 16kB)
Priority Mask	Create a mask register PRI_MASK that allows buffer entries to only be used by Priority queues. This can guarantee commands can start for high priority queue and the FIFO is not used by non-priority queues	Y,N
Maximum Transfer Size	This is the maximum size transfer that will be issued. Also, this will be the maximum address boundary that the logic will not cross.	128, 64, <i>32</i>

Table 1-2 DRU Configurations

Configuration Option	Description	Range
CLEC MMR Feedthrough Port	Add an external VBUSP master that takes address decoding bit 28 of address will send MMR request on VBUSM,C received to this external VBUSP port. Only used if Bus Type is VBUSM.C	Y,N
Maximum Number of Dimensions	This is the maximum number of ICNTs supported by the DRU. It can be reduced to 2 if only block copies of 2 dimensions are required.	4,2
Atomic Direct TR Submission	This specifies if an MMR exists that allow for a TR to be submitted with a single burst write of the TR size.	Y,N
Number of Unique Core Direct TR Submission registers	This is the number of separate registers are available for non-atomic Direct TR submission. This is to allow DirectTR submission without requiring the submission in a single burst.	0-8
Data Reformat Functions	This is the type of data reformatting that the DRU can perform on a data block move.	Repacking , Transpose, Reverse
Minimum Transpose Size	This is the minimum size in bits a transpose can be performed on. It must be 0 if the Transpose Reformat function is not used	0,32,64,128
Clocking Multiplier Pipelining	This is the amount of pipelining required in the logic, The smaller the number means more pipelining which allows for higher clock cycles but increases the latency between receiving commands and acting on them.	1-4
Virtual Address Support	The DRU will support both physical and virtual addresses in all supported transaction requests.	Y,N
Video Compression Support	Is the enhanced AFC module and bit size adaption module included to do video compress support	Y,N
Number of Compression Units	The number of parallel compression units used	4-16
PSI Split TR Input	This determines if the channels can support being a DMA split input channel. If this is set the PSI Input Buffer Entries must be set	Y,N
PSI Split TR Output	This determines if the channels can support being a DMA split output channel. If this is set a buffer of the max transfer size + 512 bits will be created for each channel.	Y,N
PSI Input Buffer Entries	This is the number of total entries across all of the channels. If the PSI Split TR Input is set.	0-1024
Safety	This determines are all the safety features included on all interfaces and if any additional logic required to meet the safety requirements are included.	Y,N
BIST Interface	This determines what bist interface should be brought out of the ports.	MCAD, GP
Embedded Firewall	This determines if a firewall is placed between the control logic and the DMA port to provide a firewall for the DMA transfers. This does NOT determine if there is a firewall for MMR accesses as those are always present	Y,N
Mem wrapper	This selects which memwrapper to use in the top level. It can use either the ECC embedded memory wrappers or standard memory wrappers with the ECC embedded in the logic.	ECC, Standard
Event Generation	The events can either be a bit_vector with a single bit for each channel or vectorized so that a request signal along with an encoded value of the channel that has the event. This applies to both the error event and the completion event	bit_vector, encoded
Maximum Trigger Counter Size	This is the number of triggers that can be received without overflowing for a channel.	1,3,7,15
Circular Buffer Addressing	Does the DRU support the Circular Buffer Addressing feature	Y,N
Static	Does the DRU support a STATIC TR that repeats until the channel is torn down	Y,N
Constant Copy	Does the DRU support using the SRC ADDR as a constant value that will be written out to the region specified by the TR	Y,N
Bit Expansion	Does the DRU support element types 8-11 which allow for 8 bit to 16 bit 12 bit to 16 bit and 16 bit to 8 bit and 16 bit to 12 bit data compression and decompression	Y,N
Multiple Buffer Offset	Does the DRU support multiple buffer offset. Using the secondary TR to allow for multiple DIM values	Y,N
Different Source and Destination Size Blocks	Does the DRU support the limited definition of type 9 and 11 TRs where the limitation that ICNT0 and DICNT0 must be equal and if the trigger size or event size is greater than 1D will require ICNT1 and DICNT1 to be the same as well	Y,N

Table 1-2 DRU Configurations

Configuration Option	Description	Range
Forced Teardown	Does the DRU support the forced teardown bit in the chcfg register to abort a channel without waiting for all triggers	Y,N
Secondary PSIL Teardown	Does PSIL support the teardown at both address 0x2 and 0x402	Y,N
Low Cost DMA Ring Support	Does the DRU support low cost DMA Architecture Rings	Y,N
End of Table 1-2		

PRELIMINARY SPECIFICATION

Overview	1
1.1 Primary Usage of DRU	2
1.2 Feature List	2
1.2.1 Features Revisions	2
1.2.2 Features Supported.....	3
1.2.3 Features Not Supported.....	4
1.2.4 Features Descoped and Planned for Future Versions	4
1.3 RTL Configuration Options.....	5
3.1 = References.....	9
3.2 Terminology	10
Functional Description	11
3.3 DRU Basic Functionality	12
3.3.1 Channel Configuration and TR Submission	12
3.3.2 Basic Transfer Request Operation	16
3.3.3 Queue Structure	17
3.3.4 Queue Arbitration	17
3.3.5 Address fetch algorithm and TR Formats	18
3.3.6 Virtualization.....	28
3.3.7 Compression and Decompression	29
3.3.8 Output Events	29
3.3.9 Input Events and Triggers.....	32
3.3.10 Memory Attribute Table	32
3.3.11 Shared Hardware Resources.....	32
3.3.12 Safety Features	33
3.3.13 Security Features.....	33
3.3.14 EDMA3 TC Compatibility	34
3.4 DRU Detailed Operation.....	34
3.4.1 Software Managed Flow.....	35
3.4.2 Active Channel Control Flow	39
3.4.3 Read Queue Control Flow	42
3.4.4 Write Queue Control Flow	43
3.4.5 Channel Clear Flow	44
Interfaces	45
4.1 Bus Type Configuration Parameter Equals C.....	45
4.1.1 MSMC_0_DRU/DRU_0_MSMC VBUSM.C Pair.....	45
4.1.2 MSMC_1_DRU/DRU_1_MSMC VBUSM.C Pair.....	52
4.2 Bus Type Configuration Parameter Equals M.....	58
4.2.1 VBUSM DMA Bus	58
4.3 PSI-L Bus Pair SOC_DRU/DRU_SOC.....	64
4.3.1 Threads.....	64
4.3.2 Configuration	64
4.3.3 Signals	64
4.4 Event Interfaces.....	64
4.5 Local Input Events.....	65
4.6 Tieoff Interfaces	65
Registers	67
5.1 General registers [Address = 0x0]	68
5.1.1 DRU_PID [Address = 0x0]	73
5.1.2 DRU_CAPABILITIES [Address = 0x8]	74
5.1.3 DRU_PRI_MASK [Address = 0x0_0040]	75

5.1.4	DRU_SHARED_EVT_SET[Address = 0x0_4000]	78
5.1.5	DRU_COMP_EVT_SET[Address = 0x0_4040]	78
5.1.6	DRU_ERR_EVT_SET[Address = 0x0_4080]	81
5.1.7	DRU_LOCAL_EVT_SET[Address = 0x0_40C0]	84
5.1.8	QUEUE0_CFG [Address = 0x0_8000]	86
5.1.9	QUEUE1_CFG [Address = 0x0_8008]	87
5.1.10	QUEUE2_CFG [Address = 0x0_8010]	88
5.1.11	QUEUE3_CFG [Address = 0x0_8018]	88
5.1.12	QUEUE4_CFG [Address = 0x0_8020]	89
5.1.13	QUEUE5_CFG [Address = 0x0_8028]	90
5.1.14	QUEUE6_CFG [Address = 0x0_8030]	91
5.1.15	QUEUE7_CFG [Address = 0x0_8038]	91
5.1.16	QUEUE0_STATUS [Address = 0x0_8040]	92
5.1.17	QUEUE1_STATUS [Address = 0x0_8048]	93
5.1.18	QUEUE2_STATUS [Address = 0x0_8050]	93
5.1.19	QUEUE3_STATUS [Address = 0x0_8058]	94
5.1.20	QUEUE4_STATUS [Address = 0x0_8060]	95
5.1.21	QUEUE5_STATUS [Address = 0x0_8068]	95
5.1.22	QUEUE6_STATUS [Address = 0x0_8070]	96
5.1.23	QUEUE7_STATUS [Address = 0x0_8078]	96
5.1.24	MMU_PID [Address = 0xA000]	98
5.1.25	TDDR [Address = 0xA100]	99
5.1.26	TDFAR [Address = 0xA140]	100
5.1.27	TLB_INV [Address = 0xA200]	101
5.1.28	TLB_INVC [Address = 0xA280]	102
5.1.29	TLB_DBG [Address = 0xA2C0]	103
5.1.30	TLB_DBG_DATA0 [Address = 0xA300]	104
5.1.31	TLB_DBG_DATA1 [Address = 0xA340]	105
5.1.32	SCR [Address = 0xA400]	106
5.1.33	TCR0 [Address = 0xA440]	107
5.1.34	TCR1 [Address = 0xA480]	108
5.1.35	TBR0 [Address = 0xA4C0]	109
5.1.36	TBR1 [Address = 0xA500]	110
5.1.37	MAR [Address = 0xA540]	111
5.1.38	TDAR [Address = 0xA580]	112
5.1.39	SCR_GS [Address = 0xA800]	113
5.1.40	TCR0_GS [Address = 0xA840]	114
5.1.41	TCR1_GS [Address = 0xA880]	115
5.1.42	TBR0_GS [Address = 0xA8C0]	116
5.1.43	TBR1_GS [Address = 0xA900]	117
5.1.44	MAR_GS [Address = 0xA940]	118
5.1.45	TDAR_GS [Address = 0xA980]	119
5.1.46	SCR_S [Address = 0xAC00]	120
5.1.47	TCR0_S [Address = 0xAC40]	121
5.1.48	TCR1_S [Address = 0xAC80]	122
5.1.49	TBR0_S [Address = 0xACC0]	123
5.1.50	TBR1_S [Address = 0xAD00]	124
5.1.51	MAR_S [Address = 0xAD40]	125
5.1.52	TDAR_S [Address = 0xAD80]	126
5.1.53	VCR [Address = 0xB800]	127

5.1.54	VTCT [Address = 0xB840]	128
5.1.55	VTBR [Address = 0xB880]	129
5.1.56	VTDR [Address = 0xB8C0]	130
5.1.57	RD_UTLB_MATCH [Address = 0xC000]	131
5.1.58	RD_UTLB_ATTR [Address = 0xD000]	132
5.1.59	WR_UTLB_MATCH [Address = 0xE000]	133
5.1.60	WR_UTLB_ATTR [Address = 0xF000]	134
5.1.61	MSMC SRAM MEMORY ATTRIBUTES [Address = 0x1_0000]	134
5.1.62	DDR MEMORY ATTRIBUTES [Address = 0x2_0000]	135
5.1.63	DDR BOOT MEMORY ATTRIBUTES [Address = 0x3_0000]	136
5.1.64	CHCFG [Address = 0x4_0000]	137
5.1.65	CHOS0 [Address = 0x4_0020]	138
5.1.66	CHRING_ADDR [Address = 0x4_0040]	139
5.1.67	CHRING_SIZE [Address = 0x4_0048]	139
5.1.68	CHST_SCHED [Address = 0x4_0060]	140
5.1.69	CHRT_CTL [Address = 0x6_0000]	140
5.1.70	CHRT_SWTRIG [Address = 0x6_0008]	141
5.1.71	CHRT_ERROR_STAT [Address = 0x6_0010]	142
5.1.72	CHRT_ERROR_CNT [Address = 0x6_0018]	143
5.1.73	CHRT_RING_FWD_DB [Address = 0x6_0020]	143
5.1.74	CHRT_RING_FWD_OCC [Address = 0x6_0028]	144
5.1.75	CHRT_RING_RVRS_DB [Address = 0x6_0040]	144
5.1.76	CHRT_RING_RVRS_OCC [Address = 0x6_0048]	145
5.1.77	ATOMIC_SUBMIT_CURR_TR_WORD0_1 [Address = 0x8_0000]	146
5.1.78	ATOMIC_SUBMIT_CURR_TR_WORD2_3 [Address = 0x8_0008]	146
5.1.79	ATOMIC_SUBMIT_CURR_TR_WORD4_5 [Address = 0x8_0010]	147
5.1.80	ATOMIC_SUBMIT_CURR_TR_WORD6_7 [Address = 0x8_0018]	147
5.1.81	ATOMIC_SUBMIT_CURR_TR_WORD8_9 [Address = 0x8_0020]	148
5.1.82	ATOMIC_SUBMIT_CURR_TR_WORD10_11 [Address = 0x8_0028]	148
5.1.83	ATOMIC_SUBMIT_CURR_TR_WORD12_13 [Address = 0x8_0030]	149
5.1.84	ATOMIC_SUBMIT_CURR_TR_WORD14_15 [Address = 0x8_0038]	149
5.1.85	NEXT_TR_WORD0_1 [Address = 0x8_0040]	150
5.1.86	NEXT_TR_WORD2_3 [Address = 0x8_0048]	150
5.1.87	NEXT_TR_WORD4_5 [Address = 0x8_0050]	151
5.1.88	NEXT_TR_WORD6_7 [Address = 0x8_0058]	151
5.1.89	NEXT_TR_WORD8_9 [Address = 0x8_0060]	152
5.1.90	NEXT_TR_WORD10_11 [Address = 0x8_0068]	152
5.1.91	NEXT_TR_WORD12_13 [Address = 0x8_0070]	153
5.1.92	NEXT_TR_WORD14_15 [Address = 0x8_0078]	153
5.1.93	NEXT_NEXT_TR_WORD0_1 [Address = 0x8_0080]	154
5.1.94	NEXT_NEXT_TR_WORD2_3 [Address = 0x8_0088]	154
5.1.95	NEXT_NEXT_TR_WORD4_5 [Address = 0x8_0090]	155
5.1.96	NEXT_NEXT_TR_WORD6_7 [Address = 0x8_0098]	155
5.1.97	NEXT_NEXT_TR_WORD8_9 [Address = 0x8_00A0]	156
5.1.98	NEXT_NEXT_TR_WORD10_11 [Address = 0x8_00A8]	156
5.1.99	NEXT_NEXT_TR_WORD12_13 [Address = 0x8_00B0]	157
5.1.100	NEXT_NEXT_TR_WORD14_15 [Address = 0x8_00B8]	157
5.1.101	CURR_STATE_WORD0_1 [Address = 0x8_00C0]	158
5.1.102	CURR_STATE_WORD2_3 [Address = 0x8_00C8]	158
5.1.103	CURR_STATE_WORD4_5 [Address = 0x8_00D0]	159

5.1.104	CURR_STATE_WORD6_7 [Address = 0x8_00D8]	159
5.1.105	CURR_STATE_WORD8_9 [Address = 0x8_00E0]	160
5.1.106	CURR_STATE_WORD10_11 [Address = 0x8_00E8]	160
5.1.107	CURR_STATE_WORD12_13 [Address = 0x8_00F0]	161
5.1.108	CURR_STATE_WORD14_15 [Address = 0x8_00F8]	161
5.1.109	CORE0_SUBMIT_WORD0_1 [Address = 0xA_0000]	162
5.1.110	CORE0_SUBMIT_WORD2_3 [Address = 0xA_0008]	162
5.1.111	CORE0_SUBMIT_WORD4_5 [Address = 0xA_0010]	163
5.1.112	CORE0_SUBMIT_WORD6_7 [Address = 0xA_0018]	163
5.1.113	CORE0_SUBMIT_WORD8_9 [Address = 0xA_0020]	164
5.1.114	CORE0_SUBMIT_WORD10_11 [Address = 0xA_0028]	164
5.1.115	CORE0_SUBMIT_WORD12_13 [Address = 0xA_0030]	165
5.1.116	CORE0_SUBMIT_WORD14_15 [Address = 0xA_0038]	165
5.1.117	CORE1_SUBMIT_WORD0_1 [Address = 0xA_0040]	166
5.1.118	CORE1_SUBMIT_WORD2_3 [Address = 0xA_0048]	166
5.1.119	CORE1_SUBMIT_WORD4_5 [Address = 0xA_0050]	167
5.1.120	CORE1_SUBMIT_WORD6_7 [Address = 0xA_0058]	167
5.1.121	CORE1_SUBMIT_WORD8_9 [Address = 0xA_0060]	168
5.1.122	CORE1_SUBMIT_WORD10_11 [Address = 0xA_0068]	168
5.1.123	CORE1_SUBMIT_WORD12_13 [Address = 0xA_0070]	169
5.1.124	CORE1_SUBMIT_WORD14_15 [Address = 0xA_0078]	169
5.1.125	CORE2_SUBMIT_WORD0_1 [Address = 0xA_0080]	170
5.1.126	CORE2_SUBMIT_WORD2_3 [Address = 0xA_0088]	170
5.1.127	CORE2_SUBMIT_WORD4_5 [Address = 0xA_0090]	171
5.1.128	CORE2_SUBMIT_WORD6_7 [Address = 0xA_0098]	171
5.1.129	CORE2_SUBMIT_WORD8_9 [Address = 0xA_00A0]	172
5.1.130	CORE2_SUBMIT_WORD10_11 [Address = 0xA_00A8]	172
5.1.131	CORE2_SUBMIT_WORD12_13 [Address = 0xA_00B0]	173
5.1.132	CORE2_SUBMIT_WORD14_15 [Address = 0xA_00B8]	173
5.1.133	CORE3_SUBMIT_WORD0_1 [Address = 0xA_00C0]	174
5.1.134	CORE3_SUBMIT_WORD2_3 [Address = 0xA_00C8]	174
5.1.135	CORE3_SUBMIT_WORD4_5 [Address = 0xA_00D0]	175
5.1.136	CORE3_SUBMIT_WORD6_7 [Address = 0xA_00D8]	175
5.1.137	CORE3_SUBMIT_WORD8_9 [Address = 0xA_00E0]	176
5.1.138	CORE3_SUBMIT_WORD10_11 [Address = 0xA_00E8]	176
5.1.139	CORE3_SUBMIT_WORD12_13 [Address = 0xA_00F0]	177
5.1.140	CORE3_SUBMIT_WORD14_15 [Address = 0xA_00F8]	177
5.1.141	CORE4_SUBMIT_WORD0_1 [Address = 0XC0XC_0000]	178
5.1.142	CORE4_SUBMIT_WORD2_3 [Address = 0XC_0008]	178
5.1.143	CORE4_SUBMIT_WORD4_5 [Address = 0XC_0010]	179
5.1.144	CORE4_SUBMIT_WORD6_7 [Address = 0XC_0018]	179
5.1.145	CORE4_SUBMIT_WORD8_9 [Address = 0XC_0020]	180
5.1.146	CORE4_SUBMIT_WORD10_11 [Address = 0XC_0028]	180
5.1.147	CORE4_SUBMIT_WORD12_13 [Address = 0XC_0030]	181
5.1.148	CORE4_SUBMIT_WORD14_15 [Address = 0XC_0038]	181
5.1.149	CORE5_SUBMIT_WORD0_1 [Address = 0XC_0040]	182
5.1.150	CORE5_SUBMIT_WORD2_3 [Address = 0XC_0048]	182
5.1.151	CORE5_SUBMIT_WORD4_5 [Address = 0XC_0050]	183
5.1.152	CORE5_SUBMIT_WORD6_7 [Address = 0XC_0058]	183
5.1.153	CORE5_SUBMIT_WORD8_9 [Address = 0XC_0060]	184

5.1.154	CORE5_SUBMIT_WORD10_11 [Address = 0XC_0068]	184
5.1.155	CORE5_SUBMIT_WORD12_13 [Address = 0XC_0070]	185
5.1.156	CORE5_SUBMIT_WORD14_15 [Address = 0XC_0078]	185
5.1.157	CORE6_SUBMIT_WORD0_1 [Address = 0XC_0080]	186
5.1.158	CORE6_SUBMIT_WORD2_3 [Address = 0XC_0088]	186
5.1.159	CORE6_SUBMIT_WORD4_5 [Address = 0XC_0090]	187
5.1.160	CORE6_SUBMIT_WORD6_7 [Address = 0XC_0098]	187
5.1.161	CORE6_SUBMIT_WORD8_9 [Address = 0XC_00A0]	188
5.1.162	CORE6_SUBMIT_WORD10_11 [Address = 0XC_00A8]	188
5.1.163	CORE6_SUBMIT_WORD12_13 [Address = 0XC_00B0]	189
5.1.164	CORE6_SUBMIT_WORD14_15 [Address = 0XC_00B8]	189
5.1.165	CORE7_SUBMIT_WORD0_1 [Address = 0XC_00C0]	190
5.1.166	CORE7_SUBMIT_WORD2_3 [Address = 0XC_00C8]	190
5.1.167	CORE7_SUBMIT_WORD4_5 [Address = 0XC_00D0]	191
5.1.168	CORE7_SUBMIT_WORD6_7 [Address = 0XC_00D8]	191
5.1.169	CORE7_SUBMIT_WORD8_9 [Address = 0XC_00E0]	192
5.1.170	CORE7_SUBMIT_WORD10_11 [Address = 0XC_00E8]	192
5.1.171	CORE7_SUBMIT_WORD12_13 [Address = 0XC_00F0]	193
5.1.172	CORE7_SUBMIT_WORD14_15 [Address = 0XC_00F8]	193
5.1.173	CAUSEX [Address = 0XE_0000]	194
Detailed Micro-Architecture		197
6.1	Transfer controllers	197
6.2	Queue Controller	198
6.2.1	TR Queues	198
6.2.2	Cache Warm Queue	203
6.3	Read Engine	203
6.3.1	Read Engine TR Handling	203
6.4	Read Arb	206
6.5	Read Command Interface	207
6.5.1	MSMC	207
6.6	Read Data Interface	208
6.6.1	MSMC	208
6.7	Read Buffer Control	208
6.8	Data buffer	209
6.9	Read Command Interface Logic	209
6.10	Write Engine Data Available Logic	209
6.10.1	Read Data Interface Logic	209
6.10.2	Write Engine Read Data Logic	209
6.11	Write Data Engine	210
6.11.1	Write Engine TR Handling	210
6.11.2	Write Engine Size Calculation	211
6.11.3	Write Engine Address Calculation	212
6.11.4	Virtual to Physical Translation	212
6.11.5	Write Engine Data Manipulation	213
6.11.6	TR Completion Handling	214
6.12	Write Arbitration	214
6.12.1	Write Controller	215
6.12.2	Space Occupied Rules	215
6.12.3	Outstanding write commands for a TR	216
6.12.4	Write Status Errors	216

6.13	Write Command Interface	216
6.13.1	Error Behavior	216
6.13.2	Detailed Functional Description	216
User Manual		239
7.1	Performance	239
7.1.1	Virtual vs. Physical	239
7.1.2	Multiple Virtual TCs	239
7.1.3	Multiple Dimensions	239
7.2	Reset and Power Down	239
7.3	Errors	239
7.3.1	Error Descriptor	240
7.3.2	Error Cases	241
7.3.3	TR and CR Format/Submission Errors	242
7.3.4	Read Transmission Errors	243
7.3.5	Write Transmission Errors and Write Completion Errors	243
7.4	Security	243
7.5	Emulation Support	243
7.6	Debug	243
8.1	Configuration History	245
8.1.1	DRU Maxwell Configuration	245
8.1.2	DRU J7ES Configuration	246
8.1.3	DRU UTC DMPAC Configuration for J7ES	247
8.1.4	DRU UTC VPAC for J7ES	248
8.1.5	DRU UTC 32CH for J7Quad	249
8.1.6	DRU UTC 64 Ch Configuration for J7Quad	250
8.1.7	DRU VC Configuration for J7Quad	252
8.1.8	DRU VC R Configuration for AM67	254
8.1.9	DRU UTC 32CH for J7Quad	255
8.1.10	DRU UTC 64 Ch R Configuration for AM67	257
9.1	Requirements Traceability	261
9.1.1	DRU Maxwell Configuration	261
9.1.2	DRU J7ES Configuration	262
9.1.3	DRU UTC DMPAC Configuration for J7ES	263
9.1.4	DRU UTC VPAC for J7ES	264
9.1.5	DRU UTC 32CH for J7Quad	265
9.1.6	DRU UTC 64 Ch Configuration for J7Quad	266
9.1.7	DRU VC Configuration for J7Quad	267
9.1.8	DRU VC R Configuration for AM67	269
9.1.9	DRU UTC 32CH R for AM67	270
9.1.10	DRU UTC 64 Ch R Configuration for J7Quad	271
UTC TR Format Specification		273
10.1	Overview	274
10.2	Addressing algorithm	275
10.3	Transfer Request Formats	277
10.4	Flags Field Definition	278
10.4.1	Type: TR Type Field	279
10.4.2	STATIC: Static Field Definition	281
10.4.3	EOL: End of Line Definition	281
10.4.4	EVENT_SIZE: Event Generation Definition	281
10.4.5	TRIGGER INFO: TR Triggers	282

10.4.6	CMD ID: Command ID Field Definition	282
10.4.7	Configuration Specific Flags Definition	282
10.5	TR Address and Size Attributes.....	282
10.5.1	ICNT0	282
10.5.2	ICNT1	283
10.5.3	ADDR.....	283
10.5.4	DIM1	283
10.5.5	ICNT2	283
10.5.6	ICNT3	283
10.5.7	DIM2	283
10.5.8	DIM3	283
10.5.9	DDIM1	283
10.5.10	DADDR.....	283
10.5.11	DDIM2	283
10.5.12	DDIM3	284
10.5.13	DICNT0	284
10.5.14	DICNT1	284
10.5.15	DICNT2	284
10.5.16	DICNT3	284
10.6	FMTFLAGS	284
10.6.1	AMODE: Addressing Mode Definition	285
10.6.2	DIR: Addressing Mode Direction Definition	286
10.6.3	ELTYPE: Element Type Definition	286
10.6.4	DFMT:Data Formatting Algorithm Definition	287
10.6.5	SECTR: Secondary Transfer Request Definition.....	287
10.6.6	AMODE SPECIFIC: Addressing Mode Field	291
10.6.7	Cache Flags.....	293
10.7	Compression and Decompression	295
10.7.1	Compression and Decompression Addressing Description	295
10.7.2	Compression and Decompression Table Formats.....	298
10.7.3	Compression and Decompression Examples.....	303
10.8	TR Response	311
10.8.1	TR Response Format	311
10.8.2	STATUS Field definition.....	311
	Accelerator Framework Compression Module	315
12.1	Requirements	317
12.1.1	Supported Data Types	317
12.1.2	Required Functions	317
12.1.3	Performance.....	317
12.1.4	Area	318
12.1.5	Safety	318
12.2	Theory of Operation.....	319
12.2.1	Zero Compression	321
12.2.2	Unsigned Exponential Golomb	331
12.2.3	Signed Exponential Golomb.....	341
12.2.4	Parallel Encode and Decode	351
12.3	Data Structures.....	353
12.3.1	Configuration Constants.....	353
12.4	Programming	367
12.4.1	Compressing data of known size and bounded compressed size	367

12.4.2	Compressing data of known size and unbounded compressed size.....	367
12.4.3	Compressing data of unknown size and bounded compressed size.....	367
12.4.4	Decompressing data of known compressed size and bounded uncompressed size	368
12.4.5	Decompressing data of known compressed size and unbounded uncompressed size	368
12.4.6	Decompressing data of unknown compressed size and bounded uncompressed size	369
12.5	Errors.....	371
12.6	Performance	379
12.7	Appendix - Exporting Tables.....	381
12.8	References	383
AFC Micro-Architecture		385
13.1	Compression	387
13.1.1	Compression Ingress	387
13.1.2	Compression Engine.....	389
13.1.3	Compression Prefix Merge Datapath.....	390
13.1.4	Compression Egress.....	393
13.2	Decompression	395
13.2.1	Decompression Ingress.....	395
13.2.2	Decompression Engine	396
13.3	Performance and Area	403
13.3.1	Compression Performance.....	403
13.3.2	Decompression Performance	403
13.3.3	Area	404
Revision History		407

3.1 = References

Table 3-1 Table of References

Document	Version	Links	Description
TPTC	1.0	https://pds.design.ti.com/cgi-bin/internaldocs?docconfigid=49&filtertitle=172	3rd Party Transfer Controller that the DRU is replacing
TPCC	1.0	https://pds.design.ti.com/cgi-bin/internaldocs?docconfigid=49&filtertitle=171	3rd Party Channel Controller that interacts with the DRU to allow for TR linking and chaining
DMA Architecture Specification	1.0.14	https://pds.design.ti.com/d/21/2180/11105/12454/11/ks3_dma_architecture_1p0p14.docx	The overall KS3 DMA Architecture that the DRU falls under
CBA Protocol	4.0	TBD	The Common Bus Architecture used in Keystone3 Architecture. T
PSI-L Protocol	1.2.2	https://pds.design.ti.com/d/21/2180/11107/12456/1/psi_link_interface_1p2p2.docx	The messaging interface used to communicate between the CC, VTU and DRU
PECC module	1.0	https://pds.design.ti.com Group = DSP, Docs/Project = MCAD	Parameterizable ECC Spec
Firewall Module	1.0.6	https://pds.design.ti.com/d/21/2207/10815/12131/10/K3_fw_module_v1p0p10.docx	Firewall Used for External DMA Transfers and Global MMR protection
CBASS	4.0.10	https://pds.design.ti.com/d/21/2181/11123/12475/5/ksbus_cbass_4p0p10.doc	CBASS Used for Firewall for Channel Registers
AFC_arch	1.0.0	"Accelerator Framework Compression Module"	Accelerator Framework Compression Architecture Specification
AFC_uarch	1.0.0	"AFC Micro-Architecture"	Accelerator Framework Compression Micro Architecture Specification
Keystone 3 Low cost DMA Architecture	1.0.28	Keystone3 Low Cost DMA Architecture	Keystone 3 DMA Architecture for Low Cost Designs
End of Table 3-1			

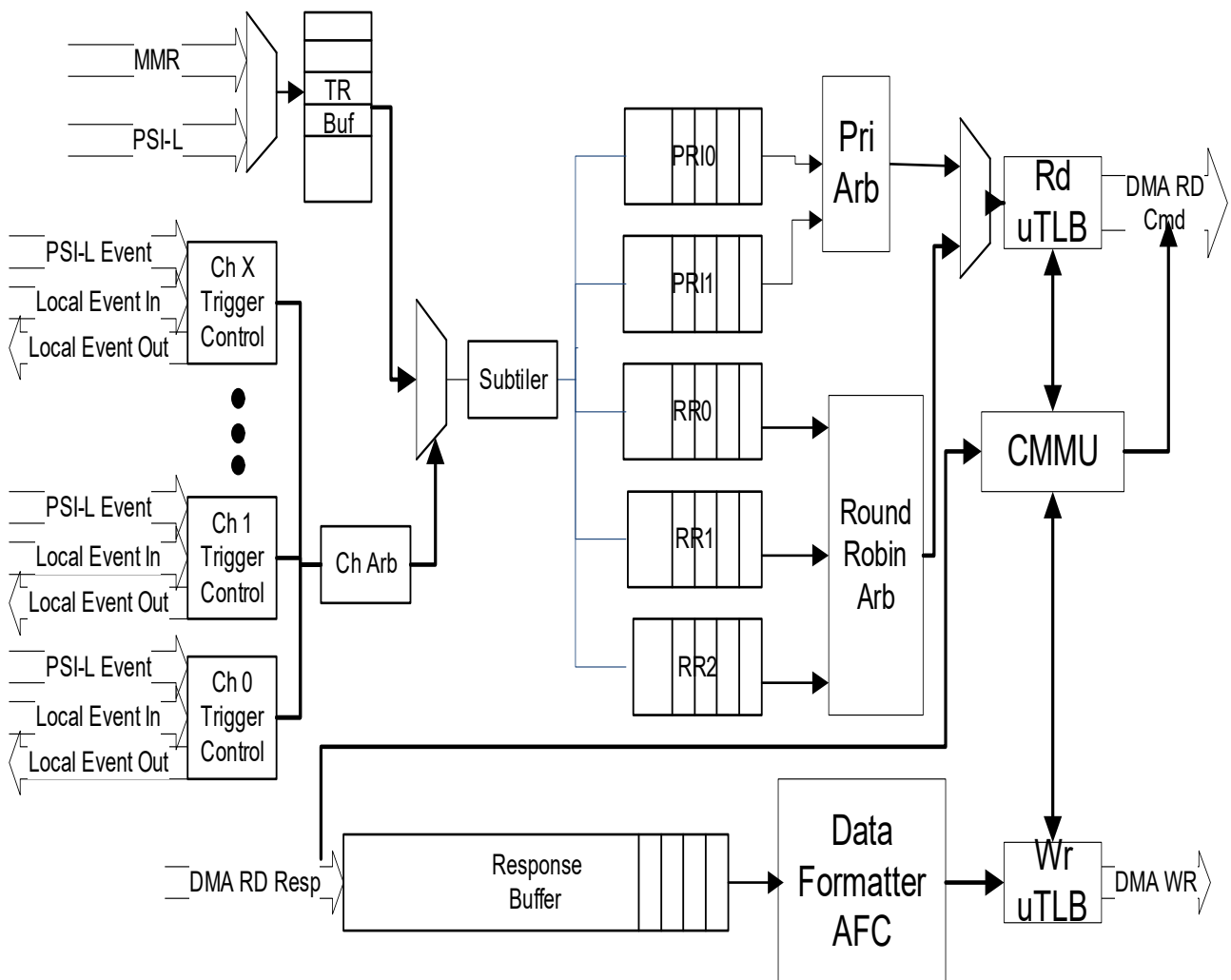
3.2 Terminology

Table 3-2 Terms and Definition

Term	Definition
Channel	An independent flow of data that can be given requests for data movement. Each channel operates independently of each other but do share common resources.
Queue	A shared unit in the controller that holds the order that channels have requested a portion of their data movement until the shared data engine can perform the actual data move. A channel is assigned a queue where its data movement requests will be placed.
TR	A transfer request to move data from a source location to a destination.
Sub TR	A one or two dimensional portion of a TR that is pushed into the queue for an active channel.
CR	A cache operation request. A request to send messages to a specified cache controller to prepare the cache for a future operation.
TC	Transfer Controller the module in the EDMA3 architecture that handled the actual movement of data requested in the TR.
CC	Channel controller in the EDMA 3 architecture. This sends TRs to the DRU and can do higher order linking of TRs and other functions.
VTC	Virtual Transfer controller. The DRU can behave as multiple TCs to a channel controller
DTR	Direct Transfer Request. Mechanism to allow CorePac and ARM to submit a TR request directly to the DRU and not have to go through a CC
DCR	Direct Cache Operation Request. Mechanism to allow a single VBUSM.C write to submit a Cache operation
VTR	Virtual Transfer request is when the addresses in the Transfer request are NOT physical addresses but must be looked up in the IOMMU. A is determined by the IOSID of the TR which is either based on the virtid signal on a DTR or on the PSI-L bus for a submitted TR.
End of Table 3-2	

Functional Description

Figure 3-1 DRU Functional Diagram



1.

3.3 DRU Basic Functionality

The DRU receives commands to transfer data from one location to another (“TR”) or to send messages to CPUs or MMUs to pre-warm logic (“CR”). The DRU can receive these commands through two mechanisms. A direct write of MMR registers or through a TR submission over PSI-L. Once the TR has been received the Channel Trigger Control detects when the triggers have been received and arbitrates with all the other channels to use the internal subtiler. The internal subtiler takes the winning channel and generates the next 1D or 2D transfer based on the trigger size, transfer type and event size specified in the TR and places the new Sub-TR into the queue for the channel. Once the Sub-TR reaches the front of its queue it will arbitrate with the other queues to send out the read request on the DMA bus. Once the read response returns for the read the data formatter will pull the data out of the response buffer and reformat the data in the proper alignment for the write transfer and send the data out the DMA write interface.

3.3.1 Channel Configuration and TR Submission

The DRU operates on having multiple channels that can run in parallel on the shared hardware but be independently triggered by events to flow as needed. Each channel must be configured before it is used. The channel configuration has two parts the non-realtime configuration, which is intended to be controlled by a single master in the system for all software processes and processors, and the real time configuration which is controlled by the software process that owns the channel.

3.3.1.1 Non-realtime Channel Configuration

The non-realtime configuration is done through MMR writes to the Channel Configuration registers located in the region 0x40000 to 0x5ffff of the DRU memory map. The address bits from 16 to 8 select the channel to be used for the configuration. If the channel does not exist in the DRU configuration a write will be met with an address error.

The non-realtime configuration consists of assigning the channel to a submission mechanism and a hardware queue and event to use for its transactions and notifications. These registers must be written to before the channel is enabled in its realtime configuration register as once a channel is enabled any write to the non-realtime configuration registers will result in the write failing with an address error.

3.3.1.2 Realtime Channel Configuration Registers

Once the channel has been configured through the non-realtime configuration register the Realtime Channel configuration register should be written. The Enable bit in the CH_RTCTL register must be written before any TRs can be written. The mechanism to write this bit is based on the ownership of the channel in the CHCFG register. If the owner is CPU then the MMR register must be used to write this register. If the owner

is UDMA than the PSI-L must be used to write the register through the PSI-L register configuration write process described in the PSI-L specification. The MMR registers are addressed the same way as the configuration register with bits 16:8 representing the channel that is being used.



WARNING—Once a channel has been enabled it can only be disabled through writing the teardown bit the same way the enable bit was written and none of the CHCFG Registers can be modified.

The CHRT_SWTRIG register can only be accessed as an MMR and is used if software wants to control the triggers for the TR.

3.3.1.3 TR Submission

DRU supports two methods to provide TRs, direct TRs through MMR accesses or through PSI-L submission through a UDMA.

TR Submission checks that a legal TR has been submitted, that the correct submission method has been used and that there is space in the channel TR FIFO for the new TR.

The following fields are checked in the TR to ensure that a TR is legal.

Table 3-3 TR Submission Error Checks

Field Check	Legal Values
Flag Type	Types supported by the DRU as specified in the CAPABILITIES Register
Flag STATIC	0 unless STATIC is specified in the CAPABILITIES Register
Flag EOL	0 unless types supported other than the Block DMA type 8,10 and 5 as specified in the CAPABILITIES Register.
ICNT0	Anything other than 0
FMTFLAGS AMODE	0 unless circular addressing is supported as specified in the CAPABILITIES Register
FMT_FLAGS ELTYPE	Values less than or equal to the value specified in the CAPABILITIES Register
FMT_FLAGS CONSTANT COPY	0 unless it is supported as specified in the CAPABILITIES Register.
FMT_FLAGS DIR	0 unless it is supported as specified in the CAPABILITIES Register
FMT_FLAGS DFMT	Values less than or equal to the value specified in the CAPABILITIES Register
FMT_FLAGS SECONDARY TR	Values less than or equal to the value specified in the CAPABILITIES Register
FMT_FLAGS AMODE SPECIFIC	Value is 0 if AMODE is 0. Otherwise the legality is based on the specific AMODE supported.
CACHE OPERATION	0 unless MMU warm feature is supported
COMPRESSION	FLAG_STATIC == 0 && FMT_FLAGS_ELTYPE == 6 && FMT_FLAGS_SEC_TR == 1 SRC_ADDR0[4:0] == 0 && SRC_DIM1[2:0] == 0 && SRC_DIM2[2:0] == 0 && SRC_DIM3[2:0] == 0
DECOMPRESSION	FLAG_STATIC == 0 && FMT_FLAGS_ELTYPE == 6 && FMT_FLAGS_SEC_TR == 1 SRC_ADDR0[4:0] == 0 && SRC_DIM1[2:0] == 0 && SRC_DIM2[2:0] == 0 && SRC_DIM3[2:0] == 0
CONSTANT COPY	FLAG_STATIC == 0 && FMT_FLAGS_ELTYPE < 8
TRANPOSE	(FLAGS_TYPE == 8 FLAG_TYPE == 10) && ALL_TRIGGERS AND EVENT SIZE != ICNT1
End of Table 3-3	

3.3.1.4 Direct TR Submission

The direct TR submission is handled through MMR writes to one of two sets of registers. The Atomic submission register and the non-atomic CoreX submission registers.

The Atomic submission register requires the write in a single 64 byte burst and will check all the values from the burst for legal values and space in the channel FIFO and then push the TR into the FIFO and return the write status with success. If any of the legal TR format checks or FIFO space check or if the channel is not owned and ENABLED will result in an address error returned.



Note—The Error status registers will not be updated in this case as the TR is never run.

The Non-atomic CoreX submission registers provide a fixed number of registers that can be written to provide a TR. Only when the TR with Word 0 and the byte enable for the Type field are asserted will all the field of the register be combined and the legal values of the TR will be checked.



WARNING—If a channel is full any write to the Core Submission register will return an address error and the data in the local register will NOT be updated



WARNING—The Non-Atomic submission registers will only look at the channel region of the address for the address that requires a submission and will not provide any check that all registers were updated for the same channel.



WARNING—The DRU does not check anything to say that the same core writes to the same register for the core submission. It is assumed that some higher level of architecture or software will assign ownership of a Core register set for a single master.

3.3.1.5 PSI-L TR Submission

The PSI-L TR submission follows the PSI-L specification for TR submission. For every TR request a TR response will be sent. If the TR fails a legal check or channel ownership or FIFO fullness check an immediate TR response will be sent back with the cause of the submission error provided in the response.

3.3.1.6 Ring TR Submission

Starting with version 3.1 of the DRU a configuration option can be set to allow for direct ring support to be handled inside the DRU. This ring support follows the mechanism as defined in the Keystone 3 Low Cost DMA specification. The DRU only support a TR size of 64 bytes and must be aligned to that boundary as well.



WARNING—Ring support configuration can only be set if the PSIL data width is set to zero for the configuration. Submission from external channel support and ring support is not supported in the same DRU.

The Ring is defined by setting a base address and a size. Once this is written the pointer for the forward and reverse ring will be set to this base address for that channel.

Upon receiving a write to the forward doorbell register the DRU will send an up event on the forward ring event and then proceed to start pulling entries into the internal FIFO until either all entries have been consumed and the forward ring occupancy is zero or the internal FIFO is full.

On the initial ring or any time a ring descriptor is completed the DRU will fetch the ring entry to get the pointer to the Transfer Request Descriptor. Upon receiving the response the DRU will fetch Transfer Request Descriptor Packet Info Size and The DRU will require that the transfer request descriptor has a Transfer Request Nominal Element Size of 2 for 64 bytes. All other values in that field will cause the TR response to be sent for the first TR based on the specified element size and last entry field. The TR response in this case will always be sent with submission error and bad descriptor type.

If the Element size is correct the DRU will process the descriptor packet info portion of the descriptor to set loop counts for the descriptor as specified. The first TR which was fetched with packet info will be pushed into the channel TR FIFO and will begin the normal TR processing steps.

As the DRU proceeds through the TR the channel will try and pull in as many entries as possible in a single fetch based on the maximum burst size and the current space in the TR FIFO. Once the channel is active it will NOT fetch additional TRs until the read active for the channel is complete. If the channel reaches the last entry and the reload count value it will check the forward occupancy count and see if another ring entry can be fetched. Once the forward ring occupancy reaches zero and the last TR from the TR request descriptor has been loaded the forward ring down event will be sent to clear the event. This means all the TR fetches have been ordered but have not been completed.

The TRs will be fetched by the ring by using the secondary TR fetch mechanism and using a CID of all 1s and upon receiving the response the data is routed either back to the channel logic to update the channel fields or written into the internal FIFO by muxing into the PSIL path into the TR FIFO if the request was for a TR.

Once a TR is complete the response will be sent by the channel write queue to the location in the TR Packet Descriptor based on the current TR index. Once the last TR response is sent for a TR Packet descriptor the reverse ring occupancy will be incremented by 1. If this increment causes the reverse ring to go from zero to non-zero then a reverse ring event will be sent

The reverse ring doorbell can be written to reduce the reverse ring occupancy if it sets the occupancy to zero then the reverse ring event will be sent with a down event to clear the external event.

WARNING—A DRU in ring mode will not consider a channel idle until the reverse occupancy has been cleared. This means the DRU clock will continue to run until all channels in ring mode have the reverse occupancy cleared.

3.3.2 Basic Transfer Request Operation

The DRU memory to memory transfers works by receiving a transfer request (TR) or Cache Request (CR) through a TR submission and placing it in a queue to be processed. The TR will configure the event handler for the given channel to wait for the trigger specified in the TR. If the trigger is none then it is assumed to be always received.

Once both triggers have been received the channel will be considered active. All active channels will arbitrate to have the next Sub-TR generated by the subtiler. The arbitration of the channels is based on the queue for the channel. All priority queues will arbitrate on a fixed priority basis with the lowest channel having the highest priority. A round robin queue will arbitrate using a round robin using the same algorithm as used in the queue arbitration.

Once a channel is the highest priority active channel it is passed to the subtiler. The subtiler will generate a sub-TR and place that sub-TR in the queue specified for the channel. The sub-TR generated can be a single dimension or two dimensions based upon the smallest trigger size and formatting commands. Once the sub-TR is generated the event control is notified and any active triggers are cleared if the sub-TR crosses the

boundary of one of the triggers. For example, if a TR has a trigger size for a 2D block the active signal will be high from when the trigger is received until the subtiler has sent the TR. If a TR has a trigger size for a 3D block and the ICNT2 count is 16. Then the active signal will clear after 16 sub-TRs are sent to the queue.

Upon reaching the top of the queue the TR is processed. Based on the information in the TR a read of the specified location and size is performed. Followed by writing the data to the specified destination.

Upon sending the last write of a loop for the entire TR the completion event will be generated. The event for the final completion will occur upon receiving of all of the write responses before sending out the completion event. The completion event output will also place a TR response in the PSI-L response FIFO if the TR has a UDMA-C type.

3.3.3 Queue Structure

Every TR that is active is placed in one of the queues. Each queue operates independently of the other queues. A queue can be a fixed priority queue or a round-robin queue. In a given queue the TRs will complete in the order they are placed by the subtiler. Each queue can be configured to have a unique DMA Bus Priority. The queues can specify the number of consecutive commands to send and the number of commands to wait before arbitration. These attributes apply to both the channel arbitration and the queue arbitration.

A maximum number of 8 queues are supported by the DRU. The queues can be fixed priority queues, round robin queues or compression/decompression queues. The lowest number queues will always be the priority queues starting at queue 0 and each higher queue number has lower priority then the next. The first queue after the priority queues would be the round robin queues. The last queues would be the compression queue and then decompression queue if required.

For example if a DRU had 2 priority queues, 4 round robin queues and compression and decompression then the queues would be as follows:

Queue 0 - highest priority all TRs processed before other queues unless disabled by Req wait logic.

Queue 1 - second priority all TRs processed when Queue 0 is empty or disabled

Queue 2-5 - Round Robin Queues will alternate priority after each TR sent by another Round Robin Queue

Queue 6 - Compression Queue - Used to send compression data

Queue 7 - Decompression Queue - Used to send decompression data

3.3.4 Queue Arbitration

The DRU can have multiple queues but only one read and one write can be occurring at a time. The DRU can have up to 8 queues that are configured during RTL creation.

The queues use the same resources and so arbitration is required. The DRU gives priority to the priority queues with the highest priority being the queue with the lowest number. Whenever arbitration occurs if there is a TR in the priority queues it will always go and the round robin queues will be held off. If the priority queues are empty

than the round robin queues will be arbitrated through a round robin such that each queue will get to send its specified number of transactions and then arbitration will get to run next. On the write side a arbitration will end if the read response has not been received for the next command when the current command is completing.

The arbitration will happen the last phase of a transfer chunk. A transfer chunk is defined for the Round Robin Queues as sending the number of read transactions in the CONSECUTIVE_COMANDS field of the QUEUEX_CONFIG Register, or completing the TR.

Additionally, when a queue has exhausted its consecutive_commands count it will then wait for the number of commands specified by REARB_WAIT field before it sets its request to try and rearbitrate again. This should allow software to tune the data flow from each queue to prevent starvation.

The arbitration algorithm will look at the queues with TRs and the last of the Round Robin queues to run and the arbitration will be based on this. If a queue is running and no other queue has a TR then it will start its next chunk without stopping. Each time the arbitration is performed for round robin queues the queue below the current queue will be highest priority and the current queue will be the lowest. The Last Queue field for the round robin only updates when a round robin queue is the winner. If a priority queue is active then the current round robin will remain the highest priority round robin queue until is has exhausted its CONSECUTIVE_COMMANDS count or is empty.

Table 3-4 Round Robin Arbitration example for 4 queues used by the Round Robin Queues

Current/Last Queue	Highest Priority	Second Priority	Third Priority	Lowest Priority
Queue A	Queue B	Queue C	Queue D	Queue A
Queue B	Queue C	Queue D	Queue A	Queue B
Queue C	Queue D	Queue A	Queue B	Queue C
Queue D	Queue A	Queue B	Queue C	Queue D

End of Table 3-4

3.3.5 Address fetch algorithm and TR Formats

The Address fetch algorithm and TR and CR formats are defined in the [UTC TR Format Specification](#). The UTC TR format specification documents all features that can be supported by the DRU but not all configurations of the DRU support all features of the TR. The [DRU_CAPABILITIES \[Address = 0x8\]](#) for the specific DRU defines what features of the TR are supported in the DRU.

3.3.5.1 Data Reformatting and Alternate Addressing Functions

The TR format does allow for the option to perform some kinds of data reformatting or to alter from the standard addressing formats. All of these features are optional and are selected through the configuration option. If the DRU configuration does not support the data reformatting function or the alternate addressing formats then the DRU will respond with a submission error when the TR is submitted with these fields set to a value that requires this functionality. Additionally, some of these functions can also place limitations on trigger or event sizes that if these requirements are not met will also result in a submission error. The following subsections define each of these data reformatting functions and alternate addressing algorithms.

3.3.5.1.1 Transpose

Linear streams work for large classes of algorithms, but not all. In some cases it is desired to transpose rows to columns and columns to rows in the data structure. In order to help in these cases the DRU supports a transpose function.

To a first order, transposed addressing looks like this:

Example 3-1 Transposed Addressing Algorithm, Baseline

```
-----
// sptr is an element pointer.
// dptr is an element pointer
for (i3 = 0; i3 < ICNT3; i3++)
{
    sptr3 = sptr; // save current position before entering next level
    dptr3 = dptr; // save current position before entering next level
    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptr2 = sptr; // save current position before entering next level
        dptr2 = dptr; // save current position before entering next level
        for (i0 = 0; i0 < ICNT0; i0++)
        {
            sptr0 = sptr; // save current position before entering next level
            dptr0 = dptr; // save current position before entering next level
            for (i1 = 0; i1 < ICNT1; i1++)
            {
                *dptr = *sptr;
                sptr = sptr + SDIM1/ELEM_SIZE;
                dptr = dptr ++;
            }
            // Update based on saved pointer for this level
            sptr = sptr0 + 1;
            dptr = dptr0 + DDIM1/ELEM_SIZE;
        }
        // Update based on saved pointer for this level
        sptr = sptr2 + SDIM2/ELEM_SIZE;
        dptr = dptr2 + SDIM2/ELEM_SIZE;
    }
    // Update based on saved pointer for this level
    sptr = sptr3 + DDIM3/ELEM_SIZE;
    dptr = dptr3 + DDIM3/ELEM_SIZE;
}
-----
```

End of Example 3-1

The above algorithm works, but results in inefficient accesses to memory. Since the DRU works on blocks of data it will attempt to fetch a row of data on a 128 byte boundary and then fetch the same number of columns to equal the number of elements in a row and then send the writes. The below

Example 3-2 Transposed Stream Addressing, with Block Fetch

```
-----
// sptr is an element pointer.
// dptr is an element pointer.
for (i3 = 0; i3 < ICNT3; i3++)
{
    sptr3 = sptr; // save current position before entering next level
    dptr3 = dptr; // save current position before entering next level
    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptr2 = sptr; // save current position before entering next level
        dptr2 = dptr; // save current position before entering next level
        for (i1 = 0; i1 < ICNT1; i1 += 128/ELEM_SIZE){
            sptr1 = sptr; // save current position before entering next level
            dptr1 = dptr; // save current position before entering next level
            for (i0 = 0; i0 < ICNT0; i0 += 128/ELEM_SIZE )
            {
                sptr0 = sptr; // save current position before entering next level
                dptr0 = dptr; // save current position before entering next level
                for (i = 0; i < 128/ELEM_SIZE; i++)
                {
                    for (j = 0; j < 128/ELEM_SIZE; j++) {
```

```

        temp[i][j] = *sptr;
        sptr++;
    }
    sptr = sptr + SDIM1/ELEM_SIZE;
}
for (i = 0; i < 128/ELEM_SIZE; i++)
{
    for (j = 0; j < 128/ELEM_SIZE; j++)
        *dptr = temp[j][i];
    dptr++;
}
dptr = dptra1 + DDIM1/ELEM_SIZE
}
sptr = sptr0 + 128/ELEM_SIZE;
dptra = dptra0 + (DDIM1/ELEM_SIZE * 128/ELEM_SIZE);
} // i0 loop
sptr = sptra1 + (SDIM1/ELEM_SIZE * 128/ELEM_SIZE);
dptra = dptra1 + 128/ELEM_SIZE;
} // i1 loop
// Update based on saved pointer for this level
sptr = sptr2 + SDIM2/ELEM_SIZE;
dptra = dptra2 + DDIM2/ELEM_SIZE;
}
// Update based on saved pointer for this level
sptr = sptr3 + SDIM3/ELEM_SIZE;
dptra = dptra3 + DDIM3/ELEM_SIZE;
}

```

End of Example 3-2

The above algorithm assumes a forward direction stream. Reverse direction streams visit dimension 0 in decreasing-address order.

Example 3-3 Reversed, Transposed Stream Addressing, with Block Fetch

```

// sptr is an element pointer.
// dptra is an element pointer.
for (i3 = 0; i3 < ICNT3; i3++)
{
    sptra3 = sptr; // save current position before entering next level
    dptra3 = dptra; // save current position before entering next level
    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptra2 = sptr; // save current position before entering next level
        dptra2 = dptra; // save current position before entering next level
        for (i1 = 0; i1 < ICNT1; i1 += 128/ELEM_SIZE) {
            sptra1 = sptr; // save current position before entering next level
            dptra1 = dptra; // save current position before entering next level
            for (i0 = 0; i0 < ICNT0; i0 += 128)
            {
                sptra0 = sptr; // save current position before entering next level
                dptra0 = dptra; // save current position before entering next level
                for (i = 0; i < 128/ELEM_SIZE; i++)
                {
                    for (j = 0; j < 128/ELEM_SIZE; j++) {
                        sptra--;
                        temp[i][j] = *sptra;
                    }
                    sptra = sptra + SDIM1/ELEM_SIZE;
                }
                for (i = 0; i < 128/ELEM_SIZE; i++)
                {
                    for (j = 0; j < 128/ELEM_SIZE; j++)
                    {
                        *dptra = temp[j][i];
                        dptra++;
                    }
                    dptra = dptra1 + DDIM1/ELEM_SIZE
                }
                sptra = sptra0 + 128/ELEM_SIZE;
                dptra = dptra0 + (DDIM1/ELEM_SIZE * 128/ELEM_SIZE);
            } // i0 loop
            sptra = sptra1 + (SDIM1/ELEM_SIZE * 128/ELEM_SIZE);
            dptra = dptra1 + 128/ELEM_SIZE;
        } // i1 loop
        // Update based on saved pointer for this level
    }
}

```

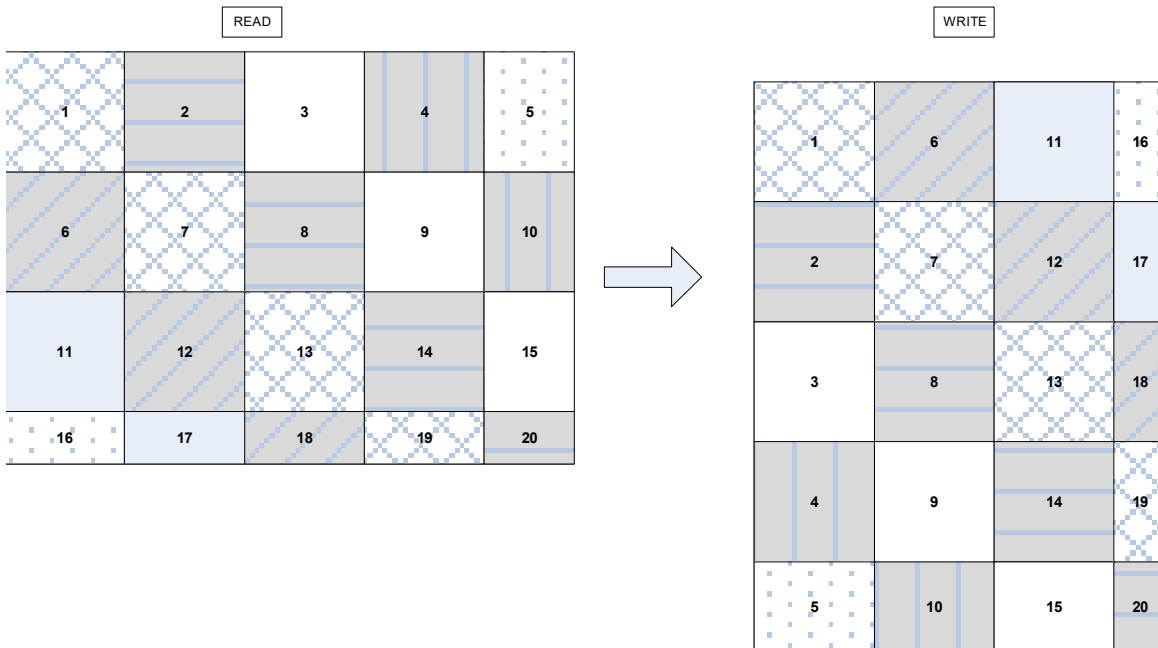
```

    sptr = sptr2 + SDIM2/ELEM_SIZE;
    dptr = dptr2 + DDIM2/ELEM_SIZE;
}
// Update based on saved pointer for this level
sptr = sptr3 + SDIM3/ELEM_SIZE;
dptr = dptr3 + DDIM3/ELEM_SIZE;
}

```

End of Example 3-3

Figure 3-2 Example 2D Breakdown of Transposed TR



The Transpose type will work on element sizes 32 bit, 64 bit or 128 bit and work by sending sub-TRs in the size of 32 elements by 32 lines for 32 bit, 16 elements by 16 lines for 64 bit and 8 elements by 8 lines for 128 bit element. The sub-TR that is being sent will always result in the maximum transfer size but if the address for the line on the read is not aligned the read request will be sent with an additional bit in the CID to mark the order of the request being sent so that the data can be properly shifted over so that for the write requests the data in the response buffer will always look as a buffer aligned to the 0 offset in the address.

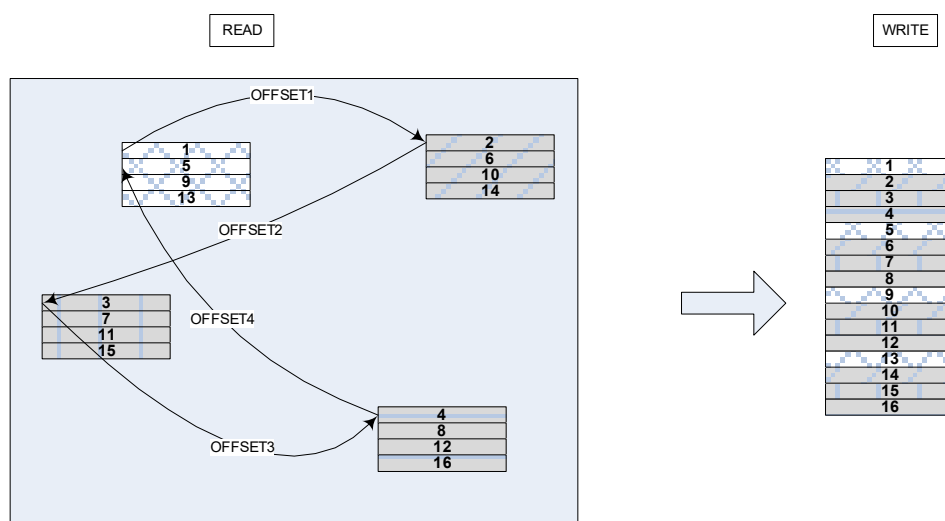
Since the transpose function requires a reorganization of the buffer the response buffer will be requested to reserve buffer space for the transpose as soon as a channel receives the trigger and the transpose is set for the active TR it will send a transpose arbitration request and will not forward a channel request until it has received the response that it has won the arbitration for the response buffer and the response buffer is in transpose mode.

Any other multiple arbitration modes such as multiple read offset will need to be won in arbitration before the channel can arbitrate for the transpose feature. The channel will release the transpose buffer and rearbitrate when it completes the smallest trigger size for the TR. This means if the TR sets the smallest trigger size to complete TR then the channel will arbitrate for the transpose and once it wins arbitration it will be free to run each sub-TR until it finishes the TR. If the trigger size is 2D then the TR will finish all of the sub-TR tiles for the 2D definition and then have to check if it had received the trigger before rearbitrating for the transpose buffer.

3.3.5.1.2 Secondary TR with Multiple Buffer Interleave

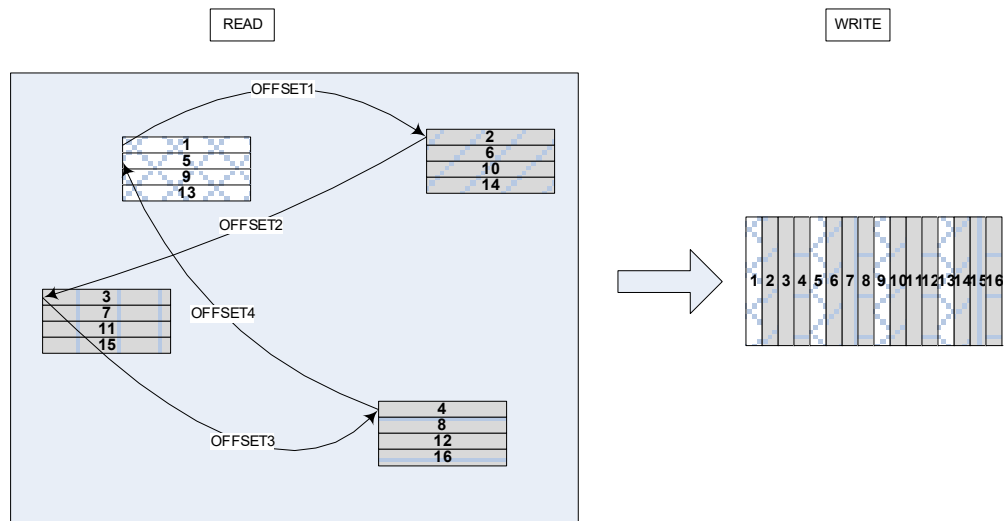
The multiple buffer interleave is used to have a single TR plus Secondary TR that can define a fetch where a single line is selected from multiple buffers and then sent as a single buffer. This can be used along with transpose of the data to present data in slices optimal for processing the same algorithm on different data sets at the same time. An example of this can be seen in the figure below.

Figure 3-3 Example of Multiple Buffer Interleaving Read with 4 Offsets



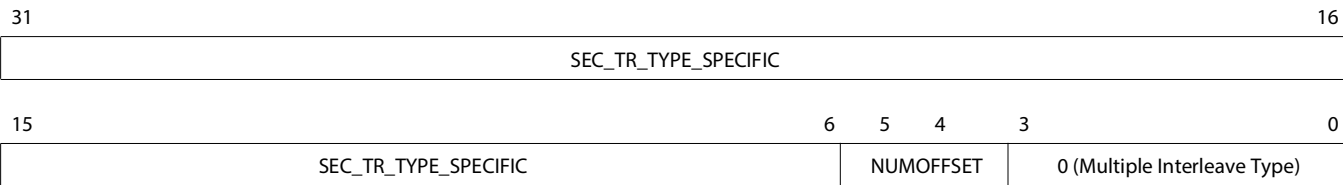
The case above provides the ability to minimize the number of TRs submitted to make the transfer but the interleaved Read main advantage comes when used with the transpose mode enabled. The figure below shows the advantage of the interleaved mode along with the transpose mode to allow for maximum size transfer on both read and write transactions.

Figure 3-4 Example of Interleaved Read with 4 Offsets and Transpose Enabled



As the figure shows the data that is written out now contains one element from each block of memory and can be read in directly to processing units.

Figure 3-5 SECONDARY TR FLAGS Field Definition



The flags field fills in the remaining details about the stream, as follows:

Table 3-5 Multiple Buffer Interleave FLAGS Field Descriptions

Bit	Field	Description	Reference
0-3	SEC_TR_TYPE	0: Multiple Buffer Interleave	"SEC_TR_TYPE: Secondary TR Type Field" in KS3 DMA Specification
4-5	NUMOFFSET	The encoded value for the number of offsets. 0: 4, 1: 8, 2: 16	
6-31	Reserved	Reserved for Future Use	

End of Table 3-5

Secondary TR TYPE from the Secondary TR Flags field.

Figure 3-6 Multiple Buffer Interleave Secondary Transfer Request Template 128 Byte Secondary TR

word 31	word 30	word 29	word 28
RESERVED			
word 27	word 26	word 25	word 24
RESERVED			

Figure 3-6 Multiple Buffer Interleave Secondary Transfer Request Template 128 Byte Secondary TR

word 23	word 22	word 21	word 20
RESERVED			
word 19	word 18	word 17	word 16
OFFSET15	OFFSET14	OFFSET13	OFFSET12
word 15	word 14	word 13	word 12
OFFSET11	OFFSET10	OFFSET9	OFFSET8
word 11	word 10	word 9	word 8
OFFSET7	OFFSET6	OFFSET5	OFFSET4
word 7	word 6	word 5	word 4
OFFSET3	OFFSET2	OFFSET1	OFFSET0
word3	word 2	word 1	word0
RESERVED	SECONDARY TR FLAGS	ADDR	

3.3.5.1.2.1 Multiple Buffer Interleave Functionality

Multiple Buffer Interleave will work by setting the SECTR field in the TR presented to the channel to be either 64 or 128 bytes depending on the number of offsets required. An offset number of 4 or 8 will require 64 bytes while 16 offsets will require 128 bytes. Prior to the channel becoming active it will see that the SECTR field is nonzero and arbitrate to use the SECTR holding space and fetch the SECTR of the size specified. The first access from the channel will be a read of the secondary TR as specified in the address field of the TR. Upon receiving the Secondary TR the channel will parse the type field and seeing that it is a Multiple Buffer interleave will load the offset index fields for the channel. Additionally it will load the ADDRESS field from the secondary TR into the Address of the actual TR to allow it to process as normal.

The first address fetch will then be to this SEC_TR_ADDRESS. After finishing the inner loop the next address will be SEC_TR_ADDRESS + OFFSET0.

Note—ICNT1 should be equal to the height of the transfer blocks times the number of offsets.

The DRU contains a single holding space for the secondary TR storage so only a single channel can be active at a time. The channel will arbitrate for the secondary TR storage when it has received the trigger. The channel will then arbitrate for the secondary TR storage and if it wins the secondary TR fetch will be sent for the channel and when the secondary TR is received the channel TR will start sending sub-TRs like its triggers have been received. The secondary TR holding space will be released by the channel when its TR event is set. If the channel also has the transpose flags set it will arbitrate for the transpose buffer only after it has won the secondary TR holding space. This will prevent a deadlock of two channels needing both resources and each winning arbitration for a separate

3.3.5.1.3 Circular Buffering

Circular addressing modifies how the UTC performs address arithmetic so that addresses remain within a power-of-2 sized window. Circular addressing works by holding upper address bits constant during an address update, while allowing the lower address bits to vary. This contrasts with the default—linear addressing—which allows all of the address bits to vary.

The UTC TR provides address mode selection for each level of loop nest. Each level can select between linear addressing or circular addressing with one of two circular block sizes.

Conceptually, selectable addressing mode support replaces all address arithmetic with a function similar to the following.

Example 3-4 Address Arithmetic with Selectable Linear / Circular Addressing Modes

```
enum addr_mode_t
{
    LINEAR,          // Linear addressing mode
    CIRC0,           // Circular addressing with block size 0
    CIRC1            // Circular addressing with block size 1
};

uint64_t circ_mask_0; // Circular addressing mask for block size 0
uint64_t circ_mask_1; // Circular addressing mask for block size 1

uint64_t address_add( uint64_t base, int32_t offset, addr_mode_t mode )
{
    uint64_t new_addr = base + offset;

    if ( mode == LINEAR )
        return new_addr;

    // Look up address mask based on selected circular buffer size.
    // Mask contains 1s for bits that remain fixed, and 0s elsewhere.
    uint64_t addr_mask = mode == CIRC0 ? circ_mask_0 : circ_mask_1;

    return ( base & addr_mask ) | ( new_addr & ~addr_mask );
}
```

End of Example 3-4

The `address_add` primitive then takes the place of plain addition for all TR address computations in this mode. The following example illustrates `address_add`.

Example 3-5 Linear Stream Addressing Algorithm with Circular Addressing Add

```
for ( i3 = 0; i3 < ICNT3; i3++ )
{
    ptr3 = ptr; // save current position before entering next level
    for ( i2 = 0; i2 < ICNT2; i2++ )
    {
        ptr2 = ptr; // save current position before entering next level
        for ( i1 = 0; i1 < ICNT1; i1++ )
        {
            ptr1 = ptr; // save current position before entering next level
            for ( i0 = 0; i0 < ICNT0; i0++ )
            {
                fetch( ptr, ELEM_BYTES );
                ptr = address_add( ptr, ELEM_BYTES, addr_mode_0 );
            }
            // Update based on saved pointer for this level
            ptr = address_add( ptr1, DIM1, addr_mode_1 );
        }
        // Update based on saved pointer for this level
        ptr = address_add( ptr2, DIM2, addr_mode_2 );
    }
}
```

```

    }
    // Update based on saved pointer for this level
    ptr = address_add( ptr3, DIM3, addr_mode_3 );
}

```

End of Example 3-5

The values for the circular addressing can be selected by using the 16 bits of the AMODE specific Field as described in [AMODE SPECIFIC: Addressing Mode Field](#) in the KS3 DMA Specification.

The AMODE Specific Fields are defined based upon the Addressing Mode being used as specified in [AMODE: Addressing Mode Definition](#) in the KS3 DMA Specification.

3.3.5.2 Circular Address Mode Specific Flags

If the circular addressing mode is selected in AMODE the definition of the upper 16 bits of the FMTFLAGS field are used to select the type of addressing for each entry in the loop and to define 2 unique masks that can be used for the circular buffering. Additionally, the DIR field in the TR will determine if the circular addressing flags will apply for the source or the destination of the block move. The other direction will always follow the standard linear addressing that is used if AMODE is not set.

The following example code shows the method used to compute the masks referred to by the `address_add` function described in [“Address Arithmetic with Selectable Linear / Circular Addressing Modes”](#) on page 3-25.

Example 3-6 Circular Address Mask Computation

```

// Circular addressing masks.
// 1 bits in the mask indicate address bits that remain fixed.
// 0 bits in the mask indicate address bits that are allowed to vary.
uint64_t circ_mask_0; // mask for circular addressing block 0
uint64_t circ_mask_1; // mask for circular addressing block 1

void compute_circular_address_masks( int cbk0, int cbk1 )
{
    int block_size_0 = cbk0 + 9; // power-of-2 of block size in bytes
    int block_size_1 = cbk0 + cbk1 + 10; // power-of-2 of block size in bytes

    if ( block_size_1 > 31 ) block_size_1 = 32; // clamp to 4GB maximum size

    circ_mask_0 = (~0ULL) << block_size_0;
    circ_mask_1 = (~0ULL) << block_size_1;
}

```

End of Example 3-6

Figure 3-7 Transfer Request Template FMTFLAGS AMODE SPECIFC Circular Addressing Field

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
AM3			AM2			AM1			AM0			CBK1			CBK0	

Table 3-6 Circular Address Mode Specific Flags Field Descriptions

Bit	Field	Description	Reference
16-19	CBK0	Specifies the circular block 0 value as used in Circular Address Mask Computation	“CBK0 and CBK1: Circular Block Size Selection” on page 3-27
20-23	CBK1	Specifies the circular block 1 value as used in Circular Address Mask Computation	“CBK0 and CBK1: Circular Block Size Selection” on page 3-27
24-25	AM0	The address mode to use with ICNT0 loop	“AMx: Addressing Mode Selection” on page 3-27
26-27	AM1	The address mode to use with ICNT1 loop	“AMx: Addressing Mode Selection” on page 3-27
28-29	AM2	The address mode to use with ICNT2 loop	“AMx: Addressing Mode Selection” on page 3-27
30-31	AM3	The address mode to use with ICNT3 loop	“AMx: Addressing Mode Selection” on page 3-27
End of Table 3-6			

3.3.5.2.1 CBK0 and CBK1: Circular Block Size Selection

The CBK0 and CBK1 fields set the circular block sizes for circular addressing. CBK0 directly determines the circular block size for block 0. CBK1 combines with CBK0 to determine the circular block size for block 1. Specifically, the streaming engine sets block 1’s size according to $CBK0 + CBK1 + 1$. Therefore, circular block 1 is always larger than circular block 0.

The following table illustrates the resulting valid block sizes for circular block 0 and 1. For circular block 0, the use the value of CBK0 directly. Circular block 0 supports block sizes ranging from 512 bytes to 16M bytes. For circular block 1, use the value of $CBK0 + CBK1 + 1$. Circular block 1 supports sizes ranging from 1K bytes to 4G bytes.

Table 3-7 Circular Block Size Decoding

Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size
0	512	8	128K	16	32M	24	Reserved
1	1K	9	256K	17	64M	25	Reserved
2	2K	10	512K	18	128M	26	Reserved
3	4K	11	1M	19	256M	27	Reserved
4	8K	12	2M	20	512M	28	Reserved
5	16K	13	4M	21	1G	29	Reserved
6	32K	14	8M	22	2G	30	Reserved
7	64K	15	16M	23	4G	31	Reserved
End of Table 3-7							

3.3.5.2.2 AMx: Addressing Mode Selection

The fields AM0 through AM3 control the addressing modes for each of the 4 dimensions. The UTC supports the following addressing modes:

Table 3-8 AMX Address Mode Selection Encoding

AMX	Meaning
00b	Linear addressing
01b	CKB0 (Circular Buffer 0 is used)

Table 3-8 AMX Address Mode Selection Encoding

AMX	Meaning
10b	CKB1 (Circular Buffer 1 is used)
11b	Reserved
End of Table 3-8	

Each dimension has its own addressing mode, independent of the other dimensions. This permits any mixture of linear and circular addressing among the 4 dimensions.

When a given dimension selects linear addressing, the UTC uses unmodified 2s complement arithmetic to update addresses for that loop level.

When a given dimension selects circular addressing via CBK0 or CBK1, the UTC uses the circular address arithmetic described above in [“Address Arithmetic with Selectable Linear / Circular Addressing Modes”](#) on page 3-25

3.3.5.3 Element Type Expansion or Contraction

If the TR selects an ELTYPE greater than 8 the element type has a different size between the source and the destination. If the source element is larger than the destination element then the number of extra bits will be dropped from the top of the original element. It is assumed that the processing step prior to the data movement will already have done any rounding or truncating so the DRU will not do anything or check if these values are nonzero but just drop them. If the source element is smaller than the destination then the DRU will place 0s in the created bits.

For example if the DRU is doing a 12 bit to 16 bit conversion and it reads 4

0x123,0x456,0x789,0xabc the DRU will send

0x0123,0x0456,0x0789,0x0abc

3.3.5.4 Repacking Data Types

The DRU implementation of the channels does not allow for dedicated buffering for an individual channel this means that when a TR is broken down to a smaller size to the queue to be operated on it requires that the source and destination block that are operated on must have the same dimensions. The block size is determined by the smallest of the trigger and the event size and a two dimensional block. Whichever is the smallest will determine the block size and the size of the block.

So if a TR specifies either of the triggers or the event to be on the decrement of ICNT1 then the requirement is that ICNT0 == DICNT0 all other count values can have any value. If the trigger size and event size are anything else then the requirement is that ICNT0 == DICNT0 && ICNT1 == DICNT1.

3.3.6 Virtualization

The DRU can support virtual addressing when through a configuration option. This option will integrate the CMMU from the C7x along with 2 uTLBs one for read commands and another for write commands.

The channel configuration will add a bit for *atype* to choose if the addresses are virtual or physical. If a channel is configured to have virtual addresses the queue will send a request to the uTLB and wait for the status response before it sends the command. If the uTLB returns with a page fault then the TR will fail and all future commands for the TR will be flushed. If the pause on error option is set for the channel then no further TRs will be sent for the channel until it is torn down and setup again. All the attributes for the command will be selected based on those returned from the uTLB.

The CMMU will be given its own *routeid* configured through a tieoff. The CMMU page walks will go through the read port of the DRU and have the highest priority between DMA commands.

3.3.7 Compression and Decompression

The DRU can support compression and decompression of data in line by integrating the Accelerator Framework Compression (AFC) module inside the response buffer path. Additionally, the AFC has two separate configurations an Analytics Compression mode and a video compression mode. The Analytics Compression mode is a subset of the video compression so the video compression will be described with differences for the analytics mode called out. For the TR and Secondary TR fields values are to be zero if only Analytics Compression is supported by the DRU configuration.

The compression logic places the AFC module between the Read Response Buffer and a dedicated write engine for compression and a dedicated write engine of decompression. The read response buffer handles any out of order responses and then the data is pushed into FIFOs inside the AFC module. The AFC module pulls the data out of the FIFOs as 128 bit elements for compression or 128 CDBs for decompression.

Once the 128 bit entry is removed from the FIFO the symbol size specified for the compression is used to split the 128 bit element into the corresponding number of symbols. If the 128 bit does not divide into an even number of symbols then the remain bits are used with the next 128 bit element to create the first symbol. Once each symbol is determined then they are passed into the compression pipeline. The first stage of the compression pipeline is the difference phase. If it is a non-differential compression algorithm then a fixed bias value is subtracted from the symbol. If it is a differential compression algorithm then the symbol is subtracted from a symbol that is from a previous slot. If there was not data at the previous slot selected it will be zero until actual data is present.

“[10.7 Compression and Decompression](#)” on page 10-295 gives a full description of how the compression and decompression work within the DRU.

3.3.8 Output Events

The DRU has four different types of output events. The output events are completion events, error completion events, local output events/ctset events and protocol violation events.

3.3.8.1 Completion Events

The completion events are defined for each channel. The completion event fires whenever a channel completes a TR successfully. It will only fire when all responses are received from the write portion of the block copy. A single write completion event can occur in a given cycle since it is based on the write responses being received to generate the response.

3.3.8.2 Error Completion Events

The error completion events are defined for each channel. The error completion event fires whenever a channel completes a TR with an error and it has flushed out all data related to that TR in the response buffer. Upon determining that a TR will not be able to complete successfully, the DRU will enter a flush mode to remove all read data received after the error is determined as well as make sure all write responses are received. Once all of this criteria has been matched the write engine will fire a completion event. If multiple error completions occur at the same time the DRU will still send them out one at a time using an arbitration scheme used for the original arbitration for the queue, either fixed priority or round robin based on the queue that has the error.

3.3.8.3 Local Output Event PSIL Global Event and CTSET Event

Each TR can set an Event Size of when it will fire an output event. This same event is provided on three different interfaces. The Local Output Event interface, the PSIL event interface and the CTSET event interface. The DRU will generate output events based on when the write interface for the block to block queues reaches the requested size.

The PSIL event interface will use the event number configured in the Output Event Steering register for the channel.

The event size specified in the TR will be triggered when the write command for the given loop level has been sent except for the TR complete size which will be generated when all the responses are received. The size of events are listed in the table below.

Table 3-9 **Event Types**

Event Size	Event Encoding	Description
0	TR Complete	The TR has completed and all responses have been received
1	ICNT1 Loop is decremented	The last write command of a row has been sent
2	ICNT2 Loop is decremented	The last write command of an array has been sent
3	ICNT3 Loop is decremented	The last write command of an array of an array has been sent

3.3.8.4 CTSET Events

For the CTSET events the DRU just exports the channel output event signals for each channel. The frequency of the event will be determined by the event size field in the TR as described in Table 3-9, “Event Types,” on page 30. The descr

Table 3-10 DRU CTSET Export

Number	Category	Event	Description	Interest Qualified	Level/Pulse
1	Global	Channel 0 Event	Event specified by channel 0	No	Pulse
2	Global	Channel 1 Event	Event specified by channel 1	No	Pulse
3	Global	Channel 2 Event	Event specified by channel 02	No	Pulse
4	Global	Channel 3 Event	Event specified by channel 3	No	Pulse
5	Global	Channel 4 Event	Event specified by channel 4	No	Pulse
6	Global	Channel 5 Event	Event specified by channel 6	No	Pulse
7	Global	Channel 6 Event	Event specified by channel 6	No	Pulse
8	Global	Channel 7 Event	Event specified by channel 7	No	Pulse
9	Global	Channel 8Event	Event specified by channel 8	No	Pulse
10	Global	Channel 9 Event	Event specified by channel 9	No	Pulse
11	Global	Channel 10 Event	Event specified by channel 10	No	Pulse
12	Global	Channel 11 Event	Event specified by channel 11	No	Pulse
13	Global	Channel 12Event	Event specified by channel 12	No	Pulse
14	Global	Channel 13 Event	Event specified by channel 13	No	Pulse
15	Global	Channel 14 Event	Event specified by channel 14	No	Pulse
16	Global	Channel 15 Event	Event specified by channel 15	No	Pulse
17	Global	Channel 16 Event	Event specified by channel 16	No	Pulse
18	Global	Channel 17 Event	Event specified by channel 17	No	Pulse
19	Global	Channel 18 Event	Event specified by channel 18	No	Pulse
20	Global	Channel 19 Event	Event specified by channel 19	No	Pulse
21	Global	Channel 20 Event	Event specified by channel 20	No	Pulse
22	Global	Channel 21 Event	Event specified by channel 21	No	Pulse
23	Global	Channel 22 Event	Event specified by channel 22	No	Pulse
24	Global	Channel 23 Event	Event specified by channel 23	No	Pulse
25	Global	Channel 24 Event	Event specified by channel 24	No	Pulse
26	Global	Channel 25 Event	Event specified by channel 25	No	Pulse
27	Global	Channel 26 Event	Event specified by channel 26	No	Pulse
28	Global	Channel 27 Event	Event specified by channel 27	No	Pulse
29	Global	Channel 28 Event	Event specified by channel 28	No	Pulse
30	Global	Channel 29 Event	Event specified by channel 29	No	Pulse
31	Global	Channel 30 Event	Event specified by channel 30	No	Pulse
32	Global	Channel 31 Event	Event specified by channel 31	No	Pulse

3.3.9 Input Events and Triggers

Each channel can select up to two triggers that activates the current TR. The triggers can be a dedicated local event, or one of two dedicated global events. The triggers work on a counter that increment every time a trigger is received. A TR does not have to be actively using that trigger for the TR to increment. This allows for triggers and TRs to come out of order. However, the overflow logic will only detect if a trigger overflow occurs on the current active TR.

The trigger logic looks at the trigger_rcvd counter for the selected trigger and sets the trigger_active bit if the count is non-zero. Each time a channel is received by the subtiler it returns a trigger_completed_size based upon where last sub-TR was. If the trigger_completed_size matches the active trigger_size as specified in the TR then the trigger_active will be cleared and the trigger_rcvd counter for the selected trigger will be decremented.

If a trigger is received when the trigger_rcvd counter is at its maximum value and the trigger is one being used by the current TR then a trigger overflow event will occur and the TR will be aborted.

3.3.10 Memory Attribute Table

In a DRU that does not have a MMU or if the TR is a physical TR and not a virtual TR if the DRU is on a coherent fabric then the memory attributes for inner and outer cacheability, domain shareability and the memory type are determined by looking up the value based on the address in the memory attribute table. The DRU has three tieoffs that define the address ranges that use this memory attribute table and the number of tables. The first table is for the MSMC SRAM which has an entry for each 64kB of memory and can support up to 8192 entries. The second table is for the DDR which has an entry for each 16 MB and has 8192 entries as well. The third table is for an optional DDR boot section that has 16 MB for each entry and can have 128 entries.

The memory attribute table entries are standard MMR entries that can only be accessed when the DRU is idle. If a read or write to the MMR for the memory attribute table occurs when the DRU is active the DRU will respond with an address error to the write and a read with all 0's.

The attribute table is written during the first 1024 cycles and any read or write will be stalled until this reset is complete.

3.3.11 Shared Hardware Resources

Some features require more hardware resources so while all channels are able to utilize the resource only a single channel can use it at a time. If the resources required are not available the channel will behave as if all the triggers have not been received. The features that require the shared resources are transpose and secondary TR. If both functions are required the secondary TR must be arbitrated and won before a channel can arbitrate for the transpose function. The secondary TR function uses a shared register set to hold the secondary TR while transpose requires that up to 32 buffers be pre-allocated for the transpose function.

The arbitration for the shared resources is based solely on the channel number with the lower channel going before higher number channels.

Upon winning the secondary TR arbitration the channel will be able to fetch the secondary TR and store it in the internal storage for the secondary TR. Arbitration for the secondary TR logic will not occur again until the TR for the channel has completed and then the next cycle a new arbitration between all other channels that need to do a secondary TR will occur.

For the transpose function as soon as a channel starts to request the transpose buffer the response buffer will begin reserving the lower buffer space for the TR based on the largest side needed. If a 64 bit element transpose is being requested then the transpose logic will force that the lower 16 response buffer entries will no longer be used for other channels. Once the 16 buffers are free the arbitration between all channels will occur and the winner selected based on the lowest channel number. If during this time a larger buffer requirement is chosen then the arbitration will not occur until the required number of buffers are free regardless of the number of buffers required by the winning channel. So for example if channel 0 is trying to do a transpose of a 128 bit element that requires 8 buffers the logic will stop issuing requests to buffers 0-7. If while waiting for this to occur channel 1 requests a 32 bit element that requires 32 buffers. The logic will now stop issuing request to buffers 8-31 as well. The arbitration will not occur until all 32 buffers are free even though only 8 buffers are needed for the channel that will win arbitration.

3.3.12 Safety Features

The DRU architecture is designed with safety requirements in place. Additional, safety features can easily be added to meet higher level safety classification requirements. Please refer to the safety documentation for more information on the safety.

3.3.13 Security Features

The DRU contains firewalls to allow for a secure environment. The DRU has a firewall on both the read and write command for all DMAs that share a single reference table. The number of regions and number of privids per region are specified as configuration parameter for the DRU.

The DRU also contains firewalls to allow for the protection of the MMR registers used for configuration and submission. The DRU has two firewalls a region based firewall that has a single priv and privid that protects all the registers in a programmable range and then a channelized firewall also protects the Realtime Configuration registers region and all the submission registers with a single priv and privid for each channel.



Note—For hardware simplification an MMR access to the DRU is mapped to a channelized region or non-channelized region. All channelized regions will be mapped to 0x80000 for the firewall checks and then the original address is restored after the firewall check. This allows for a single firewall entry for a DRU channel even though the registers for that channel can appear in multiple regions. The firewall logging report though would always show the failure with the upper 3 bits of the address replaced with a 3'b100. So all register accesses from 0x60000 to 0xDFFFF are considered channelized registers and will follow this behavior.

The region based firewall should be configured to cover the range from 0x4000 to 0x5FFFF. This will allow for a single master to control the memory attribute channels and channel ownership. If the region based firewall is made to overlap the channel based firewall then both firewall checks will have to pass for the register update to

occur. If the firewall check fails the DRU will return a PROTECTION ERROR status to the request. The second region of the firewall should be configured to be a background region that covers 0x00000 to 0xFFFFF that allows all privids and all accesses. Since enabling the region based firewall requires that all addresses fall in a region. This will ensure that only the channelized firewall will be used for the Real Time configuration registers and the submission registers. The PID and Error Status registers are read only and can safely be accessed by anyone.



Note—PROTECTION ERROR will take precedence over an ADDRESS ERROR so if the address falls into something protected by the firewall then a PROTECTION ERROR will be returned. For Channelized firewall only the channels that exist in the system have actual entries so only those will get a PROTECTION ERROR



Note—If the recommend address range is used for the region based firewall than the addresses from 0x0 to 0x3FFF and 0xE0000-0xFFFFF will be unprotected. These are all read only registers and do not require protection.

3.3.13.1 DMA Credentials

The DRU generates the credential signals (secure, priv and privid) based upon the way it determines the submitter. For Direct TR and PSIL Submission the credentials are stored with the transaction that submits the TR to the queue. For PSIL transactions these are the values given on the PSIL bus with the TR data. For direct TR it will be the write to the TR Type field in ether the ATOMIC or CORE SUBMIT registers. For ring mode the first transaction to ring the doorbell after the channel is enabled will capture the credentials from that write and store them to be used for all DMA transactions for the ring until the ring is torn down and re-enabled.

3.3.14 EDMA3 TC Compatibility

The DRU is designed to be the EDMA3 TC replacement for Keystone 3. Wherever, possible it has been designed to be compatible with the original TC. However, because of the new features in the DRU or architecture changes in Keystone some of the register field have had to expand and caused them to shift with one another this section describes the differences between the DRU and the EDMA3 TC.

3.3.14.1 Status Register Changes

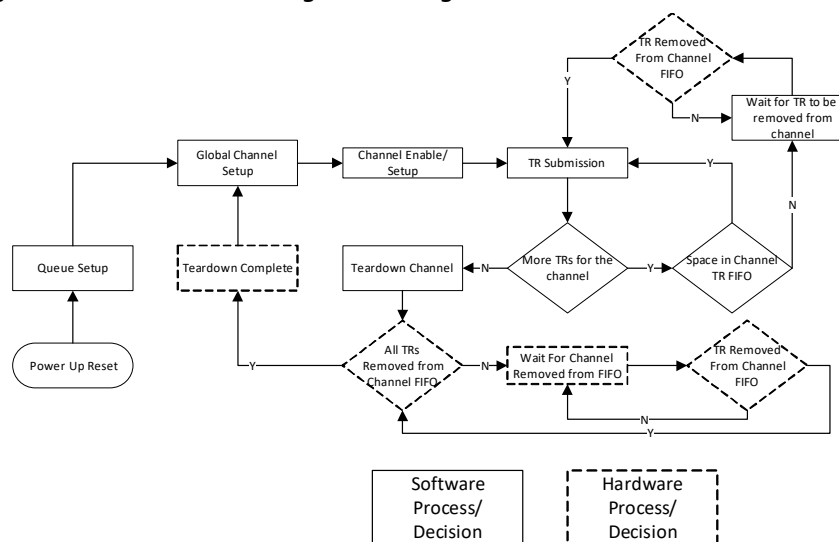
The status register has been changed to reflect the changes in the DRU architecture to reflect the fact that the DRU is replacing both the TC and the IDMA for the DSP. This change requires that the DRU has a much larger FIFO structure for the queues. Additionally, the state structure has been changed.

3.4 DRU Detailed Operation

The DRU has five different types of flows that are running simultaneously. The first flow is the software managed flow of channel configuration, TR submission and channel tear down flow. A second flow is the hardware managed channel control flow. A third flow is the hardware controlled read command flow. The fourth flow is the hardware controlled write command flow. The final flow is the hardware controlled channel removal flow. The following sections will describe each of these flows and how they interact with one another.

3.4.1 Software Managed Flow

Figure 3-8 Software Managed Flow Diagram



The flow diagram above shows the interaction between the software and the hardware to setup the DRU and submit TRs and then releasing the channel to be used by a different resource. The sections below explain in more detail what is occurring in each process or decision.

3.4.1.1 Queue Setup

The DRU queue setup is done by a single software process writing to the registers in the Queue registers. There is a dedicated register for each queue that can control the parameters used for the queue. These include the QoS and Orderid attributes for the DMA bus as well as the number of commands to send before releasing the bus and the number of commands to wait before trying to request the bus. This should be done prior to setting up any channel as changes to this while channels are active can cause unknown behavior. This setup is protected by a region based firewall that allows a single master to modify these registers.

The Memory Attribute Table can only be updated during this time. If any channels are active a write to the memory attribute table will generate an error as the memory is owned by the hardware at that time and can not be modified by the software.

3.4.1.2 Global Channel Setup

The DRU global channel setup is accomplished by a single software process writing to the registers in the Channel Non-Real time registers for each channel. These registers are unique for each channel and select the attributes of the channel. This includes the queue the channel should use, what global event it can generate and how the channel can be controlled.

The channel queue determines what resource is used to send out the commands and its transaction will assume the parameters assigned to the queue that was setup during the queue setup process.

The global event for a channel determines what event it will send out on the PSI-L output event bus that goes into Navigator and can be routed to other DMA controllers or IP inside the SOC.

The channel control is used to specify how the channel enable and setup process must be handled as well as how the TR will be sent to the DRU for that channel. These channel types are a Direct TR (QDMA channel) or a UDMA-C channel. The DRU always powers up with the channel mapped as a Direct TR even if the DRU configuration does NOT support Direct TR submission. This must be changed to a UDMA-C type channel prior to the PSI-L pairing being initiated.

3.4.1.3 Channel Enable Setup

The channel enable setup phase is when the Real Time configuration register is updated and the channel can become locked. The channel enable depends on if the channel was previously configured as a QDMA channel or a UDMA-C channel.

3.4.1.3.1 QDMA channel enable

A QDMA channel is enabled by writing to the realtime configuration register in the MMR space of the DRU as defined in [CHRT_CTL \[Address = 0x6_0000\]](#). Once software has written the Enable bit in this register the channel is locked until the Teardown sequence is complete.

3.4.1.3.2 UDMA-C channel enable

A UDMA-C channel enable is done through the pairing mechanism defined in the PSI-L specification. Once the channel has been successfully paired with a Channel Controller the channel will remain locked until the Teardown sequence is complete.

3.4.1.4 TR Submission

The DRU submission TRs can be received through two mechanisms. Through direct TR submission using the internal memory mapped registers (MMR) or through the PSI-L interface. The interface used is based on how the channel has been configured, QDMA channels will use Direct TR submission and UDMA-C configured channels will use PSI-L TR submission.

3.4.1.4.1 Direct TR Submission

The direct TR submission is performed by writing to one of the sets of DRU submission registers. The DRU can support both an atomic and non-atomic submission method. The atomic method is in the atomic submission and debug region. This submission register is required to be an atomic write of the TR size in a single burst. Any write other than this size will return an address error. The another method supported by the DRU is to have a number of non-atomic submission registers that are intended to be used for cores that cannot perform a large burst write in a single cycle. Each of these non-atomic cores are in the CORE_SUBMISSION region. The non-atomic submission registers will only submit the data to the selected queue when the 64 bit register COREX_SUBMIT0 register is written. There is a single set of submission registers for each core regardless of the number of channel so writing to the offset for a core will be updated regardless of the channel addresses selected.

3.4.1.4.2 PSI-L Submission

The DRU PSI-L submissions are used for TR submissions from an external UDMA-C.

The DRU has a single submission thread for each channel supported. The channel must be configured as a UDMA-C type channel prior to the PSI-L pairing. The PSI-L capabilities register will show 0 TRs supported for channel until the type is change to UDMA-C.

The PSI-L pairing, submission and response follow the mechanisms defined in the PSI-L specification.

3.4.1.4.3 Submission Errors

The DRU checks for illegal conditions in a TR that could prevent cause the logic to hang or perform other unexpected functions. The table below shows the different type of submission errors that can occur and includes the configuration parameter that is required to make the submission error check present. For instance if the RTL configuration is setup to not support transpose than a TR with a transpose set in the FMTFLAGS field will generate a submission error. While if the RTL configuration does support the transpose feature this will not generate a submission error. If the TR is submitted through a Direct TR submission then the TR submission write will be returned with an address error on the response. A PSIL submission will return the TR Response immediately with a submission error and the type as defined in the TR response section of the KS3 DMA Specification.

Table 3-11 Submission Error Cases

RTL Configuration	Illegal Conditions
Any	ICNT0 = 0
STATIC_SUPPORT = 0	FLAGS.STATIC = 1
!PSI_SPLIT_IN && !PSI_SPLIT_OUT	FLAGS.EOL = 1 TYPE = 0 - 4
CACHE_WARM = 0	FLAGS.TYPE == 5
REPACKING == 1 && STATIC_SUPPORT = 1	(FLAG_TYPES == 9 FLAG_TYPES == 11) & FLAGS.STATIC == 1 & ICNT0 == DICNT0 & (1D_TRIGGER 1D_EVENT_SIZE ICNT1 == DICNT1)
REPACKING == 0	FLAGS.TYPE = 9 FLAGS.TYPE == 1
CIRCULAR_ADDR = 0	FMTFLAGS.AMODE != 0
CIRCULAR_ADDR = 1	FMT_FLAGS.AMODE > 1 IF FMT_FLAGS.AMODE = 1 then FMT_FLAGS.AM3 = 3 FMT_FLAGS.AM2 = 3 FMT_FLAGS.AM1 = 3 FMT_FLAGS.AM0 = 3 IF FMT_FLAGS.AMODE == 0 then FMT_FLAGS.DIR = 0
BIT_EXPAND = 0	FMT_FLAGS.ELTYPE > 7
BIT_EXPAND = 1	FMT_FLAGS.ELTYPE > 11 (FMT_FLAGS.ELTYPE = 0xA FMT_FLAGS.ELTYPE = 0xB) && (SRC_ADDR[0] = 1 SRC_DIM1[0] = 1 SRC_DIM2[0] = 1 SRC_DIM3[0] = 1)
CONSTANT_COPY = 0	FMT_FLAGS.DFMT = 1
TRANPOSE = 0	FMT_FLAGS.DFMT = 2
TRANPOSE = 1	IF FMT_FLAGS.DFMT = 2 FMT_FLAGS.ELTYPE < 4 FMT_FLAGS.ELTYPE > 6 IF FMT_FLAGS.DFMT = 2 && FMT_FLAGS.ELTYPE = 4 SRC_ADDR{1:0} != 0 && SRC_DIM1{1:0} != 0 && SRC_DIM2{1:0} != 0 && SRC_DIM3{1:0} != 0 IF FMT_FLAGS.DFMT = 2 && FMT_FLAGS.ELTYPE = 5 SRC_ADDR{2:0} != 0 & SRC_DIM1{2:0} != 0 && SRC_DIM2{2:0} != 0 && SRC_DIM3{2:0} != 0 IF FMT_FLAGS.DFMT = 2 && FMT_FLAGS.ELTYPE = 6 SRC_ADDR{3:0} != 0 & SRC_DIM1{3:0} != 0 && SRC_DIM2{3:0} != 0 && SRC_DIM3{3:0} != 0
REVERSE = 0	FMT_FLAGS.DFMT = 3
REVERSE = 0 TRANPOSE = 0	FMT_FLAGS.DFMT = 4
ANY	FMT_FLAGS.DFMT > 4
MULTIPLE_BUF_OFFS ET = 0	FMT_FLAGS.SEC_TR != 0
MULTIPLE_BUF_OFFS ET = 1	FMT_FLAGS.SEC_TR == 3 FMT_FLAGS.SEC_TR == 0 && SRC_ADDR[5:0] != 0 FMT_FLAGS.SEC_TR == 1 && SRC_ADDR[6:0] != 0 FMT_FLAGS.SEC_TR == 2 && SRC_ADDR[7:0] != 0
End of Table 3-11	

3.4.1.5 TR Removed from Channel

After all requested write responses are received from a TR the DRU will send the completion event for the channel that completed the TR. Software can use this event to determine when a TR can be added to a channel FIFO. Each channel has a small FIFO to allow for multiple TRs to be preloaded into a channel to allow for a channel to maximize the throughput. The completion event signifies that an entry has become available in the FIFO.

3.4.1.6 Channel Tear Down

A channel must be torn down through the register `CHRT_CTL` [Address = `0x6_0000`] in order to allow for the channel to be reconfigured. The tear down bit in the register will force the channel to stop sending any new requests not already in the pipeline and when all outstanding requests have been received clear the enable bit and all other bits in the register. If the pause on error bit is set and an error occurs the tear down mechanism can be used to restart the channel. The teardown bit being set will also make the TR STATIC field in any future reads from the TR memory to be set to 0 no matter what the value was when the TR is submitted. This is the mechanism used to stop a STATIC TR.

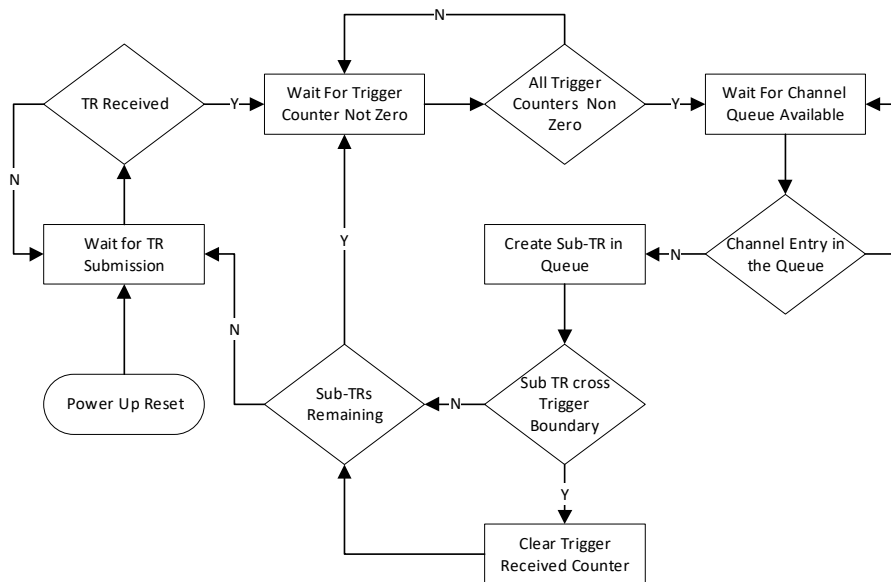
3.4.1.7 Tear Down Complete

The Tear Down Complete process is when software has set the tear down bit so no new TRs can be added to the channel but when previous TRs that had been added to the channel are still running. Once the TR FIFO for the channel is empty the hardware will clear the teardown and enable signals. The channel is now back at the global channel setup process and can be reused as needed by the system.

3.4.2 Active Channel Control Flow

The active channel control flow is the hardware logic that is present for each channel to handle submitting portions of the TR to run into the queue for the channel.

Figure 3-9 Active Channel Flow Diagram



Below the each of the processes in rectangles in the flow are described.

3.4.2.1 Wait for TR Submission

The channel logic will always wait until there is at least one entry in the channel TR FIFO. Once a TR has been received then it will begin looking at the other requirements to send a TR to the queue.

3.4.2.2 Wait for Trigger

The Triggers consist of counters that count each of the three triggers that can be received by a channel. The three triggers are the local event and 2 global events. The local event is based on the bit in the local event in bus. The global event buses use the PSI-L Event Input bus and use the lower bit to select between global event 0 and global event 1. The bits required for the number of channels of the DRU down to bit 1 are used to select which channel the global event is intended. For example a 64 channel DRU will use bits 7 down to 1 for the channel addressing.

When a given trigger is received the counter for that trigger type is incremented. The counter size is part of the DRU configuration and can be from 1 to 3 bits. For a given TR it selects from up to two of the counters and if they are not zero then it can advance to the next check. These active counters will be decremented when it reaches the clear trigger counter.

If a trigger causes the counter to overflow it will place the channel in an error state that will cause the active write TR to be aborted.

3.4.2.3 Wait for Channel Queue Slot

Once a channel has both a TR and all of the required triggers are received it must make sure it has space in the queue logic before it will generate the TR. The queue ordering logic has two portions, before the subtiler and in the Queue active FIFO. The queue active FIFO is a small 2 entry FIFO in each queue that holds all data from the sub TR required for the read or write engine. The FIFO before the subtiler is handled using a link list structure so each channel is only allowed a single place until it has gone through the subtiler.

This means that for a given channel it can send a request for a single sub-TR and it can not send another until the previous request has been processed by the subtiler. The channel logic will wait for notification that the previous sub-TR request has advance to the queue active FIFO before the next request can be sent.

3.4.2.4 Create Subtiled TR

If a queue has space in it Queue active FIFO it will request the next sub-TR from the top of the list. The subtiler will build a sub-TR based on four possible aspects of the TR.

1. The trigger0 size
2. The trigger1 size
3. The event size
4. Transpose formatting enabled

The subtiler will look at what level is being decremented to tell the write queue if the output event should be generated based on the TR specified criteria.

3.4.2.4.1 Non-Transpose Formatting

If the transpose is not set the size of the sub-TR will either be sent with ICNT0 set the same as the original TR and all other counts set as 1 or it will send ICNT0 and ICNT1 set the same as the original TR and all other counts set as 1. The case where it only sends ICNT0 is if any of the trigger size or event size is set as on ICNT1 decrement.

The subtiler logic keeps track of the decrement of all other counters so for a two dimensional block it will decrement ICNT2 by 1 and if the current ICNT2 is 1 then it will decrement ICNT3 by 1 and set ICNT2 back to the original TR ICNT2. If ICNT3 is 1 and ICNT2 is 1 then it will mark the TR as the last one.

3.4.2.4.2 Transpose Formatting

If the TR has transpose set then the TR will be composed of a TR transpose sub tile based on the element size. See the transpose description in the UTC TR Specification for details.

3.4.2.5 Clear Trigger Counters

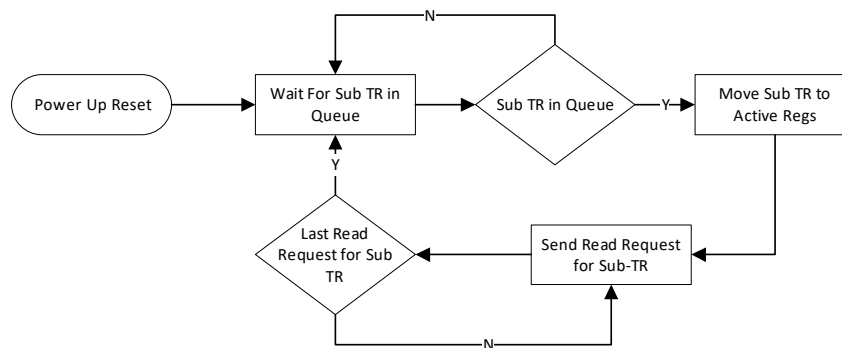
Upon sending a sub-TR down to the Queue Active FIFO the level of decrement is checked against all of the trigger levels specified in the TR. If the level of the trigger is met then the trigger counter will be decremented by 1. For example, if a TR has specified a ICNT2 Decrement by 1 for the trigger type and trigger of local then if the subtiler sends the sub-TR to the Queue Active FIFO and the subtiler determines that ICNT2 of the next TR is decremented by 1 from the current TR then it will send the signal back to the channel logic to decrement the trigger counter for the local event.

If the sub-TR if the sub-tiler determines that it is the last sub-TR for the current TR, then it will return to the wait for TR state and begin looking at the next entry in the channel TR FIFO.

3.4.3 Read Queue Control Flow

The read queue logic is described below. The read queue pops a sub-TR out of its FIFO and then breaks the TR into a series of aligned reads and when the last read is sent it will repeat the process.

Figure 3-10 Read Queue Flow Diagram



3.4.3.1 Wait for Sub-TR in Queue Active FIFO

The read queue waits until there is an entry in the Queue Active FIFO. As long as the Queue Active FIFO is NOT Full the read engine will keep asking for a sub-TR from the subtiler. Once the Queue Active FIFO is NOT Empty then the read queue will move to the active registers and begin sending read commands.

3.4.3.2 Move Sub-TR to Active Registers

Upon receiving an entry in the Queue Active FIFO the read queue will pop the top entry out of the FIFO and place it in its active registers to start doing the address calculations.

3.4.3.3 Send Read Requests for Sub-TR

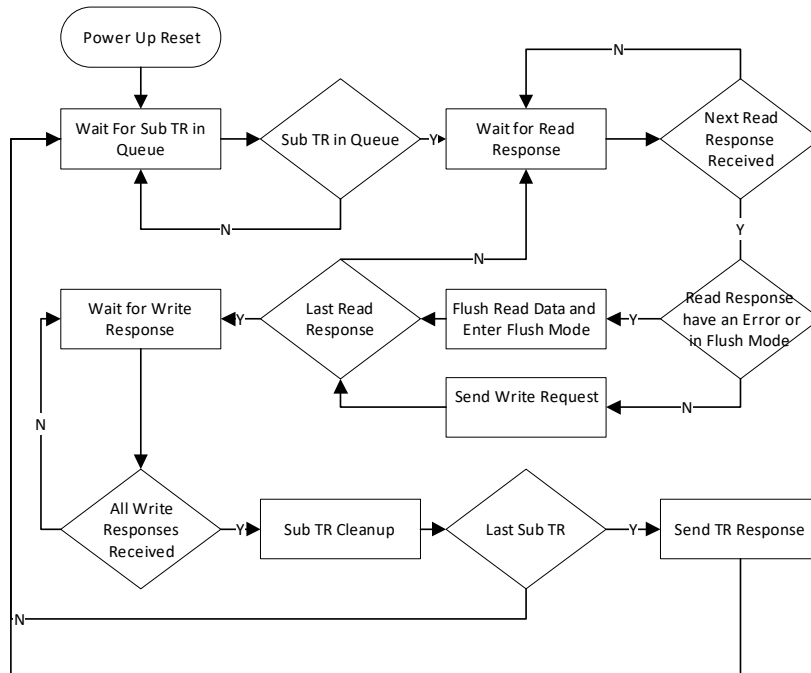
Once the read queue has a TR in its active register it will try and send a read request every cycle that it has the memory attributes for the command. The reads will be arbitrated by the arbitration logic based on the queue arbitration scheme. The arbitration will occur as long as there is space in the response buffer and the commands are sent out on the read DMA bus.

Each read request will receive the memory attributes either through an IOMMU or the memory attribute table if it exists. The page size will be returned by either the IOMMU or the memory attribute table and any time a read address crosses the address range of the page it will request a new read from the table to get the new attributes and stop sending reads until the new page attributes are received. If the DRU configuration does not have a memory attribute table or IOMMU then it will have default memory attributes.

3.4.4 Write Queue Control Flow

The write queue flow control unlike the read also has to handle any errors that can happen in the transfer. In the case of the write any time it determines that an error has occurred on the sub-TR that it is working the queue will enter a flush mode where it will wait for the last read response and empty out any read data from the response buffer but it will not send any commands. The read response buffer marks any error along with data availability so the write queue can see it before it reads the data. The write queue can also enter the flush mode if it receives a write response that is not successful. The below diagram shows how the write queue command flow works.

Figure 3-11 Write Queue Control Flow Diagram



3.4.4.1 Wait for Sub TR in Queue

3.4.4.2 Wait for Read Response Data

3.4.4.3 Send Write Request

3.4.4.4 Flush Read Data

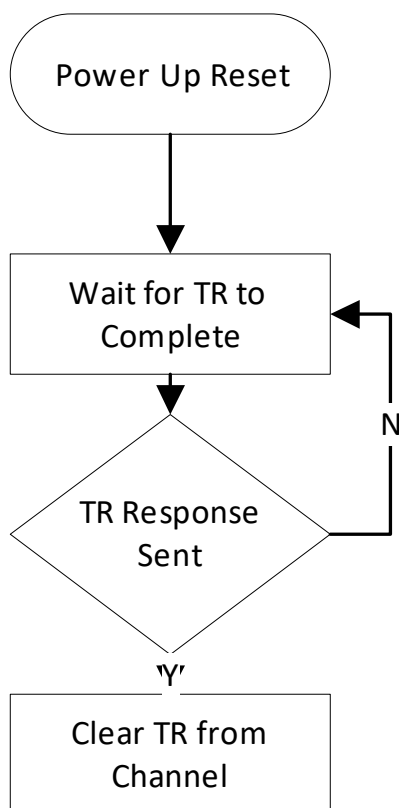
3.4.4.5 Wait for Write Response

3.4.4.6 Sub TR Cleanup

3.4.4.7 Send TR Response

3.4.5 Channel Clear Flow

Figure 3-12



3.4.5.1 Wait for TR Complete

3.4.5.2 Clear TR from Channel

Interfaces

The DRU has the following external interfaces 2 pairs of VBUSMC ports or 2 VBUSM buses and a VBUSP configuration port based on the bus type configuration parameter. A pair of PSI-L buses, an interrupt bus, a local input events, local output events and a set of tieoffs. The description of the functions and limitations of each interface will be described below.

4.1 Bus Type Configuration Parameter Equals C

The configuration parameter of Bus Type selects what the interface should be for the DMA operations. If the parameter equals C then the bus type for both the read and the write ports are VBUSM.C. Additionally, the MMR configuration port will be split between threads on the two ports with MMR configuration reads going to the DMA Write port and MMR configuration writes going to the DMA read port. This is to minimize area and take advantage of the threading supported in the VBUSM.C architecture.

4.1.1 MSMC_0_DRU/DRU_0_MSMC VBUSM.C Pair

This VBUSMC interfaces are used to perform the DMA read commands and also the MMR writes will also be directed to this port. The tables below show the VBUSM.C configuration of this pair of buses. This pair of VBUSM.C interfaces has an input and output CMD Interface, an input data interface and an output write response interface.

4.1.1.1 Threads

To minimize the number of data buses the DRU logic assumes that routing of configuration commands on the port will be routed based on the direction of the transaction to steer the transaction to the port that has the data bus in the correct direction.

The table below shows the breakdown of the threads and what criteria is assumed for the routing. The selection criteria has been chosen to minimize the amount of compares to select the correct thread.

Table 4-1 Thread Selection for MSMC_0_DRU/DRU_0_MSMC bus

Thread	Purpose	Selection Criteria
0: CPU	DMA read commands that were sent on Thread 0	copcode == Read Response and !thread 2
1: DMA	MMR CFG Write to non submission register	copcode != Read Response and thread 1

Table 4-1 Thread Selection for MSMC_0_DRU/DRU_0_MSMC bus

Thread	Purpose	Selection Criteria
2: CPU Hi	DMA read commands that were sent on Thread 2	copcode == Read Response and Thread 2
3: DMA HI	MMR CFG write to non submission registers	copcode != Read Response and thread 1
End of Table 4-1		

4.1.1.2 Configuration

4.1.1.3 Signals

Table 4-2 DRU_0_MSMC VBUSM.C CMD (Output) Interface Signals (Part 1 of 2)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR Address
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccredittype	2	All Indicate kind of credit-type being spent. The DRU will use thread based credits only so this interface will always be 0x1.
cdir	1	Command Indicate read vs. write. This signal will always be tied to 0x1 as this port will only issue read commands
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	1	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. This id matches the buffer id allocated to the read so the number of bits used is only based on the maximum number of in-flight commands. All other pins are tied to zero.
cinterest	1	Command Debug marker. This signal is set if a DTR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a VTR then this value is tied to 0
cmsband	CFG	Command User defined sideband signal that is mirrored on the response. This interface is always tied off to zero
cnogap	1	Command Contiguous byte enable attribute. This is always tied high as all transactions will not have a gap.
copcode	6	Command Coherence opcode. This can only send the RD_ONCE or RD_NO_SNOOP attribute which will be based on if the memory space is shareable or not.
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.

Table 4-2 DRU_0_MSMC VBUSM.C CMD (Output) Interface Signals (Part 2 of 2)

Signal	Width	Description
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR .
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
cqos	3	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_rd_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physica
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. On a VTR the if the page comes back with security demotion then it will reflect the value from the page.
csubthread	1	Command Identify the sub-thread associated with this transaction. This is always tied high since it only issues blocking commands.
cthread	Number of Queues + 2	Command Identify the thread associated with this transaction. The thread used will always be thread 0. The threads are for DTR submission and are used in the MSMC_0_DRU CMD Interface. Therefore this interface will always be set with the LSB tied high and all other bits are 0.
cgreturn	1	Credit return Global credit return required Slave. This signal is not used as only thread credits are used.
cs0return	Number of Queues + 2	Credit return Subthread0 (blocking) credit return bit-vector, one bit for each valid thread required Slave. This signal is not used as only thread credits are used.
cs1return	Number of Queues + 2	Credit return Subthread1 (non-blocking) credit return bit-vector, one bit for each valid. This signal is not used as only thread credits are used.
ctreturn	Number of Queues + 2	Credit return Thread credit return bit-vector, one bit for each valid thread. required Slave. Only a thread return for thread 0 is expected as that is the only thread set in cthread and credittype is tied to threaded.
End of Table 4-2		

Table 4-3 DRU_0_MSMC VBUSM.C WRC (Output) Interface Signals

Signal Width Transaction Description	Width	Description
sbytecnt	10	Number of bytes to which this status corresponds. This will reflect back the cbytecnt that can be sent in and can be from 1-128.
screditttype	2	Credit type selection. This interface will only use global credits and so it is tied to 0.
sid	12	Loopback ID for dedicated write response interface. This will match the cid from the write configuration command.
sinterest	1	Debug marker. This will match the cinterest from the write configuration command.
smsband	CFG	Loopback (mirrored) sideband signal. Thie will match the cmsband from the write configuration command.
sorderid	4	Loopback from command corderid. This will match the corderid from the write configuration command
sparity	1	Parity bit on Write Response Channel. This signal is present but currently just tied low until required for a safety application.
sreq	1	Indicate presence of write response.
srouteid	12	Loopback master ID for the write response. This will match the crouteid from the write configuration command.
ssband	0	User defined sideband signal. This is not present on this port
sstatus	3	Write status for dedicated write response interface. This interface will return the status based on the configuration write. It can support Success, Addressing Error or Data Error
ssubthread	1	Identify the sub-thread to which the given transaction belongs (per thread). This will match the csubthread of the configuration write.
sthread	Number of Queues + 2	Identify the thread to which the given data phase belongs required Master
strace	4	Debug trace information. This interface is just tied low as it is the response to a register.
stype	1	Identify the type of response. This only support sending a normal write response so it is tied low.
sgreturn	1	Credit return Credit return strobe. This is the only credit that should be returned.
ss0return	Number of Queues + 2	Credit return Return subthread0 (blocking) credit for the given thread. This should never be set as no credits of this type will be sent.
ss1return	Number of Queues + 2	Credit return Return subthread1 (non-blocking) credit for the given thread This should never be set as no credits of this type will be sent.
streturn	Number of Queues + 2	Credit return Return thread credit. This should never be set as no credits of this type will be sent.
End of Table 4-3		

Table 4-4 MSMC_0_DRU VBUSM.C CMD (Input) Interface Signals (Part 1 of 2)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR address fields
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccredittype	2	All Indicate kind of credit-type being spent. The DRU will use thread based credits only so this interface will always be 0x1.
cdir	1	Command Indicate read vs. write. This signal will always be tied to 0x1 as this port will only issue read commands
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	1	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. This id matches the buffer id allocated to the read so the number of bits used is only based on the maximum number of in-flight commands. All other pins are tied to zero.
cinterest	1	Command Debug marker. This signal is set if a DTR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a VTR then this value is tied to 0
cmsband	CFG	Command User defined sideband signal that is mirrored on the response. This interface is always tied off to zero
cnogap	1	Command Contiguous byte enable attribute. This is always tied high as all transactions will not have a gap.
copcode	6	Command Coherence opcode. This can only send the RD_ONCE or RD_NO_SNOOP attribute which will be based on if the memory space is shareable or not.
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR.
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.

Table 4-4 MSMC_0_DRU VBUSM.C CMD (Input) Interface Signals (Part 2 of 2)

Signal	Width	Description
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
cqos	3	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_rd_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physica
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
csubthread	1	Command Identify the sub-thread associated with this transaction. This is always tied high since it only issues blocking commands.
cthread	Number of Queues + 2	Command Identify the thread associated with this transaction. The thread used will always be thread 0. The threads are for DTR submission and are used in the MSMC_0_DRU CMD Interface. Therefore this interface will always be set with the LSB tied high and all other bits are 0.
cgreturn	1	Credit return Global credit return required Slave. This signal is not used as only thread credits are used.
cs0return	Number of Queues + 2	Credit return Subthread0 (blocking) credit return bit-vector, one bit for each valid thread required Slave. This signal is not used as only thread credits are used.
cs1return	Number of Queues + 2	Credit return Subthread1 (non-blocking) credit return bit-vector, one bit for each valid. This signal is not used as only thread credits are used.
ctreturn	Number of Queues + 2	Credit return Thread credit return bit-vector, one bit for each valid thread. required Slave. Only a thread return for thread 0 is expected as that is the only thread set in cthread and credittype is tied to threaded.
End of Table 4-4		

Table 4-5 MSMC_0_DRU VBUSM.C Data (Input) Interface Signals (Part 1 of 3)

Signal	Width	Transaction Description
dalign	10	Responses Data alignment. Bit 6 of this is used to determine the lowest bit of the address for the response buffer.
dbfirst	1	First data phase in a burst. This is not used in the DRU.
dblast	1	Last data phase in a burst. This is not used in the DRU.
dbytecnt	10	Burst data transfer byte count. This is not used in the DRU.
dbyten	64	Indicate valid data bytes for the entire data window in the current cycle. This is used both in the MMR config and the response buffer to only write the specified bytes.
dcoh	2	Response Indicate coherence state for this response transaction. This is not used in the DRU,

Table 4-5 MSMC_0_DRU VBUSM.C Data (Input) Interface Signals (Part 2 of 3)

Signal	Width	Transaction Description
dcredittype	2	Credit type selection. The DRU will only allow thread type credits so this should always be 0x1 when a transaction is received.
ddata	512	Data for current data phase
ddatavalid	1	Indicates validity of the ddata signal. This is not used in the DRU
ddirty	1	Indicate the response data is dirty with respect to main memory. This is not used in DRU
dedc	20	EDC hamming and parity data fields computed on a 256 bit granularity. DRU will calculate this on the full ddata bus and not just on the ddata that has dbyten asserted.
depriority	3	Elevated priority. This is not used in DRU.
dfirst	1	First data phase in the entire data transfer. This is not used in DRU
did	12	Transaction ID for the current data transfer. For read responses the lower bits are used to generate the buffer address combined with Bit 6 of the dalgn. In the write configuration case any value can be received and will be refelected on the Write Response Channel
dinterest	1	Debug marker. This is not used in DRU.
dlast	1	Indicates the last data phase for the current data transfer. This is not used in DRU.
dmsband	5	Mirrored sideband signal for response transactions. Matches cmsband of originating command transaction. This is not used in the DRU.
dopcode	6	Response opcode. DRU uses this and if it is a read response it handles it as a read return for the response buffer. All other opcodes will be forwarded to the MMR to be responded to appropriately.
dorderid	4	Ordering ID for the transaction. On responses, loopback from command. This is not used in the DRU
dpar	0	Parity vector generated per 32 bit datum for entire data window
dparity	1	Single parity bit for all signals on the channel (except ddata, dedc). This is not used in the DRU.
dpriority	3	Priority for data transfer. This is not used in the DRU.
dreq	1	Indicate presence of a valid data phase
drouteid	12	Master ID for the current data transfer. This is not used in the DRU
dsband	0	User defined sideband signal. This is not used in the DRU
dstatus	3	Response success/failure status per data phase. If the status is not Success then the DRU will trigger an error for that slot so the TR can be aborted at the appropriate time.
dsubthread	1	Identify the sub-thread classification for this transaction (per thread). This is not used in the DRU
dthread	Number of Queues + 2	Identify the thread to which the given data phase belongs. The DRU uses this to control that the number of TRs submitted to a queue does not exceed the amount of storage. Each thread is intended to be mapped to the address range of the specified queue,
dtrace	4	Debug trace information for read responses. This is not used in the DRU
dxcnt	8	Indicates the number of bytes transferred by the current data phase This is not used in the DRU
dgreturn	1	Global credit return. This is not used in the DRU

Table 4-5 MSMC_0_DRU VBUSM.C Data (Input) Interface Signals (Part 3 of 3)

Signal	Width	Transaction Description
ds0return	Number of Queues + 2	Return subthread0 (blocking) credit for the given thread. This is not used in the DRU
ds1return	Number of Queues + 2	Return subthread1 (non-blocking) credit for the given thread. This is not used in the DRU
dtreturn	Number of Queues + 2	Return thread credit this is the only type of credit that will be returned as the DRU only expects thread based credits to be used on this interface.
End of Table 4-5		

4.1.2 MSMC_1_DRU/DRU_1_MSMC VBUSM.C Pair

These VBUSMC interfaces are used to perform the DMA write commands, Send the Cache and MMU warm message and also the MMR reads will also be directed to this port. The tables below show the VBUSM.C configuration of this pair of buses. This pair of VBUSM.C interfaces has an input and output CMD Interface, an output data interface and an input write response interface.

4.1.2.1 Threads

The DRU uses threads to separate between MMR read requests and write data requests.

Table 4-6 Thread Selection for MSMC_1_DRU/DRU_1_MSMC bus

Thread	Purpose
0: CPU	DMA write commands that TR were sent on Thread 0
1: DMA	MMR CFG Read
2: CPU Hi	DMA write commands that TR were sent on Thread 2
3: DMA HI	MMR CFG read using the QoS Thread
End of Table 4-6	

4.1.2.2 Configuration

The configuration options for the VBUSM.C is the data width and the types of copcodes will be determined by the cache warm feature enabled and virtual TRs.

4.1.2.3 Signals

Table 4-7 DRU_1_MSMC VBUSM.C CMD (Output) Interface Signals (Part 1 of 3)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR DADDR fields. In the case of a cache warm request this will be driven by address in the CR . In the case of a read response this will be tied to 0.
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.

Table 4-7 DRU_1_MSMC VBUSM.C CMD (Output) Interface Signals (Part 2 of 3)

Signal	Width	Description
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR or read responses this signal is forced to zero. For Cache operation it will be driven by the MMU results.
ccredittype	2	All Indicate kind of credit-type being spent. The DRU will use thread based credits only so this interface will always be 0x1.
cdir	1	Command Indicate read vs. write. This signal will be driven low for the DMA write commands and MMU Warm Messages and it will be high for Cache Warm Messages
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	0	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. The lower 8 bits are a rotating counter that is shared by all queues. The upper four bits are used to identify which queue sent it with urgent queue being 0, queue A being 1 etc and the cache warm after that.
cinterest	1	Command Debug marker. This signal is set if a DTR or a DCR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a CR or VTR then this value is tied to 0
cmsband	5	Command User defined sideband signal that is mirrored on the response. This interface is always tied off to zero for a DMA or Read response. For the Message types it will be based on the conversion of the ID in the CR to the port id inside of MSMC.
cnogap	1	Command Contiguous byte enable attribute. This is always tied high as all transactions will not have a gap.
copcode	6	Command Coherence opcode. DMAs will send the WR_UNIQUE or WR_NO_SNOOP attribute which will be based on if the memory space is shareable or not. The read response will send READ_RESPONSE and the CR will send a CacheWarmShared, CacheWarmUnique or MMUWarm based on the CR
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR .
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.

Table 4-7 DRU_1_MSMC VBUSM.C CMD (Output) Interface Signals (Part 3 of 3)

Signal	Width	Description
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.
cqos	0	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_wr_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physical TR
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. On a VTR the if the page comes back with security demotion then it will reflect the value from the page.
csubthread	1	Command Identify the sub-thread associated with this transaction. This is low for read response command and high for all others commands.
cthread	Number of Queues + 2	Command Identify the thread associated with this transaction. The thread used will always be thread 0 for the DMA WR commands and messages. The read response will always be sent on thread 1. The threads are for DTR submission and are used in the MSMC_0_DRU CMD Interface. All other bits are 0.
cgreturn	1	Credit return Global credit return required Slave. This signal is not used as only thread credits are used.
cs0return	Number of Queues + 2	Credit return Subthread0 (blocking) credit return bit-vector, one bit for each valid thread required Slave. This signal is not used as only thread credits are used.
cs1return	Number of Queues + 2	Credit return Subthread1 (non-blocking) credit return bit-vector, one bit for each valid. This signal is not used as only thread credits are used.
ctreturn	Number of Queues + 2	Credit return Thread credit return bit-vector, one bit for each valid thread. required Slave. A thread return for thread 0 is used for the write and message credit counter while a thread return on credit 1 will be used in the read response credit.
End of Table 4-7		

Table 4-8 DRU_1_MSMC VBUSM.C Data (Output) Interface Signals (Part 1 of 3)

Signal	Width	Transaction Description
dalign	10	Responses Data alignment. Bit 6 of this is used to determine the lowest bit of the address for the response buffer.
dbfirst	1	First data phase in a burst. This is not used in the DRU.
dblast	1	Last data phase in a burst. This is not used in the DRU.
dbytecnt	10	Burst data transfer byte count. This is not used in the DRU.
dbyten	64	Indicate valid data bytes for the entire data window in the current cycle. This is used both in the MMR config and the response buffer to only write the specified bytes.

Table 4-8 DRU_1_MSMC VBUSM.C Data (Output) Interface Signals (Part 2 of 3)

Signal	Width	Transaction Description
dcoh	2	Response Indicate coherence state for this response transaction. This is not used in the DRU,
dcreditttype	2	Credit type selection. The DRU will only allow thread type credits so this should always be 0x1 when a transaction is received.
ddata	512	Data for current data phase
ddatavalid	1	Indicates validity of the ddata signal. This is not used in the DRU
ddirty	1	Indicate the response data is dirty with respect to main memory. This is not used in DRU
dedc	20	EDC hamming and parity data fields computed on a 256 bit granularity. DRU will calculate this on the full ddata bus and not just on the ddata that has dbyten asserted.
depriority	3	Elevated priority. This is not used in DRU.
dfirst	1	First data phase in the entire data transfer. This is not used in DRU
did	12	Transaction ID for the current data transfer. For read responses the lower bits are used to generate the buffer address combined with Bit 6 of the dalign. In the write configuration case any value can be received and will be refelected on the Write Response Channel
dinterest	1	Debug marker. This is not used in DRU.
dlast	1	Indicates the last data phase for the current data transfer. This is not used in DRU.
dmsband	5	Mirrored sideband signal for response transactions. Matches cmsband of originating command transaction. This is not used in the DRU.
dopcode	6	Response opcode. DRU uses this and if it is a read response it handles it as a read return for the response buffer. All other opcodes will be forwarded to the MMR to be responded to appropriately.
dorderid	4	Ordering ID for the transaction. On responses, loopback from command. This is not used in the DRU
dpar	0	Parity vector generated per 32 bit datum for entire data window
dparity	1	Single parity bit for all signals on the channel (except ddata, dedc). This is not used in the DRU.
dpriority	3	Priority for data transfer. This is not used in the DRU.
dreq	1	Indicate presence of a valid data phase
drouteid	12	Master ID for the current data transfer. This is not used in the DRU
dsband	0	User defined sideband signal. This is not used in the DRU
dstatus	3	Response success/failure status per data phase. If the status is not Success then the DRU will trigger an error for that slot so the TR can be aborted at the appropriate time.
dsubthread	1	Identify the sub-thread classification for this transaction (per thread). This is not used in the DRU
dthread	Number of Queues + 2	Identify the thread to which the given data phase belongs. The DRU uses this to control that the number of TRs submitted to a queue does not exceed the amount of storage. Each thread is intended to be mapped to the address range of the specified queue,
dtrace	4	Debug trace information for read responses. This is not used in the DRU

Table 4-8 DRU_1_MSMC VBUSM.C Data (Output) Interface Signals (Part 3 of 3)

Signal	Width	Transaction Description
dxcnt	8	Indicates the number of bytes transferred by the current data phase This is not used in the DRU
dgreturn	1	Global credit return. This is not used in the DRU
ds0return	Number of Queues + 2	Return subthread0 (blocking) credit for the given thread. This is not used in the DRU
ds1return	Number of Queues + 2	Return subthread1 (non-blocking) credit for the given thread. This is not used in the DRU
dtreturn	Number of Queues + 2	Return thread credit this is the only type of credit that will be returned as the DRU only expects thread based credits to be used on this interface.
End of Table 4-8		

Table 4-9 MSMC_1_DRU VBUSM.C CMD (Input) Interface Signals (Part 1 of 2)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR Address fields
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccredittype	2	All Indicate kind of credit-type being spent. The DRU will use thread based credits only so this interface will always be 0x1.
cdir	1	Command Indicate read vs. write. This signal will always be tied to 0x1 as this port will only issue read commands
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	1	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. This id matches the buffer id allocated to the read so the number of bits used is only based on the maximum number of in-flight commands. All other pins are tied to zero.
cinterest	1	Command Debug marker. This signal is set if a DTR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a VTR then this value is tied to 0
cmsband	CFG	Command User defined sideband signal that is mirrored on the response. This interface is always tied off to zero
cnogap	1	Command Contiguous byte enable attribute. This is always tied high as all transactions will not have a gap.
copcode	6	Command Coherence opcode. This can only send the RD_ONCE or RD_NO_SNOOP attribute which will be based on if the memory space is shareable or not.
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.

Table 4-9 MSMC_1_DRU VBUSM.C CMD (Input) Interface Signals (Part 2 of 2)

Signal	Width	Description
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR .
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
cqos	3	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_rd_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physica
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface.
csubthread	1	Command Identify the sub-thread associated with this transaction. This is always tied high since it only issues blocking commands.
cthread	Number of Queues + 2	Command Identify the thread associated with this transaction. The thread used will always be thread 0. The threads are for DTR submission and are used in the MSMC_0_DRU CMD Interface. Therefore this interface will always be set with the LSB tied high and all other bits are 0.
cgreturn	1	Credit return Global credit return required Slave. This signal is not used as only thread credits are used.
cs0return	Number of Queues + 2	Credit return Subthread0 (blocking) credit return bit-vector, one bit for each valid thread required Slave. This signal is not used as only thread credits are used.
cs1return	Number of Queues + 2	Credit return Subthread1 (non-blocking) credit return bit-vector, one bit for each valid. This signal is not used as only thread credits are used.
ctreturn	Number of Queues + 2	Credit return Thread credit return bit-vector, one bit for each valid thread. required Slave. Only a thread return for thread 0 is expected as that is the only thread set in cthread and credittype is tied to threaded.
End of Table 4-9		

Table 4-10 MSMC_1_DRU VBUSM.C WRC (Input) Interface Signals

Signal Width Transaction Description	Width	Description
sbytecnt	10	Number of bytes to which this status corresponds. This will reflect back the cbytecnt that can be sent in and can be from 1-128.
scredittype	2	Credit type selection. This interface will only use global credits and so it is tied to 0.
sid	12	Loopback ID for dedicated write response interface. This will match the cid from the write configuration command.
sinterest	1	Debug marker. This will match the cinterest from the write configuration command.
smsband	CFG	Loopback (mirrored) sideband signal. This will match the cmsband from the write configuration command.
sorderid	4	Loopback from command corderid. This will match the corderid from the write configuration command.
sparity	1	Parity bit on Write Response Channel. This signal is present but currently just tied low until required for a safety application.
sreq	1	Indicate presence of write response.
srouteid	12	Loopback master ID for the write response. This will match the crouteid from the write configuration command.
ssband	0	User defined sideband signal. This is not present on this port.
sstatus	3	Write status for dedicated write response interface. This interface will return the status based on the configuration write. It can support Success, Addressing Error or Data Error.
ssubthread	1	Identify the sub-thread to which the given transaction belongs (per thread). This will match the csubthread of the configuration write.
stthread	Number of Queues + 2	Identify the thread to which the given data phase belongs required Master.
strace	4	Debug trace information. This interface is just tied low as it is the response to a register.
stype	1	Identify the type of response. This only support sending a normal write response so it is tied low.
sgreturn	1	Credit return Credit return strobe. This is the only credit that should be returned.
ss0return	Number of Queues + 2	Credit return Return subthread0 (blocking) credit for the given thread. This should never be set as no credits of this type will be sent.
ss1return	Number of Queues + 2	Credit return Return subthread1 (non-blocking) credit for the given thread. This should never be set as no credits of this type will be sent.
streturn	Number of Queues + 2	Credit return Return thread credit. This should never be set as no credits of this type will be sent.
End of Table 4-10		

4.2 Bus Type Configuration Parameter Equals M

The configuration parameter of Bus Type selects what the interface should be for the DMA operations. If the parameter equals M then the bus type for both the read and the write ports are VBUSM. Additionally, an extra VBUSP port will be added to support the MMR configuration accesses.

4.2.1 VBUSM DMA Bus

The VBUSM DMA bus is configured based on the data width parameter.

4.2.1.1 DRU_0_MSMC VBUSM Bus

The DRU_0_MSMC bus is used for send the read commands.

Table 4-11 DRU_0_MSMC VBUSM CMD (Output) Interface Signals (Part 1 of 2)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR DADDR fields. In the case of a cache warm request this will be driven by Address in the CR. In the case of a read response this will be tied to 0.
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR or read responses this signal is forced to zero. For Cache operation it will be driven by the MMU results.
cdir	1	Command Indicate read vs. write. This signal will be driven low for the DMA write commands and MMU Warm Messages and it will be high for Cache Warm Messages
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	0	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. The lower 8 bits are a rotating counter that is shared by all queues. The upper four bits are used to identify which queue sent it with urgent queue being 0, queue A being 1 etc and the cache warm after that.
cinterest	1	Command Debug marker. This signal is set if a DTR or a DCR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a CR or VTR then this value is tied to 0
cdir	1	Direction
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR.
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.

Table 4-11 DRU_0_MSMC VBUSM CMD (Output) Interface Signals (Part 2 of 2)

Signal	Width	Description
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.
cqos	0	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_wr_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physical TR
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. On a VTR the if the page comes back with security demotion then it will reflect the value from the page.
ralign	10	Responses Data alignment. Bit 6 of this is used to determine the lowest bit of the address for the response buffer.
rbfirst	1	First data phase in a burst. This is not used in the DRU.
rblast	1	Last data phase in a burst. This is not used in the DRU.
rbytecnt	10	Burst data transfer byte count. This is not used in the DRU.
rdata	512	Data for current data phase
redc	20	EDC hamming and parity data fields computed on a 256 bit granularity. DRU will calculate this on the full ddata bus and not just on the ddata that has dbyten asserted.
repriority	3	Elevated priority. This is not used in DRU.
rfirst	1	First data phase in the entire data transfer. This is not used in DRU
rid	12	Transaction ID for the current data transfer. For read responses the lower bits are used to generate the buffer address combined with Bit 6 of the dalign. In the write configuration case any value can be received and will be refelected on the Write Response Channel
rinterest	1	Debug marker. This is not used in DRU.
rlast	1	Indicates the last data phase for the current data transfer. This is not used in DRU.
rpriority	3	Priority for data transfer. This is not used in the DRU.
rreq	1	Indicate presence of a valid data phase
rstatus	3	Response success/failure status per data phase. If the status is not Success then the DRU will trigger an error for that slot so the TR can be aborted at the appropriate time.
rxcnt	8	Indicates the number of bytes transferred by the current data phase This is not used in the DRU
End of Table 4-11		

4.2.1.2 DRU_1_MSMC VBUSM Bus

The DRU_1_MSMC bus is used for sending the write commands.

Table 4-12 DRU_0_MSMC VBUSM CMD (Output) Interface Signals (Part 1 of 2)

Signal	Width	Description
caddress	CFG	Command Transaction address. The DRU can do full byte alignment based on the submitted TR DADDR fields. In the case of a cache warm request this will be driven by Address in the CR. In the case of a read response this will be tied to 0.
cbytecnt	10	Command Transaction size. The DRU can do from 1 to 128 bytes in a transfer. The upper 3 bits will always be 0.
ccinner	2	Command Cache allocation policy for inner cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR this signal is forced to zero.
ccouter	2	Command Cache allocation policy for outer cacheability domain, as indicated by MMU. This can only be controlled for a VTR for a physical TR or read responses this signal is forced to zero. For Cache operation it will be driven by the MMU results.
cdir	1	Command Indicate read vs. write. This signal will be driven low for the DMA write commands and MMU Warm Messages and it will be high for Cache Warm Messages
cdtype	2	Command Data type this will always be driven to the value for a DMA or 0x2.
cemudbg	1	Command Emulation level of the transaction. This value will always be 0x0.
cepriority	3	Command Elevated priority level. This will be the priority of the urgent queue if it is active or it will be the priority of the active queue.
cexcl	0	Command Exclusive access attribute. This signal is always tied low.
cid	12	All Transaction ID. The lower 8 bits are a rotating counter that is shared by all queues. The upper four bits are used to identify which queue sent it with urgent queue being 0, queue A being 1 etc and the cache warm after that.
cinterest	1	Command Debug marker. This signal is set if a DTR or a DCR is received with the cinterest bit set.
cmemtype	2	Command Memory type as indicated by MMU. If it is not a CR or VTR then this value is tied to 0
cdir	1	Direction
corderid	4	Command Order ID used by the slave to order transactions. This is always tied off to zero.
cpable	1	Command Prefetchable attribute as indicated by MMU. It is tied 0 for a physical TR.
cparity	1	Command Parity bit on Transaction Attribute Channel signals. The signal is present but just tied off for now. It is a placeholder for future use in safety applications.
cpriority	3	Command Priority level of the transaction. This is controlled by the priority of the queue sending the transaction which is set by writing to the priority field in the Queue configuration register << TBD add CR once registers can be generated
cpriv	2	Command Privilege level of the transaction master. This is driven with the priv given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.

Table 4-12 DRU_0_MSMC VBUSM CMD (Output) Interface Signals (Part 2 of 2)

Signal	Width	Description
cprivid	5	Command Privilege ID used for security architecture. This is driven with the privid given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. This is tied to zero for read responses.
cqos	0	Command Quality of service levels. This signal is currently tied low.
creq	1	Command Indicate presence of transaction.
crouteid	12	All Master ID to identify the master that originated the transaction this is driven by the tieoff_wr_routeid signal. << TBD Add Crossref
crsel	0	Command Region select driven by infrastructure. This signal is not present
csband	8	Command User defined sideband signal that is not mirrored on the response. This is tied off to zero
csdomain	2	Command Shareability domain as indicated by MMU. It is tied 0 for a physical TR
csecure	1	Command Secure-level of the transaction master. This is driven with the secure given on the TR submission either from the Write config write in a DTR or in the TR itself in a TR submitted through PSI-L interface. On a VTR the if the page comes back with security demotion then it will reflect the value from the page.
walign	10	Responses Data alignment. Bit 6 of this is used to determine the lowest bit of the address for the response buffer.
wbfirst	1	First data phase in a burst. This is not used in the DRU.
wblast	1	Last data phase in a burst. This is not used in the DRU.
wdata	512	Data for current data phase
wedc	20	EDC hamming and parity data fields computed on a 256 bit granularity. DRU will calculate this on the full ddata bus and not just on the ddata that has dbyten asserted.
wepriority	3	Elevated priority. This is not used in DRU.
wfirst	1	First data phase in the entire data transfer. This is not used in DRU
winterest	1	Debug marker. This is not used in DRU.
wlast	1	Indicates the last data phase for the current data transfer. This is not used in DRU.
wpriority	3	Priority for data transfer. This is not used in the DRU.
wreq	1	Indicate presence of a valid data phase
wxcnt	8	Indicates the number of bytes transferred by the current data phase This is not used in the DRU
sinterest	1	Debug marker. This will match the cinterest from the write configuration command.
sparity	1	Parity bit on Write Response Channel. This signal is present but currently just tied low until required for a safety application.
sreq	1	Indicate presence of write response.
sstatus	3	Write status for dedicated write response interface. This interface will return the status based on the configuration write. It can support Success, Addressing Error or Data Error
End of Table 4-12		

4.2.1.3 MMR_CFG VBUSP Port

The MMR_CFG port is used to write the MMR ports if the DMA bus is of type VBUSM or VBUSP. The data buses can be configured by the MMR data width parameter.

Table 4-13 MMR_CFG VBUSP Interface Signals

Signal	Width	Description
orderid	4	Order ID
routeid	12	Route Id
xid	12	Transaction ID
req	1	Request
dir	1	Direction
address	20	Address
bytecnt	mmr_cfg_xcnt	Bytecnt
first	1	First
last	1	Last
xcnt	3	Good Byte Count
byten	4	Byte Enables
wdata	mmr_cfg_data_width	Write Data
wready	1	Write Ready
rdatap	mmr_cfg_data_width	Read Data (Pipelined)
rstatus	3	Read Status
rready	1	Read Ready
amode	2	Addressing Mode
clsize	3	Cacheline size
priv	2	Privilege Attribute
privid	8	Privilege ID
secure	1	Secure Attribute
emudbg	1	Emulation Debug Attribute
interest	1	Debug Marker
virtid	12	Virtual ID attribute
sorderid	4	Status Order ID
srouteid	12	Status Route ID
sid	12	Status Transaction ID
sreq	1	Status Request
sbytecnt	10	Status Done Indicator
sstatus	3	Status Information
sinterest	1	Debug Marker
sready	1	Status Ready
End of Table 4-13		

4.3 PSI-L Bus Pair SOC_DRU/DRU_SOC

The PSI-L bus follows the PSI-L standard and is used to submit “TR” and to submit “CR” it is also used to forward table lookup request to the IOMMU. It can also be used to send and receive data from other UTCs within the system.

4.3.1 Threads

The number of threads on the PSI-L is configured by the number of channels. An additional channel will be added if split TR support is being used.

4.3.2 Configuration

The PSI-L is configured based on the number of channels and if split_tr support is required. Additionally, the width is selected through a configuration parameter.

4.3.3 Signals

The signals for the PSI-L interfaces are all required signals as specified in the PSI-L

4.4 Event Interfaces

The DRU has 2 event interfaces, the completion event and the error event. Both interfaces are per channel and only one can fire at a time. The interfaces can be configured to be a bit vector with a bit for each channel or encoded to have a single req signal and a channel to specify which channel.

Table 4-14 Event Bit Vector Interface Signals

Signal	Width	Description
complete_intr	NUM_CHS	The TR for CH bit has completed successfully
error_intr	NUM_CHS	The TR for CH bit has completed with and error
End of Table 4-14		

Table 4-15 Event Encoded Interface Signals

Signal	Width	Description
complete_intr	1	A TR has completed successfully the event_ch specifies which TR
error_intr	1	A TR has completed with an error the event_ch specifies which TR
event_ch	NUM_CHS_ENCODED	The channel that has completed
End of Table 4-15		

4.5 Local Input Events

The local input events are single pulse events that are used to trigger a local event that can be selected by the TR. The signal is a bit vector the size of the number of channels. There is no limitation on the number of local events that can be received in a given cycle.

Table 4-16 Local Event Bit Vector Interface Signals

Signal	Width	Description
local_in_event_intr	NUM_CHS	The TR for CH bit has completed successfully
End of Table 4-16		

4.6 Tieoff Interfaces

The tieoff interface is used for pseudo static configuration signals that change some aspect of the DRU performance. The signals are assumed to be held constant during the operation of the DRU.

Table 4-17 Tieoff Interface Signals

Signal	Width	Description
dru_core_tieoff_rd_routeid	12	This is the routeid that will be driven out on the read command DMA bus.
dru_core_tieoff_wr_routeid	12	This is the routeid that will be driven out on the write command DMA bus
dru_core_tieoff_mmu_routeid	12	This signal only exists if the DRU include an IOMMU and then it is the routeid that will e driven when a MMU based transaction is on the bus.
dru_core_tieoff_memmap_int_addr	15	This is the upper bits of the comparator for the location of the MSMC SRAM and is used by the memory attribute table to determine if the address falls in the SRAM region.
dru_core_tieoff_memmap_ext_addr	9	This is the upper bits of the comparator for the location of the DDR and is used by the memory attribute table to determine if the address falls in the DDR region.
dru_core_tieoff_memmap_ext_32b_enable	1	This is used by the memory attribute table to determine if the DDR boot region should be used or if the default values should be used.
End of Table 4-17		

Registers

5.1 "General registers [Address = 0x0]" on page 5-68

5.1 General registers [Address = 0x0]

General registers (Part 1 of 6)

Offset	Acronym	Register Description	Section
0x0	DRU_PID	Peripheral ID Register	5.1.1
0x8	DRU_CAPABILITIES	DRU Capabilities: Lists the capabilities of the channel for TR TYPE and formatting functions	5.1.2
0x40	DRU_PRI_MASK	DRU Priority Mask allows to use portions of the response buffer only for the priority queues	5.1.3
0x4000	DRU_SHARED_EVT_SET	Set the shared events	5.1.4
0x4040-0x404F	DRU_COMP_EVT_SETX	Set the completion events for each channel	5.1.5
0x4080-0x408F	DRU_ERR_EVT_SETX	Set the error event for each channel	5.1.6
0x40C0-0x40CF	DRU_LOCAL_EVT_SETX	Set the local output event for each channel	5.1.7
0x8000	QUEUE0_CFG	Configuration Register for Queue 0	5.1.5
0x8008	QUEUE1_CFG	Configuration Register for Queue 1	5.1.9
0x8010	QUEUE2_CFG	Configuration Register for Queue 2	5.1.10
0x8018	QUEUE3_CFG	Configuration Register for Queue 3	5.1.11
0x8020	QUEUE4_CFG	Configuration Register for Queue 4	5.1.12
0x8028	QUEUE5_CFG	Configuration Register for Queue 5	5.1.13
0x8030	QUEUE6_CFG	Configuration Register for Queue 6	5.1.14
0x8038	QUEUE7_CFG	Configuration Register for Queue 7	5.1.15
0x8040	QUEUE0_STATUS	Status Register for Queue 0	5.1.16
0x8048	QUEUE1_STATUS	Status Register for Queue 0	5.1.17
0x8050	QUEUE2_STATUS	Status Register for Queue 0	5.1.18
0x8058	QUEUE3_STATUS	Status Register for Queue 0	5.1.19
0x8060	QUEUE4_STATUS	Status Register for Queue 0	5.1.20
0x8068	QUEUE5_STATUS	Status Register for Queue 0	5.1.21
0x8070	QUEUE6_STATUS	Status Register for Queue 0	5.1.22
0x8078	QUEUE7_STATUS	Status Register for Queue 0	5.1.23
0xA000	MMU_PID	Physical ID Register	5.1.24
0xA100	TDRR	Translation Debug Response Register	5.1.25
0xA140	TDFAR	Translation Debug Fault Address Register	5.1.26
0xA200	TLB_INV	TLB Invalidation Register	5.1.27
0xA280	TLB_INV_C	TLB Invalidation Complete Register	5.1.28
0xA2C0	TLB_DBG	TLB Debug Access Register	5.1.29
0xA300	TLB_DBG_DATA0	TLB Debug Data 0 Register	5.1.30
0xA340	TLB_DBG_DATA1	TLB Debug Data 1 Register	5.1.31
0xA400	SCR	System Control Register	5.1.32
0xA440	TCR0	Table Control Register 0	5.1.33
0xA480	TCR1	Table Control Register 1	5.1.34
0xA4C0	TBR0	Table Base Register 0	5.1.35
0xA500	TBR1	Table Base Register 1	5.1.36

General registers (Part 2 of 6)

Offset	Acronym	Register Description	Section
0xA540	MAR	Memory Attribute Register	5.1.37
0xA580	TDAR	Translation Debug Address Register	5.1.38
0xA800	SCR_GS	System Control Register - Non-Secure Guest Supervisor	5.1.39
0xA840	TCR0_GS	Table Control Register 0 - Non-Secure Guest Supervisor	5.1.40
0xA880	TCR1_GS	Table Control Register 1 - Non-Secure Guest Supervisor	5.1.41
0xA8C0	TBR0_GS	Table Base Register 0 - Non-Secure Guest Supervisor	5.1.42
0xA900	TBR1_GS	Table Base Register 1 - Non-Secure Guest Supervisor	5.1.43
0xA940	MAR_GS	Memory Attribute Register - Non-Secure Guest Supervisor	5.1.44
0xA980	TDAR_GS	Translation Debug Address Register - Non-Secure Guest Supervisor	5.1.45
0xAC00	SCR_S	System Control Register - Non-Secure Supervisor	5.1.46
0xAC40	TCR0_S	Table Control Register 0 - Non-Secure Supervisor	5.1.47
0xAC80	TCR1_S	Table Control Register 1 - Non-Secure Supervisor	5.1.48
0xACC0	TBR0_S	Table Base Register 0 - Non-Secure Supervisor	5.1.49
0xAD00	TBR1_S	Table Base Register 1 - Non-Secure Supervisor	5.1.50
0xAD40	MAR_S	Memory Attribute Register - Non-Secure Supervisor	5.1.51
0xAD80	TDAR_S	Translation Debug Address Register - Non-Secure Supervisor	5.1.52
0xB800	VCR	Virtualization Control Register	5.1.53
0xB840	VTCT	Virtualization Table Control Register	5.1.54
0xB880	VTBR	Virtualization Table Base Register	5.1.55
0xB8C0	VTDA	Virtualization Translation Debug Address Register	5.1.56
0xC000	RD_UTLB_MATCH	Indexed uTLB Page Match Entry	5.1.57
0xD000	RD_UTLB_ATTR	Indexed uTLB Page Attribute Entry	5.1.58
0xE000	WR_UTLB_MATCH	Indexed uTLB Page Match Entry	5.1.59
0xF000	WR_UTLB_ATTR	Indexed uTLB Page Attribute Entry	5.1.60
0x10000	MSMC SRAM MEMORY ATTRIBUTES	The memory attributes for transactions that are to the MSMC SRAM	5.1.24
0x20000	DDR MEMORY ATTRIBUTES	The memory attributes for transactions that are to the DDR	5.1.62
0x30000	DDR BOOT MEMORY ATTRIBUTES	The memory attributes for the transaction that are to the boot region of the DDR.	5.1.63
0x40000	CHCFG	Channel Configuration Register. The Channel Configuration Register is used to initialize static mode settings for the DMA Channel. The register may only be written when the channel is disabled in the ch_enable field in the REALTIME CONTROL register is 0.	5.1.64
0x40020	CHOE0	The Output Event Steering Registers are used to specify a global event number to generate anytime the required event generation criteria specified in a TR are met. A single event with the event number set equal to the value in the corresponding register will be generated. This register is provided in order to allow security SW to lock down which events in the global space any given channel/thread is allowed to generate	5.1.65
0x040040	CHRING_ADDR	The Channel Ring Base Address register contains the 48-bit base address for the ring.	5.1.66
0x040048	CHRING_SIZE	The Ring Size Register contains the element size and element counts for the ring.	5.1.67

General registers (Part 3 of 6)

Offset	Acronym	Register Description	Section
0x40060	CHST_SCHED	Channel Static Scheduler Config Register	5.1.68
0x60000	CHRT_CTL	The channel realtime control register contains real-time control and status information for the DMA Channel. The fields in this register can be changed while the channel is in operation.	5.1.69
0x60008	CHRT_SWTRIG	The Software Trigger Register provides a mechanism by which software can directly trigger the channel in a secure way.	5.1.70
0x60010	CHRT_ERR_STAT	The error status register provides the information if an error occurs on a TR. This information stores the first error that has occurred for a TR.	5.1.71
0x60018	CHRT_ERR_CNT	The counts of the write engine when an error was received for a TR. This information stores the first error that has occurred for a TR.	5.1.72
0x60020	CHRT_RING_FWD_DB	The Ring Forward Doorbell Register is written by software to increment the number of entries on the submitting ring. One or more entries as specified by the entry_cnt field can be added to a ring with a single write operation. Note: Since the occupancy number is 17 bits and the doorbell entry count is a 8 bit 2's complement number it can take multiple writes to clear the reverse occupancy.	5.1.73
0x60028	CHRT_RING_FWD_OCC	The Ring Forward Occupancy Register can be read by software to determine the total number of valid entries on a ring.	5.1.74
0x60040	CHRT_RING_RVRS_DB	The Ring Reverse Doorbell Register is written by software to increment the number of entries on the submitting ring. One or more entries as specified by the entry_cnt field can be added to a ring with a single write operation. Note: Since the occupancy number is 17 bits and the doorbell entry count is a 8 bit 2's complement number it can take multiple writes to clear the reverse occupancy.	5.1.75
0x60048	CHRT_RING_RVRS_OCC	The Ring Forward Occupancy Register can be read by software to determine the total number of valid entries on a ring.	5.1.76
0x80000	ATOMIC_SUBMIT_CURR_TR_WORD0_1	The first TR submission word	5.1.77
0x80008	ATOMIC_SUBMIT_CURR_TR_WORD2_3	The second TR submission word	5.1.78
0x80010	ATOMIC_SUBMIT_CURR_TR_WORD4_5	The third TR submission word	5.1.79
0x80018	ATOMIC_SUBMIT_CURR_TR_WORD6_7	The fourth TR submission word	5.1.80
0x80020	ATOMIC_SUBMIT_CURR_TR_WORD8_9	The fifth TR submission word	5.1.81
0x80028	ATOMIC_SUBMIT_CURR_TR_WORD10_11	The sixth TR submission word	5.1.82
0x80030	ATOMIC_SUBMIT_CURR_TR_WORD12_13	The seventh TR submission word	5.1.83
0x80038	ATOMIC_SUBMIT_CURR_TR_WORD14_15	The eight TR submission word	5.1.84
0x80040	NEXT_TR_WORD0_1	The first TR submission word	5.1.85
0x80048	NEXT_TR_WORD2_3	The second TR submission word	5.1.86
0x80050	NEXT_TR_WORD4_5	The third TR submission word	5.1.87
0x80058	NEXT_TR_WORD6_7	The fourth TR submission word	5.1.88
0x80060	NEXT_TR_WORD8_9	The fifth TR submission word	5.1.89
0x80068	NEXT_TR_WORD10_11	The sixth TR submission word	5.1.90
0x80070	NEXT_TR_WORD12_13	The seventh TR submission word	5.1.91
0x80078	NEXT_TR_WORD14_15	The eight TR submission word	5.1.92
0x80080	NEXT_NEXT_TR_WORD0_1	The first TR submission word	5.1.93
0x80088	NEXT_NEXT_TR_WORD2_3	The second TR submission word	5.1.94

General registers (Part 4 of 6)

Offset	Acronym	Register Description	Section
0x80090	NEXT_NEXT_TR_WORD4_5	The third TR submission word	5.1.95
0x80098	NEXT_NEXT_TR_WORD6_7	The fourth TR submission word	5.1.96
0x800A0	NEXT_NEXT_TR_WORD8_9	The fifth TR submission word	5.1.97
0x800A8	NEXT_NEXT_TR_WORD10_11	The sixth TR submission word	5.1.98
0x800B0	NEXT_NEXT_TR_WORD12_13	The seventh TR submission word	5.1.99
0x800B8	NEXT_NEXT_TR_WORD14_15	The eight TR submission word	5.1.100
0x800C0	CURR_STATE_WORD0_1	The first TR submission word	5.1.101
0x800C8	CURR_STATE_WORD2_3	The second TR submission word	5.1.102
0x800D0	CURR_STATE_WORD4_5	The third TR submission word	5.1.103
0x800D8	CURR_STATE_WORD6_7	The fourth TR submission word	5.1.104
0x800E0	CURR_STATE_WORD8_9	The fifth TR submission word	5.1.105
0x800E8	CURR_STATE_WORD10_11	The sixth TR submission word	5.1.106
0x800F0	CURR_STATE_WORD12_13	The seventh TR submission word	5.1.107
0x800F8	CURR_STATE_WORD14_15	The eight TR submission word	5.1.108
0xA0000	CORE0_SUBMIT_WORD0_1	The first TR submission word	5.1.109
0xA0008	CORE0_SUBMIT_WORD2_3	The second TR submission word	5.1.110
0xA0010	CORE0_SUBMIT_WORD4_5	The third TR submission word	5.1.111
0xA0018	CORE0_SUBMIT_WORD6_7	The fourth TR submission word	5.1.112
0xA0020	CORE0_SUBMIT_WORD8_9	The fifth TR submission word	5.1.113
0xA0028	CORE0_SUBMIT_WORD10_11	The sixth TR submission word	5.1.114
0xA0030	CORE0_SUBMIT_WORD12_13	The seventh TR submission word	5.1.115
0xA0038	CORE0_SUBMIT_WORD14_15	The eight TR submission word	5.1.116
0xA0040	CORE1_SUBMIT_WORD0_1	The first TR submission word	5.1.117
0xA0048	CORE1_SUBMIT_WORD2_3	The second TR submission word	5.1.118
0xA0050	CORE1_SUBMIT_WORD4_5	The third TR submission word	5.1.119
0xA0058	CORE1_SUBMIT_WORD6_7	The fourth TR submission word	5.1.120
0xA0060	CORE1_SUBMIT_WORD8_9	The fifth TR submission word	5.1.121
0xA0068	CORE1_SUBMIT_WORD10_11	The sixth TR submission word	5.1.122
0xA0070	CORE1_SUBMIT_WORD12_13	The seventh TR submission word	5.1.123
0xA0078	CORE1_SUBMIT_WORD14_15	The eight TR submission word	5.1.124
0xA0080	CORE2_SUBMIT_WORD0_1	The first TR submission word	5.1.125
0xA0088	CORE2_SUBMIT_WORD2_3	The second TR submission word	5.1.126
0xA0090	CORE2_SUBMIT_WORD4_5	The third TR submission word	5.1.127
0xA0098	CORE2_SUBMIT_WORD6_7	The fourth TR submission word	5.1.128
0xA00A0	CORE2_SUBMIT_WORD8_9	The fifth TR submission word	5.1.129
0xA00A8	CORE2_SUBMIT_WORD10_11	The sixth TR submission word	5.1.130
0xA00B0	CORE2_SUBMIT_WORD12_13	The seventh TR submission word	5.1.131
0xA00B8	CORE2_SUBMIT_WORD14_15	The eight TR submission word	5.1.132

General registers (Part 5 of 6)

Offset	Acronym	Register Description	Section
0xA00C0	CORE3_SUBMIT_WORD0_1	The first TR submission word	5.1.133
0xA00C8	CORE3_SUBMIT_WORD2_3	The second TR submission word	5.1.134
0xA00D0	CORE3_SUBMIT_WORD4_5	The third TR submission word	5.1.135
0xA00D8	CORE3_SUBMIT_WORD6_7	The fourth TR submission word	5.1.136
0xA00E0	CORE3_SUBMIT_WORD8_9	The fifth TR submission word	5.1.137
0xA00E8	CORE3_SUBMIT_WORD10_11	The sixth TR submission word	5.1.138
0xA00F0	CORE3_SUBMIT_WORD12_13	The seventh TR submission word	5.1.139
0xA00F8	CORE3_SUBMIT_WORD14_15	The eight TR submission word	5.1.140
0xC0000	CORE4_SUBMIT_WORD0_1	The first TR submission word	5.1.141
0xC0008	CORE4_SUBMIT_WORD2_3	The second TR submission word	5.1.142
0xC0010	CORE4_SUBMIT_WORD4_5	The third TR submission word	5.1.143
0xC0018	CORE4_SUBMIT_WORD6_7	The fourth TR submission word	5.1.144
0xC0020	CORE4_SUBMIT_WORD8_9	The fifth TR submission word	5.1.145
0xC0028	CORE4_SUBMIT_WORD10_11	The sixth TR submission word	5.1.146
0xC0030	CORE4_SUBMIT_WORD12_13	The seventh TR submission word	5.1.147
0xC0038	CORE4_SUBMIT_WORD14_15	The eight TR submission word	5.1.148
0xC0040	CORE5_SUBMIT_WORD0_1	The first TR submission word	5.1.149
0xC0048	CORE5_SUBMIT_WORD2_3	The second TR submission word	5.1.150
0xC0050	CORE5_SUBMIT_WORD4_5	The third TR submission word	5.1.151
0xC0058	CORE5_SUBMIT_WORD6_7	The fourth TR submission word	5.1.152
0xC0060	CORE5_SUBMIT_WORD8_9	The fifth TR submission word	5.1.153
0xC0068	CORE5_SUBMIT_WORD10_11	The sixth TR submission word	5.1.154
0xC0070	CORE5_SUBMIT_WORD12_13	The seventh TR submission word	5.1.155
0xC0078	CORE5_SUBMIT_WORD14_15	The eight TR submission word	5.1.156
0xC0080	CORE6_SUBMIT_WORD0_1	The first TR submission word	5.1.157
0xC0088	CORE6_SUBMIT_WORD2_3	The second TR submission word	5.1.158
0xC0090	CORE6_SUBMIT_WORD4_5	The third TR submission word	5.1.159
0xC0098	CORE6_SUBMIT_WORD6_7	The fourth TR submission word	5.1.160
0xC00A0	CORE6_SUBMIT_WORD8_9	The fifth TR submission word	5.1.161
0xC00A8	CORE6_SUBMIT_WORD10_11	The sixth TR submission word	5.1.162
0xC00B0	CORE6_SUBMIT_WORD12_13	The seventh TR submission word	5.1.163
0xC00B8	CORE6_SUBMIT_WORD14_15	The eight TR submission word	5.1.164
0xC00C0	CORE7_SUBMIT_WORD0_1	The first TR submission word	5.1.165
0xC00C8	CORE7_SUBMIT_WORD2_3	The second TR submission word	5.1.166
0xC00D0	CORE7_SUBMIT_WORD4_5	The third TR submission word	5.1.167
0xC00D8	CORE7_SUBMIT_WORD6_7	The fourth TR submission word	5.1.168
0xC00E0	CORE7_SUBMIT_WORD8_9	The fifth TR submission word	5.1.169
0xC00E8	CORE7_SUBMIT_WORD10_11	The sixth TR submission word	5.1.170

General registers (Part 6 of 6)

Offset	Acronym	Register Description	Section
0xC00F0	CORE7_SUBMIT_WORD12_13	The seventh TR submission word	5.1.171
0xC00F8	CORE7_SUBMIT_WORD14_15	The eight TR submission word	5.1.172
0xE0000	CAUSE0	Error Register cause for channels 0 to 15	5.1.173
0xE0008	CAUSE1	Error Register cause for channels 16 to 31	5.1.173
0xE0010	CAUSE2	Error Register cause for channels 32 to 47	5.1.173
0xE0018	CAUSE3	Error Register cause for channels 48 to 63	5.1.173
0xE0020	CAUSE4	Error Register cause for channels 64 to 79	5.1.173
0xE0028	CAUSE5	Error Register cause for channels 80 to 95	5.1.173
0xE0030	CAUSE6	Error Register cause for channels 96 to 111	5.1.173
0xE0038	CAUSE7	Error Register cause for channels 112 to 127	5.1.173
0xE0040	CAUSE8	Error Register cause for channels 128 to 143	5.1.173
0xE0048	CAUSE9	Error Register cause for channels 144 to 159	5.1.173
0xE0050	CAUSE10	Error Register cause for channels 160 to 175	5.1.173
0xE0058	CAUSE11	Error Register cause for channels 176 to 191	5.1.173
0xE0060	CAUSE12	Error Register cause for channels 192 to 207	5.1.173
0xE0068	CAUSE13	Error Register cause for channels 208 to 223	5.1.173
0xE0070	CAUSE14	Error Register cause for channels 224 to 239	5.1.173
0xE0078	CAUSE15	Error Register cause for channels 240 to 255	5.1.173

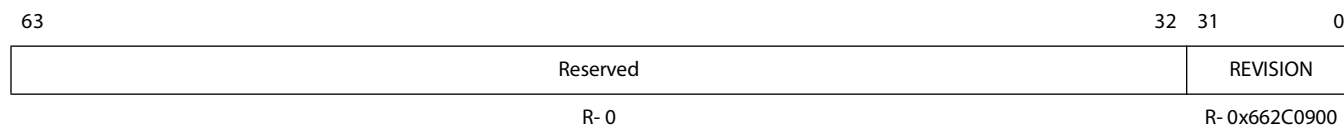
5.1.1 DRU_PID [Address = 0x0]

Peripheral ID Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-1 DRU_PID



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-1 DRU_PID Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved
31-0	REVISION	PID Revision

5.1.2 DRU_CAPABILITIES [Address = 0x8]

DRU Capabilities: Lists the capabilities of the channel for TR TYPE and formatting functions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-2 DRU_CAPABILITIES

63	47	46	43	42	39	38	35	34	32	31	20	19	18	17	16
Reserved	SECTR	DFMT	ELTYPE	AMODE	RSVD_CONF_SPEC	GLOBAL_TRIG	LOCAL_TRIG	EOL	STATIC						
R-0	R-0x1	R-0x2	R-0x7	R-0x0	R-0	R-0x1	R-0x1	R-0	R-1						
15	14	13	12	11	10	9	8								
TYPE15	TYPE14	TYPE13	TYPE12	TYPE11	TYPE10	TYPE9	TYPE8								
R-0	R-0	R-0	R-0	R-0	R-1	R-0	R-1								
7	6	5	4	3	2	1	0								
TYPE7	TYPE6	TYPE5	TYPE4	TYPE3	TYPE2	TYPE1	TYPE0								
R-0	R-0	R-0x1	R-0	R-0	R-0	R-0	R-0								

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-2 DRU_CAPABILITIES Field Descriptions

Bits	Name	Description
63-47	Reserved	Reserved
46-43	SECTR	Maximum second TR function that is supported
42-39	DFMT	Maximum data reformatting function that is supported
38-35	ELTYPE	Maximum element type value that is supported
34-32	AMODE	The maximum AMODE that is supported. If AMODE is supported then DIR field must be supported for that AMODE.
31-20	RSVD_CONF_SPEC	Reserved for Configuration Specific Features. This implementation has no configuration specific features.
19	GLOBAL_TRIG	Global Triggers 0 and 1 are supported
18	LOCAL_TRIG	Dedicated Local Trigger is supported
17	EOL	EOL Field is supported
16	STATIC	STATIC Field is supported
15	TYPE15	Type 15 TR is supported
14	TYPE14	Type 14 TR is supported
13	TYPE13	Type 13 TR is supported
12	TYPE12	Type 12 TR is supported
11	TYPE11	Type 11 TR is supported

Table 5-2 DRU_CAPABILITIES Field Descriptions

Bits	Name	Description
10	TYPE10	Type 10 TR is supported
9	TYPE9	Type 9 TR is supported
8	TYPE8	Type 8 TR is supported
7	TYPE7	Type 7 TR is supported
6	TYPE6	Type 6 TR is supported
5	TYPE5	Type 5 TR is supported
4	TYPE4	Type 4 TR is supported
3	TYPE3	Type 3 TR is supported
2	TYPE2	Type 2 TR is supported
1	TYPE1	Type 1 TR is supported
0	TYPE0	Type 0 TR is supported

5.1.3 DRU_PRI_MASK [Address = 0x0_0040]

Priority Buffer Mask. Allows the buffer entry to only be used for the priority queues. For buffer entries that do not exist the field will be treated as reserved.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-3 DRU_PRI_MASK

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
BUF63	BUF62	BUF61	BUF60	BUF59	BUF58	BUF57	BUF56	BUF55	BUF54	BUF53	BUF52	BUF51	BUF50	BUF49	BUF48
RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
BUF47	BUF46	BUF45	BUF44	BUF43	BUF42	BUF41	BUF40	BUF39	BUF38	BUF37	BUF36	BUF35	BUF34	BUF33	BUF32
RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BUF31	BUF30	BUF29	BUF28	BUF27	BUF26	BUF25	BUF24	BUF23	BUF22	C21	BUF20	BUF19	BUF18	BUF17	BUF16
RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BUF15	BUF14	BUF13	BUF12	BUF11	BUF10	BUF9	BUF8	BUF7	BUF6	BUF5	BUF4	BUF3	BUF2	BUF1	BUF0
RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0	RW-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-3 DRU_PRI_MASK Field Descriptions

Bits	Name	Description
63	BUF63	Only the Priority queues can use buffer 63
62	BUF62	Only the Priority queues can use buffer 62
61	BUF61	Only the Priority queues can use buffer 61
60	BUF60	Only the Priority queues can use buffer 60
59	BUF59	Only the Priority queues can use buffer 59
58	BUF58	Only the Priority queues can use buffer 58
57	BUF57	Only the Priority queues can use buffer 57
56	BUF56	Only the Priority queues can use buffer 56
55	BUF55	Only the Priority queues can use buffer 55
54	BUF54	Only the Priority queues can use buffer 54
53	BUF53	Only the Priority queues can use buffer 53
52	BUF52	Only the Priority queues can use buffer 52
51	BUF51	Only the Priority queues can use buffer 51
50	BUF50	Only the Priority queues can use buffer 50
49	BUF49	Only the Priority queues can use buffer 49
48	BUF48	Only the Priority queues can use buffer 48
47	BUF47	Only the Priority queues can use buffer 47
46	BUF46	Only the Priority queues can use buffer 46
45	BUF45	Only the Priority queues can use buffer 45
44	BUF44	Only the Priority queues can use buffer 44
43	BUF43	Only the Priority queues can use buffer 43
42	BUF42	Only the Priority queues can use buffer 42
41	BUF41	Only the Priority queues can use buffer 41
40	BUF40	Only the Priority queues can use buffer 40
39	BUF39	Only the Priority queues can use buffer 39
38	BUF38	Only the Priority queues can use buffer 38
37	BUF37	Only the Priority queues can use buffer 37
36	BUF36	Only the Priority queues can use buffer 36
35	BUF35	Only the Priority queues can use buffer 35
34	BUF34	Only the Priority queues can use buffer 34
33	BUF33	Only the Priority queues can use buffer 33
32	BUF32	Only the Priority queues can use buffer 32
31	BUF31	Only the Priority queues can use buffer 31

Table 5-3 DRU_PRI_MASK Field Descriptions

Bits	Name	Description
30	BUF30	Only the Priority queues can use buffer 30
29	BUF29	Only the Priority queues can use buffer 29
28	BUF28	Only the Priority queues can use buffer 28
27	BUF27	Only the Priority queues can use buffer 27
26	BUF26	Only the Priority queues can use buffer 26
25	BUF25	Only the Priority queues can use buffer 25
24	BUF24	Only the Priority queues can use buffer 24
23	BUF23	Only the Priority queues can use buffer 23
22	BUF22	Only the Priority queues can use buffer 22
21	BUF21	Only the Priority queues can use buffer 21
20	BUF20	Only the Priority queues can use buffer 20
19	BUF19	Only the Priority queues can use buffer 19
18	BUF18	Only the Priority queues can use buffer 18
17	BUF17	Only the Priority queues can use buffer 17
16	BUF16	Only the Priority queues can use buffer 16
15	BUF15	Only the Priority queues can use buffer 15
14	BUF14	Only the Priority queues can use buffer 14
13	BUF13	Only the Priority queues can use buffer 13
12	BUF12	Only the Priority queues can use buffer 12
11	BUF11	Only the Priority queues can use buffer 11
10	BUF10	Only the Priority queues can use buffer 10
9	BUF9	Only the Priority queues can use buffer 9
8	BUF8	Only the Priority queues can use buffer 8
7	BUF7	Only the Priority queues can use buffer 7
6	BUF6	Only the Priority queues can use buffer 6
5	BUF5	Only the Priority queues can use buffer 5
4	BUF4	Only the Priority queues can use buffer 4
3	BUF3	Only the Priority queues can use buffer 3
2	BUF2	Only the Priority queues can use buffer 2
1	BUF1	Only the Priority queues can use buffer 1
0	BUF0	Only the Priority queues can use buffer 0

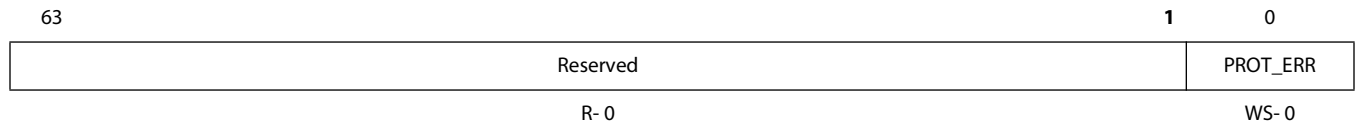
5.1.4 DRU_SHARED_EVT_SET[Address = 0x0_4000]

Shared Event Set Register for any events that are not channel based but common to the DRU.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-4 DRU_SHARED_EVT_SET



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-4 DRU_SHARED_EVT_SET Field Descriptions

Bits	Name	Description
63-1	Reserved	RESERVED
0	PROT_ERR	Set the protocol error event

5.1.5 DRU_COMP_EVT_SET[Address = 0x0_4040]

Completion Event Set Register for Channel 0-63 additional channels will follow the same in additional 64 bit registers with 1 bit per channel. For channels that do not exist the field will be treated as reserved.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-5 DRU_COMP_EVT_SET

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
CH63	CH62	CH61	CH60	CH59	CH58	CH57	CH56	CH55	CH54	CH53	CH52	CH51	CH50	CH49	CH48
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
CH47	CH46	CH45	CH44	CH43	CH42	CH41	CH40	CH39	CH38	CH37	CH36	CH35	CH34	CH33	CH32
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CH31	CH30	CH29	CH28	CH27	CH26	CH25	CH24	CH23	CH22	CH21	CH20	CH19	CH18	CH17	CH16
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8	CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-5 DRU_COMP_EVT_SET Field Descriptions

Bits	Name	Description
63	CH63	Set the completion event for channel 63
62	CH62	Set the completion event for channel 62
61	CH61	Set the completion event for channel 61
60	CH60	Set the completion event for channel 60
59	CH59	Set the completion event for channel 59
58	CH58	Set the completion event for channel 58
57	CH57	Set the completion event for channel 57
56	CH56	Set the completion event for channel 56
55	CH55	Set the completion event for channel 55
54	CH54	Set the completion event for channel 54
53	CH53	Set the completion event for channel 53
52	CH52	Set the completion event for channel 52
51	CH51	Set the completion event for channel 51
50	CH50	Set the completion event for channel 50
49	CH49	Set the completion event for channel 49
48	CH48	Set the completion event for channel 48
47	CH47	Set the completion event for channel 47
46	CH46	Set the completion event for channel 46
45	CH45	Set the completion event for channel 45

Table 5-5 DRU_COMP_EVT_SET Field Descriptions

Bits	Name	Description
44	CH44	Set the completion event for channel 44
43	CH43	Set the completion event for channel 43
42	CH42	Set the completion event for channel 42
41	CH41	Set the completion event for channel 41
40	CH40	Set the completion event for channel 40
39	CH39	Set the completion event for channel 39
38	CH38	Set the completion event for channel 38
37	CH37	Set the completion event for channel 37
36	CH36	Set the completion event for channel 36
35	CH35	Set the completion event for channel 35
34	CH34	Set the completion event for channel 34
33	CH33	Set the completion event for channel 33
32	CH32	Set the completion event for channel 32
31	CH31	Set the completion event for channel 31
30	CH30	Set the completion event for channel 30
29	CH29	Set the completion event for channel 29
28	CH28	Set the completion event for channel 28
27	CH27	Set the completion event for channel 27
26	CH26	Set the completion event for channel 26
25	CH25	Set the completion event for channel 25
24	CH24	Set the completion event for channel 24
23	CH23	Set the completion event for channel 23
22	CH22	Set the completion event for channel 22
21	CH21	Set the completion event for channel 21
20	CH20	Set the completion event for channel 20
19	CH19	Set the completion event for channel 19
18	CH18	Set the completion event for channel 18
17	CH17	Set the completion event for channel 17
16	CH16	Set the completion event for channel 16
15	CH15	Set the completion event for channel 15
14	CH14	Set the completion event for channel 14
13	CH13	Set the completion event for channel 13
12	CH12	Set the completion event for channel 12
11	CH11	Set the completion event for channel 11

Table 5-5 DRU_COMP_EVT_SET Field Descriptions

Bits	Name	Description
10	CH10	Set the completion event for channel 10
9	CH9	Set the completion event for channel 9
8	CH8	Set the completion event for channel 8
7	CH7	Set the completion event for channel 7
6	CH6	Set the completion event for channel 6
5	CH5	Set the completion event for channel 5
4	CH4	Set the completion event for channel 4
3	CH3	Set the completion event for channel 3
2	CH2	Set the completion event for channel 2
1	CH1	Set the completion event for channel 1
0	CH0	Set the completion event for channel 0

5.1.6 DRU_ERR_EVT_SET[Address = 0x0_4080]

Error Event Set Register for Channel 0-63 additional channels will follow the same in additional 64 bit registers with 1 bit per channel. For channels that do not exist the field will be treated as reserved.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-6 DRU_ERROR_EVT_SET

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
CH63	CH62	CH61	CH60	CH59	CH58	CH57	CH56	CH55	CH54	CH53	CH52	CH51	CH50	CH49	CH48
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
CH47	CH46	CH45	CH44	CH43	CH42	CH41	CH40	CH39	CH38	CH37	CH36	CH35	CH34	CH33	CH32
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CH31	CH30	CH29	CH28	CH27	CH26	CH25	CH24	CH23	CH22	CH21	CH20	CH19	CH18	CH17	CH16
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8	CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-6 DRU_ERROR_EVT_SET Field Descriptions

Bits	Name	Description
63	CH63	Set the error event for channel 63
62	CH62	Set the error event for channel 62
61	CH61	Set the error event for channel 61
60	CH60	Set the error event for channel 60
59	CH59	Set the error event for channel 59
58	CH58	Set the error event for channel 58
57	CH57	Set the error event for channel 57
56	CH56	Set the error event for channel 56
55	CH55	Set the error event for channel 55
54	CH54	Set the error event for channel 54
53	CH53	Set the error event for channel 53
52	CH52	Set the error event for channel 52
51	CH51	Set the error event for channel 51
50	CH50	Set the error event for channel 50
49	CH49	Set the error event for channel 49
48	CH48	Set the error event for channel 48
47	CH47	Set the error event for channel 47
46	CH46	Set the error event for channel 46
45	CH45	Set the error event for channel 45
44	CH44	Set the error event for channel 44
43	CH43	Set the error event for channel 43
42	CH42	Set the error event for channel 42
41	CH41	Set the error event for channel 41
40	CH40	Set the error event for channel 40
39	CH39	Set the error event for channel 39
38	CH38	Set the error event for channel 38
37	CH37	Set the error event for channel 37
36	CH36	Set the error event for channel 36
35	CH35	Set the error event for channel 35
34	CH34	Set the error event for channel 34
33	CH33	Set the error event for channel 33
32	CH32	Set the error event for channel 32
31	CH31	Set the error event for channel 31

Table 5-6 DRU_ERROR_EVT_SET Field Descriptions

Bits	Name	Description
30	CH30	Set the error event for channel 30
29	CH29	Set the error event for channel 29
28	CH28	Set the error event for channel 28
27	CH27	Set the error event for channel 27
26	CH26	Set the error event for channel 26
25	CH25	Set the error event for channel 25
24	CH24	Set the error event for channel 24
23	CH23	Set the error event for channel 23
22	CH22	Set the error event for channel 22
21	CH21	Set the error event for channel 21
20	CH20	Set the error event for channel 20
19	CH19	Set the error event for channel 19
18	CH18	Set the error event for channel 18
17	CH17	Set the error event for channel 17
16	CH16	Set the error event for channel 16
15	CH15	Set the error event for channel 15
14	CH14	Set the error event for channel 14
13	CH13	Set the error event for channel 13
12	CH12	Set the error event for channel 12
11	CH11	Set the error event for channel 11
10	CH10	Set the error event for channel 10
9	CH9	Set the error event for channel 9
8	CH8	Set the error event for channel 8
7	CH7	Set the error event for channel 7
6	CH6	Set the error event for channel 6
5	CH5	Set the error event for channel 5
4	CH4	Set the error event for channel 4
3	CH3	Set the error event for channel 3
2	CH2	Set the error event for channel 2
1	CH1	Set the error event for channel 1
0	CH0	Set the error event for channel 0

5.1.7 DRU_LOCAL_EVT_SET[Address = 0x0_40C0]

Local Output Event Set Register for Channel 0-63 additional channels will follow the same in additional 64 bit registers with 1 bit per channel. For channels that do not exist the field will be treated as reserved.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-7 DRU_LOCAL_EVT_SET

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
CH63	CH62	CH61	CH60	CH59	CH58	CH57	CH56	CH55	CH54	CH53	CH52	CH51	CH50	CH49	CH48
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
CH47	CH46	CH45	CH44	CH43	CH42	CH41	CH40	CH39	CH38	CH37	CH36	CH35	CH34	CH33	CH32
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CH31	CH30	CH29	CH28	CH27	CH26	CH25	CH24	CH23	CH22	CH21	CH20	CH19	CH18	CH17	CH16
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8	CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0
WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0	WS-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-7 DRU_LOCAL_EVT_SET Field Descriptions

Bits	Name	Description
63	CH63	Set the local event for channel 63
62	CH62	Set the local event for channel 62
61	CH61	Set the local event for channel 61
60	CH60	Set the local event for channel 60
59	CH59	Set the local event for channel 59
58	CH58	Set the local event for channel 58
57	CH57	Set the local event for channel 57
56	CH56	Set the local event for channel 56
55	CH55	Set the local event for channel 55
54	CH54	Set the local event for channel 54
53	CH53	Set the local event for channel 53
52	CH52	Set the local event for channel 52

Table 5-7 DRU_LOCAL_EVT_SET Field Descriptions

Bits	Name	Description
51	CH51	Set the local event for channel 51
50	CH50	Set the local event for channel 50
49	CH49	Set the local event for channel 49
48	CH48	Set the local event for channel 48
47	CH47	Set the local event for channel 47
46	CH46	Set the local event for channel 46
45	CH45	Set the local event for channel 45
44	CH44	Set the local event for channel 44
43	CH43	Set the local event for channel 43
42	CH42	Set the local event for channel 42
41	CH41	Set the local event for channel 41
40	CH40	Set the local event for channel 40
39	CH39	Set the local event for channel 39
38	CH38	Set the local event for channel 38
37	CH37	Set the local event for channel 37
36	CH36	Set the local event for channel 36
35	CH35	Set the local event for channel 35
34	CH34	Set the local event for channel 34
33	CH33	Set the local event for channel 33
32	CH32	Set the local event for channel 32
31	CH31	Set the local event for channel 31
30	CH30	Set the local event for channel 30
29	CH29	Set the local event for channel 29
28	CH28	Set the local event for channel 28
27	CH27	Set the local event for channel 27
26	CH26	Set the local event for channel 26
25	CH25	Set the local event for channel 25
24	CH24	Set the local event for channel 24
23	CH23	Set the local event for channel 23
22	CH22	Set the local event for channel 22
21	CH21	Set the local event for channel 21
20	CH20	Set the local event for channel 20
19	CH19	Set the local event for channel 19
18	CH18	Set the local event for channel 18

Table 5-7 DRU_LOCAL_EVT_SET Field Descriptions

Bits	Name	Description
17	CH17	Set the local event for channel 17
16	CH16	Set the local event for channel 16
15	CH15	Set the local event for channel 15
14	CH14	Set the local event for channel 14
13	CH13	Set the local event for channel 13
12	CH12	Set the local event for channel 12
11	CH11	Set the local event for channel 11
10	CH10	Set the local event for channel 10
9	CH9	Set the local event for channel 9
8	CH8	Set the local event for channel 8
7	CH7	Set the local event for channel 7
6	CH6	Set the local event for channel 6
5	CH5	Set the local event for channel 5
4	CH4	Set the local event for channel 4
3	CH3	Set the local event for channel 3
2	CH2	Set the local event for channel 2
1	CH1	Set the local event for channel 1
0	CH0	Set the local event for channel 0

5.1.8 QUEUE0_CFG [Address = 0x0_8000]

Configuration Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-8 QUEUE0_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT		CONSECUTIVE_TRANS			Reserved	QOS	ORDERID	Reserved	PRI				
R,W-0	R,W-0		R,W-0			R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0			

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-8 QUEUE0_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.

Table 5-8 QUEUE0_CFG Field Descriptions

Bits	Name	Description
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE0. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE0.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE0. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.9 QUEUE1_CFG [Address = 0x0_8008]

Configuration Register for Queue 1

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-9 QUEUE1_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUTIVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-9 QUEUE1_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE1. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE1.

Table 5-9 QUEUE1_CFG Field Descriptions

Bits	Name	Description
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE1. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.10 QUEUE2_CFG [Address = 0x0_8010]

Configuration Register for Queue 2

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-10 QUEUE2_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUITVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-10 QUEUE2_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUITVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE2. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE2.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE2. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.11 QUEUE3_CFG [Address = 0x0_8018]

Configuration Register for Queue 3

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-11 QUEUE3_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUTIVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-11 QUEUE3_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE3. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE3.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE3. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.12 QUEUE4_CFG [Address = 0x0_8020]

Configuration Register for Queue 4

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-12 QUEUE4_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUTIVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-12 QUEUE4_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.

Table 5-12 QUEUE4_CFG Field Descriptions

Bits	Name	Description
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE4. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE4.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE4. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.13 QUEUE5_CFG [Address = 0x0_8028]

Configuration Register for Queue 5

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-13 QUEUE5_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUTIVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-13 QUEUE5_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE5. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE5.

Table 5-13 QUEUE5_CFG Field Descriptions

Bits	Name	Description
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE5. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.14 QUEUE6_CFG [Address = 0x0_8030]

Configuration Register for Queue 6

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-14 QUEUE6_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT	CONSECUITVE_TRANS	Reserved	QOS	ORDERID	Reserved	PRI							
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-14 QUEUE6_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUITVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE6. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE6.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE6. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.15 QUEUE7_CFG [Address = 0x0_8038]

Configuration Register for Queue 7

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-15 QUEUE7_CFG

63	32	31	24	23	16	15	11	10	8	7	4	3	2	0
Reserved	REARB_WAIT		CONSECUTIVE_TRANS			Reserved	QOS	ORDERID	Reserved	PRI				
R,W-0	R,W-0		R,W-0			R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0			

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-15 QUEUE7_CFG Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved.
31-24	REARB_WAIT	This is the number of commands that will be sent by other queues before allowing the queue to arbitrate again for the right to send commands. This is only started when a queue exhausted its consecutive trans count.
23-16	CONSECUTIVE_TRANS	This is the number of consecutive transactions that will be sent before allowing another queue of equal level to arbitrate to send commands. This is the maximum number of commands that it can send. If the queue has any delays such as virtual address lookup then the arbitration will be forced regardless of the number of commands.
15-11	Reserved	Reserved.
10-8	QOS	This configures the QOS for QUEUE7. This should only be set for fixed priority queues and the lower queue should have the lower QoS.
7-4	ORDERID	This configures the orderid for QUEUE7.
3	Reserved	Reserved.
2-0	PRI	This configures the priority for QUEUE7. This will be the priority that will be presented on the External bus for all commands from this queue.

5.1.16 QUEUE0_STATUS [Address = 0x0_8040]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-16 QUEUE0_STATUS

63	36	35	27	26	18	17	9	8	0
Reserved				RD_TOTAL	RD_TOP	WR_TOTAL	WR_TOP		
R-0				R-0	R-0	R-0	R-0		

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-16 QUEUE0_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved

Table 5-16 QUEUE0_STATUS Field Descriptions

Bits	Name	Description
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

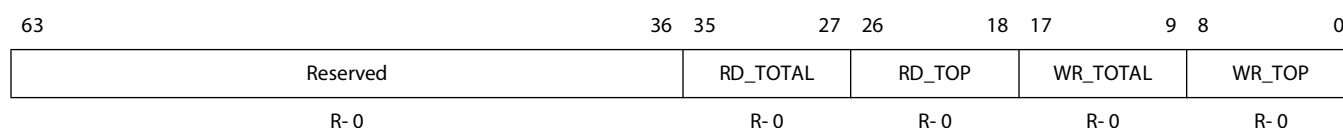
5.1.17 QUEUE1_STATUS [Address = 0x0_8048]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-17 QUEUE1_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-17 QUEUE1_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

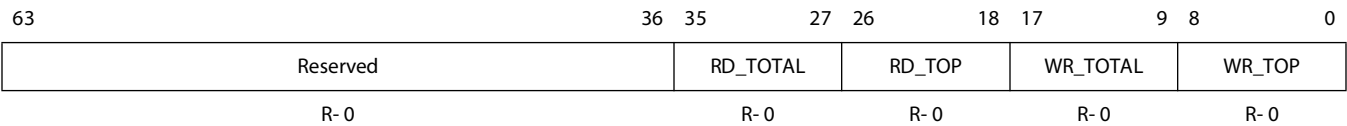
5.1.18 QUEUE2_STATUS [Address = 0x0_8050]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-18 QUEUE2_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-18 QUEUE2_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

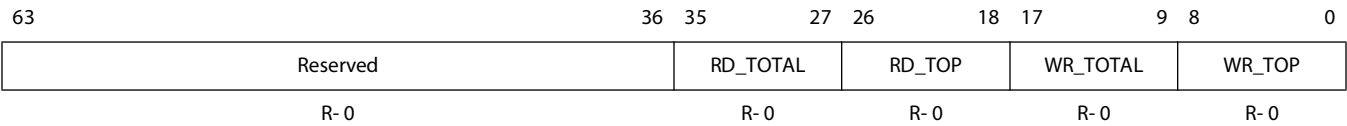
5.1.19 QUEUE3_STATUS [Address = 0x0_8058]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-19 QUEUE3_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-19 QUEUE3_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

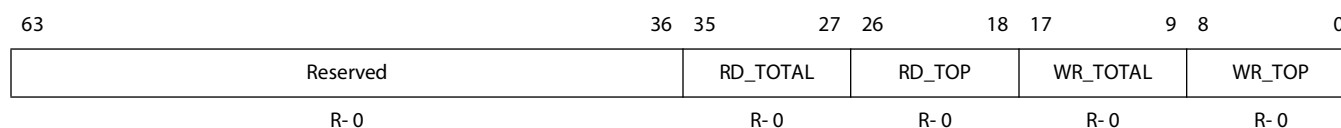
5.1.20 QUEUE4_STATUS [Address = 0x0_8060]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-20 QUEUE4_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-20 QUEUE4_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

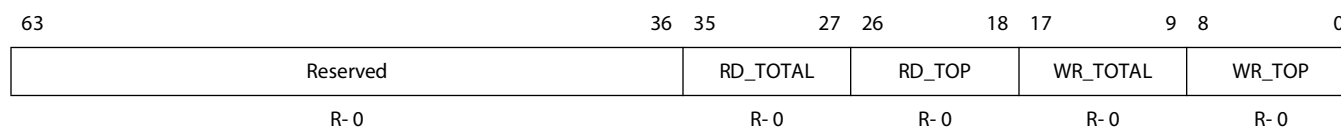
5.1.21 QUEUE5_STATUS [Address = 0x0_8068]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-21 QUEUE5_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-21 QUEUE5_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.

Table 5-21 QUEUE5_STATUS Field Descriptions

Bits	Name	Description
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

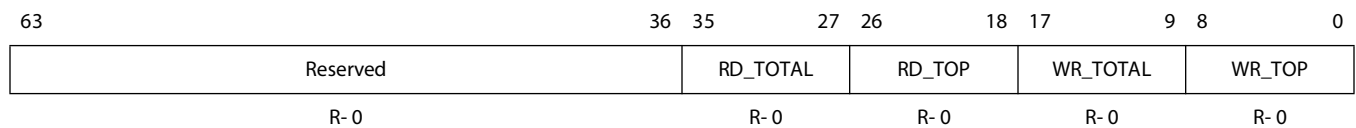
5.1.22 QUEUE6_STATUS [Address = 0x0_8070]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-22 QUEUE6_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-22 QUEUE6_STATUS Field Descriptions

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

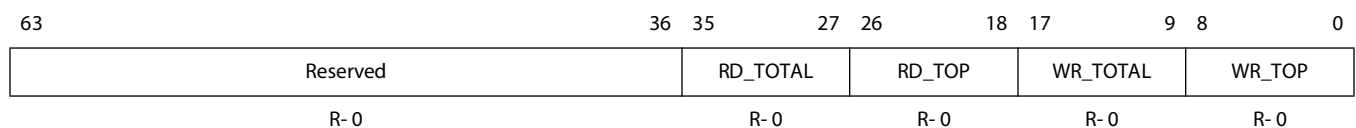
5.1.23 QUEUE7_STATUS [Address = 0x0_8078]

Status Register for Queue 0

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-23 QUEUE7_STATUS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-23 **QUEUE7_STATUS Field Descriptions**

Bits	Name	Description
63-36	Reserved	Reserved
35-27	RD_TOTAL	This is the channel that the read half is currently working on.
26-18	RD_TOP	This is the channel that the read half is currently working on.
17-9	WR_TOTAL	This is the channel that the read half is currently working on.
8-0	WR_TOP	This is the channel that the write half is currently working on.

5.1.24 MMU_PID [Address = 0xA000]

Physical ID Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	R	-	R	-	R

Figure 5-24 MMU_PID

63	32	31	30	29	28	27	16	15	11	10	8	7	6	5	0
Reserved	SCHEME		BU		FUNC		R		X		CUSTOM		Y		
R- 0	R- 0x1		R- 0x2		R- 0x0F0		R- 0		R- 0		R- 0		R- 0		

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-1 MMU_PID Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved
31-30	SCHEME	PID naming scheme
29-28	BU	Business Unit
27-16	FUNC	MMU Function code
15-11	R	Minor Revision
10-8	X	Architecture Revision
7-6	CUSTOM	Reuseable/Custom
5-0	Y	Configuration Revision

5.1.25 TDRR [Address = 0xA100]

Translation Debug Response Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-25 TDRR

63	62	57	56	55	54	53	52	51	50	49	48	47	40	39	12	11	0
COMP	Reserved	PREF	OUTER	INNER	MEMTYPE	SHARE	Reserved	ADDR	STATUS								
R,W- 0	R- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R- 0	R,W- 0	R,W- 0								

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-2 TDRR Field Descriptions

Bits	Name	Description
63	COMP	Translation Completion Bit
62-57	Reserved	Reserved
56	PREF	Prefetchable
55-54	OUTER	Outer Cacheability
53-52	INNER	Inner Cacheability
51-50	MEMTYPE	Memory Type
49-48	SHARE	Shareability
47-40	Reserved	Reserved
39-12	ADDR	Output Address, Faulting Stage-2 IPA
11-0	STATUS	Translation Response Status

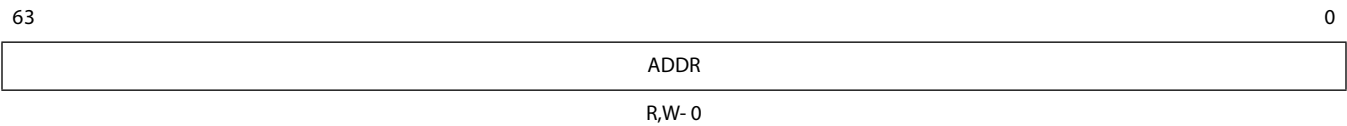
5.1.26 TDFAR [Address = 0xA140]

Translation Debug Fault Address Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-26 TDFAR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-3 TDFAR Field Descriptions

Bits	Name	Description
63-0	ADDR	Faulted Input Address

5.1.27 TLB_INV [Address = 0xA200]

TLB Invalidation Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	W	-	W	-	W

Figure 5-27 TLB_INV

63	56	55	48	47	43	42	40	39	38	37	36	0
Reserved	ID	Reserved	INV_TYPE	ASID	VA	LL	ADDR					
R-0	W-0	R-0	W-0	W-0	W-0	W-0	W-0					

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-4 TLB_INV Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ID	ASID Value
47-43	Reserved	Reserved
42-40	INV_TYPE	Invalidation Type
39	ASID	ASID Match
38	VA	VA Match
37	LL	Last Level Only
36-0	ADDR	VA/IPA

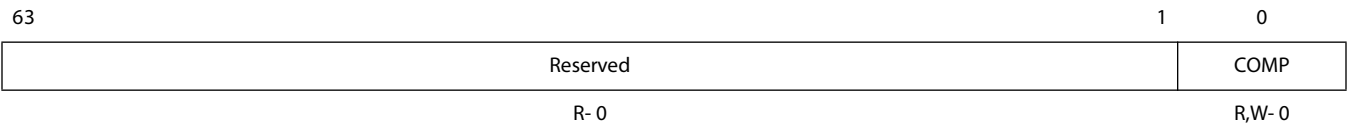
5.1.28 TLB_INV [Address = 0xA280]

TLB Invalidation Complete Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-28 TLB_INV



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-5 TLB_INV Field Descriptions

Bits	Name	Description
63-1	Reserved	Reserved
0	COMP	Invalidation Completed

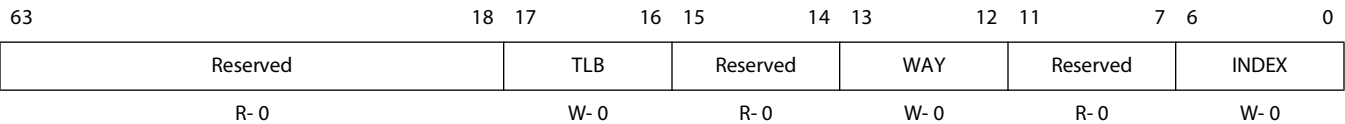
5.1.29 TLB_DBG [Address = 0xA2C0]

TLB Debug Access Register

Access Permissions

Non-Secure Debug	Secure Debug
W	W

Figure 5-29 TLB_DBG



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-6 TLB_DBG Field Descriptions

Bits	Name	Description
63-18	Reserved	Reserved
17-16	TLB	TLB Type
15-14	Reserved	Reserved
13-12	WAY	Way
11-7	Reserved	Reserved
6-0	INDEX	Index

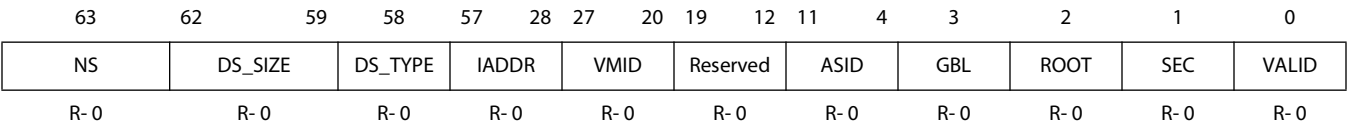
5.1.30 TLB_DBG_DATA0 [Address = 0xA300]

TLB Debug Data 0 Register

Access Permissions

Non-Secure Debug	Secure Debug
R	R

Figure 5-30 TLB_DBG_DATA0



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-7 TLB_DBG_DATA0 Field Descriptions

Bits	Name	Description
63	NS	Non-Secure Page, NSTable accumulation
62-59	DS_SIZE	Descriptor Size
58	DS_TYPE	Descriptor Type
57-28	IADDR	Input Address
27-20	VMID	VMID
19-12	Reserved	Reserved
11-4	ASID	ASID
3	GBL	Global Page
2	ROOT	Root Context
1	SEC	Security Context
0	VALID	Valid Entry

5.1.31 TLB_DBG_DATA1 [Address = 0xA340]

TLB Debug Data 1 Register

Access Permissions

Non-Secure Debug	Secure Debug
R	R

Figure 5-31 TLB_DBG_DATA1

63	48	47	46	45	44	43	40	39	36	35	33	32	28	27	0
Reserved	SHARE	S2_LVL	S2_MEM_TYPE	S2_PERM	S1_MEM_INDEX	S1_PERM	OADDR								
R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0								

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-8 TLB_DBG_DATA1 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-46	SHARE	Shareability
45-44	S2_LVL	Stage 2 Level
43-40	S2_MEM_TYPE	Stage 2 Memory Type
39-36	S2_PERM	S2 Access Permissions
35-33	S1_MEM_INDEX	Stage 1 Memory Attribute Index
32-28	S1_PERM	Stage 1 Access Permissions
27-0	OADDR	Output Address

5.1.32 SCR [Address = 0xA400]

System Control Register

Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-32 SCR

63	62	8	7	6	5	4	3	2	1	0
HW	Reserved	INSTC	DATAAC	FAULT	ASEL	WXN	ENDIAN1	ENDIAN0	S1_EN	
R,W- 0	R- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-9 SCR Field Descriptions

Bits	Name	Description
63	HW	HW Mode
62-8	Reserved	Reserved
7	INSTC	Stage 1 Instruction Caching
6	DATAAC	Stage 1 Data Caching
5	FAULT	Caching faults in uTLBs
4	ASEL	ASID Selection
3	WXN	Writeable memory Execute Never
2	ENDIAN1	Stage 1 Region 1 Endian
1	ENDIAN0	Stage 1 Region 0 Endian
0	S1_EN	Enable Stage 1 Address Translation

5.1.33 TCR0 [Address = 0xA440]

Table Control Register 0

Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-33 TCR0

63	17	16	15	14	13	12	11	10	9	8	7	6	1	0
Reserved	MEMTYPE	COUTER	CINNER	SHARE	GRANULE	TBL_SZ	WALK_EN							
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-10 TCR0 Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

5.1.34 TCR1 [Address = 0xA480]

Table Control Register 1

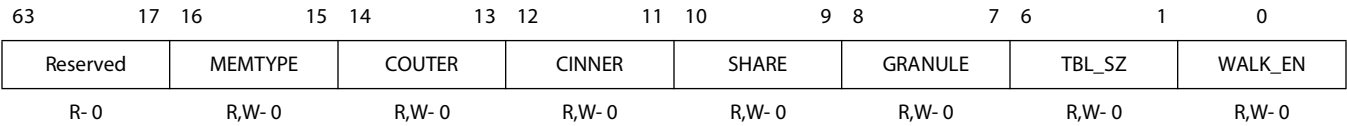
Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-34 TCR1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-11 TCR1 Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

5.1.35 TBR0 [Address = 0xA4C0]

Table Base Register 0

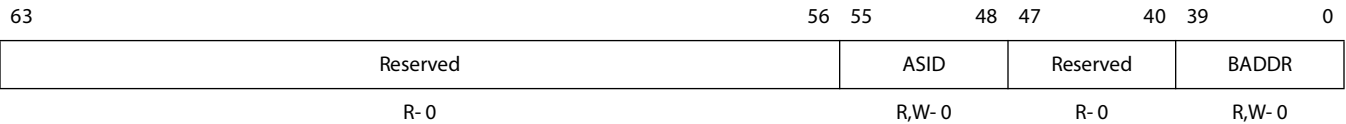
Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-35 TBR0



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-12 TBR0 Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

5.1.36 TBR1 [Address = 0xA500]

Table Base Register 1

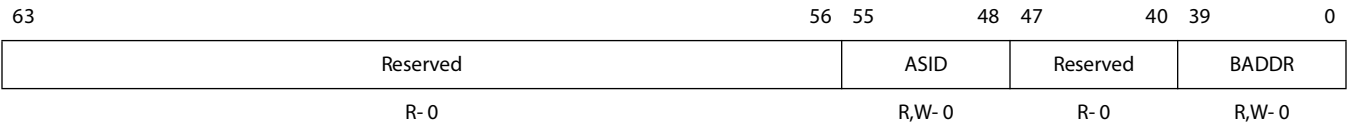
Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-36 TBR1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-13 TBR1 Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

5.1.37 MAR [Address = 0xA540]

Memory Attribute Register

Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Figure 5-37 MAR

63	56	55	48	47	40	39	32	31	24	23	16	15	8	7	0
ATTR7	ATTR6	ATTR5	ATTR4	ATTR3	ATTR2	ATTR1	ATTR0								
R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0								

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-14 MAR Field Descriptions

Bits	Name	Description
63-56	ATTR7	Memory Attribute Index 7
55-48	ATTR6	Memory Attribute Index 6
47-40	ATTR5	Memory Attribute Index 5
39-32	ATTR4	Memory Attribute Index 4
31-24	ATTR3	Memory Attribute Index 3
23-16	ATTR2	Memory Attribute Index 2
15-8	ATTR1	Memory Attribute Index 1
7-0	ATTR0	Memory Attribute Index 0

5.1.38 TDAR [Address = 0xA580]

Translation Debug Address Register

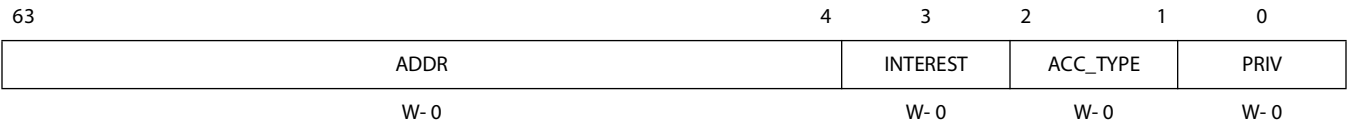
Banked

Guest Supervisor	Supervisor	Secure Supervisor
------------------	------------	-------------------

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	W	-	W	-	W

Figure 5-38 TDAR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-15 TDAR Field Descriptions

Bits	Name	Description
63-4	ADDR	Input Address
3	INTEREST	Interest Flag
2-1	ACC_TYPE	Access Type
0	PRIV	Supervisor Access

5.1.39 SCR_GS [Address = 0xA800]

System Control Register - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-39 SCR_GS

63	62	8	7	6	5	4	3	2	1	0
HW	Reserved	INSTC	DATAAC	FAULT	ASEL	WXN	ENDIAN1	ENDIAN0	S1_EN	
R,W- 0	R- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-16 SCR_GS Field Descriptions

Bits	Name	Description
63	HW	HW Mode
62-8	Reserved	Reserved
7	INSTC	Stage 1 Instruction Caching
6	DATAAC	Stage 1 Data Caching
5	FAULT	Caching faults in uTLBs
4	ASEL	ASID Selection
3	WXN	Writeable memory Execute Never
2	ENDIAN1	Stage 1 Region 1 Endian
1	ENDIAN0	Stage 1 Region 0 Endian
0	S1_EN	Enable Stage 1 Address Translation

5.1.40 TCR0_GS [Address = 0xA840]

Table Control Register 0 - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-40 TCR0_GS

63	17	16	15	14	13	12	11	10	9	8	7	6	1	0
Reserved	MEMTYPE	COUTER	CINNER	SHARE	GRANULE	TBL_SZ	WALK_EN							
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-17 TCR0_GS Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

5.1.41 TCR1_GS [Address = 0xA880]

Table Control Register 1 - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-41 TCR1_GS

63	17	16	15	14	13	12	11	10	9	8	7	6	1	0
Reserved	MEMTYPE	COUTER	CINNER	SHARE	GRANULE	TBL_SZ	WALK_EN							
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-18 TCR1_GS Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

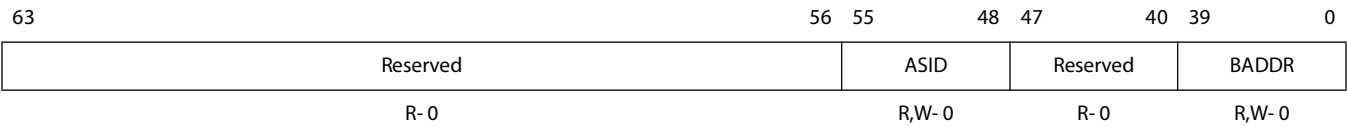
5.1.42 TBR0_GS [Address = 0xA8C0]

Table Base Register 0 - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-42 TBR0_GS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-19 TBR0_GS Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

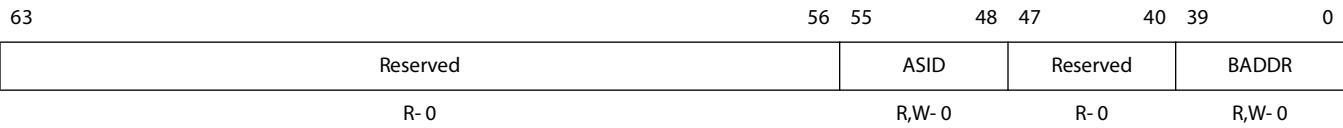
5.1.43 TBR1_GS [Address = 0xA900]

Table Base Register 1 - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-43 TBR1_GS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-20 TBR1_GS Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

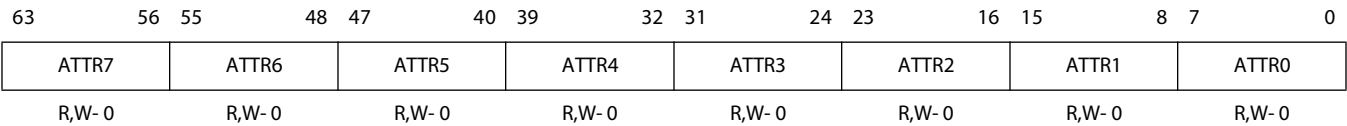
5.1.44 MAR_GS [Address = 0xA940]

Memory Attribute Register - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-44 MAR_GS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-21 MAR_GS Field Descriptions

Bits	Name	Description
63-56	ATTR7	Memory Attribute Index 7
55-48	ATTR6	Memory Attribute Index 6
47-40	ATTR5	Memory Attribute Index 5
39-32	ATTR4	Memory Attribute Index 4
31-24	ATTR3	Memory Attribute Index 3
23-16	ATTR2	Memory Attribute Index 2
15-8	ATTR1	Memory Attribute Index 1
7-0	ATTR0	Memory Attribute Index 0

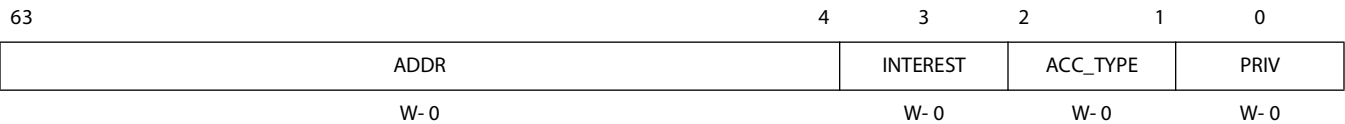
5.1.45 TDAR_GS [Address = 0xA980]

Translation Debug Address Register - Non-Secure Guest Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	W	-	W

Figure 5-45 TDAR_GS



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-22 TDAR_GS Field Descriptions

Bits	Name	Description
63-4	ADDR	Input Address
3	INTEREST	Interest Flag
2-1	ACC_TYPE	Access Type
0	PRIV	Supervisor Access

5.1.46 SCR_S [Address = 0xAC00]

System Control Register - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-46 SCR_S

63	62	8	7	6	5	4	3	2	1	0
HW	Reserved	INSTC	DATAAC	FAULT	ASEL	WXN	ENDIAN1	ENDIAN0	S1_EN	
R,W- 0	R- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-23 SCR_S Field Descriptions

Bits	Name	Description
63	HW	HW Mode
62-8	Reserved	Reserved
7	INSTC	Stage 1 Instruction Caching
6	DATAAC	Stage 1 Data Caching
5	FAULT	Caching faults in uTLBs
4	ASEL	ASID Selection
3	WXN	Writeable memory Execute Never
2	ENDIAN1	Stage 1 Region 1 Endian
1	ENDIAN0	Stage 1 Region 0 Endian
0	S1_EN	Enable Stage 1 Address Translation

5.1.47 TCR0_S [Address = 0xAC40]

Table Control Register 0 - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-47 TCR0_S

63	17	16	15	14	13	12	11	10	9	8	7	6	1	0
Reserved	MEMTYPE	COUTER	CINNER	SHARE	GRANULE	TBL_SZ	WALK_EN							
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-24 TCR0_S Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

5.1.48 TCR1_S [Address = 0xAC80]

Table Control Register 1 - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-48 TCR1_S

63	17	16	15	14	13	12	11	10	9	8	7	6	1	0
Reserved	MEMTYPE	COUTER	CINNER	SHARE	GRANULE	TBL_SZ	WALK_EN							
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0							

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-25 TCR1_S Field Descriptions

Bits	Name	Description
63-17	Reserved	Reserved
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	WALK_EN	Enable Translation Table Walks

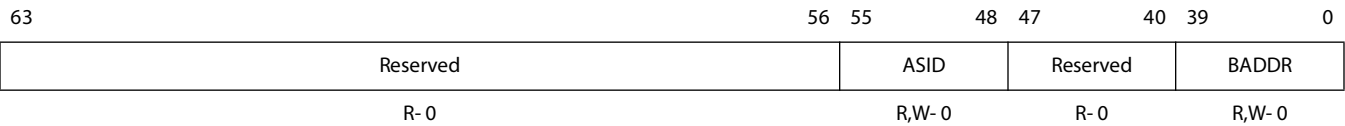
5.1.49 TBR0_S [Address = 0xACC0]

Table Base Register 0 - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-49 TBR0_S



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-26 TBR0_S Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

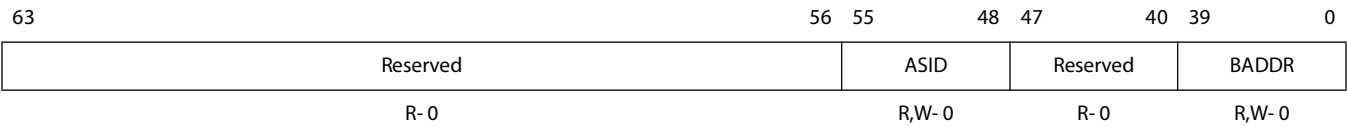
5.1.50 TBR1_S [Address = 0xAD00]

Table Base Register 1 - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-50 TBR1_S



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-27 TBR1_S Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	ASID	Application Space ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

5.1.51 MAR_S [Address = 0xAD40]

Memory Attribute Register - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	RW

Figure 5-51 MAR_S

63	56	55	48	47	40	39	32	31	24	23	16	15	8	7	0
ATTR7		ATTR6		ATTR5		ATTR4		ATTR3		ATTR2		ATTR1		ATTR0	
R,W- 0		R,W- 0		R,W- 0		R,W- 0		R,W- 0		R,W- 0		R,W- 0		R,W- 0	

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-28 MAR_S Field Descriptions

Bits	Name	Description
63-56	ATTR7	Memory Attribute Index 7
55-48	ATTR6	Memory Attribute Index 6
47-40	ATTR5	Memory Attribute Index 5
39-32	ATTR4	Memory Attribute Index 4
31-24	ATTR3	Memory Attribute Index 3
23-16	ATTR2	Memory Attribute Index 2
15-8	ATTR1	Memory Attribute Index 1
7-0	ATTR0	Memory Attribute Index 0

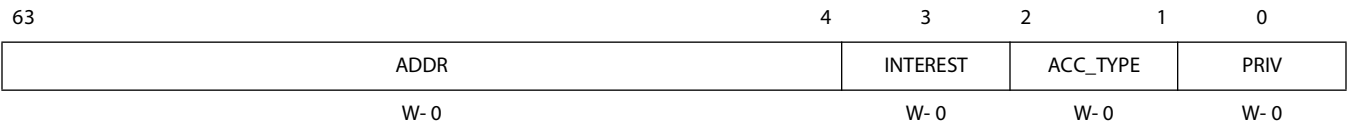
5.1.52 TDAR_S [Address = 0xAD80]

Translation Debug Address Register - Non-Secure Supervisor

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	-	-	W

Figure 5-52 TDAR_S



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-29 TDAR_S Field Descriptions

Bits	Name	Description
63-4	ADDR	Input Address
3	INTEREST	Interest Flag
2-1	ACC_TYPE	Access Type
0	PRIV	Supervisor Access

5.1.53 VCR [Address = 0xB800]

Virtualization Control Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-53 VCR

63	6	5	4	3	2	1	0
Reserved	ID	CD	DC	PROT	ENDIAN	S2_EN	
R-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-30 VCR Field Descriptions

Bits	Name	Description
63-6	Reserved	Reserved
5	ID	Stage 2 Instruction Cache Disable
4	CD	Stage 2 Data Cache Disable
3	DC	Default Cacheable Mode
2	PROT	Protected Table Walk
1	ENDIAN	Stage 2 Endian
0	S2_EN	Enable Stage 2 Address Translation

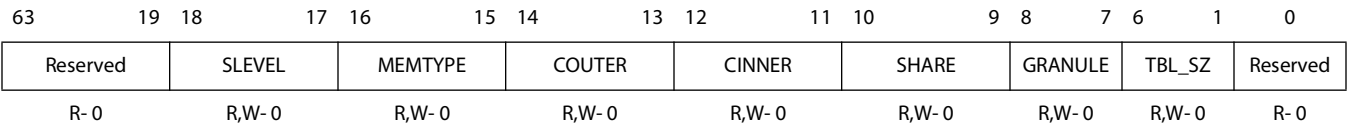
5.1.54 VTCR [Address = 0xB840]

Virtualization Table Control Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-54 VTCR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-31 VTCR Field Descriptions

Bits	Name	Description
63-19	Reserved	Reserved
18-17	SLEVEL	Stage 2 Starting Translation Level
16-15	MEMTYPE	Memory Type of Translation Table Walks
14-13	COUTER	Outer Cacheability attribute of memory for Translation Table Walks
12-11	CINNER	Inner Cacheability attribute of memory for Translation Table Walks
10-9	SHARE	Shareability attribute of memory for Translation Table Walks
8-7	GRANULE	Translation Table Page Granule Size
6-1	TBL_SZ	Translation Table Region Size
0	Reserved	Reserved

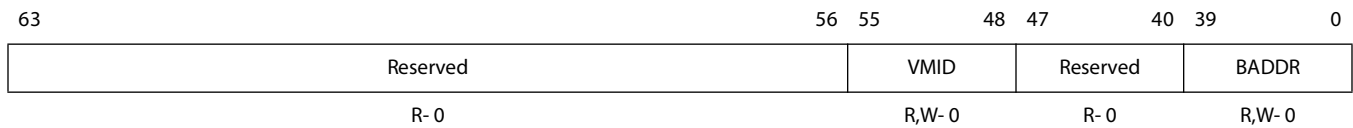
5.1.55 VTBR [Address = 0xB880]

Virtualization Table Base Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	RW	-	RW

Figure 5-55 VTBR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-32 VTBR Field Descriptions

Bits	Name	Description
63-56	Reserved	Reserved
55-48	VMID	Virtual Machine ID
47-40	Reserved	Reserved
39-0	BADDR	Table Base Address

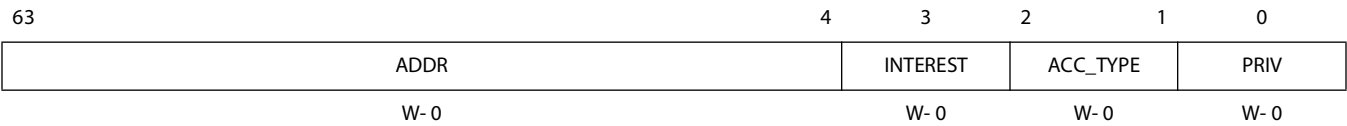
5.1.56 VTDAR [Address = 0xB8C0]

Virtualization Translation Debug Address Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	-	-	W	-	W

Figure 5-56 VTDAR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-33 VTDAR Field Descriptions

Bits	Name	Description
63-4	ADDR	Input Address
3	INTEREST	Interest Flag
2-1	ACC_TYPE	Access Type
0	PRIV	Supervisor Access

5.1.57 RD_UTLB_MATCH [Address = 0xC000]

Indexed uTLB Page Match Entry

Access Permissions in SW Mode

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Access Permissions in HW Mode

Non-Secure Debug	Secure Debug
R	R

Figure 5-57 RD_UTLB_MATCH

63	62	61	60	59	52	51	44	43	42	40	39	37	36	0
VALID	SECURE	ROOT	GBL	VMID	ASID	Reserved	PG_SIZE	Reserved	IADDR					
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R- 0	R,W- 0	R- 0	R,W- 0					

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-34 RD_UTLB_MATCH Field Descriptions

Bits	Name	Description
63	VALID	Entry Valid
62	SECURE	Secure Page
61	ROOT	Root Page
60	GBL	Global Page
59-52	VMID	VMID
51-44	ASID	ASID
43	Reserved	Reserved
42-40	PG_SIZE	Page Size
39-37	Reserved	Reserved
36-0	IADDR	Input Address

5.1.58 RD_UTLB_ATTR [Address = 0xD000]

Indexed uTLB Page Attribute Entry

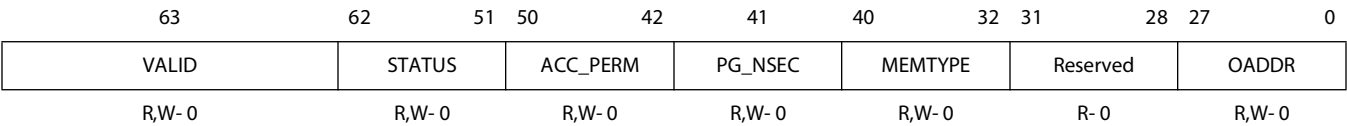
Access Permissions in SW Mode

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Access Permissions in HW Mode

Non-Secure Debug	Secure Debug
R	R

Figure 5-58 RD_UTLB_ATTR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-35 SEO_UTLB_ATTR Field Descriptions

Bits	Name	Description
63	VALID	Entry Valid
62-51	STATUS	TLB Status
50-42	ACC_PERM	Access Permissions
41	PG_NSEC	Page Non-Secure
40-32	MEMTYPE	Page Memory Type
31-28	Reserved	Reserved
27-0	OADDR	Output Address

5.1.59 WR_UTLB_MATCH [Address = 0xE000]

Indexed uTLB Page Match Entry

Access Permissions in SW Mode

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Access Permissions in HW Mode

Non-Secure Debug	Secure Debug
R	R

Figure 5-59 WR_UTLB_MATCH

63	62	61	60	59	52	51	44	43	42	40	39	37	36	0
VALID	SECURE	ROOT	GBL	VMID	ASID	Reserved	PG_SIZE	Reserved	IADDR					
R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R,W- 0	R- 0	R,W- 0	R- 0	R,W- 0					

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-36 WR_UTLB_MATCH Field Descriptions

Bits	Name	Description
63	VALID	Entry Valid
62	SECURE	Secure Page
61	ROOT	Root Page
60	GBL	Global Page
59-52	VMID	VMID
51-44	ASID	ASID
43	Reserved	Reserved
42-40	PG_SIZE	Page Size
39-37	Reserved	Reserved
36-0	IADDR	Input Address

5.1.60 WR_UTLB_ATTR [Address = 0xF000]

Indexed uTLB Page Attribute Entry

Access Permissions in SW Mode

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
-	RW	-	RW	-	RW

Access Permissions in HW Mode

Non-Secure Debug	Secure Debug
R	R

Figure 5-60 WR_UTLB_ATTR

63	62	51	50	42	41	40	32	31	28	27	0
VALID	STATUS	ACC_PERM	PG_NSEC	MEMTYPE	Reserved	OADDR					
R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R,W-0	R-0					R,W-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-37 WR_UTLB_ATTR Field Descriptions

Bits	Name	Description
63	VALID	Entry Valid
62-51	STATUS	TLB Status
50-42	ACC_PERM	Access Permissions
41	PG_NSEC	Page Non-Secure
40-32	MEMTYPE	Page Memory Type
31-28	Reserved	Reserved
27-0	OADDR	Output Address

5.1.61 MSMC SRAM MEMORY ATTRIBUTES [Address = 0x1_0000]

The MSMC SRAM Memory Attributes is a table that has up to 8192. Each 64 bit register contains two entries for the table and each entry represents the attributes for a 64kB page. The register can only be written prior to any TR submissions. Once TRs have been submitted the register will return write errors for any intended write.

So 0x10000 has the entry for 0 and 1 and 0x10008 has the entry for 2 and 3.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-61 MSMC SRAM Memory Attributes

63	40	39	38	37	36	35	34	33	32	31	8	7	6	5	4	3	2	1	0
Reserved	Entry Hi Memtype	Entry Hi Sdomain	Entry Hi Outer	Entry Hi Inner	Reserved	Entry Lo Memtype	Entry Lo Sdomain	Entry Lo Outer	Entry Lo Inner										
R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0	R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0										

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-24 MSMC SRAM Memory Field Descriptions

Bits	Name	Description
63-40	Reserved	RESERVED
39-38	ENTRY_HI_MEMTYPE	This sets the memtype for transactions that fall in this region.
37-36	ENTRY_HI_SDOMAIN	This sets the sdomain for transactions that fall in this region.
35-34	ENTRY_HI_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
33-32	ENTRY_HI_INNER	This sets the inner cacheability attribute for transactions that fail in this region
31-8	Reserved	Reserved
7-6	ENTRY_LO_MEMTYPE	This sets the memtype for transactions that fall in this region.
5-4	ENTRY_LO_SDOMAIN	This sets the sdomain for transactions that fall in this region.
3-2	ENTRY_LO_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
1-0	ENTRY_LO_INNER	This sets the inner cacheability attribute for transactions that fail in this region

5.1.62 DDR MEMORY ATTRIBUTES [Address = 0x2_0000]

The MSMC SRAM Memory Attributes is a table that has up to 8192. Each 64 bit register contains two entries for the table and each entry represents the attributes for a 16 MB page. The register can only be written prior to any TR submissions. Once TRs have been submitted the register will return write errors for any intended write.

So 0x20000 has the entry for 0 and 1 and 0x20008 has the entry for 2 and 3.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-62 DDR Memory Attributes

63	40	39	38	37	36	35	34	33	32	31	8	7	6	5	4	3	2	1	0
Reserved	Entry Hi Memtype	Entry Hi Sdomain	Entry Hi Outer	Entry Hi Inner	Reserved	Entry Lo Memtype	Entry Lo Sdomain	Entry Lo Outer	Entry Lo Inner										
R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0	R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0										

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-25 DDR Memory Field Descriptions

Bits	Name	Description
63-40	Reserved	RESERVED
39-38	ENTRY_HI_MEMTYPE	This sets the memtype for transactions that fall in this region.
37-36	ENTRY_HI_SDOMAIN	This sets the sdomain for transactions that fall in this region.
35-34	ENTRY_HI_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
33-32	ENTRY_HI_INNER	This sets the inner cacheability attribute for transactions that fail in this region
31-8	Reserved	Reserved
7-6	ENTRY_LO_MEMTYPE	This sets the memtype for transactions that fall in this region.
5-4	ENTRY_LO_SDOMAIN	This sets the sdomain for transactions that fall in this region.
3-2	ENTRY_LO_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
1-0	ENTRY_LO_INNER	This sets the inner cacheability attribute for transactions that fail in this region

5.1.63 DDR BOOT MEMORY ATTRIBUTES [Address = 0x3_0000]

The MSMC SRAM Memory Attributes is a table that has up to 128. Each 64 bit register contains two entries for the table and each entry represents the attributes for a 16MB page. The register can only be written prior to any TR submissions. Once TRs have been submitted the register will return write errors for any intended write.

So 0x30000 has the entry for 0 and 1 and 0x30008 has the entry for 2 and 3.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-63 DDR Boot Memory Attributes

63	40	39	38	37	36	35	34	33	32	31	8	7	6	5	4	3	2	1	0
Reserved	Entry Hi Memtype	Entry Hi Sdomain	Entry Hi Outer	Entry Hi Inner	Reserved	Entry Lo Memtype	Entry Lo Sdomain	Entry Lo Outer	Entry Lo Inner										
R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0	R- 0	R,W- 1	R,W- 1	R,W- 3	R,W- 0										

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-26 DDR Boot Memory Field Descriptions

Bits	Name	Description
63-40	Reserved	RESERVED
39-38	ENTRY_HI_MEMTYPE	This sets the memtype for transactions that fall in this region.
37-36	ENTRY_HI_SDOMAIN	This sets the sdomain for transactions that fall in this region.
35-34	ENTRY_HI_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
33-32	ENTRY_HI_INNER	This sets the inner cacheability attribute for transactions that fail in this region

Table 5-26 DDR Boot Memory Field Descriptions

Bits	Name	Description
31-8	Reserved	Reserved
7-6	ENTRY_LO_MEMTYPE	This sets the memtype for transactions that fall in this region.
5-4	ENTRY_LO_SDOMAIN	This sets the sdomain for transactions that fall in this region.
3-2	ENTRY_LO_OUTER	This sets the outer cacheability attribute for transactions that fall in this region
1-0	ENTRY_LO_INNER	This sets the inner cacheability attribute for transactions that fall in this region

5.1.64 CHCFG [Address = 0x4_0000]

Channel Configuration Register. The Channel Configuration Register is used to initialize static mode settings for the DMA Channel. The register may only be written when the channel is disable in the ch_enable field in the REALTIME CONTROL register is 0.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-64 CHCFG

63	32	31	30	26	25	26	20	19	18	16	15	0
Reserved	PAUSE_ON_ERR	Reserved	ATYPE	Reserved	CHAN_TYPE_OWNER	CHAN_TYPE	Reserved					
R-0	R,W-0	R-0	R,W-0	R-0	R,W-0	R-0	R-0					

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-27 CHCFG Field Descriptions

Bits	Name	Description
63-32	Reserved	RESERVED
31	PAUSE_ON_ERR	Pause on Error. This field controls what the channel will do if an error or exception occurs during a data transfer. This field is encoded as follows: 0 = Channel will drop current work and move on 1 = Channel will pause and wait for SW to investigate and un-pause the channel.
30-26	Reserved	Reserved
25	ATYPE	Address type. This field controls how the pointers are interpreted for TRs on this channel. This field is encoded as follows: 0 = Pointers are physical addresses 1 = Pointers are virtual addresses which require virtual to physical translation before they can be decoded. If the DRU configuration does not support Virtual TRs this field is Resved
24-20	Reserved	Reserved

Table 5-27 CHCFG Field Descriptions

Bits	Name	Description
19	CHAN_TYPE_OWNER	This field controls how the TR is received by the UTC. If it is 0 then the SUBMISSION registers must be written to submit it. If it is a 1 then the TR will be received through PSIL or through ring submission. A DRU can either support TRs through PSIL or Rings but not both. So a value of 1 will signify using the non-direct method of TR submission.
18-16	CHAN_TYPE	This field states the TR type that is being used it along with CHAN_TYPE_OWNER field make up the 4 bit CHAN_TYPE for a KS3 DMA UTC. The value of this is all zeroes. To reflect that the UTC DRU only does TRs through pass by value mechanisms.
15-0	Reserved	Reserved

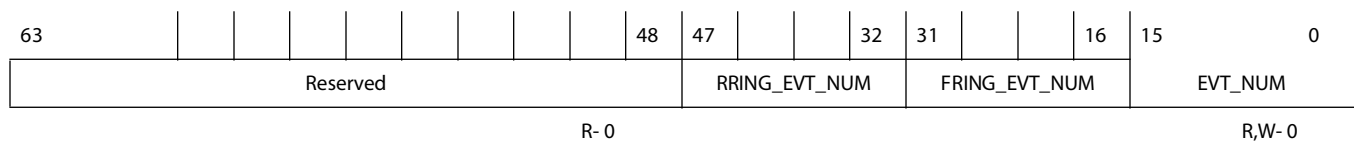
5.1.65 CHOES0 [Address = 0x4_0020]

The Output Event Steering Registers are used to specify a global event number to generate anytime the required event generation criteria specified in a TR are met. A single event with the event number set equal to the value in the corresponding register will be generated. This register is provided in order to allow security SW to lock down which events in the global space any given channel/ thread is allowed to generate

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-65 CHOES0



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-28 CHOES0 Field Descriptions

Bits	Name	Description
63-48	Reserved	RESERVED
47-32	RRING_EVT_NUM	This is the event that is sent for a reverse ring and event up will be sent when the reverse ring occupancy changes from 0 to non-zero. An event down will be sent when the reverse occupancy goes from non-zero to zero. This is only present if ring mode is supported. A value of 0xFFFF is a null event and will result in no EVENT being sent
31-16	FRING_EVT_NUM	This is the event that is sent for a forward ring and event up will be sent when the reverse ring occupancy changes from 0 to non-zero. An event down will be sent when the reverse occupancy goes from non-zero to zero. This is only present if ring mode is supported. A value of 0xFFFF is a null event and will result in no EVENT being sent
15-0	EVT_NUM	This is the global event number to be generated. A value of 0xFFFF is a null event and will result in no EVENT being sent

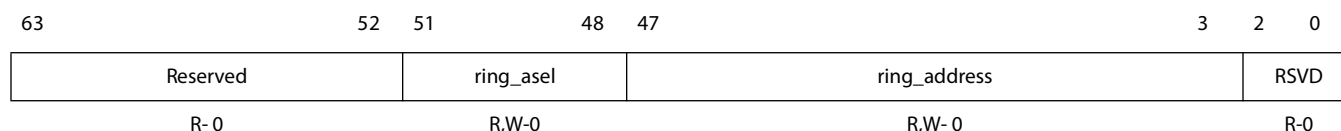
5.1.66 CHRING_ADDR [Address = 0x4_0040]

The Channel Ring Address Registers are used to specify the address of the top of the ring if the channel is configured for ring mode.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-66 CHRING_ADDR



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-29 CHRING_ADDR Field Descriptions

Bits	Name	Description
63-52	Reserved	RESERVED
51-48	ring_asel	Address space select for ring base. Note- This is only used for VBUSM interfaces and not if it is in VBUSM.C
47-3	ring_address	Ring base address. The width of this field may be less than 48 bits if the system address space is configured to be less than 48 bits.
2-0	Reserved	Reserved

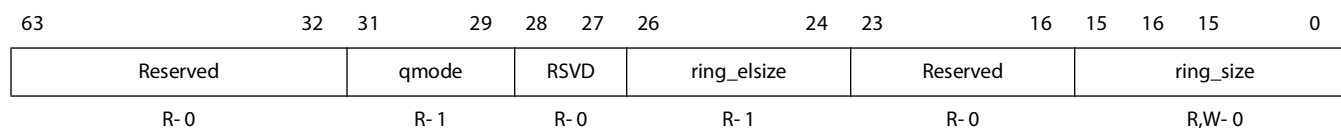
5.1.67 CHRING_SIZE [Address = 0x4_0048]

The Ring Size Register contains the element size and element counts for the ring

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-67 CHRING_SIZE



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-30 CHRING_SIZE Field Descriptions

Bits	Name	Description
63-32	Reserved	RESERVED

Table 5-30 CHRING_SIZE Field Descriptions

Bits	Name	Description
31-29	qmode	Defines the mode for this ring or queue. Field values: DUAL_RING_MODE(0): exposed ring mode for SW direct access
28-27	Reserved	RESERVED
26-24	ring_elsize	Ring element size. This field is hardcoded as: 1 = 8 bytes
23-16	Reserved	RESERVED
15-0	ring_size	Ring size. This field configures the size of the ring in elements.

5.1.68 CHST_SCHED [Address = 0x4_0060]

Channel Static Scheduler Config Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-68 CHST_SCHED

63	3	2	0
			QUEUE
			R,W- 0

n/a

R,W- 0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-31 CHST_SCHED Field Descriptions

Bits	Name	Description
63-3	Reserved	Undefined
2-0	QUEUE	Thus is the queue number that is written

5.1.69 CHRT_CTL [Address = 0x6_0000]

The channel realtime control register contains real-time control and status information for the DMA Channel. The fields in this register can be changed while the channel is in operation.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-69 CHRT_CTL

63	32	31	30	29	28	27	0
Reserved		ENABLE	TEARDOWN	PAUSE	FORCED TEARDOWN		
R- 0		R,W- 0	R,W- 0	R,W- 0	R,W- 0	R- 0	

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-32 CHRT_CTL Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved
31	ENABLE	This field enables or disables the channel. Disabling a channel halts operation on the channel after the current block transfer is completed. Disabling a channel in the middle of a packet transfer may result in underflow conditions in the attached application block and data loss. This field is encoded as follows: 0 = channel is disabled 1 = channel is enabled This field will be cleared after a channel teardown is complete.
30	TEARDOWN	Channel teardown: Setting this bit will request the channel to be torn down. This field will remain set after a channel teardown is complete.
29	PAUSE	Channel pause: Setting this bit will request the channel to pause processing at the next packet boundary. This is a more graceful method of halting processing than disabling the channel as it will not allow any current packets to underflow.
28	FORCED_TEARDOWN	This will force a teardown to occur. All triggers will be disabled and all TRs that have not been fully triggered will return a response of ABORTED. The enable will be cleared once all the TRs have been removed from internal FIFOs and all triggered TRs have finished. Unlike a regular teardown the TRs will not complete and no transfers that have already been triggered will be performed. Additionally, this teardown will always complete no matter the TR triggers or types that have been submitted.
28-0	Reserved	Reserved

5.1.70 CHRT_SWTRIG [Address = 0x6_0008]

The Software Trigger Register provides a mechanism by which software can directly trigger the channel in a secure way.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-70 CHRT_SWTRIG

63	3	2	1	0
Reserved	LOCAL_TRIGGER0		GLOBAL_TRIGGER1	GLOBAL_TRIGGER0
R- 0	R,WS- 0		R,WS- 0	R,WS- 0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-33 CHRT_SWTRIG Field Descriptions

Bits	Name	Description
63-3	Reserved	Reserved
2	LOCAL_TRIGGER0	Trigger: writing this bit with a value of 1 will cause the trigger event to be sent to this channel. This will trigger LOCAL Event
1	GLOBAL_TRIGGER1	Trigger: writing this bit with a value of 1 will cause the trigger event to be sent to this channel. This will trigger Global Event 1
0	GLOBAL_TRIGGER0	Trigger: writing this bit with a value of 1 will cause the trigger event to be sent to this channel. This will trigger Global Event 0

5.1.71 CHRT_ERROR_STAT [Address = 0x6_0010]

The Channel Error Status Register

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-71 CHRT_ERROR_STAT

63	62	61	60	32	31	24	23	16	15	8	7	4	3	0
IDLE	WR_ACTIVE	RD_ACTIVE	RSVD	TR_QUEUE_CNT	TR_CH_CNT	CMD_ID	INFO	TYPE						
R- 1	R- 0	R- 0	R- 0	R- 0	R- 0	R- 0	R- 0	R- 0						

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-34 CHRT_ERROR_STAT Field Descriptions

Bits	Name	Description
63	idle	The channel is currently idle
62	wr_active	The current active TR for the channel is doing writes
61	rd_active	The current active TR for the channel is doing reads
60-32	Reserved	Reserved
31-24	TR_QUEUE_CNT	The number of TRs that are no longer in the channel FIFO but are still in the queue for the channel
23-16	TR_CH_CNT	The number of TRs that are still to be processed by the channel
15-8	CMD_ID	The command id of the TR that had an error.
7-4	INFO	The information of the type of status. See the UTC TR Response format for the definition.
3-0	TYPE	The type of error that occurred. See the UTC TR Response format for the definition. Submission Errors are not captured in this register

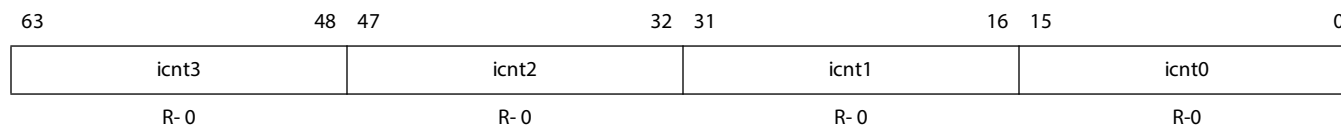
5.1.72 CHRT_ERROR_CNT [Address = 0x6_0018]

The Channel Error Count Register stores the count of the write engine when the error was received.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-72 CHRT_ERROR_CNT



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-35 CHRT_ERROR_CNT Field Descriptions

Bits	Name	Description
63-48	ICNT3	Value of outermost count when error occurred
47-32	ICNT2	Value of second outermost count when error occurred
31-16	ICNT1	The value of the second innermost count when error occurred
15-8	ICNT0	The value of the innermost count when error occurred.

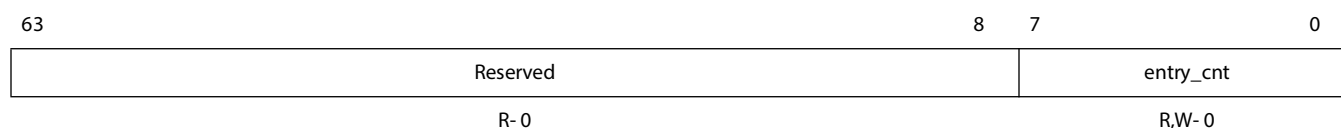
5.1.73 CHRT_RING_FWD_DB [Address = 0x6_0020]

The Ring N Doorbell Register is written by software to increment the number of entries on a ring. One or more entries as specified by the entry_cnt field can be added to a ring with a single write operation.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-73 CHRT_RING_FWD_DB



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-36 CHRT_RING_FWD_DB Field Descriptions

Bits	Name	Description
63-8	Reserved	Reserved

Table 5-36 CHRT_RING_FWD_DB Field Descriptions

Bits	Name	Description
7-0	entry_cnt	Number of entries by which to increment (or decrement) the ring occupancy. This is a twos compliment signed number.

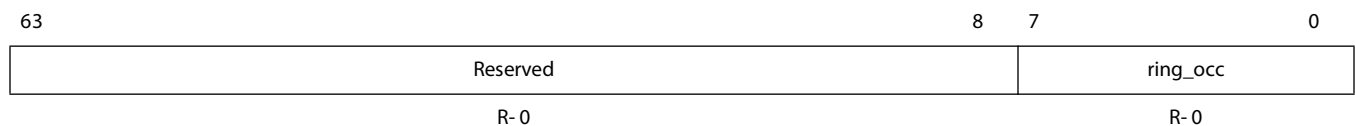
5.1.74 CHRT_RING_FWD_OCC [Address = 0x6_0028]

The Ring Occupancy Register can be read by software to determine the total number of valid entries on a ring. The contents of each of these registers are unary-ORed in order to create a pending signal for the ring which can be used for triggering hardware operations and/or for generating interrupts to the host..

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-74 CHRT_RING_FWD_OCC



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-37 CHRT_RING_FWD_OCC Field Descriptions

Bits	Name	Description
63-8	Reserved	Reserved
7-0	ring_occ	Total number of valid entries on the ring. This value is reset to 0 anytime the ring configuration registers are modified

5.1.75 CHRT_RING_RVRS_DB [Address = 0x6_0040]

The Ring N Doorbell Register is written by software to decrement the number of entries on a ring. One or more entries as specified by the entry_cnt field can be added to a ring with a single write operation.

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-75 CHRT_RING_RVRS_DB

63	32	31	30	8	7	0
Reserved	tdown_ack	Reserved	entry_cnt			
R-0	WS-0	R-0	R,W-0			

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-38 CHRT_RING_RVSR_DB Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved
31	tdown_ack	This bit is set to 1 to acknowledge (and clear) the tdown_complete bit in the corresponding Ring N Occupancy Register.
30-8	Reserved	Reserved
7-0	entry_cnt	Number of entries by which to increment (or decrement) the ring occupancy. This is a twos complement signed number.

5.1.76 CHRT_RING_RVRS_OCC [Address = 0x6_0048]

The Ring Occupancy Register can be read by software to determine the total number of valid entries on a ring. The contents of each of these registers are unary-ORed in order to create a pending signal for the ring which can be used for triggering hardware operations and/or for generating interrupts to the host..

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-76 CHRT_RING_RVRS_OCC

63	32	31	30	8	7	0
Reserved	tdown_complete	Reserved	ring_occ			
R-0	R-0	R-0	R-0			

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-39 CHRT_RING_RVSR_OCC Field Descriptions

Bits	Name	Description
63-32	Reserved	Reserved
31	tdown_complete	This bit when set indicates that a teardown is complete on the channel. This bit is cleared anytime the tdown_ack bit is written as a 1 in the corresponding Ring N Doorbell Register
30-8	Reserved	Reserved
7-0	ring_occ	Total number of valid entries on the ring. This value is reset to 0 anytime the ring configuration registers are modified

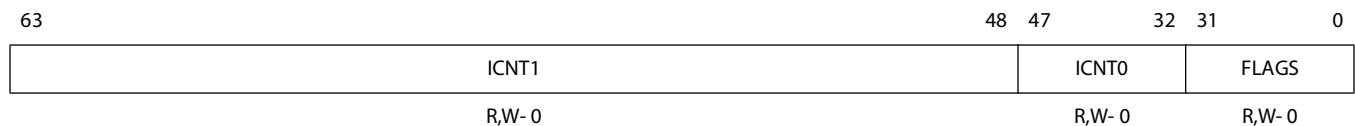
5.1.77 ATOMIC_SUBMIT_CURR_TR_WORD0_1 [Address = 0x8_0000]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-77 ATOMIC_SUBMIT_CURR_TR_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-40 ATOMIC_SUBMIT_CURR_TR_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

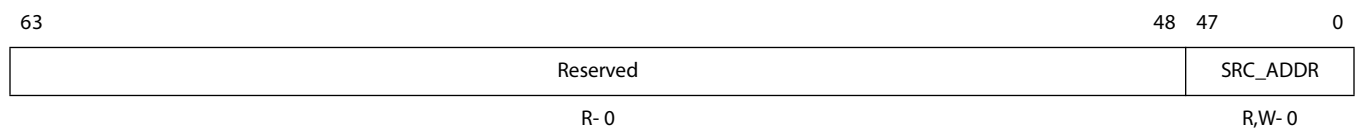
5.1.78 ATOMIC_SUBMIT_CURR_TR_WORD2_3 [Address = 0x8_0008]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-78 ATOMIC_SUBMIT_CURR_TR_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-41 ATOMIC_SUBMIT_CURR_TR_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

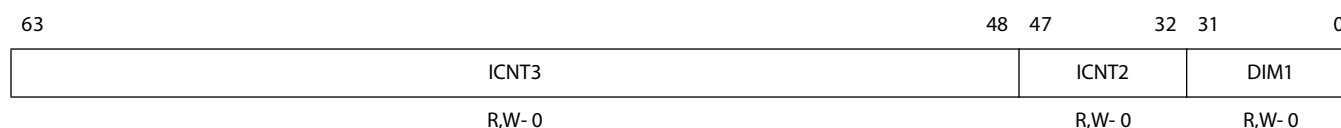
5.1.79 ATOMIC_SUBMIT_CURR_TR_WORD4_5 [Address = 0x8_0010]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-79 ATOMIC_SUBMIT_CURR_TR_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-42 ATOMIC_SUBMIT_CURR_TR_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

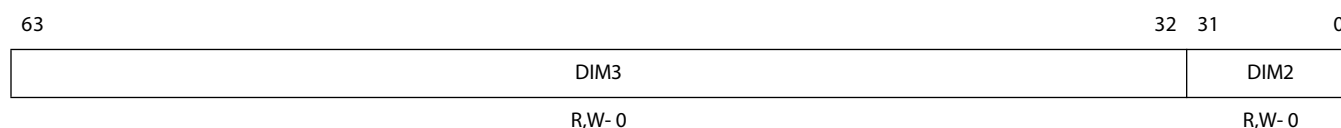
5.1.80 ATOMIC_SUBMIT_CURR_TR_WORD6_7 [Address = 0x8_0018]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-80 ATOMIC_SUBMIT_CURR_TR_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-43 ATOMIC_SUBMIT_CURR_TR_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

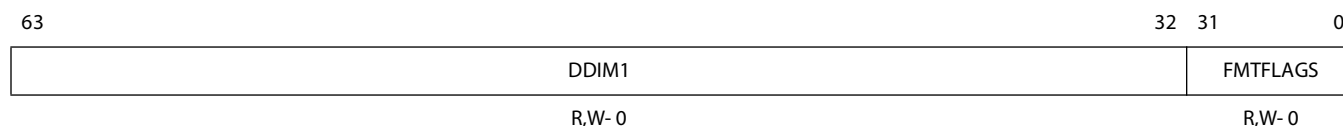
5.1.81 ATOMIC_SUBMIT_CURR_TR_WORD8_9 [Address = 0x8_0020]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-81 ATOMIC_SUBMIT_CURR_TR_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-44 ATOMIC_SUBMIT_CURR_TR_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

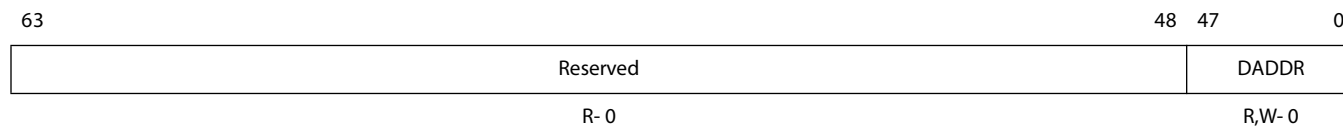
5.1.82 ATOMIC_SUBMIT_CURR_TR_WORD10_11 [Address = 0x8_0028]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-82 ATOMIC_SUBMIT_CURR_TR_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-45 ATOMIC_SUBMIT_CURR_TR_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

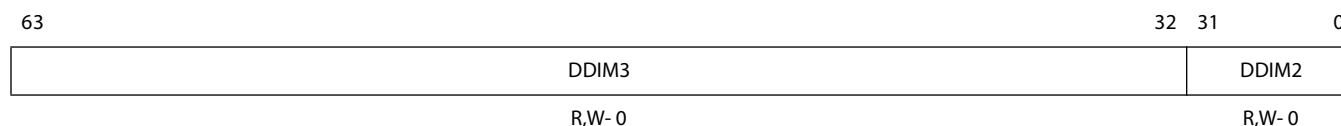
5.1.83 ATOMIC_SUBMIT_CURR_TR_WORD12_13 [Address = 0x8_0030]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-83 ATOMIC_SUBMIT_CURR_TR_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-46 ATOMIC_SUBMIT_CURR_TR_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

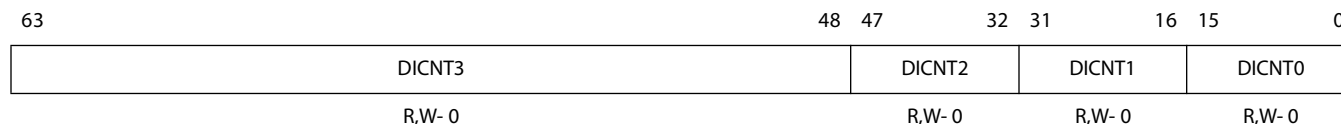
5.1.84 ATOMIC_SUBMIT_CURR_TR_WORD14_15 [Address = 0x8_0038]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-84 ATOMIC_SUBMIT_CURR_TR_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-47 ATOMIC_SUBMIT_CURR_TR_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

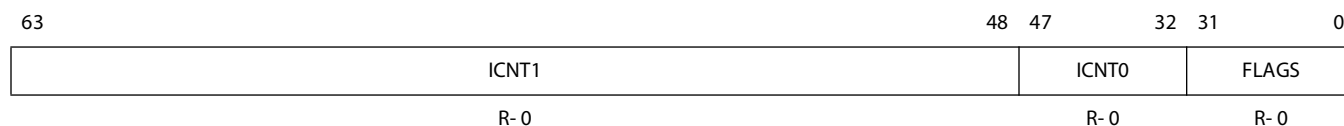
5.1.85 NEXT_TR_WORD0_1 [Address = 0x8_0040]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-85 NEXT_TR_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-48 NEXT_TR_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

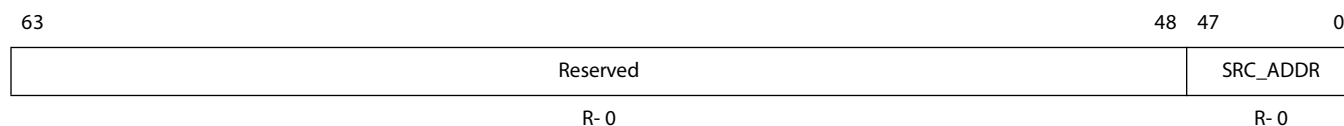
5.1.86 NEXT_TR_WORD2_3 [Address = 0x8_0048]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-86 NEXT_TR_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-49 NEXT_TR_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

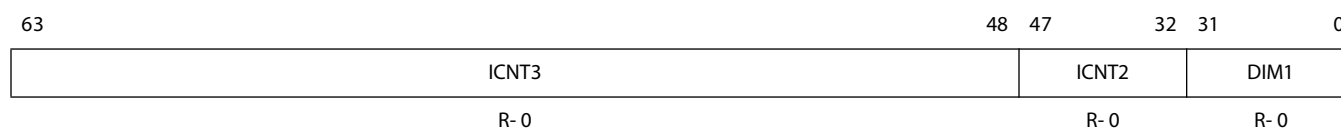
5.1.87 NEXT_TR_WORD4_5 [Address = 0x8_0050]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-87 NEXT_TR_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-50 NEXT_TR_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

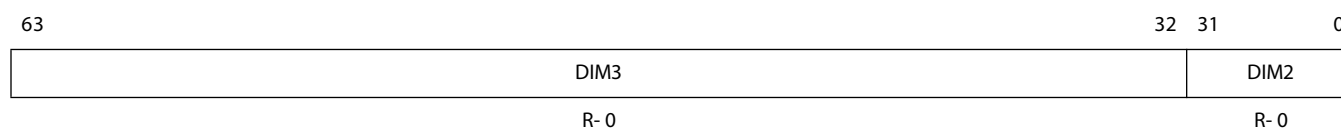
5.1.88 NEXT_TR_WORD6_7 [Address = 0x8_0058]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-88 NEXT_TR_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-51 NEXT_TR_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

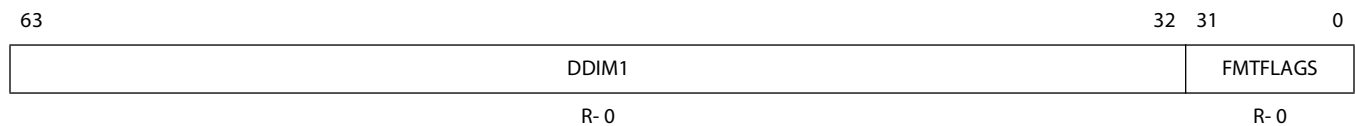
5.1.89 NEXT_TR_WORD8_9 [Address = 0x8_0060]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-89 NEXT_TR_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-52 NEXT_TR_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

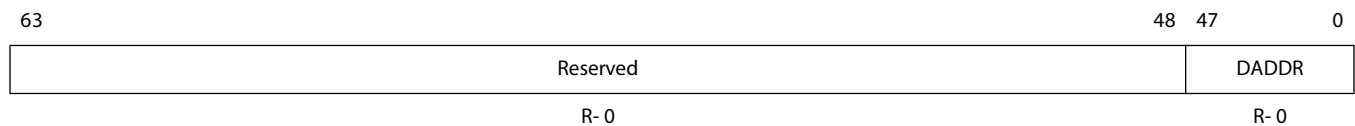
5.1.90 NEXT_TR_WORD10_11 [Address = 0x8_0068]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-90 NEXT_TR_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-53 NEXT_TR_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

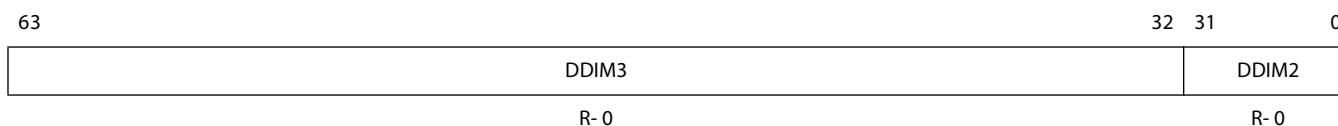
5.1.91 NEXT_TR_WORD12_13 [Address = 0x8_0070]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-91 NEXT_TR_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-54 NEXT_TR_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

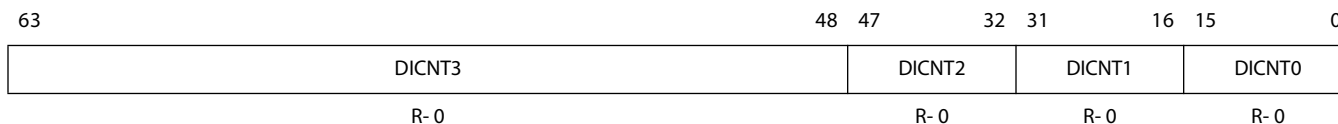
5.1.92 NEXT_TR_WORD14_15 [Address = 0x8_0078]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-92 NEXT_TR_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-55 NEXT_TR_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

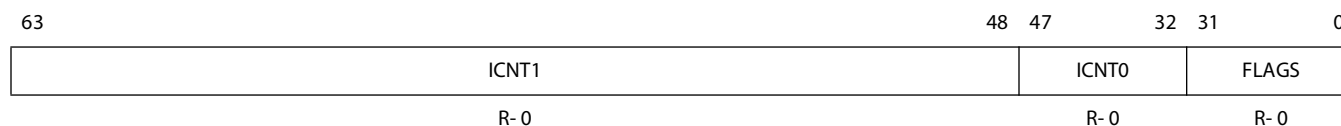
5.1.93 NEXT_NEXT_TR_WORD0_1 [Address = 0x8_0080]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-93 NEXT_NEXT_TR_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-56 NEXT_NEXT_TR_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

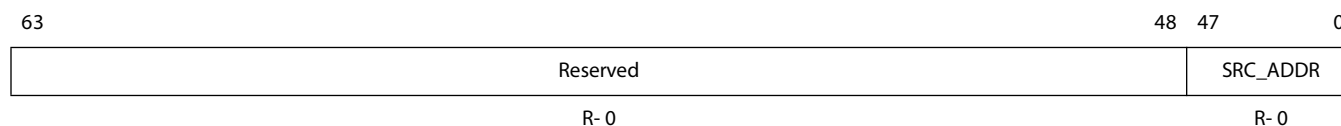
5.1.94 NEXT_NEXT_TR_WORD2_3 [Address = 0x8_0088]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-94 NEXT_NEXT_TR_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-57 NEXT_NEXT_TR_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

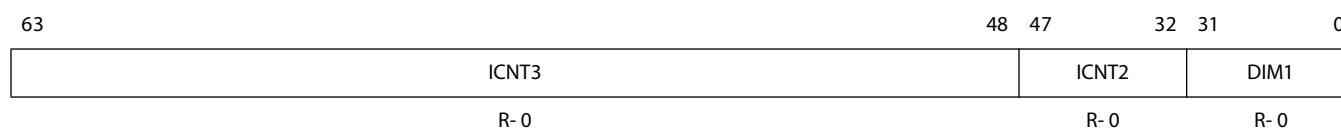
5.1.95 NEXT_NEXT_TR_WORD4_5 [Address = 0x8_0090]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-95 NEXT_NEXT_TR_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-58 NEXT_NEXT_TR_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

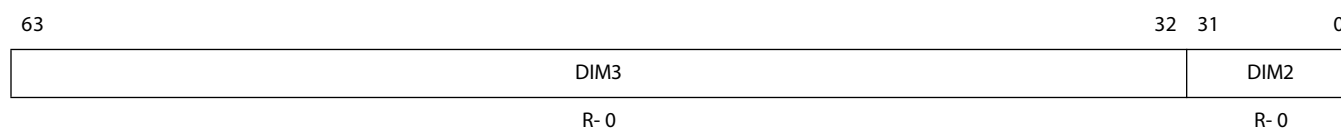
5.1.96 NEXT_NEXT_TR_WORD6_7 [Address = 0x8_0098]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-96 NEXT_NEXT_TR_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-59 NEXT_NEXT_TR_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

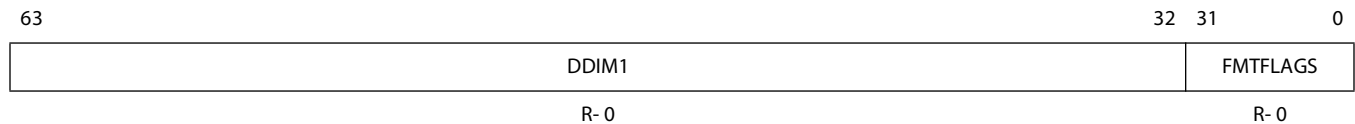
5.1.97 NEXT_NEXT_TR_WORD8_9 [Address = 0x8_00A0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-97 NEXT_NEXT_TR_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-60 NEXT_NEXT_TR_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

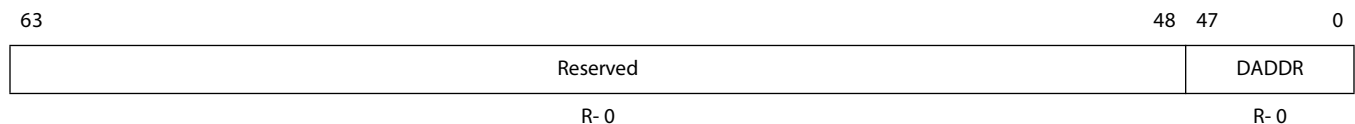
5.1.98 NEXT_NEXT_TR_WORD10_11 [Address = 0x8_00A8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-98 NEXT_NEXT_TR_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-61 NEXT_NEXT_TR_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

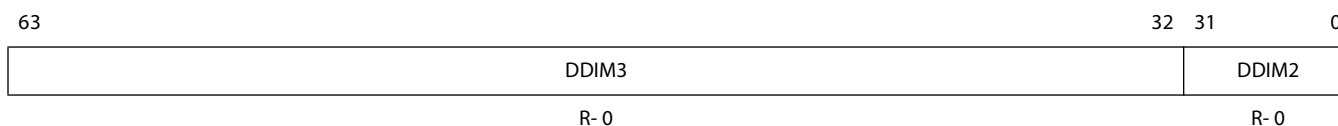
5.1.99 NEXT_NEXT_TR_WORD12_13 [Address = 0x8_00B0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-99 NEXT_NEXT_TR_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-62 NEXT_NEXT_TR_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

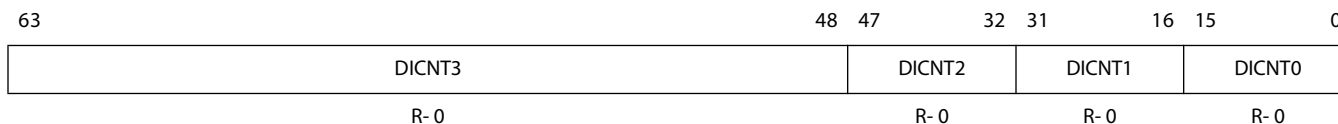
5.1.100 NEXT_NEXT_TR_WORD14_15 [Address = 0x8_00B8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-100 NEXT_NEXT_TR_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-63 NEXT_NEXT_TR_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

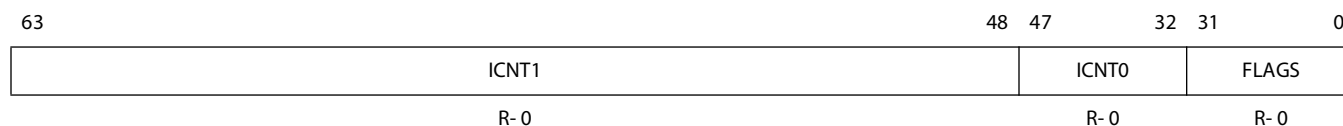
5.1.101 CURR_STATE_WORD0_1 [Address = 0x8_00C0]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-101 CURR_STATE_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-64 CURR_STATE_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

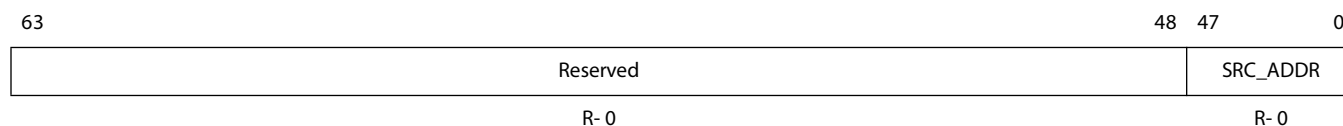
5.1.102 CURR_STATE_WORD2_3 [Address = 0x8_00C8]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-102 CURR_STATE_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-65 CURR_STATE_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

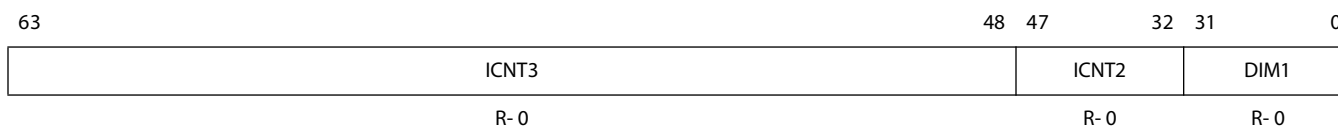
5.1.103 CURR_STATE_WORD4_5 [Address = 0x8_00D0]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-103 CURR_STATE_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-66 CURR_STATE_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

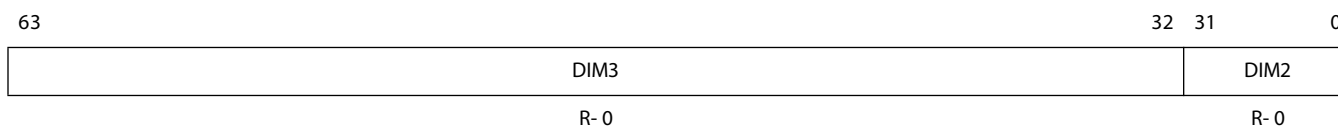
5.1.104 CURR_STATE_WORD6_7 [Address = 0x8_00D8]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-104 CURR_STATE_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-67 CURR_STATE_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

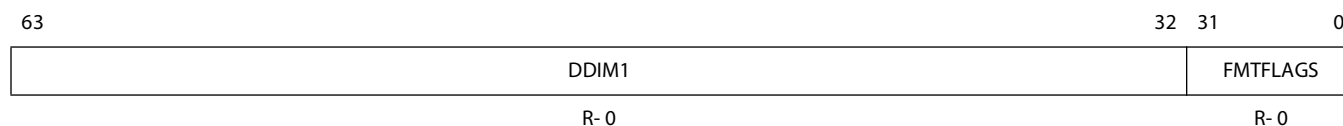
5.1.105 CURR_STATE_WORD8_9 [Address = 0x8_00E0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-105 CURR_STATE_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-68 CURR_STATE_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

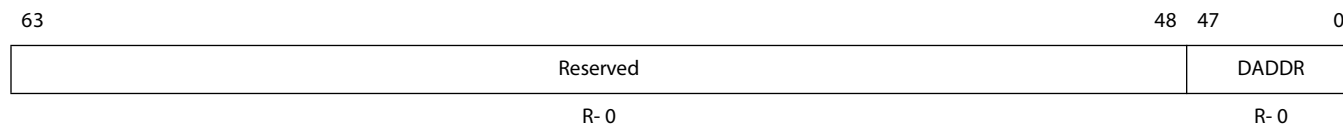
5.1.106 CURR_STATE_WORD10_11 [Address = 0x8_00E8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-106 CURR_STATE_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-69 CURR_STATE_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

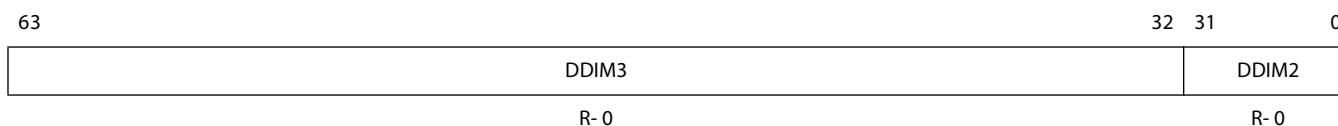
5.1.107 CURR_STATE_WORD12_13 [Address = 0x8_00F0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-107 CURR_STATE_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-70 CURR_STATE_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

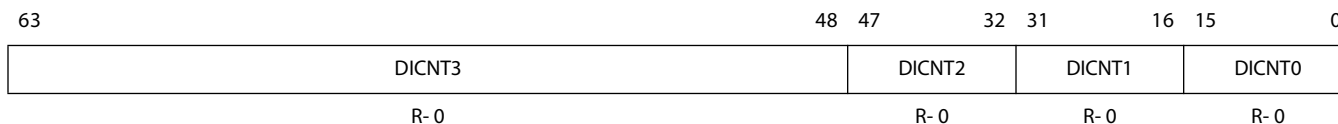
5.1.108 CURR_STATE_WORD14_15 [Address = 0x8_00F8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-108 CURR_STATE_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-71 CURR_STATE_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

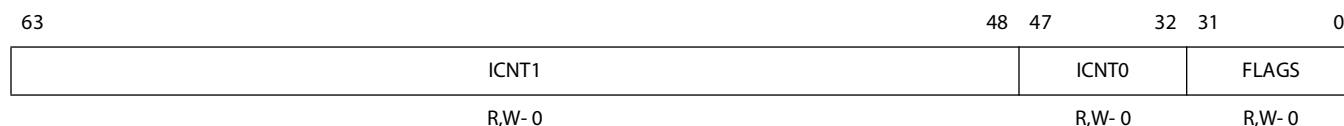
5.1.109 CORE0_SUBMIT_WORD0_1 [Address = 0xA_0000]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-109 CORE0_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-72 CORE0_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

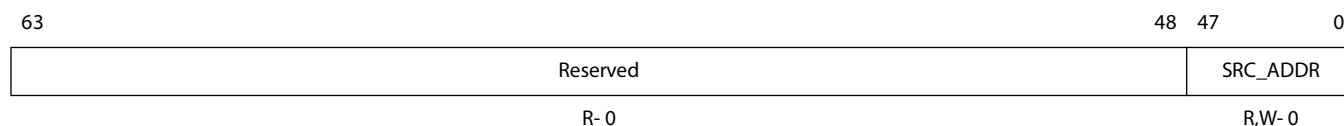
5.1.110 CORE0_SUBMIT_WORD2_3 [Address = 0xA_0008]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-110 CORE0_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-73 CORE0_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

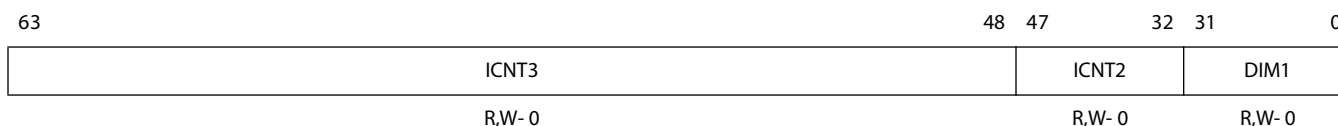
5.1.111 CORE0_SUBMIT_WORD4_5 [Address = 0xA_0010]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-111 CORE0_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-74 CORE0_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

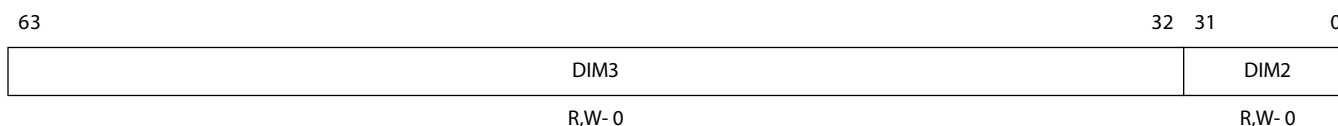
5.1.112 CORE0_SUBMIT_WORD6_7 [Address = 0xA_0018]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-112 CORE0_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-75 CORE0_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

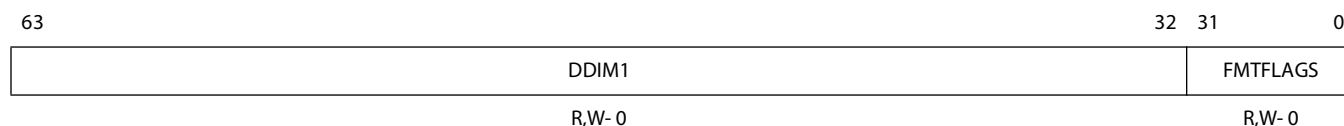
5.1.113 CORE0_SUBMIT_WORD8_9 [Address = 0xA_0020]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-113 CORE0_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-76 CORE0_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

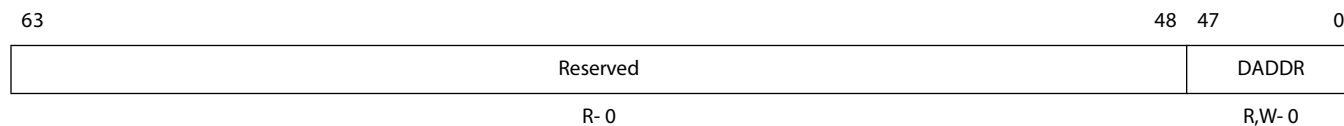
5.1.114 CORE0_SUBMIT_WORD10_11 [Address = 0xA_0028]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-114 CORE0_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-77 CORE0_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

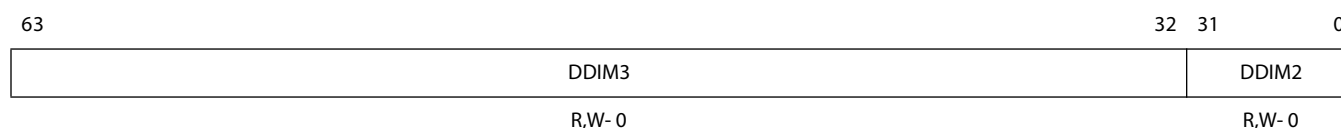
5.1.115 CORE0_SUBMIT_WORD12_13 [Address = 0xA_0030]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-115 CORE0_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-78 CORE0_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

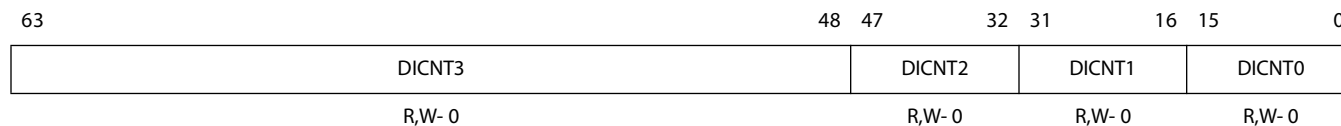
5.1.116 CORE0_SUBMIT_WORD14_15 [Address = 0xA_0038]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-116 CORE0_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-79 CORE0_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

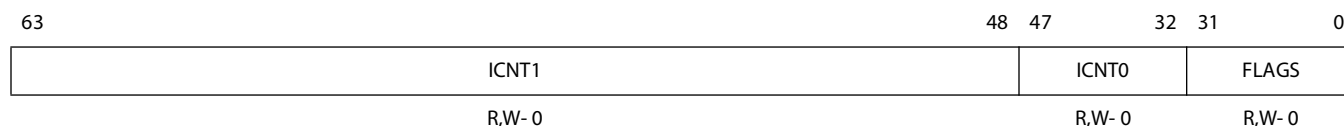
5.1.117 CORE1_SUBMIT_WORD0_1 [Address = 0xA_0040]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-117 CORE1_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-80 CORE1_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

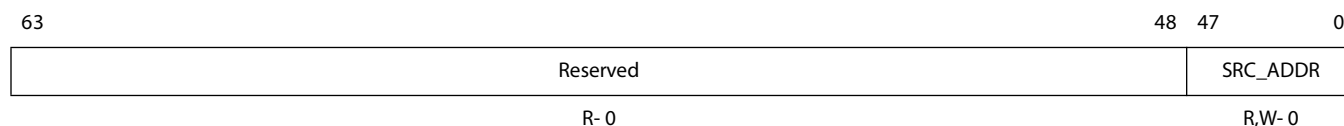
5.1.118 CORE1_SUBMIT_WORD2_3 [Address = 0xA_0048]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-118 CORE1_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-81 CORE1_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

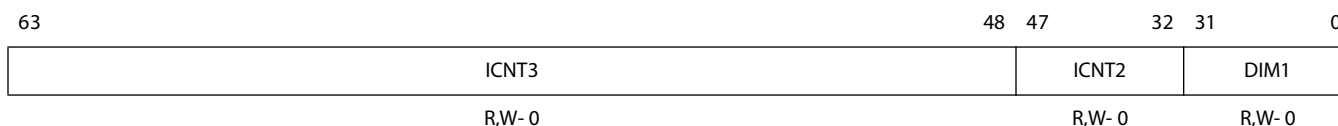
5.1.119 CORE1_SUBMIT_WORD4_5 [Address = 0xA_0050]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-119 CORE1_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-82 CORE1_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

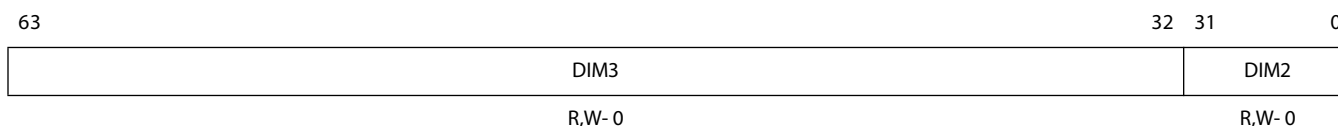
5.1.120 CORE1_SUBMIT_WORD6_7 [Address = 0xA_0058]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-120 CORE1_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-83 CORE1_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

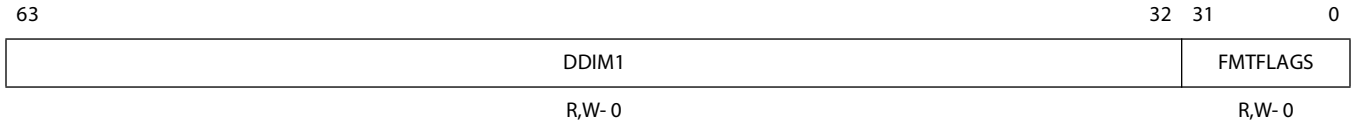
5.1.121 CORE1_SUBMIT_WORD8_9 [Address = 0xA_0060]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-121 CORE1_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-84 CORE1_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

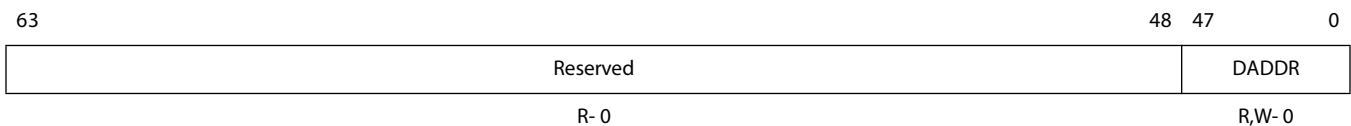
5.1.122 CORE1_SUBMIT_WORD10_11 [Address = 0xA_0068]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-122 CORE1_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-85 CORE1_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

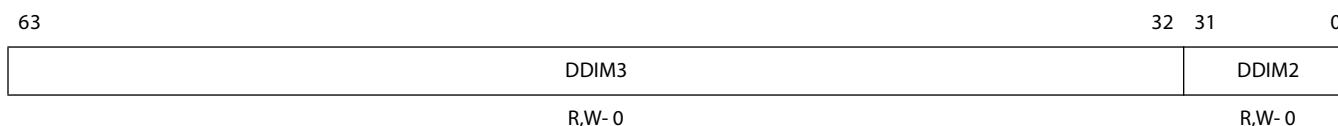
5.1.123 CORE1_SUBMIT_WORD12_13 [Address = 0xA_0070]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-123 CORE1_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-86 CORE1_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

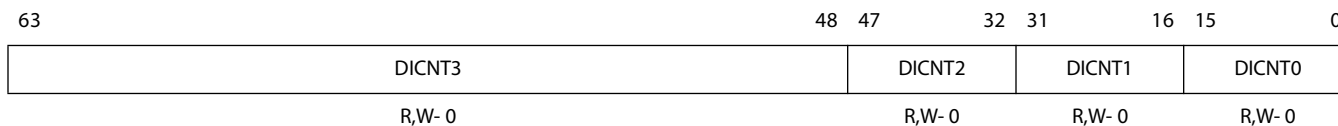
5.1.124 CORE1_SUBMIT_WORD14_15 [Address = 0xA_0078]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-124 CORE1_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-87 CORE1_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

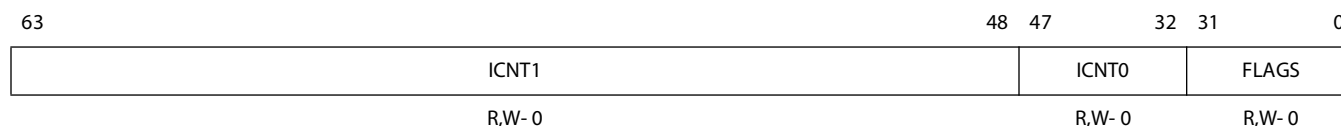
5.1.125 CORE2_SUBMIT_WORD0_1 [Address = 0xA_0080]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-125 CORE2_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-88 CORE2_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

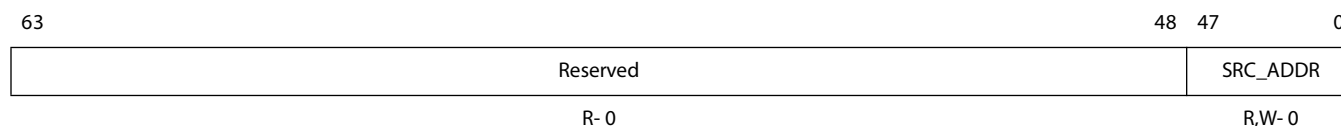
5.1.126 CORE2_SUBMIT_WORD2_3 [Address = 0xA_0088]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-126 CORE2_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-89 CORE2_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

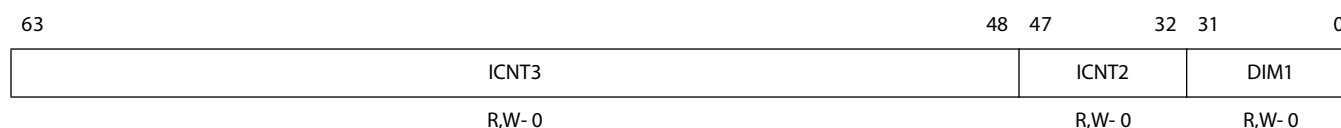
5.1.127 CORE2_SUBMIT_WORD4_5 [Address = 0xA_0090]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-127 CORE2_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-90 CORE2_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

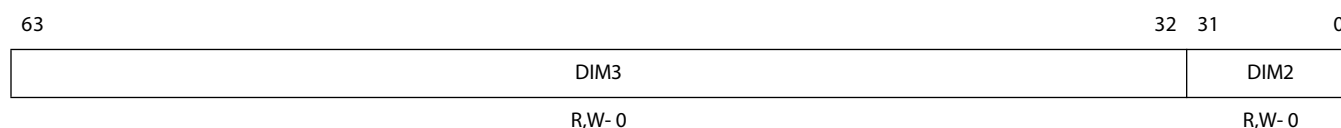
5.1.128 CORE2_SUBMIT_WORD6_7 [Address = 0xA_0098]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-128 CORE2_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-91 CORE2_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

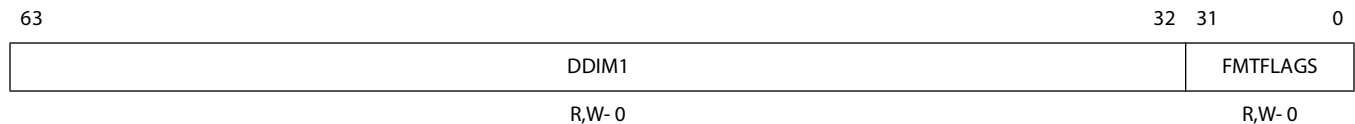
5.1.129 CORE2_SUBMIT_WORD8_9 [Address = 0xA_00A0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-129 CORE2_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-92 CORE2_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

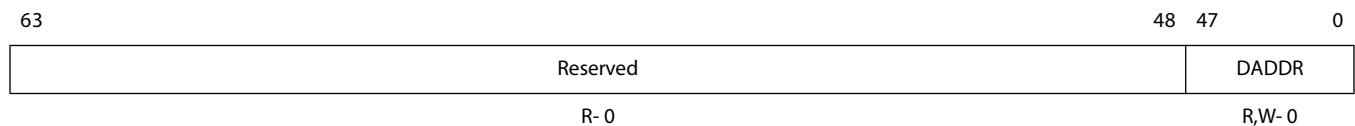
5.1.130 CORE2_SUBMIT_WORD10_11 [Address = 0xA_00A8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-130 CORE2_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-93 CORE2_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

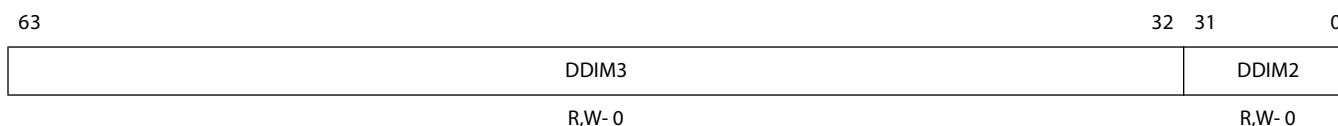
5.1.131 CORE2_SUBMIT_WORD12_13 [Address = 0xA_00B0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-131 CORE2_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-94 CORE2_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

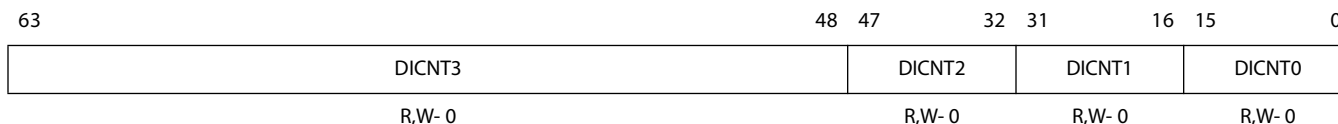
5.1.132 CORE2_SUBMIT_WORD14_15 [Address = 0xA_00B8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-132 CORE2_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-95 CORE2_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

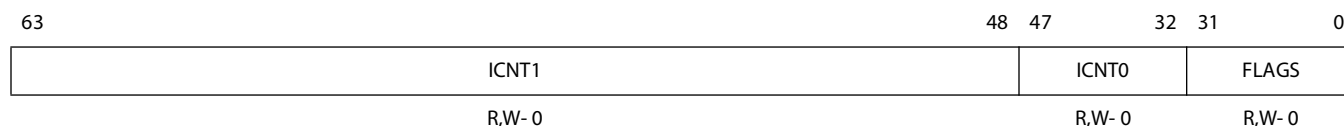
5.1.133 CORE3_SUBMIT_WORD0_1 [Address = 0xA_00C0]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-133 CORE3_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-96 CORE3_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

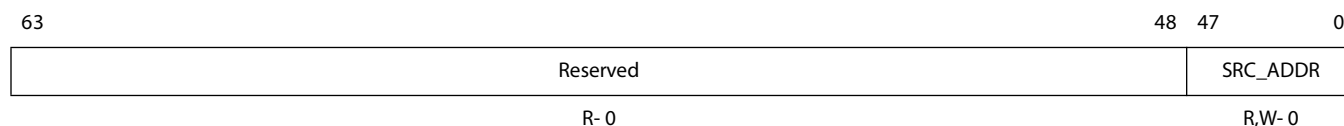
5.1.134 CORE3_SUBMIT_WORD2_3 [Address = 0xA_00C8]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-134 CORE3_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-97 CORE3_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

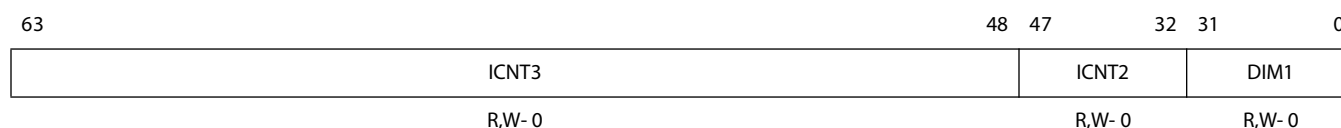
5.1.135 CORE3_SUBMIT_WORD4_5 [Address = 0xA_00D0]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-135 CORE3_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-98 CORE3_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

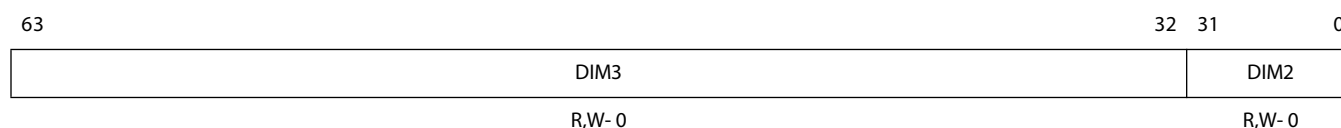
5.1.136 CORE3_SUBMIT_WORD6_7 [Address = 0xA_00D8]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-136 CORE3_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-99 CORE3_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

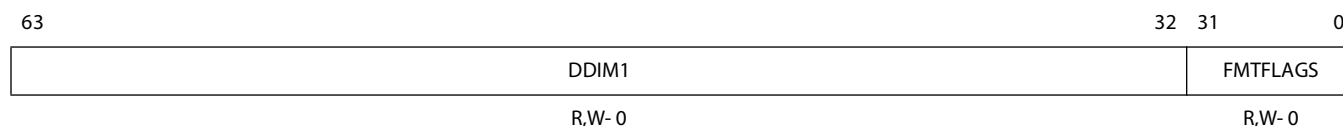
5.1.137 CORE3_SUBMIT_WORD8_9 [Address = 0xA_00E0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-137 CORE3_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-100 CORE3_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

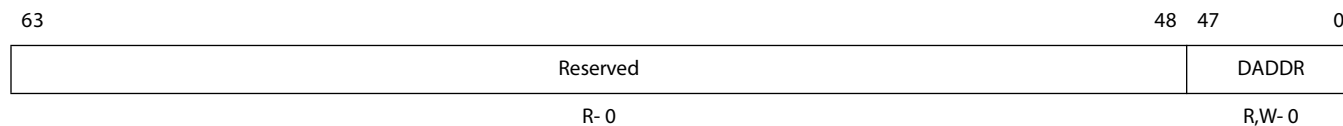
5.1.138 CORE3_SUBMIT_WORD10_11 [Address = 0xA_00E8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-138 CORE3_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-101 CORE3_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

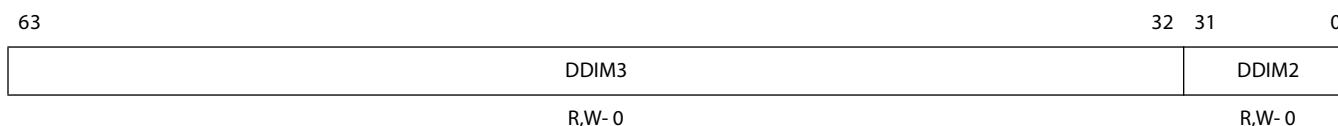
5.1.139 CORE3_SUBMIT_WORD12_13 [Address = 0xA_00F0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-139 CORE3_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-102 CORE3_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

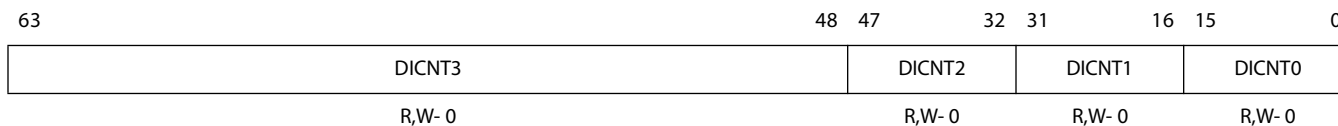
5.1.140 CORE3_SUBMIT_WORD14_15 [Address = 0xA_00F8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-140 CORE3_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-103 CORE3_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

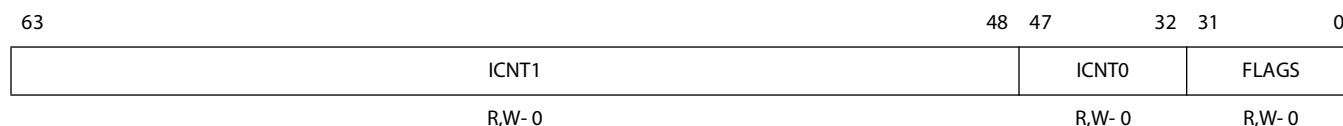
5.1.141 CORE4_SUBMIT_WORD0_1 [Address = 0XC0XC_0000]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-141 CORE4_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-104 CORE4_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

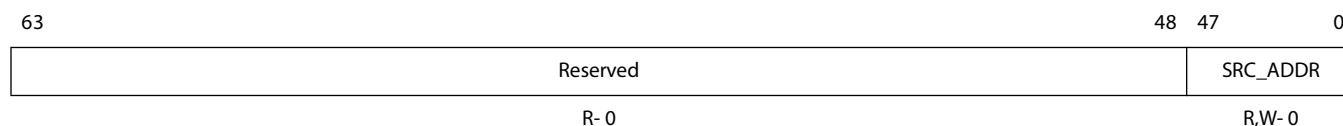
5.1.142 CORE4_SUBMIT_WORD2_3 [Address = 0XC_0008]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-142 CORE4_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-105 CORE4_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

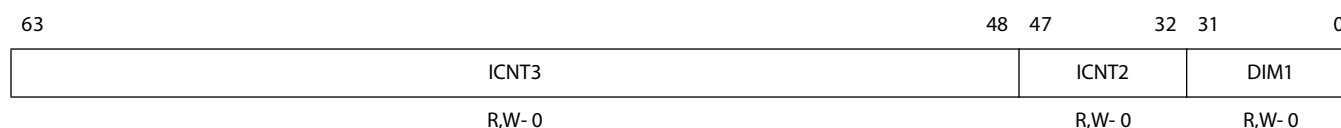
5.1.143 CORE4_SUBMIT_WORD4_5 [Address = 0XC_0010]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-143 CORE4_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-106 CORE4_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

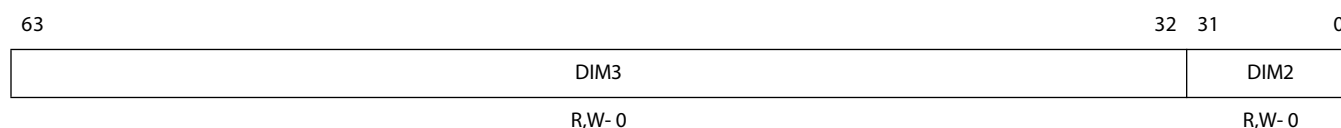
5.1.144 CORE4_SUBMIT_WORD6_7 [Address = 0XC_0018]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-144 CORE4_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-107 CORE4_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

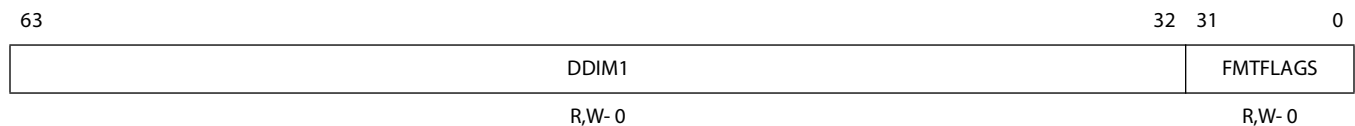
5.1.145 CORE4_SUBMIT_WORD8_9 [Address = 0XC_0020]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-145 CORE4_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-108 CORE4_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

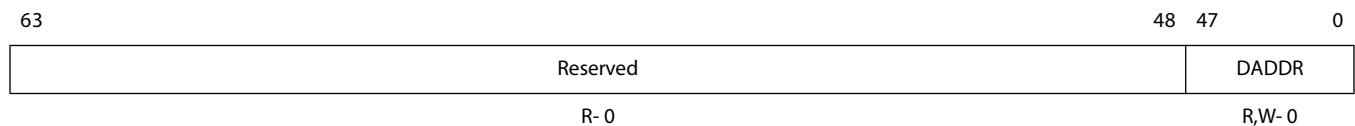
5.1.146 CORE4_SUBMIT_WORD10_11 [Address = 0XC_0028]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-146 CORE4_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-109 CORE4_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

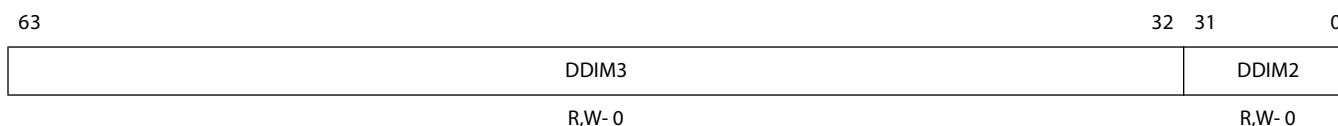
5.1.147 CORE4_SUBMIT_WORD12_13 [Address = 0XC_0030]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-147 CORE4_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-110 CORE4_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

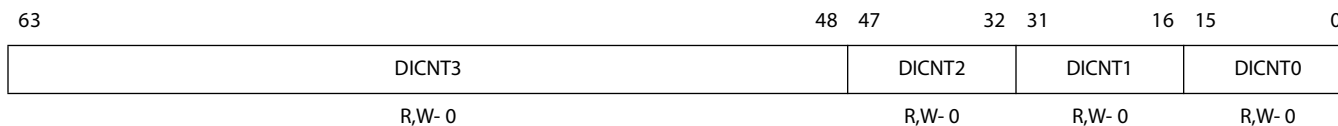
5.1.148 CORE4_SUBMIT_WORD14_15 [Address = 0XC_0038]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-148 CORE4_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-111 CORE4_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

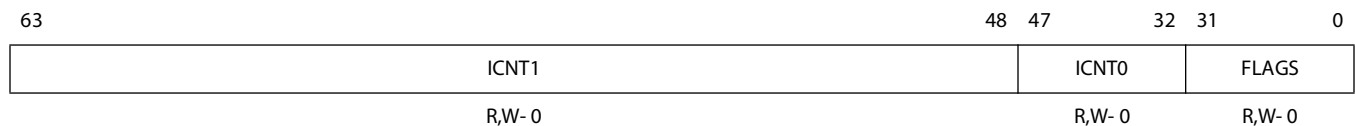
5.1.149 CORE5_SUBMIT_WORD0_1 [Address = 0XC_0040]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-149 CORE5_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-112 CORE5_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

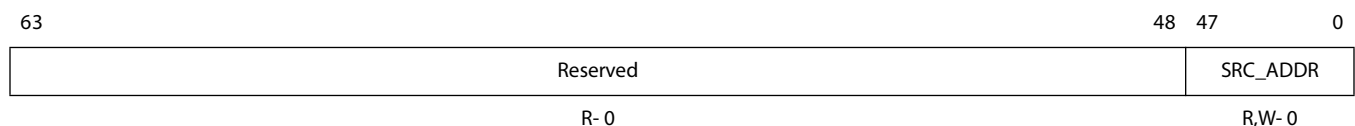
5.1.150 CORE5_SUBMIT_WORD2_3 [Address = 0XC_0048]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-150 CORE5_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-113 CORE5_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

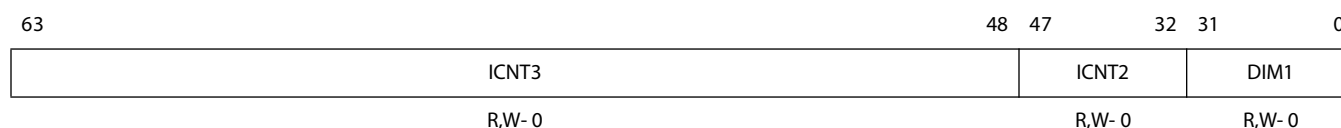
5.1.151 CORE5_SUBMIT_WORD4_5 [Address = 0XC_0050]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-151 CORE5_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-114 CORE5_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

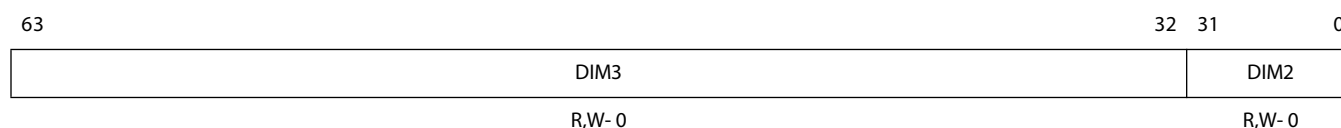
5.1.152 CORE5_SUBMIT_WORD6_7 [Address = 0XC_0058]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-152 CORE5_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-115 CORE5_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

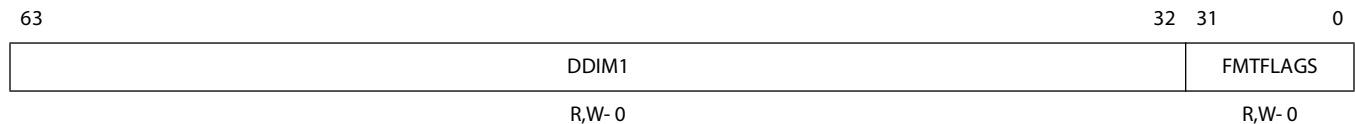
5.1.153 CORE5_SUBMIT_WORD8_9 [Address = 0XC_0060]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-153 CORE5_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-116 CORE5_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

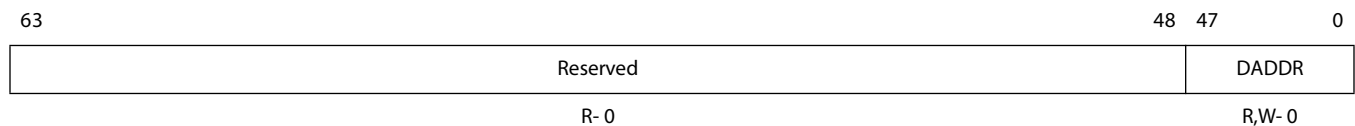
5.1.154 CORE5_SUBMIT_WORD10_11 [Address = 0XC_0068]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-154 CORE5_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-117 CORE5_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

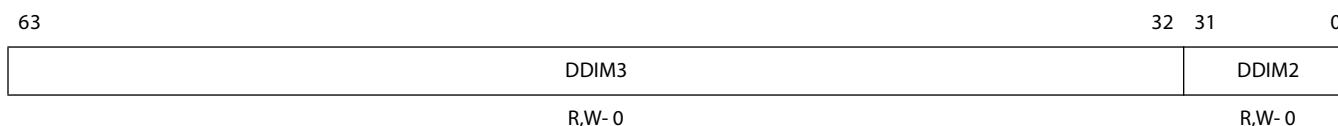
5.1.155 CORE5_SUBMIT_WORD12_13 [Address = 0XC_0070]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-155 CORE5_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-118 CORE5_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

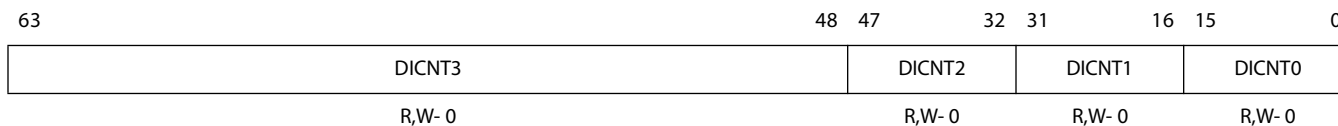
5.1.156 CORE5_SUBMIT_WORD14_15 [Address = 0XC_0078]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-156 CORE5_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-119 CORE5_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

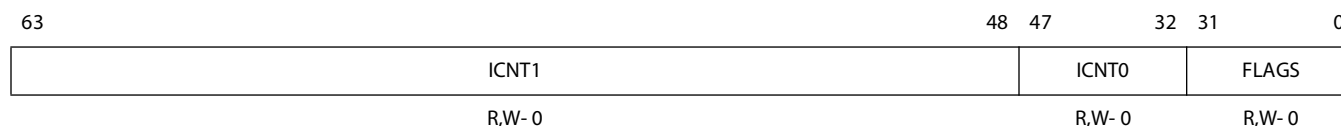
5.1.157 CORE6_SUBMIT_WORD0_1 [Address = 0XC_0080]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-157 CORE6_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-120 CORE6_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

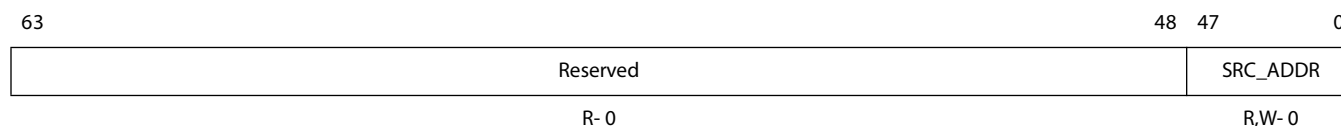
5.1.158 CORE6_SUBMIT_WORD2_3 [Address = 0XC_0088]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-158 CORE6_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-121 CORE6_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

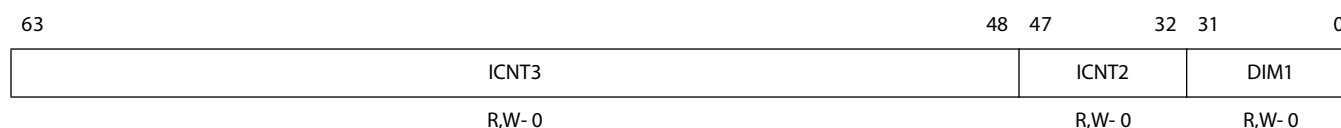
5.1.159 CORE6_SUBMIT_WORD4_5 [Address = 0XC_0090]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-159 CORE6_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-122 CORE6_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

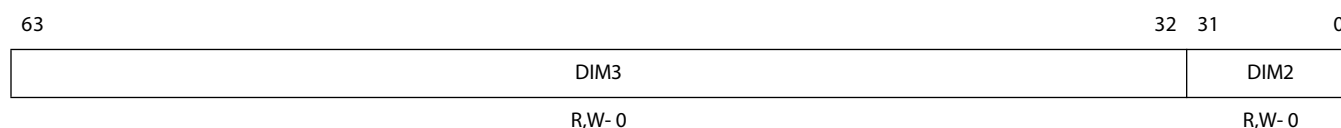
5.1.160 CORE6_SUBMIT_WORD6_7 [Address = 0XC_0098]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-160 CORE6_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-123 CORE6_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

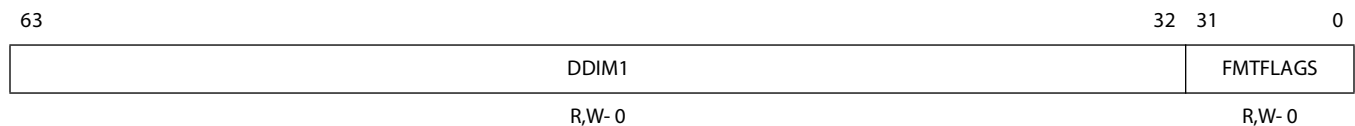
5.1.161 CORE6_SUBMIT_WORD8_9 [Address = 0XC_00A0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-161 CORE6_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-124 CORE6_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

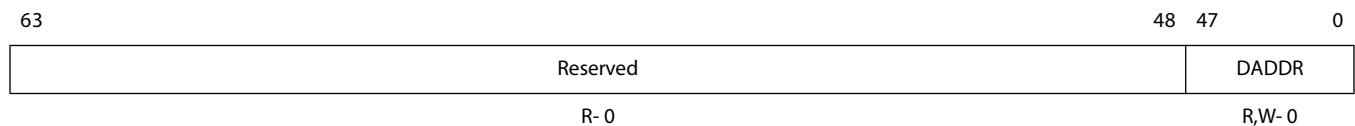
5.1.162 CORE6_SUBMIT_WORD10_11 [Address = 0XC_00A8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-162 CORE6_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-125 CORE6_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

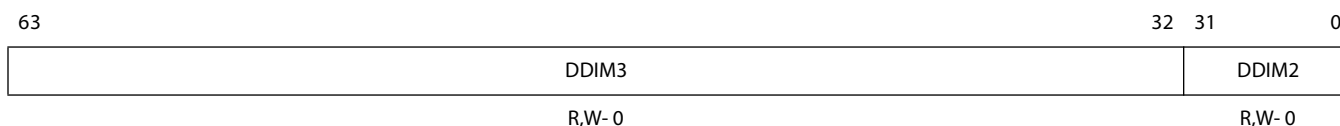
5.1.163 CORE6_SUBMIT_WORD12_13 [Address = 0XC_00B0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-163 CORE6_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-126 CORE6_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

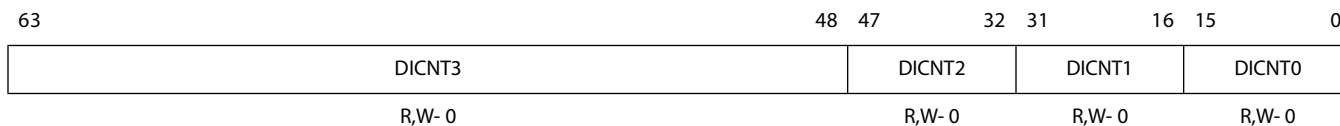
5.1.164 CORE6_SUBMIT_WORD14_15 [Address = 0XC_00B8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-164 CORE6_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-127 CORE6_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

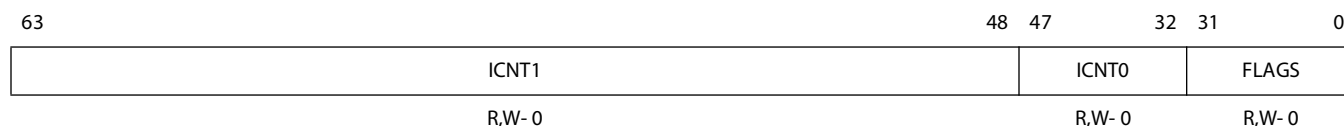
5.1.165 CORE7_SUBMIT_WORD0_1 [Address = 0XC_00C0]

The first TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-165 CORE7_SUBMIT_WORD0_1



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-128 CORE7_SUBMIT_WORD0_1 Field Descriptions

Bits	Name	Description
63-48	ICNT1	Lines in a transfer
47-32	ICNT0	Bytes in a transfer
31-0	FLAGS	Flags for the operation and type of descriptor

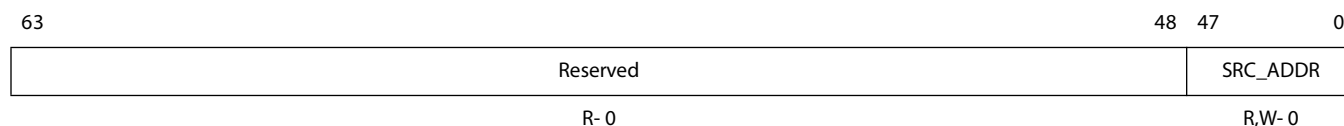
5.1.166 CORE7_SUBMIT_WORD2_3 [Address = 0XC_00C8]

The second TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-166 CORE7_SUBMIT_WORD2_3



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-129 CORE7_SUBMIT_WORD2_3 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	SRC_ADDR	Source transfer address virtual or physical allowed

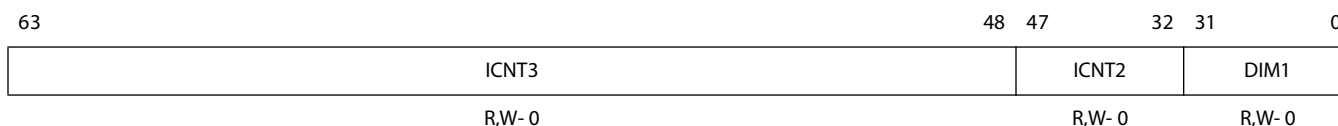
5.1.167 CORE7_SUBMIT_WORD4_5 [Address = 0XC_00D0]

The third TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-167 CORE7_SUBMIT_WORD4_5



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-130 CORE7_SUBMIT_WORD4_5 Field Descriptions

Bits	Name	Description
63-48	ICNT3	The size of the 4th loop in the TR transfer
47-32	ICNT2	The size of the 3rd loop in the TR transfer
31-0	DIM1	The first dimension width of the source data

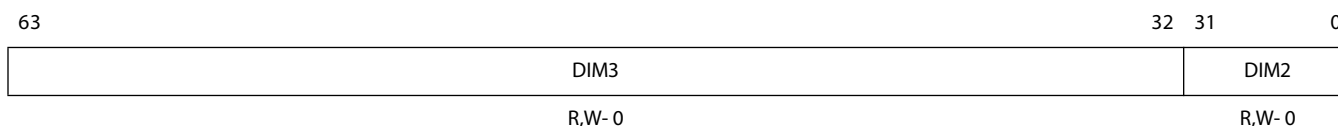
5.1.168 CORE7_SUBMIT_WORD6_7 [Address = 0XC_00D8]

The fourth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-168 CORE7_SUBMIT_WORD6_7



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-131 CORE7_SUBMIT_WORD6_7 Field Descriptions

Bits	Name	Description
63-32	DIM3	The third dimension width of the source data
31-0	DIM2	The second dimension width of the source data

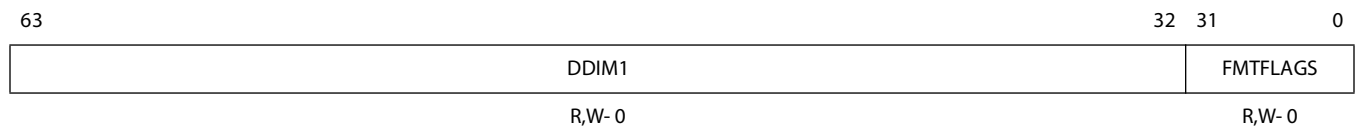
5.1.169 CORE7_SUBMIT_WORD8_9 [Address = 0XC_00E0]

The fifth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-169 CORE7_SUBMIT_WORD8_9



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-132 CORE7_SUBMIT_WORD8_9 Field Descriptions

Bits	Name	Description
63-32	DDIM1	The first dimension width of the destination data
31-0	FMTFLAGS	The data formatting flags to be used for the TR

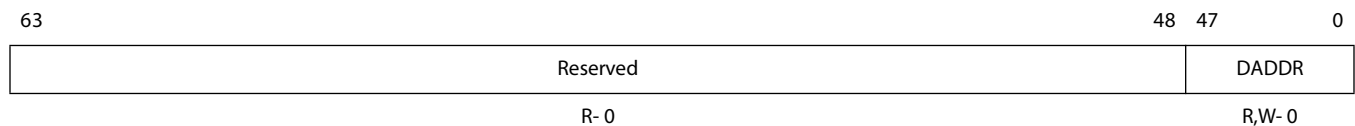
5.1.170 CORE7_SUBMIT_WORD10_11 [Address = 0XC_00E8]

The sixth TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-170 CORE7_SUBMIT_WORD10_11



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-133 CORE7_SUBMIT_WORD10_11 Field Descriptions

Bits	Name	Description
63-48	Reserved	Reserved
47-0	DADDR	Destination transfer address virtual or physical allowed

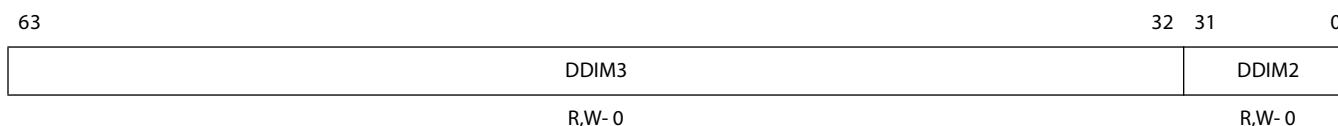
5.1.171 CORE7_SUBMIT_WORD12_13 [Address = 0XC_00F0]

The seventh TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-171 CORE7_SUBMIT_WORD12_13



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-134 CORE7_SUBMIT_WORD12_13 Field Descriptions

Bits	Name	Description
63-32	DDIM3	The third dimension width of the destination data
31-0	DDIM2	The second dimension width of the destination data

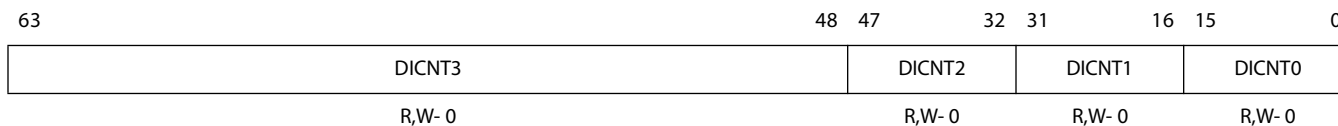
5.1.172 CORE7_SUBMIT_WORD14_15 [Address = 0XC_00F8]

The eight TR submission word

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
RW	RW	RW	RW	RW	RW

Figure 5-172 CORE7_SUBMIT_WORD14_15



Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-135 CORE7_SUBMIT_WORD14_15 Field Descriptions

Bits	Name	Description
63-48	DICNT3	The fourth count of the destination if different than the source
47-32	DICNT2	The third count of the destination if different than the source
31-16	DICNT1	The second innermost count of the destination if different than the source
15-0	DICNT0	The innermost count of the destination if different than the source

5.1.173 CAUSEX [Address = 0XE_0000]

Error Register cause for channels 0 to 15

Access Permissions

Guest User	Guest Supervisor	User	Supervisor	Secure User	Secure Supervisor
R	R	R	R	R	R

Figure 5-173 CAUSE0

63	62	61	60	59	58	57	56
CH15 Local Overflow	CH15 TR Error	CH15 Global1 Overflow	CH15 Global0 Overflow	CH14 Local Overflow	CH14 TR Error	CH14 Global1 Overflow	CH14 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
55	54	53	52	51	50	49	48
CH13 Local Overflow	CH13 TR Error	CH13 Global1 Overflow	CH13 Global0 Overflow	CH12 Local Overflow	CH12 TR Error	CH12 Global1 Overflow	CH12 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
47	46	45	44	43	42	41	40
CH11 Local Overflow	CH11 TR Error	CH11 Global1 Overflow	CH11 Global0 Overflow	CH10 Local Overflow	CH10 TR Error	CH10 Global1 Overflow	CH10 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
39	38	37	36	35	34	33	32
CH9 Local Overflow	CH9 TR Error	CH9 Global1 Overflow	CH9 Global0 Overflow	CH8 Local Overflow	CH8 TR Error	CH8 Global1 Overflow	CH8 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
31	30	29	28	27	26	25	24
CH7 Local Overflow	CH7 TR Error	CH7 Global1 Overflow	CH7 Global0 Overflow	CH6 Local Overflow	CH6 TR Error	CH6 Global1 Overflow	CH6 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
23	22	21	20	19	18	17	16
CH5 Local Overflow	CH5 TR Error	CH5 Global1 Overflow	CH5 Global0 Overflow	CH4 Local Overflow	CH4 TR Error	CH4 Global1 Overflow	CH4 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
15	14	13	12	11	10	9	8
CH3 Local Overflow	CH3 TR Error	CH3 Global1 Overflow	CH3 Global0 Overflow	CH2 Local Overflow	CH2 TR Error	CH2 Global1 Overflow	CH2 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0
7	6	5	4	3	2	1	0
CH0 Local Overflow	CH0 TR Error	CH0 Global1 Overflow	CH0 Global0 Overflow	CH0 Local Overflow	CH0 TR Error	CH0 Global1 Overflow	CH0 Global0 Overflow
R-0	R-0	R-0	R-0	R,-0	R-0	R-0	R-0

Legend: R = Read only; RC = Read to Clear; W = Write only; WS = Write to Set; WC = Write to Clear; -n = value after reset

Table 5-136 CAUSE0 Field Descriptions

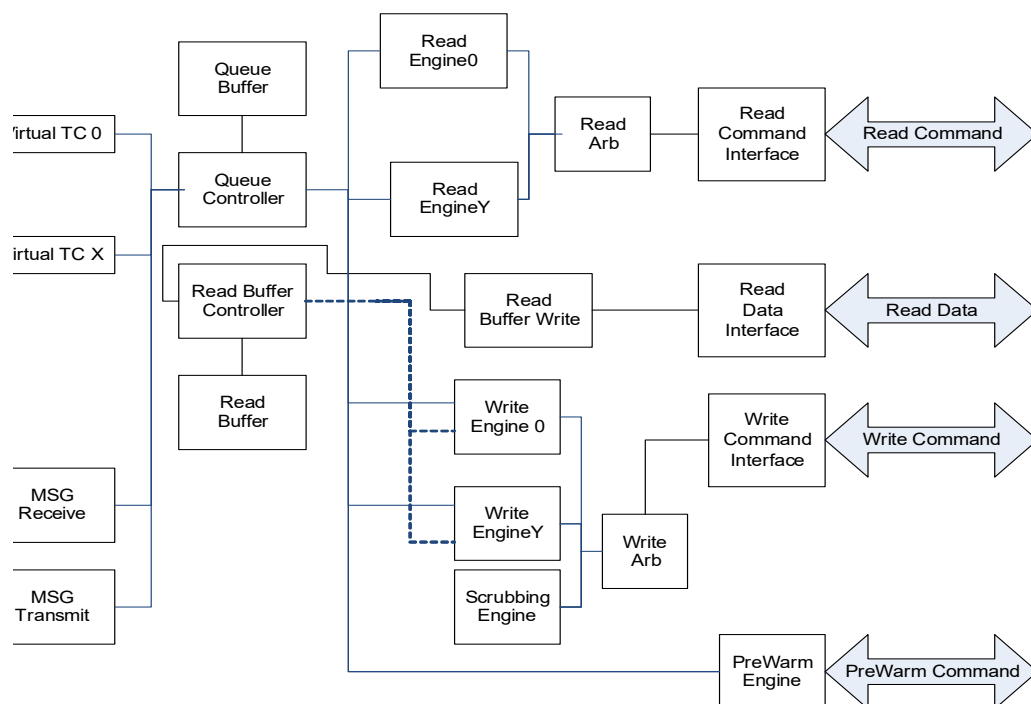
Bits	Name	Description
63	CH15_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 15
62	CH15_TR_ERROR	The last TR ran for Channel 15 received an error
61	CH15_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 15
60	CH15_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 15
59	CH14_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 14
58	CH14_TR_ERROR	The last TR ran for Channel 14 received an error
57	CH14_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 14
56	CH14_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 14
55	CH13_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 13
54	CH13_TR_ERROR	The last TR ran for Channel 13 received an error
53	CH13_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 13
52	CH13_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 13
51	CH12_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 12
50	CH12_TR_ERROR	The last TR ran for Channel 12 received an error
49	CH12_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 12
48	CH12_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 12
47	CH11_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 11
46	CH11_TR_ERROR	The last TR ran for Channel 11 received an error
45	CH11_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 11
44	CH11_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 11
43	CH10_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 10
42	CH10_TR_ERROR	The last TR ran for Channel 10 received an error
41	CH10_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 10
40	CH10_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 10
39	CH9_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 9
38	CH9_TR_ERROR	The last TR ran for Channel 9 received an error
37	CH9_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 9
36	CH9_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 9
35	CH8_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 8
34	CH8_TR_ERROR	The last TR ran for Channel 8 received an error
33	CH8_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflowed for Channel 8
32	CH8_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflowed for Channel 8
31	CH7_LOCAL_OVERFLOW	The local trigger counter has overflowed for Channel 7

Table 5-136 CAUSE0 Field Descriptions

Bits	Name	Description
30	CH7_TR_ERROR	The last TR ran for Channel 7 received an error
29	CH7_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 7
28	CH7_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 7
27	CH6_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 6
26	CH6_TR_ERROR	The last TR ran for Channel 6 received an error
25	CH6_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 6
24	CH6_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 6
23	CH5_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 5
22	CH5_TR_ERROR	The last TR ran for Channel 5 received an error
21	CH5_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 5
20	CH5_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 5
19	CH4_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 4
18	CH4_TR_ERROR	The last TR ran for Channel 4 received an error
17	CH4_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 4
16	CH4_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 4
15	CH3_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 3
14	CH3_TR_ERROR	The last TR ran for Channel 3 received an error
13	CH3_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 3
12	CH3_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 3
11	CH2_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 2
10	CH2_TR_ERROR	The last TR ran for Channel 2 received an error
9	CH2_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 2
8	CH2_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 2
7	CH1_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 1
6	CH1_TR_ERROR	The last TR ran for Channel 1 received an error
5	CH1_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 1
4	CH1_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 1
3	CH0_LOCAL_OVERFLOW	The local trigger counter has overflown for Channel 0
2	CH0_TR_ERROR	The last TR ran for Channel 0 received an error
1	CH0_GLOBAL1_OVERFLOW	The global trigger 1 counter has overflown for Channel 0
0	CH0_GLOBAL0_OVERFLOW	The global trigger 0 counter has overflown for Channel 0

Detailed Micro-Architecture

Figure 6-1 DRU Implementation Block Diagram



6.1 Transfer controllers

EDMA Channel Controller (CC) can directly interact with the Data router unit (DRU) for efficiently transferring the data between the Memory mapped slave endpoints. Typically each EDMA Channel controller can be connected to 2 to 4 EDMA Transfer controllers (TC). EDMA TC is the actual data movement engine and CC submits the Transfer request to TC.

To support more than one TC like interfaces to EDMA CC, DRU has four virtual TC like command queues. These queues have different priority ranging from High to Low. Since DRU mimics the TC structure, EDMA CC can submit Transfer Request (TR), with various priority ranges and DRU assembles the command in the proper queue.

The transfer controllers communicate with the Queue Controller to load the submitted TR into the Queue Buffer. The corresponding Read and Write Engine for the Queue requested by the Virtual TC

6.2 Queue Controller

The Queue controller handles managing all the information for all the queues. It keeps track of which queues are available as well as feeding the current TR requested to each Read and Write Engine. The queue controller also arbitrates between all of the Virtual TCs and the messaging interface. It can do one read of a queue and one write of a queue per cycle. The queue controller will pass a TR queue to the read and write engine if the current Queue is empty. Otherwise, the read and write engine will request the next queue structure.

6.2.1 TR Queues

The DRU has a configurable number of command queues and a configurable queue size. The queue size is shared between each of the command queues and cache warm queues. As soon as the transfer request is completely received, it will be transferred to the queue memory assuming there is space and the MMRs will be available for programming another request. Each queue is assigned with fixed priority with Queue 0's priority > Queue 1 priority > Queue 2 priority > Queue 3 priority. These priorities are used to load the queue memory when multiple command queues are received.

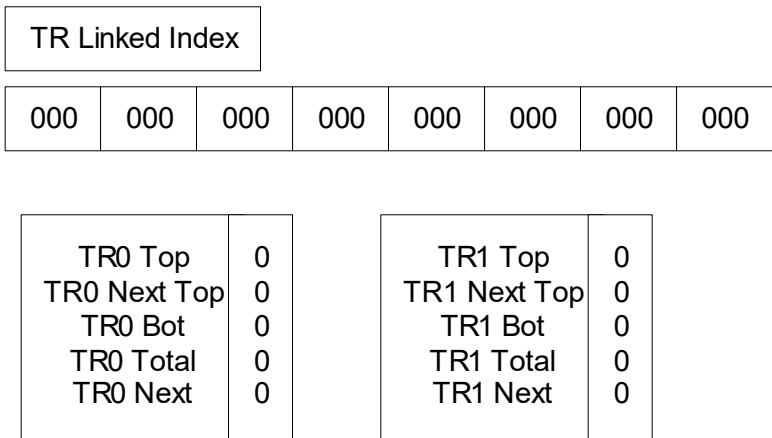
Table 6-1 TR Queue Pointer Format

Field	Size	Description
top	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory to the top of the queue
next top	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory of the second to the top of the queue
bot	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory of the last entry in the queue
total	Number of bits +1 for the size of the queue (7 for 64 TRs)	This is the total number of TRs in the queue
add	1	This signifies that a TR is being added to the queue
next	Number of bits for the size of the queue (6 for 64 TRs)	This is the location of the entry that is being added. It will become the bot the cycle after add is set
remove	1	This signifies that the top of the queue is being removed
End of Table 6-1		

Each entry in queue will be added when a request from the Virtual TC or the MSG interface generate a new TR request. The queue controller will then determine the lowest available queue location and insert the new queue. The queue controller internally keeps track of the linked nature of the queue so that it can generate next_top to the engines as required. The internal linked structure is simply a register vector that is updated anytime a TR is added to a queue.

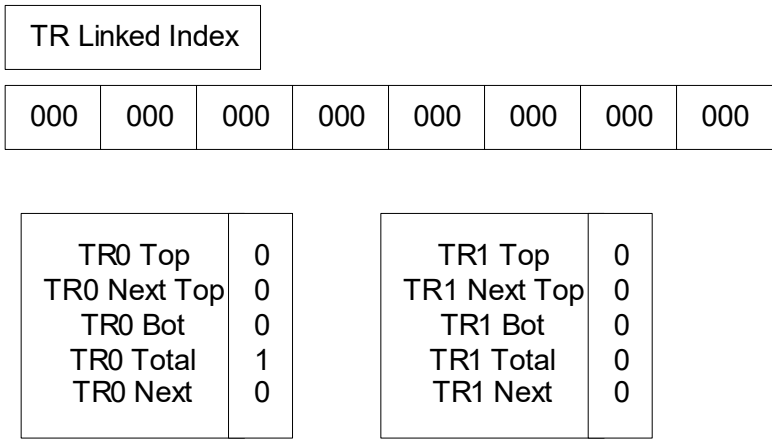
The below figures illustrate the TR linked index structure in a simple DRU with 8 TRs and 2 queues.

Figure 6-2 DRU Queue Pointer Example Reset State



The next figure is after reset a TR is added to TR queue 0 shown as TR0. Since there is not a TR in position 0 the Next for both lists will be 0 and the new TR will be written to location 0 in the queue memory. The structures will be as shown in the figure below.

Figure 6-3 DRU Queue Pointer Example TR added to queue 0



The next figure is after a TR is added to TR queue 1 shown as TR1. The TR will go in queue position 1 since the previous TR is in position 0. Notice how both the top and bottom of TR1 are both update as well as Next Top.

Figure 6-4 DRU Queue Pointer Example TR added to queue 1

TR Linked Index							
000	000	000	000	000	000	000	000

TR0 Top	0	TR1 Top	1
TR0 Next Top	0	TR1 Next Top	1
TR0 Bot	0	TR1 Bot	1
TR0 Total	1	TR1 Total	1
TR0 Next	1	TR1 Next	1

The next another TR is added to queue 0. Notice how Next Top and Bot both get updated with the value of Next as the new TR is written to queue position 2. Also note that the value in TR Linked index for entry 0 is now updated showing that entry zero is followed by entry 2. When a TR is added the TR pointed by bottom if total is greater than 1 will be updated with the next value as well.

Figure 6-5 DRU Queue Pointer Example seconded TR added to queue 0

TR Linked Index							
010	000	000	000	000	000	000	000

TR0 Top	0	TR1 Top	1
TR0 Next Top	2	TR1 Next Top	1
TR0 Bot	2	TR1 Bot	1
TR0 Total	2	TR1 Total	1
TR0 Next	2	TR1 Next	2

Next a third TR is added to queue 0. This time only Bot gets updated as well as the entry of TR linked index is updated with the pointer to the next position in entry 2 since that was the bottom of the list before it was updated.

Figure 6-6 DRU Queue Pointer Example Third TR added to queue 0

TR Linked Index							
010	000	011	000	000	000	000	000

TR0 Top	0	TR1 Top	1
TR0 Next Top	2	TR1 Next Top	1
TR0 Bot	3	TR1 Bot	1
TR0 Total	3	TR1 Total	1
TR0 Next	3	TR1 Next	3

The next state assumes that in the same cycle the first entry in TR0 is removed while a new entry in TR0 is removed. In this case TOP will get the value of Next Top and Next Top will get its value from the TR Linked Indexed pointed to by the current Next Top which in this case is the value in TR Linked Index 2 which is 3. Since the list is being added to a new entry will be written in position 4 which will update bottom and the value in TR Linked Index. The Total will remain the same since a TR is both being added and removed.

Figure 6-7 DRU Queue Pointer Example First TR removed from queue 0 Fourth TR added to queue 0

TR Linked Index							
010	000	011	100	000	000	000	000

TR0 Top	2	TR1 Top	1
TR0 Next Top	3	TR1 Next Top	1
TR0 Bot	4	TR1 Bot	1
TR0 Total	3	TR1 Total	1
TR0 Next	4	TR1 Next	4

The next state assumes that a new TR is added to queue 1. Since position 0 is now open then position 0 will be used again. Since the Total in queue 1 is going from 1 to 2 the TR1 Next Top will be updated with the new bottom value as well.

Figure 6-8 DRU Queue Pointer Example Second TR added to Queue 1

TR Linked Index							
010	000	011	100	000	000	000	000

TR0 Top	2	TR1 Top	1
TR0 Next Top	3	TR1 Next Top	0
TR0 Bot	4	TR1 Bot	0
TR0 Total	3	TR1 Total	2
TR0 Next	0	TR1 Next	0

The final state in the example is when a TR is added to queue 0 and removed from queue 1. In this case the TR0 Bot will be updated as well as the TR Linked index for the previous Bot value. In TR1 the Top will change and the Next Top. The Next Top will change but the value is a DON'T CARE since the Total is now 1 and with the next removal the list will be empty.

Figure 6-9 DRU Queue Pointer Example Fifth TR added to queue 0 and First TR removed from queue 1

TR Linked Index							
010	000	011	100	101	000	000	000

TR0 Top	2	TR1 Top	0
TR0 Next Top	3	TR1 Next Top	2
TR0 Bot	5	TR1 Bot	0
TR0 Total	4	TR1 Total	1
TR0 Next	5	TR1 Next	5

The queue controller is only concerned on if data is in the queue memory. The format of the TR in the queue memory is based on the needs of the Read and Write engines so the format of the TR in the queue is described in the same section. The Read and Write engine will receive the add signal from the queue controller and the Write engine will return the remove. The Read Engine will contain its own queue pointer structure to allow it to remove TRs that the reads have completed.

6.2.2 Cache Warm Queue

The Cache Warm queue is used by the messaging interface and the CorePacs to allow for ordering of Cache Warm messages to be sent to any of the Coherent masters in the system. As soon as the warm request is completely received, it will be transferred to the queue memory. The Cache Warm requests will have a higher priority than the command queue to write into the queue memory. The CacheWarm queue is unique from other queues in that it only interfaces with the Write Command Interface and the Queue controller

As far as the Queue controller is concerned the Cache warm queue is handled identical to the TR Command Queues. The same pointer structure and everything is used. The only difference in the Cache Warm Queue is the format of the data that is packed into the Queue memory is the data required by the Cache Warm Engine. The format of the data will be described in the Cache Warm engine section below.

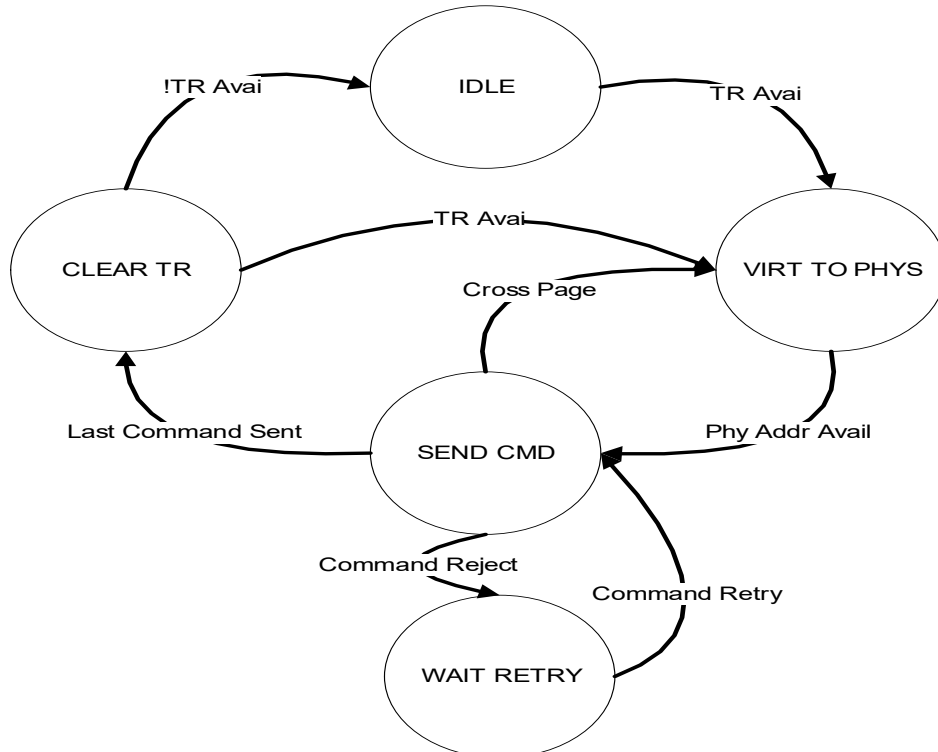
6.3 Read Engine

The Read Engine is responsible for handling a queue of TRs and issuing the read commands as specified by the TR. The read engine has four main functions the TR handling, the address calculation, the size calculation and the virtual to physical translation through the uTLB interface.

6.3.1 Read Engine TR Handling

The TR Handling in the Read Engine is managed by the simple state machine below.

Figure 6-10 TR Handling State Machine Read Engine



The table below explains the states and conditions of the state machine.

Table 6-2 Read Engine TR Handling State Machine Description

State or Condition	Description
IDLE State	This is the reset state or whenever the Read Engine is not performing any function.
VIRT TO PHY State	This is state when the Read Engine is waiting for the response from the uTLB or it is loading all of the attributes for the command to be sent to the Read Command Interface.
SEND CMD State	This is when the Line Counter and Address Calculation are active and it will be issuing a command to the Read Command Interface block.
WAIT RETRY	Since the Read Engine does not know what other modules are requesting and what resources are available this state allows the Read Command Interface block to halt commands from the read engine. If conditions change the Read Command Interface block will issue a command retry and commands will be sent again.
CLEAR TR State	This is the state when the TR is removed from the Read Queue. If another TR is in the Read Queue the next transaction will begin.
TR Avail	This is looking at the Total count for the read queue and is set if the Total count is not 0.
Phy Addr Avail	This is set if either the uTLB has returned the response to the request or if a new TR is a physical transaction then it will be set the next cycle.
Cross Page	This is if the field of the Next Address do not match the Page Address which is loaded with the uTLB. The size of the page will be provided by the uTLB so the number of bits that are compared will vary based on the page size. This allows for support of page crossing both in a burst or across a line.
Last Command Sent	This is set if the ACNT == Byte Cnt and BCNT == 1 or 0 which signifies that the last line is being transferred
Command Reject	The read command interface has rejected the command. The address and counters should not be issues and the command can be resent after the Command Retry has been sent.
Command Retry	The previous command can be sent again to see if the proper resources are now available.
End of Table 6-2	

6.3.1.1 Read Engine Size Calculation

It has two internal counters that it keeps track of the ACNT and BCNT. The ACNT is used to count the number of bytes in a line and then every time it is reset to the original value the BCNT is decremented.

The read engine is optimized for accesses on a 128 byte boundary. As such it will ensure that no command crosses a 128 byte boundary. It uses this requirement in the size calculation. The table below shows the Size Calculation functions.

Table 6-3 Size Calculation Functions

Count Condition	Calculation
New Line	Byte Cnt0 = 128 - Current Address[7:0]
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Byte Cnt1
During Burst	Byte Cnt0 = 128
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Cyte Cnt1
ACNT != ByteCnt	ACNT = ACNT - ByteCnt
ACNT == ByteCnt	ACNT == Original ACNT
	BCNT == BCNT - 1
	Last Command Sent = BCNT == 1 BCNT == 0
End of Table 6-3	

6.3.1.2 Read Engine Address Calculation

The read engine has two addresses that it must keep track of the next line address and the next burst address. The address calculations have three states where they have unique calculations. The three states are when a new TR is loaded into the Read Engine, the end of a burst when ACNT > byte cnt, and when ACNT == Byte Count which is the end of the Line. Depending on the current ACNT value in the read engine will select which of the two addresses will be used. The table below shows how Current Address and Next Line Address are calculate for the three conditions.

Table 6-4 Address Calculation Functions

Address Condition	Calculation
New TR	Current Address = TR Address
	Next Line Address = TR Address + TR SRCDIM
End of Burst	Current Address = Current Address + Byte Count
	Next Line Address = Next Line Address
End of Line	Current Address = Next Line Address
	Next Line Address = Next Line Address + SRCDIM
End of Table 6-4	

6.3.1.3 Virtual to Physical Translation

The Virtual to Physical translation function in the read engine works with the uTLB to update all of the parameters that can be specified by the MMU and pass them to the Read Command Interface module.

Upon entering the VIRT TO PHY state in the TR Handling state machine the Read Engine will wait until either the uTLB request is returned or it determines that the TR is a physical TR.

If the uTLB returns any type of fault the Read Engine will signal an error and mark the read complete field for the TR to allow the write engine to properly complete any outstanding transactions and clear them from the buffer.

The table below shows the memory type attributes that are received from the uTLB interface and the value of those signals if the TR is a physical TR.

Table 6-5 Memory Type Attributes

Signal		Physical TR
cmemtype	pg_mem_type[1:0]	0 (Device Memory Type)
ccinner	pg_mem_type[3:2]	0 (Non-allocatable)
ccouter	pg_mem_type[5:4]	0 (Non-allocatable)
csdomain	pg_mem_type[7]	0 (Don't Care since it won't be shareable)
shareable	pg_mem_type[7:6] != 0	0 (Not Shareable)
cpable	pg_mem_type[8]	0 (Non-prefetchable)
End of Table 6-5		

6.4 Read Arb

The command queues follow a dynamic priority scheme for reads that attempts to fairly allocate the number of access opportunities and bandwidth among the requestors. There is core fair-share arbitration logic for the command queues. Every queue is allowed to send only one access in a cycle to the arbitration logic, and the scheme determines who will be the winner.

The winner is selected based on the queue priority, so queue 0 will have the higher priority than other queues. But when there is a heavy traffic on Queue 0 that may make the other queues to indefinitely starve for the access. To address this every queue has a starvation bound register. They can be programmed to some value in terms of MSMC cycles. The starvation register also has a minimum cycles count. This will allow for a queue to send a small burst of commands when starvation is reached.



Note—The starvation counters are only decremented when a queue is presenting a command. So if a queue is blocked from sending commands because of a Reject from the Read Block Interface because of a lack of resources such as credits to complete the transaction the time spent in the WAIT RETRY state will NOT count towards starvation.

DRU has a starvation bound counter for each of the queues. The DRU also has a minimum cycle count. The starvation counter is decremented every cycle that a request is presented to the arbiter. Every time the queue wins arbitration the minimum cycle count is decremented by 1. When the minimum cycle count reaches zero then the starvation count and minimum cycle count for the queue are reset to the values in the Memory Mapped Registers. When the starvation counter for a queue reaches zero, the access presented from that queue will be arbitrated in the starvation arbitration logic.

The starvation arbitration has the same fixed arbitration used in the regular arbitration the only difference is that only queues that have a starvation count of zero will be seen as having a request in the starvation arbitration.

The final arbitration is that if there is a request on the starvation arbitration it beats a request for the starvation arbitration.

See the appendix on the DRU Memory Mapped Registers for the fields of the registers for the arbitration.

6.5 Read Command Interface

The Read Command Interface handles converting the general read request from the winning read engine to the specific protocol of the interface being used on the external read bus. This is the one of the few modules that should have to change if the external read bus changes. The general read format assumes that read commands and read data are separate and that the access will not cross a 128 byte boundary. Logic can be added in this module for any mapping or control state machine required. The sub-sections below discuss the current implemented Read Command Interface modules.

6.5.1 MSMC

The MSMC interface is based off of the MBA specification with minor changes required to simplify the coherency information. The MSMC interface is intended to be a direct connect of the signals and introduces no additional cycles to the initial request time.

The table below shows the MBA signals and how they are generated.

Table 6-6 DRU to MBA TAC Mapping

DRU Signal/Logic	MBA
Transaction Presentt	cvalid
See credit function	ccredit
read_buffer_available	cid
ReadNoSnoop	ctype
Tied to DRU cmstid tieoff/parameter	cmstid
Tied to 0 as this is read only port	cdata
pri	cpriority
address	caddress
cmemtype	cmemtype
cpable	cpable
ccinner	ccinner
ccouter	ccouter
csdomain	csdomain
bytecnt	cbytecnt
Tied to 0 as this is not an emulation port	cemudbg
secure	csecure
priv	cpriv
End of Table 6-6	

The internal MSMC interface uses a valid and taken pairing so a command is held until taken is used. This means that the reject and retry signals back to the read engine will be tied low.

Writer's Comment (6.1): The credit function will be fleshed out once the MSMC credit scheme is finalized. Currently it is assumed that a single credit type exists

6.6 Read Data Interface

The Read Data Interface module handles the protocol specific information to convert the specific read response data to the internal read buffer write. The read buffer write assumes a 512 bit interface plus 20 bits of parity and the memory address based on which response is being received.

Table 6-7 Read Data Interface Signals

Signal	Description
Req	Write the data into the buffer at the specified location.
Address	The address in the read buffer that should be written
Data	The 512 bits of data that should be written
Parity	The 20 bit parity for 2 256 bit words
Byten	The byte enables for the write
Confirm	Confirm that the response is complete
End of Table 6-7	

The sub-sections below discuss the current implemented Read Data Interface modules

6.6.1 MSMC

The Read Data Interface for MSMC is a simple mapping of an MBA TAC and MBA TDC. It assumes that the command and data are always received in the same cycle and it will respond with a taken when the data has been moved into the pipeline to be read.

The parity field is taken directly from the MSMC bus. If a partial word is received then a read of the previous parity will be done and the parity will be XORed with the new parity. The final parity in the memory will always be for the full 512 bit word and it will be checked before going into the write engine.

6.6.1.0.0.1 Read Status Errors

When read data is returned, the return status bus is monitored for error conditions (rstat non-zero). If an error occurs, this will cause the Read Data Interface to signal an error received and the type of error will also be sent to the write engine.

The read data is stored in the data buffer and will be written to the destination. This data may be invalid as dictated by the specific error type.

6.7 Read Buffer Control

The read buffer control is responsible for handling the read buffer management. It communicates with the read command interface to know the available memory locations. It also communicates with the write engine providing the received flag for the specific received flag for the next required data buffer. It also handles arbitrating the read requests from the write engine and doing the parity check and correction before providing the data to the write engine.

6.8 Data buffer

DRU has an 8KB buffer to temporarily store the read return data from the memory mapped slave. The data buffer is organized as eight banks of 128 bits, with 64 entries per bank. So structure looks like a 64x128 Byte RAM. Buffer has separate read and write ports. Each request is treated as going to one of the 64 Byte Buffers in the RAM. The RAM additionally has extra bits to store the parity bits which are stored with write requests.

The read buffer controller interfaces to the RAM banks and arbitrates between the various read requests from the write engines.

6.9 Read Command Interface Logic

The read command interface is required to know the data buffer location to use for each command. The read buffer controller uses the same linked list structure as the queue to determine which buffer to use. It always uses the smallest pointer to a buffer that is available. When the command is taken a flag indicating that the read command has been sent for the buffer is set and will remain set until the write engine uses the buffer after it has been received.

6.10 Write Engine Data Available Logic

The write engine requires to know when data has been received in the return buffer. The write engine will always be waiting for data in the order they were sent by the matching read engine and will maintain a linked list of the data pointers that it is expecting. The write engine will provide the buffer id of the next location that it is expecting and the read buffer control will provide a signal when the data is available.

6.10.1 Read Data Interface Logic

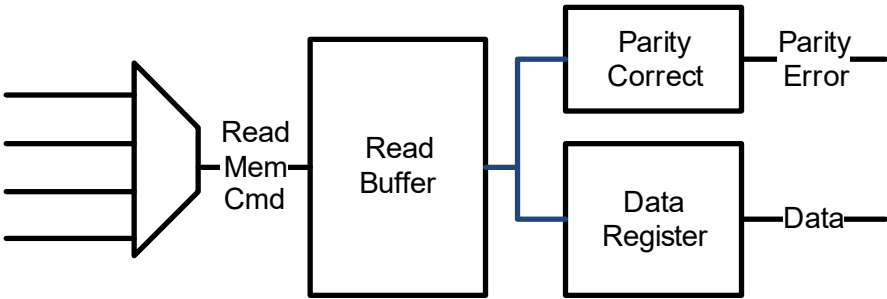
The read data interface logic will communicate with the Read Buffer Control when data is received. It will let the read buffer control know when data the transfer is complete and for the id that the transfer that is complete. Upon receiving the information that the transfer is complete the Read Buffer Controller will set the data received flag to notify any write engine waiting for that buffer that the data is now available in the queue.

Additionally the read buffer controller will look at each access and determine if it is a partial word access then it will initiate a read to XOR the contents of the parity with the previous received parity. The read of the parity bits will have the highest priority and could stall a read from a write engine.

6.10.2 Write Engine Read Data Logic

The Read Buffer Control arbitrates the data requests from the write engines by using a simple fixed priority scheme. Since a write engine will only request to read data when it is available and the starvation counters on the read arbitration have ensured that a single write engine will not take all of the bandwidth a simple arbitration scheme is used. After reading from the winning location the parity stored in the memory is checked with a calculated parity and the SECDEC function will be performed on the data. This allows for a centralized location for the parity correction logic as well as allowing to delay checking of the parity on the response until the data is used. The read buffer control will present the data bus and a status field to the write engine when the parity logic is complete.

Figure 6-11 Write Engine Read Data Logic Functional Diagram



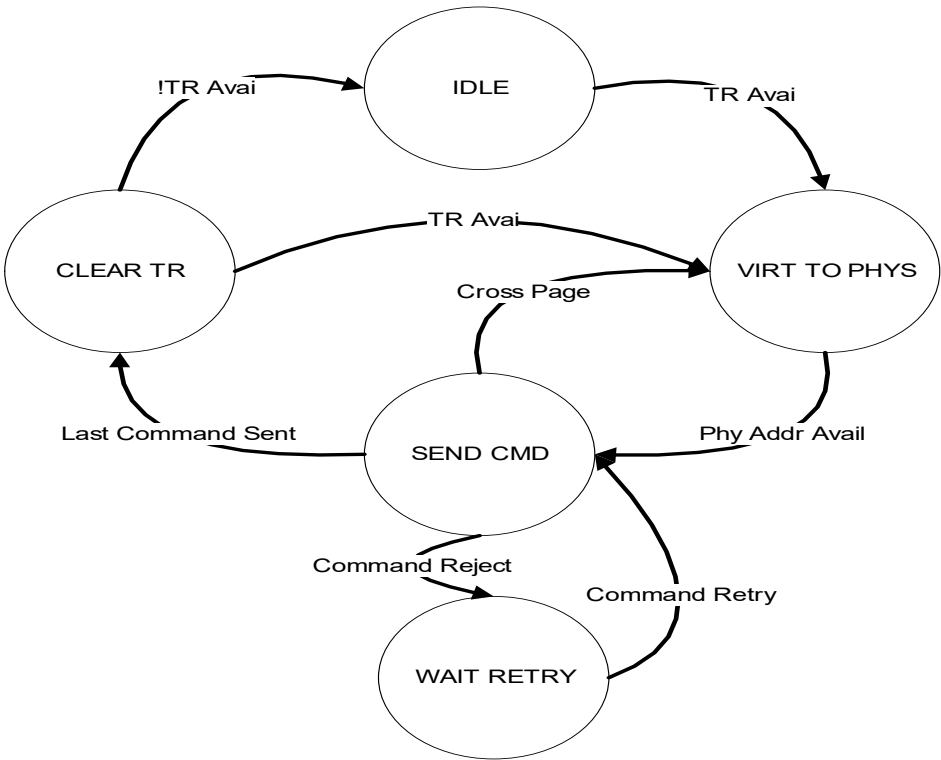
6.11 Write Data Engine

The Write Data Engine is responsible for sending the write commands for the matching read engine. The write engine has six main functions the TR handling, the address calculation, the size calculation, the virtual to physical translation through the uTLB interface, the data manipulation and the TR completion handling.

6.11.1 Write Engine TR Handling

The TR Handling in the Write Engine is managed by the simple state machine below.

Figure 6-12 TR Handling State Machine Write Engine



The table below explains the states and conditions of the state machine.

Table 6-8 Write Engine TR Handling State Machine Description

State or Condition	Description
IDLE State	This is the reset state or whenever the Write Engine is not performing any function.
VIRT TO PHY State	This is state when the Write Engine is waiting for the response from the uTLB or it is loading all of the attributes for the command to be sent to the Write Command Interface.
SEND CMD State	This is when the Line Counter and Address Calculation are active and it will be issuing a command to the Write Command Interface block when ever sufficient data is available as determined by the data shifter.
WAIT RETRY	Since the Write Engine does not know what other modules are requesting and what resources are available this state allows the Write Command Interface block to halt commands from the write engine. If conditions change the Write Command Interface block will issue a command retry and commands will be sent again.
CLEAR TR State	This is the state when the TR is removed from the Queue Memory. If another TR is in the Queue the next TR will be loaded.
TR Avail	This is looking at the Total count for the queue and is set if the Total count is not 0.
Phy Addr Avail	This is set if either the uTLB has returned the response to the request or if a new TR is a physical transaction then it will be set the next cycle.
Cross Page	This is if the field of the Next Address do not match the Page Address which is loaded with the uTLB. The size of the page will be provided by the uTLB so the number of bits that are compared will vary based on the page size. This allows for support of page crossing both in a burst or across a line.
Last Command Sent	This is set if the ACNT == Byte Cnt and BCNT == 1 or 0 which signifies that the last line is being transferred
Command Reject	The write command interface has rejected the command. The address and counters should not be issues and the command can be resent after the Command Retry has been sent.
Command Retry	The previous command can be sent again to see if the proper resources are now available.
End of Table 6-8	

6.11.2 Write Engine Size Calculation

It has two internal counters that it keeps track of the ACNT and BCNT. The ACNT is used to count the number of bytes in a line and then every time it is reset to the original value the BCNT is decremented.

The write engine is optimized for accesses on a 128 byte boundary. As such it will ensure that no command crosses a 128 byte boundary. It uses this requirement in the size calculation. The table below shows the Size Calculation functions.

Table 6-9 Size Calculation Functions

Count Condition	Calculation
New Line	Byte Cnt0 = 128 - Current Address[7:0]
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Byte Cnt1
During Burst	Byte Cnt0 = 128
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Cyte Cnt1
ACNT != ByteCnt	ACNT = ACNT - ByteCnt
ACNT == ByteCnt	ACNT == Original ACNT
	BCNT == BCNT - 1
	Last Command Sent = BCNT == 1 BCNT == 0
End of Table 6-9	

6.11.3 Write Engine Address Calculation

The write engine has two addresses that it must keep track of the next line address and the next burst address. The address calculations have three states where they have unique calculations. The three states are when a new TR is loaded into the Write Engine, the end of a burst when ACNT > byte cnt, and when ACNT == Byte Count which is the end of the Line. Depending on the current ACNT value in the read engine will select which of the two addresses will be used. The table below shows how Current Address and Next Line Address are calculate for the three conditions.

Table 6-10 Address Calculation Functions

Address Condition	Calculation
New TR	Current Address = TR Address
	Next Line Address = TR Address + TR SRCDIM
End of Burst	Current Address = Current Address + Byte Count
	Next Line Address = Next Line Address
End of Line	Current Address = Next Line Address
	Next Line Address = Next Line Address + SRCDIM
End of Table 6-10	

6.11.4 Virtual to Physical Translation

The Virtual to Physical translation function in the write engine works with the uTLB to update all of the parameters that can be specified by the MMU and pass them to the Write Command Interface module.

Upon entering the VIRT TO PHY state in the TR Handling state machine the Write Engine will wait until either the uTLB request is returned or it determines that the TR is a physical TR.

If the uTLB returns any type of fault the Write Engine will signal an error and mark the read complete field for the TR to allow the write engine to properly complete any outstanding transactions and clear them from the buffer.

The table below shows the memory type attributes that are received from the uTLB interface and the value of those signals if the TR is a physical TR.

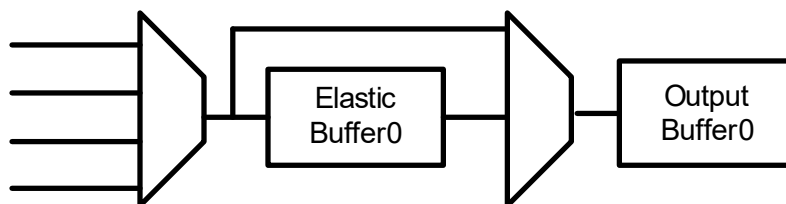
Table 6-11 Memory Type Attributes

Signal		Physical TR
cmemtype	pg_mem_type[1:0]	0 (Device Memory Type)
ccinner	pg_mem_type[3:2]	0 (Non-allocatable)
ccouter	pg_mem_type[5:4]	0 (Non-allocatable)
csdomain	pg_mem_type[7]	0 (Don't Care since it won't be shareable)
shareable	pg_mem_type[7:6] != 0	0 (Not Shareable)
cpable	pg_mem_type[8]	0 (Non-prefetchable)
End of Table 6-11		

6.11.5 Write Engine Data Manipulation

The write engine data manipulation looks at the data received from the read requests and reformats it to match the requirements of the destination request. The address calculation logic and the byte count logic will determine the amount and location of the data to send while the data manipulation logic will determine when sufficient data has been received to send the command.

Figure 6-13 Write Data Engine Shifter Structure



The figure above shows the basic structure of the data shifter. There are two buffers equal to the width of the memory. A mux in front of the buffer can select any byte from the data input from the read buffer controller and place it in any byte location of the read. Additionally this word can also be loaded into the output buffer. The output buffer will take either the input to the elastic buffer or the output of the elastic buffer. This allows for storage of data that is not required for the current required output and use it with the next data.

The only function supported by the write data engine is a simple shift based on the address differences between the write address and the read address. Therefore the selection of the muxes is controlled by simply comparing the read address and write address of the current block.

Table 6-12 Shifting Function

Term	Function
shift	write_address - read_address. This is a 6bit subtracter for 64 byte data width that rolls over for negative case.
read_address	read address is the lower 6bits of the read address from the TR and is updated every line by adding the lower 6bits of dimension to know the 128 byte alignment of the line
write_address	write address is the lower 6 bits of the write address from the TR and is updated with the lower 6 bits of next_line_address from the address calculation so the adder does not have to be replicated
load	same as shift since the shift will determine which words are in the current word and which are in the next
read_data_avail	128-read_address if new line otherwise it is 128 bytes.
data_avail	data_avail + read_data_avail cleared on a new line
write_cmd_avail	data_avail > Byte Count

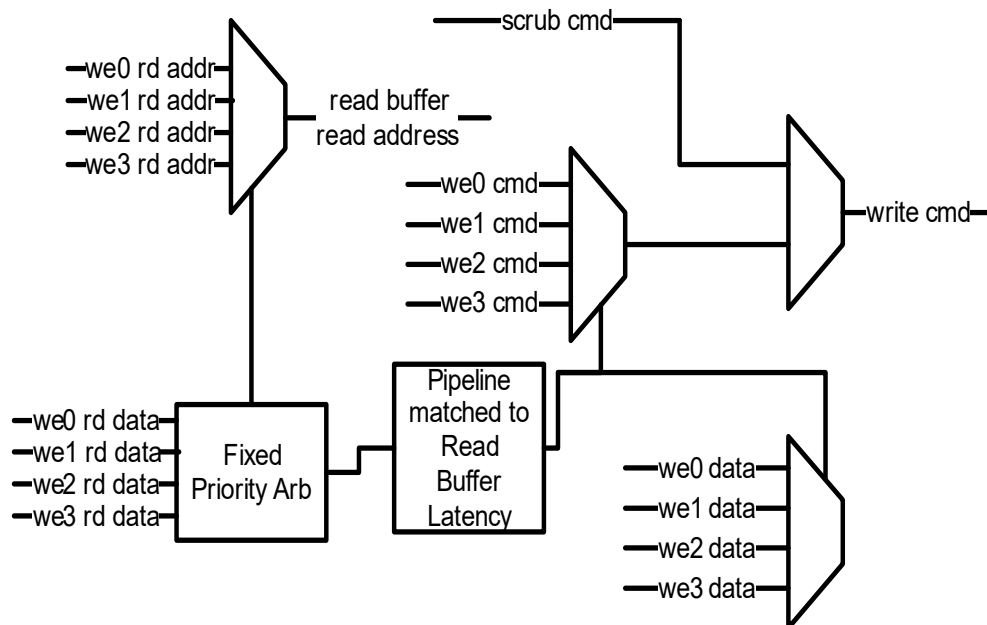
6.11.6 TR Completion Handling

The Write Engine is also responsible for sending out the completion response. The completion response engine looks at the TR Handling engine and when it sends the last command it waits until the response is received for the specified write command and then will send a command to the MSG Transmit interface to send the completion code for the TR and clear the TR from the queue. The TR completion engine will update the TR status register anytime an error is received. The status registers are stored in the queue controller and will be cleared on a new TR being received and only the first error received will be set. If a read and write error are received at the same time the read error will be stored.

6.12 Write Arbitration

The write arbitration is actually pre-scheduled on the data fetches to the read buffer control. Once a write engine wins arbitration to the read buffer control it will hold the arbitration until it completes the reads and send out as many commands as required. The selection for the muxes is a pipeline delay of the arbitration to the read buffer control. The write arbitration logic to the read buffer control is a simple fixed arbitration with write engine 0 being highest priority and write engine 1 being lower and so forth. Since a write command can only be sent every other cycle in most cases the write command port is shared with the PreWarming Cache and the Scrubbing Engine. Since both of these only issue commands that do not require data they will arbitrate separately and if there is no write command that phase then the command arbitration will go to the seperate functions. A representation of this scheme can be seen in the figure below.

Figure 6-14 Write Arb Logic



6.12.1 Write Controller

The Write Controller is responsible for issuing optimally sized write command. The data buffer state dictates when a write command can be scheduled. In addition, the write controller must wait to issue any new write command until the final data phase of a write in progress.

The Write Controller is also responsible for issuing the scrubbing commands and the messages for cache warming

6.12.2 Space Occupied Rules

The Write Controller must (conceptually) track the amount of data occupied in the data buffer. The write controller can have a counter initialized to a value of zero. The counter value can be incremented when read return data lands in the data buffer. Every time a write command is issued the counter value is decremented according to the amount of data that has been scheduled to be written. The Write Controller can only issue an optimally sized write command when there is that amount of data available in the buffer. The write command and data should be available at the same time to get the write command going, so DRU will hold a write command until all its data is available.

6.12.3 Outstanding write commands for a TR

Based on the aforementioned rules there will never be more than 2 outstanding write commands relative to write data. This does not take into account write status. The number of outstanding write commands will be based on the credit availability and also the data availability.

6.12.4 Write Status Errors

When write status is returned, the wstatus bus is monitored for error conditions (wstat non-zero). If an error occurs, this will cause the DRU error register BUSERR bit to be set and will result in the DRU error register being set to the value detected on the wstatus bus along with the TCC, TCINTEN and TCCHEN settings for that transfer. No further action is taken, if the command is still in progress it continues to be serviced as normal.

6.13 Write Command Interface

The Write Command Interface handles converting the general write request from the winning write engine to the specific protocol of the interface being used on the external write bus. This is the one of the few modules that should have to change if the external write bus changes. The general write format assumes that read commands and write data are separate and that the access will not cross a 128 byte boundary. Logic can be added in this module for any mapping or control state machine required. The sub-sections below discuss the current implemented Write Command Interface modules.

6.13.1 Error Behavior

The DRU has different behavior if an error occurs for a TR. In general the behavior should be to notify the system through an interrupt and continue on to the next TR in the respected queue. In all cases and error in a TR in one queue will not effect another queue. The DRU offers a recovery mechanism to flush a TR which will abort the current write TR. It will wait until all outstanding read requests for the queue are complete before completing the flush sequence.

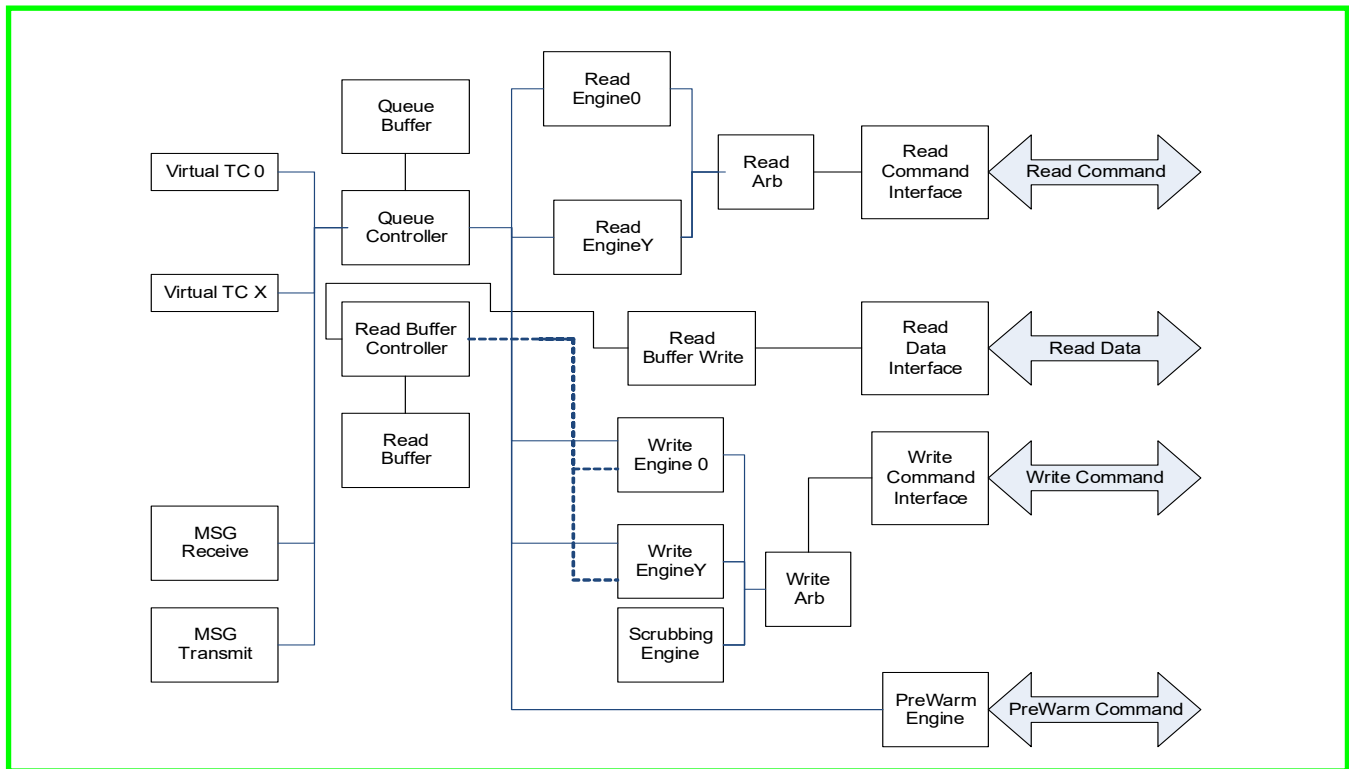
6.13.2 Detailed Functional Description

6.13.2.1 Configurable Parameters

Table 6-13 DRU Configurable Parameters

Parameter	Possible Values	First Implementation
Number of Queues	1-8	4
Number of TRs	1 or greater	64
Number of Outstanding commands	1 or greater	64
Read Bus Protocol	MSMC	MSMC
Write Bus Protocol	MSMC	MSMC
Number of Messaging Queues	1 or greater	1
End of Table 6-13		

Figure 6-15 DRU Functional Block Diagram



6.13.2.2 Virtual Transfer controllers

EDMA Channel Controller (CC) can directly interact with the Data router unit (DRU) for efficiently transferring the data between the Memory mapped slave endpoints. Typically each EDMA Channel controller can be connected to 2 to 4 EDMA Transfer controllers (TC). EDMA TC is the actual data movement engine and CC submits the Transfer request to TC.

To support more than one TC like interfaces to EDMA CC, DRU has four virtual TC like command queues. These queues have different priority ranging from High to Low. Since DRU mimics the TC structure, EDMA CC can submit Transfer Request (TR), with various priority ranges and DRU assembles the command in the proper queue.

The Virtual Transfer controllers communicate with the Queue Controller to load the submitted TR into the Queue Buffer. The corresponding Read and Write Engine for the Queue requested by the Virtual TC will then process the TR when it reaches the front of the queue.

6.13.2.3 Queue Controller

The Queue controller handles managing all the information for all the queues. It keeps track of which queues are available as well as feeding the current TR requested to each Read and Write Engine. The queue controller also arbitrates between all of the Virtual TCs and the messaging interface. It can do one read of a queue and one write of a queue per cycle. The queue controller will pass a TR queue to the read and write engine if the current Queue is empty. Otherwise, the read and write engine will request the next queue structure.

6.13.2.3.1 MMR/MSG to Queue Mapping

The Queue controller handles mapping of the received TRs based on the type and priority of the TR. A Pre-Warm message will go into the Pre-Warm queue. All other TRs are mapped to a queue based on the upper 2 bits of the 3 bit priority field in the TR. With a 0 in the upper two bits mapping to Queue 0 and the other queues mapped by the equivalent value in the upper 2 bits of the priority field.

6.13.2.3.2 TR Queues

The DRU has a configurable number of command queues and a configurable queue size. The queue size is shared between each of the command queues and cache warm queues. As soon as the transfer request is completely received, it will be transferred to the queue memory assuming there is space and the MMRs will be available for programming another request. Each queue is assigned with fixed priority with Queue 0's priority > Queue 1 priority > Queue 2 priority > Queue 3 priority. These priorities are used to load the queue memory when multiple TRs are received.

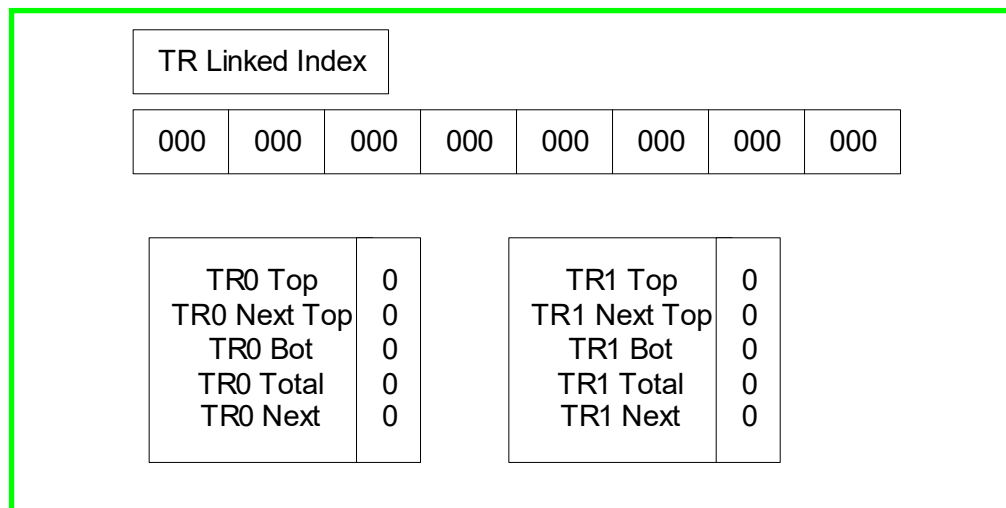
Table 6-14 TR Queue Pointer Format

Field	Size	Description
top	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory to the top of the queue
next top	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory of the second to the top of the queue
bot	Number of bits for the size of the queue (6 for 64 TRs)	This is the location in the queue memory of the last entry in the queue
total	Number of bits + 1 for the size of the queue (7 for 64 TRs)	This is the total number of TRs in the queue
add	1	This signifies that a TR is being added to the queue
next	Number of bits for the size of the queue (6 for 64 TRs)	This is the location of the entry that is being added. It will become the bot the cycle after add is set
remove	1	This signifies that the top of the queue is being removed
End of Table 6-14		

Each entry in queue will be added when a request from the Virtual TC or the MSG interface generate a new TR request. The queue controller will then determine the lowest available queue location and insert into the new queue. The queue controller internally keeps track of the linked nature of the queue so that it can generate next_top to the engines as required. The internal linked structure is simply a register vector that is updated anytime a TR is added to a queue.

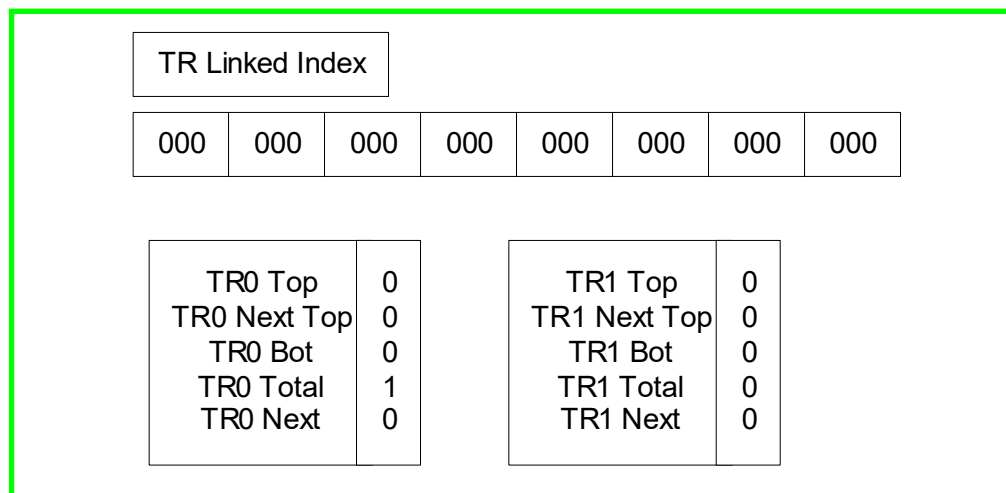
The below figures illustrate the TR linked index structure in a simple DRU with 8 TRs and 2 queues.

Figure 6-16 DRU Queue Pointer Example Reset State



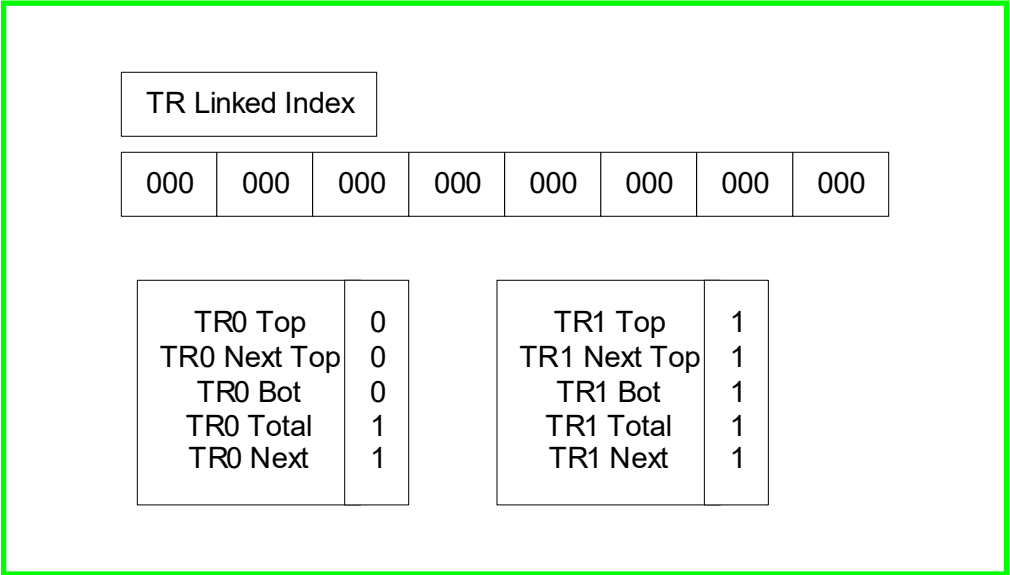
The next figure is after reset a TR is added to TR queue 0 shown as TR0. Since there is not a TR in position 0 the Next for both lists will be 0 and the new TR will be written to location 0 in the queue memory. The structures will be as shown in the figure below.

Figure 6-17 DRU Queue Pointer Example TR added to queue 0



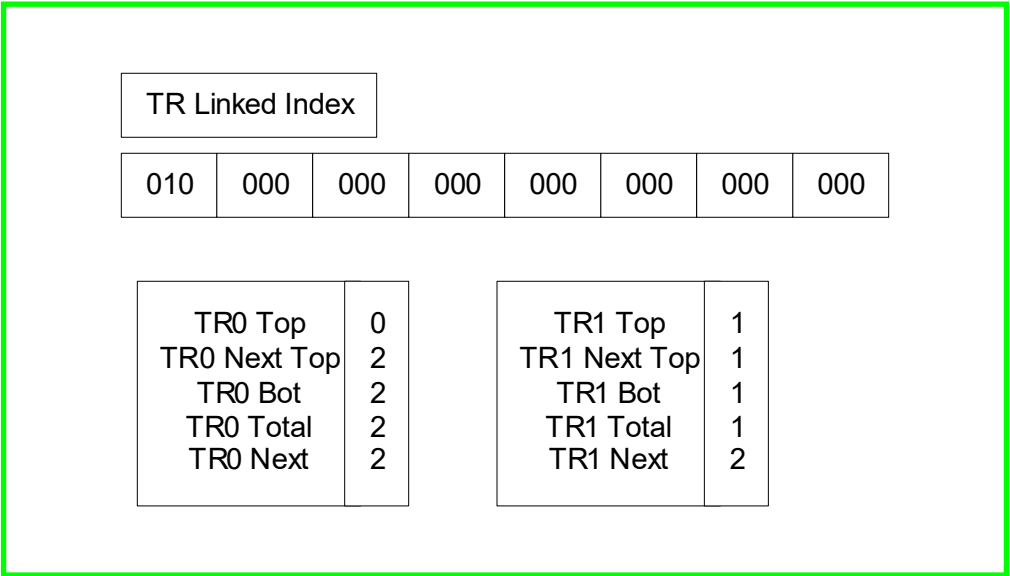
The next figure is after a TR is added to TR queue 1 shown as TR1. The TR will go in queue position 1 since the previous TR is in position 0. Notice how both the top and bottom of TR1 are both update as well as Next Top.

Figure 6-18 DRU Queue Pointer Example TR added to queue 1



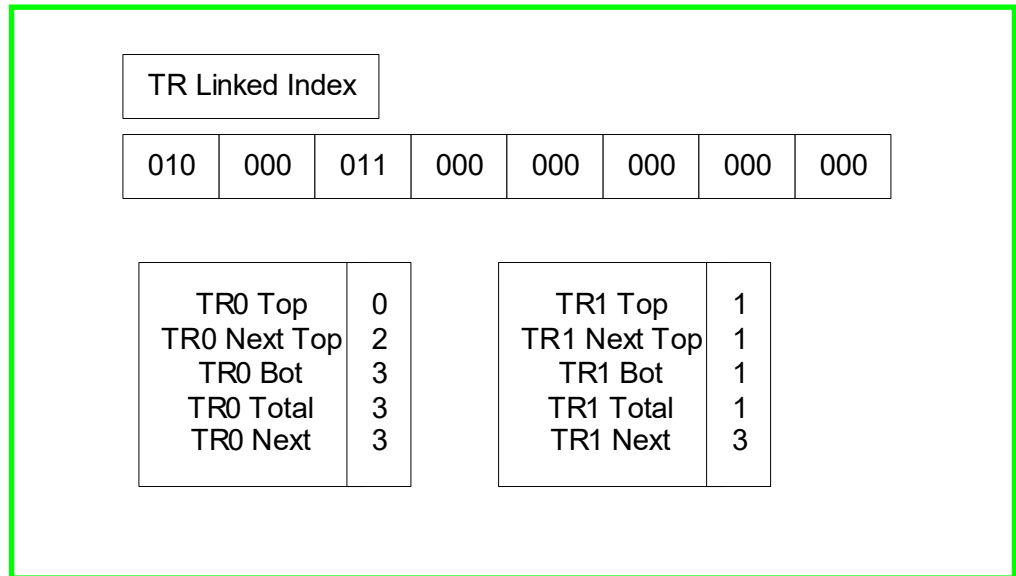
The next another TR is added to queue 0. Notice how Next Top and Bot both get updated with the value of Next as the new TR is written to queue position 2. Also note that the value in TR Linked index for entry 0 is now updated showing that entry zero is followed by entry 2. When a TR is added the TR pointed by bottom if total is greater than 1 will be updated with the next value as well.

Figure 6-19 DRU Queue Pointer Example seconded TR added to queue 0



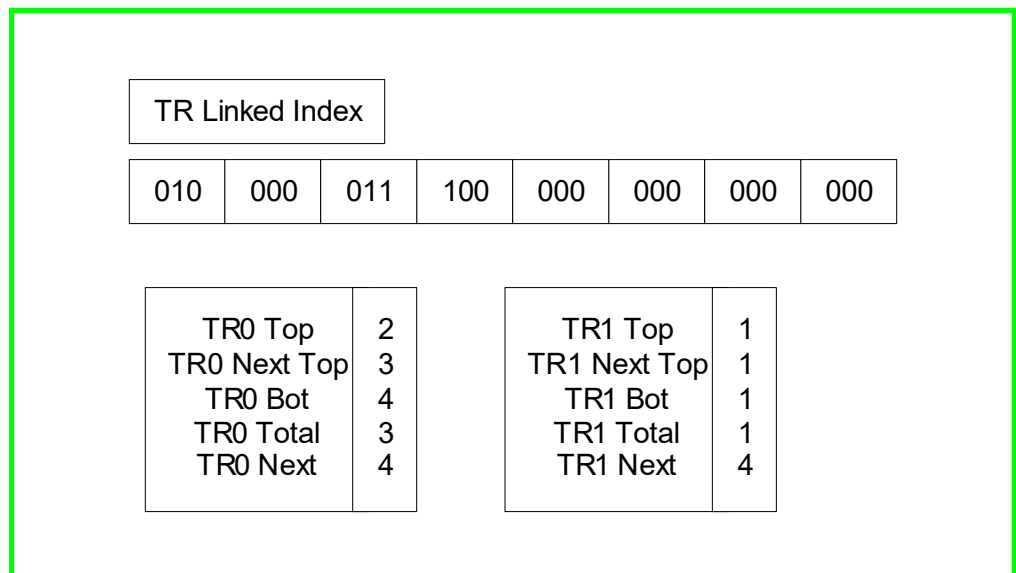
Next a third TR is added to queue 0. This time only Bot gets updated as well as the entry of TR linked index is updated with the pointer to the next position in entry 2 since that was the bottom of the list before it was updated.

Figure 6-20 DRU Queue Pointer Example Third TR added to queue 0



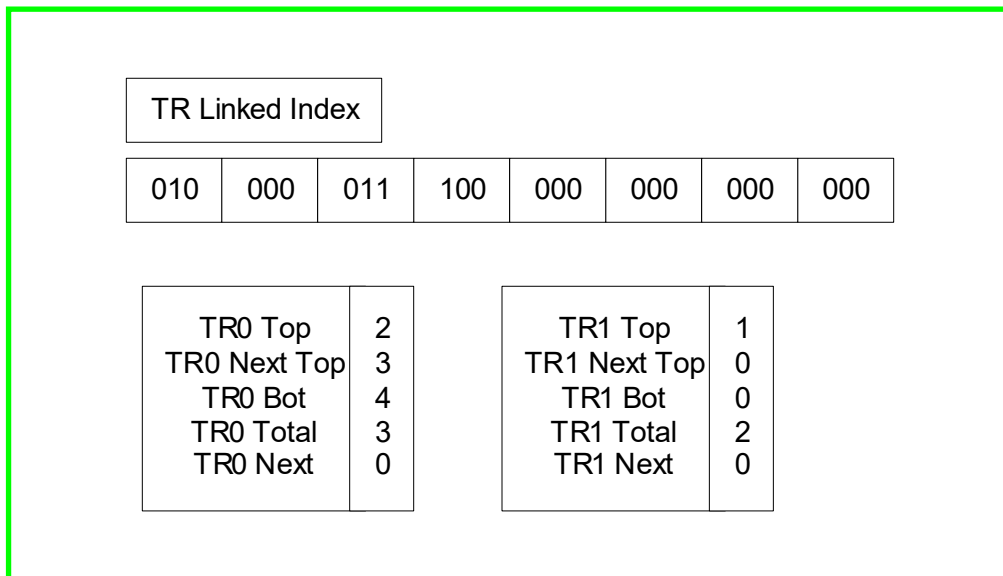
The next state assumes that in the same cycle the first entry in TR0 is removed while a new entry in TR0 is removed. In this case TOP will get the value of Next Top and Next Top will get its value from the TR Linked Indexed pointed to by the current Next Top which in this case is the value in TR Linked Index 2 which is 3. Since the list is being added to a new entry will be written in position 4 which will update bottom and the value in TR Linked Index. The Total will remain the same since a TR is both being added and removed.

Figure 6-21 DRU Queue Pointer Example First TR removed from queue 0 Fourth TR added to queue 0



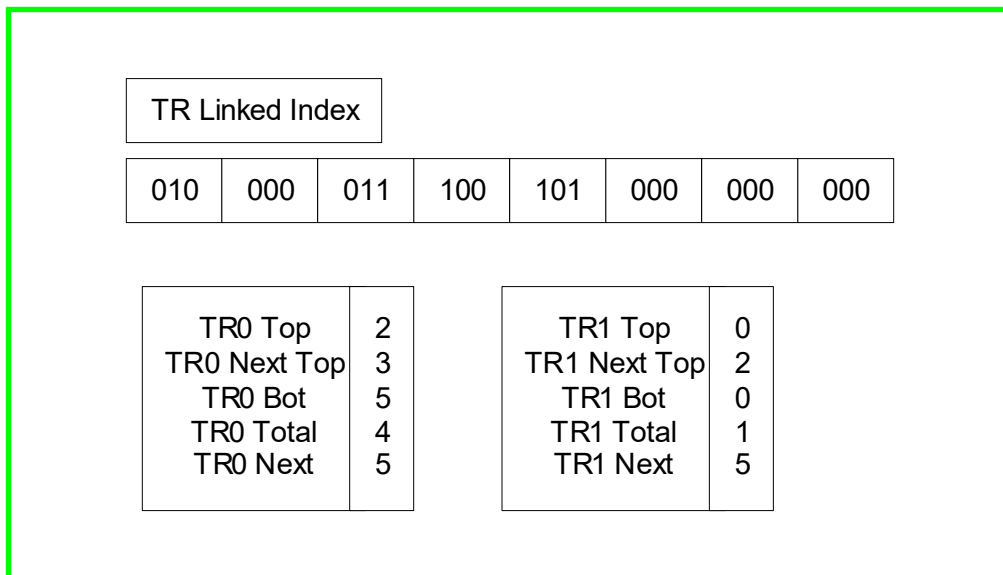
The next state assumes that a new TR is added to queue 1. Since position 0 is now open then position 0 will be used again. Since the Total in queue 1 is going from 1 to 2 the TR1 Next Top will be updated with the new bottom value as well.

Figure 6-22 DRU Queue Pointer Example Second TR added to Queue 1



The final state in the example is when a TR is added to queue 0 and removed from queue 1. In this case the TR0 Bot will be updated as well as the TR Linked index for the previous Bot value. In TR1 the Top will change and the Next Top. The Next Top will change but the value is a DON'T CARE since the Total is now 1 and with the next removal the list will be empty.

Figure 6-23 DRU Queue Pointer Example Fifth TR added to queue 0 and First TR removed from queue 1



The queue controller is only concerned on if data is in the queue memory. The format of the TR in the queue memory is based on the needs of the Read and Write engines so the format of the TR in the queue is described in the same section. The Read and Write engine will receive the add signal from the queue controller and the Write engine will return the remove. The Read Engine will contain its own queue pointer structure to allow it to remove TRs that the reads have completed.

6.13.2.3.3 Cache Warm Queue

The Cache Warm queue is used by the messaging interface and the CorePacs to allow for ordering of Cache Warm messages to be sent to any of the Coherent masters in the system. As soon as the warm request is completely received, it will be transferred to the queue memory. The Cache Warm requests will have a higher priority than the command queue to write into the queue memory. The CacheWarm queue is unique from other queues in that it only interfaces with the Write Command Interface and the Queue controller

As far as the Queue controller is concerned the Cache warm queue is handled identical to the TR Command Queues. The same pointer structure and everything is used. The only difference in the Cache Warm Queue is the format of the data that is packed into the Queue memory is the data required by the Cache Warm Engine. The format of the data will be described in the Cache Warm engine section below.

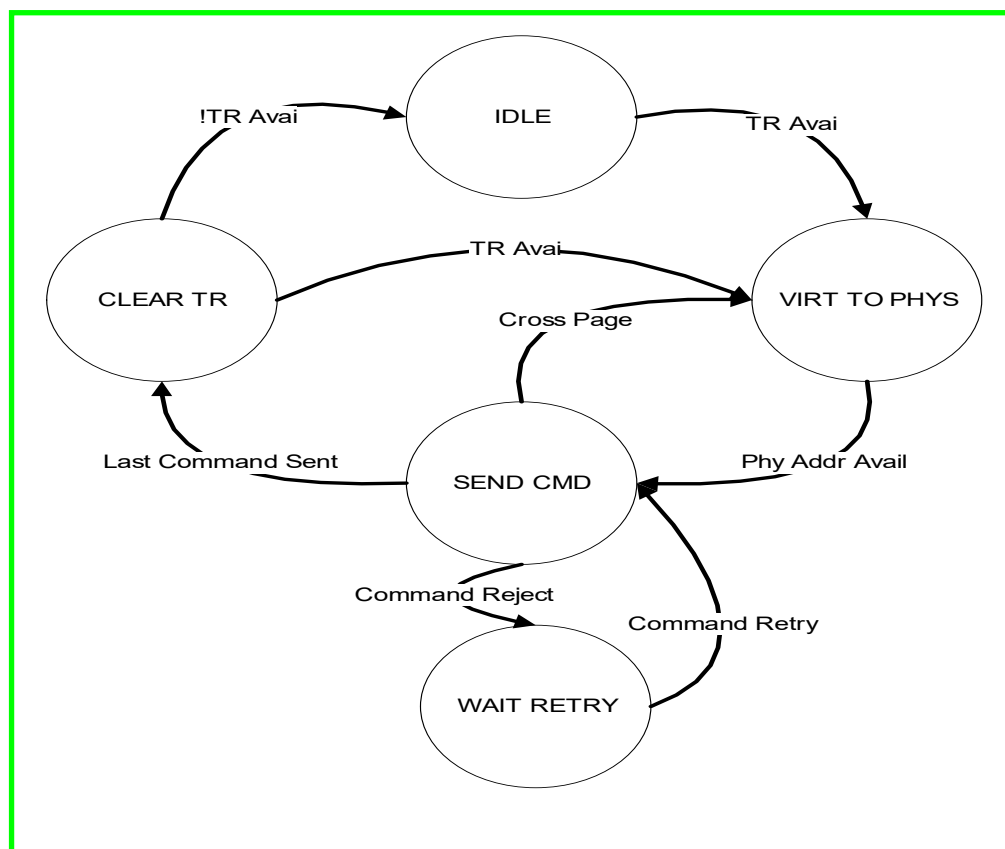
6.13.2.4 Read Engine

The Read Engine is responsible for handling a queue of TRs and issuing the read commands as specified by the TR. The read engine has four main functions the TR handling, the address calculation, the size calculation and the virtual to physical translation through the uTLB interface.

6.13.2.4.1 Read Engine TR Handling

The TR Handling in the Read Engine is managed by the simple state machine below.

Figure 6-24 TR Handling State Machine Read Engine



The table below explains the states and conditions of the state machine.

Table 6-15 Read Engine TR Handling State Machine Description

State or Condition	Description
IDLE State	This is the reset state or whenever the Read Engine is not performing any function.
VIRT TO PHY State	This is state when the Read Engine is waiting for the response from the uTLB or it is loading all of the attributes for the command to be sent to the Read Command Interface.
SEND CMD State	This is when the Line Counter and Address Calculation are active and it will be issuing a command to the Read Command Interface block.
WAIT RETRY	Since the Read Engine does not know what other modules are requesting and what resources are available this state allows the Read Command Interface block to halt commands from the read engine. If conditions change the Read Command Interface block will issue a command retry and commands will be sent again.
CLEAR TR State	This is the state when the TR is removed from the Read Queue. If another TR is in the Read Queue the next transaction will begin.
TR Avail	This is looking at the Total count for the read queue and is set if the Total count is not 0.
Phy Addr Avail	This is set if either the uTLB has returned the response to the request or if a new TR is a physical transaction then it will be set the next cycle.
Cross Page	This is if the field of the Next Address do not match the Page Address which is loaded with the uTLB. The size of the page will be provided by the uTLB so the number of bits that are compared will vary based on the page size. This allows for support of page crossing both in a burst or across a line.
Last Command Sent	This is set if the ACNT == Byte Cnt and BCNT == 1 or 0 which signifies that the last line is being transferred
Command Reject	The read command interface has rejected the command. The address and counters should not be issues and the command can be resent after the Command Retry has been sent.
Command Retry	The previous command can be sent again to see if the proper resources are now available.
End of Table 6-15	

6.13.2.4.2 Read Engine Size Calculation

It has two internal counters that it keeps track of the ACNT and BCNT. The ACNT is used to count the number of bytes in a line and then every time it is reset to the original value the BCNT is decremented.

The read engine is optimized for accesses on a 128 byte boundary. As such it will ensure that no command crosses a 128 byte boundary. It uses this requirement in the size calculation. The table below shows the Size Calculation functions.

Table 6-16 Size Calculation Functions

Count Condition	Calculation
New Line	Byte Cnt0 = 128 - Current Address[7:0]
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Byte Cnt1
During Burst	Byte Cnt0 = 128
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Cyte Cnt1
ACNT != ByteCnt	ACNT = ACNT - ByteCnt
ACNT == ByteCnt	ACNT == Original ACNT
	BCNT == BCNT - 1
	Last Command Sent = BCNT == 1 BCNT == 0
End of Table 6-16	

6.13.2.4.3 Read Engine Address Calculation

The read engine has two addresses that it must keep track of the next line address and the next burst address. The address calculations have three states where they have unique calculations. The three states are when a new TR is loaded into the Read Engine, the end of a burst when $ACNT > \text{byte cnt}$, and when $ACNT == \text{Byte Count}$ which is the end of the Line. Depending on the current ACNT value in the read engine will select which of the two addresses will be used. The table below shows how Current Address and Next Line Address are calculate for the three conditions.

Table 6-17 Address Calculation Functions

Address Condition	Calculation
New TR	Current Address = TR Address
	Next Line Address = TR Address + TR SRCDIM
End of Burst	Current Address = Current Address + Byte Count
	Next Line Address = Next Line Address
End of Line	Current Address = Next Line Address
	Next Line Address = Next Line Address + SRCDIM
End of Table 6-17	

6.13.2.4.4 Virtual to Physical Translation

The Virtual to Physical translation function in the read engine works with the uTLB to update all of the parameters that can be specified by the MMU and pass them to the Read Command Interface module.

Upon entering the VIRT TO PHY state in the TR Handling state machine the Read Engine will wait until either the uTLB request is returned or it determines that the TR is a physical TR. A TR is considered to be a physical transaction if the IOSID in the TR is all 0's.

If the uTLB returns any type of fault the Read Engine will signal an error and mark the read complete field for the TR to allow the write engine to properly complete any outstanding transactions and clear them from the buffer.

The table below shows the memory type attributes that are received from the uTLB interface and the value of those signals if the TR is a physical TR.

Table 6-18 Memory Type Attributes

Signal	Virtual TR	Physical TR
cmemtype	pg_mem_type[1:0]	0 (Device Memory Type)
ccinner	pg_mem_type[3:2]	0 (Non-allocatable)
ccouter	pg_mem_type[5:4]	0 (Non-allocatable)
csdomain	pg_mem_type[7]	0 (Don't Care since it won't be shareable)
shareable	pg_mem_type[7:6] != 0	0 (Not Shareable)
cpable	pg_mem_type[8]	0 (Non-prefetchable)
End of Table 6-18		

6.13.2.5 Read Arb

The command queues follow a dynamic priority scheme for reads that attempts to fairly allocate the number of access opportunities and bandwidth among the requestors. There is core fair-share arbitration logic for the command queues. Every queue is allowed to send only one access in a cycle to the arbitration logic, and the scheme determines who will be the winner.

The winner is selected based on the queue priority, so queue 0 will have the higher priority than other queues. But when there is a heavy traffic on Queue 0 that may make the other queues to indefinitely starve for the access. To address this every queue has a starvation bound register. They can be programmed to some value in terms of MSMC cycles. The starvation register also has a minimum cycles count. This will allow for a queue to send a small burst of commands when starvation is reached.



Note—The starvation counters are only decremented when a queue is presenting a command. So if a queue is blocked from sending commands because of a Reject from the Read Block Interface because of a lack of resources such as credits to complete the transaction the time spent in the WAIT RETRY state will NOT count towards starvation.

DRU has a starvation bound counter for each of the queues. The DRU also has a minimum cycle count. The starvation counter is decremented every cycle that a request is presented to the arbiter. Every time the queue wins arbitration the minimum cycle count is decremented by 1. When the minimum cycle count reaches zero then the starvation count and minimum cycle count for the queue are reset to the values in the Memory Mapped Registers. When the starvation counter for a queue reaches zero, the access presented from that queue will be arbitrated in the starvation arbitration logic.

The starvation arbitration has the same fixed arbitration used in the regular arbitration the only difference is that only queues that have a starvation count of zero will be seen as having a request in the starvation arbitration.

The final arbitration is that if there is a request on the starvation arbitration it beats a request for the starvation arbitration.

See the appendix on the DRU Memory Mapped Registers for the fields of the registers for the arbitration.

6.13.2.6 Read Command Interface

The Read Command Interface handles converting the general read request from the winning read engine to the specific protocol of the interface being used on the external read bus. This is the one of the few modules that should have to change if the external read bus changes. The general read format assumes that read commands and read data are separate and that the access will not cross a 128 byte boundary. Logic can be added in this module for any mapping or control state machine required. The sub-sections below discuss the current implemented Read Command Interface modules.

6.13.2.6.1 MSMC

The MSMC interface is based off of the MBA specification with minor changes required to simplify the coherency information. The MSMC interface is intended to be a direct connect of the signals and introduces no additional cycles to the initial request time.

The table below shows the MBA signals and how they are generated.

Table 6-19 DRU to MBA TAC Mapping

DRU Signal/Logic	MBA
Transaction Presentt	cvalid
See credit function	ccredit
read_buffer_available	cid
ReadNoSnoop	ctype
Tied to DRU cmstid tieoff/parameter	cmstid
Tied to 0 as this is read only port	cdata
pri	cpriority
address	caddress
cmemtype	cmemtype
cpable	cpable
ccinner	ccinner
ccouter	ccouter
csdomain	csdomain
bytecnt	cbytecnt
Tied to 0 as this is not an emulation port	cemudbg
secure	csecure
priv	cpriv
End of Table 6-19	

The internal MSMC interface uses a valid and taken pairing so a command is held until taken is used. This means that the reject and retry signals back to the read engine will be tied low.

Writer's Comment (6.2): The credit function will be fleshed out once the MSMC credit scheme is finalized. Currently it is assumed that a single credit type exists

6.13.2.7 Read Data Interface

The Read Data Interface module handles the protocol specific information to convert the specific read response data to the internal read buffer write. The read buffer write assumes a 512 bit interface plus 20 bits of parity and the memory address based on which response is being received. The parity algorithm is based on the MCAD PECC specified in the MBA protocol please refer to the Parity Encoder specification for details.

Table 6-20 Read Data Interface Signals

Signal	Description
Req	Write the data into the buffer at the specified location.
Address	The address in the read buffer that should be written
Data	The 512 bits of data that should be written
Parity	The 20 bit parity for 2 256 bit words
Byten	The byte enables for the write
Confirm	Confirm that the response is complete
End of Table 6-20	

The sub-sections below discuss the current implemented Read Data Interface modules

6.13.2.7.1 MSMC

The Read Data Interface for MSMC is a simple mapping of an MBA TAC and MBA TDC. It assumes that the command and data are always received in the same cycle and it will respond with a taken when the data has been moved into the pipeline to be read.

The parity field is taken directly from the MSMC bus. If a partial word is received then a read of the previous parity will be done and the parity will be XORed with the new parity. The final parity in the memory will always be for the full 512 bit word and it will be checked before going into the write engine.

6.13.2.7.1.1 Read Status Errors

When read data is returned, the return status bus is monitored for error conditions (rstat non-zero). If an error occurs, this will cause the Read Data Interface to signal an error received and the type of error will also be sent to the write engine.

The read data is stored in the data buffer and will be written to the destination. This data may be invalid as dictated by the specific error type.

6.13.2.8 Read Buffer Control

The read buffer control is responsible for handling the read buffer management. It communicates with the read command interface to know the available memory locations. It also communicates with the write engine providing the received flag for the next required data buffer for the write engine. It also handles arbitrating the read requests from the write engine and doing the parity check and correction before providing the data to the write engine.

6.13.2.8.1 Data buffer

DRU has an 8KB buffer to temporarily store the read return data from the memory mapped slave. The data buffer is organized as eight banks of 128 bits, with 64 entries per bank. So structure looks like a 64x128 Byte RAM. Buffer has separate read and write ports. Each request is treated as going to one of the 64 Byte Buffers in the RAM. The RAM additionally has extra bits to store the parity bits which are stored with write requests.

The read buffer controller interfaces to the RAM banks and arbitrates between the various read requests from the write engines.

6.13.2.8.2 Read Command Interface Logic

The read command interface is required to know the data buffer location to use for each command. The read buffer controller uses the same linked list structure as the queue to determine which buffer to use. It always uses the smallest pointer to a buffer that is available. When the command is taken a flag indicating that the read command has been sent for the buffer is set and will remain set until the write engine uses the buffer after it has been received.

6.13.2.8.3 Write Engine Data Available Logic

The write engine requires to know when data has been received in the return buffer. The write engine will always be waiting for data in the order they were sent by the matching read engine and will maintain a linked list of the data pointers that it is expecting. The write engine will provide the buffer id of the next location that it is expecting and the read buffer control will provide a signal when the data is available.

6.13.2.8.4 Read Data Interface Logic

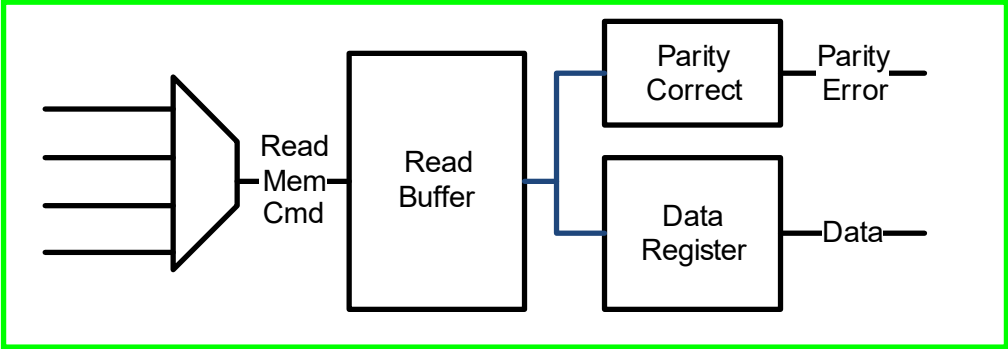
The read data interface logic will communicate with the Read Buffer Control when data is received. It will let the read buffer control know when data the transfer is complete and for the id that the transfer that is complete. Upon receiving the information that the transfer is complete the Read Buffer Controller will set the data received flag to notify any write engine waiting for that buffer that the data is now available in the queue.

Additionally the read buffer controller will look at each access and determine if it is a partial word access then it will initiate a read to XOR the contents of the parity with the previous received parity. The read of the parity bits will have the highest priority and could stall a read from a write engine.

6.13.2.8.5 Write Engine Read Data Logic

The Read Buffer Control arbitrates the data requests from the write engines by using a simple fixed priority scheme. Since a write engine will only request to read data when it is available and the starvation counters on the read arbitration have ensured that a single write engine will not take all of the bandwidth a simple arbitration scheme is used. After reading from the winning location the parity stored in the memory is checked with a calculated parity and the SECDEC function will be performed on the data. This allows for a centralized location for the parity correction logic as well as allowing to delay checking of the parity on the response until the data is used. The read buffer control will present the data bus and a status field to the write engine when the parity logic is complete.

Figure 6-25 Write Engine Read Data Logic Functional Diagram



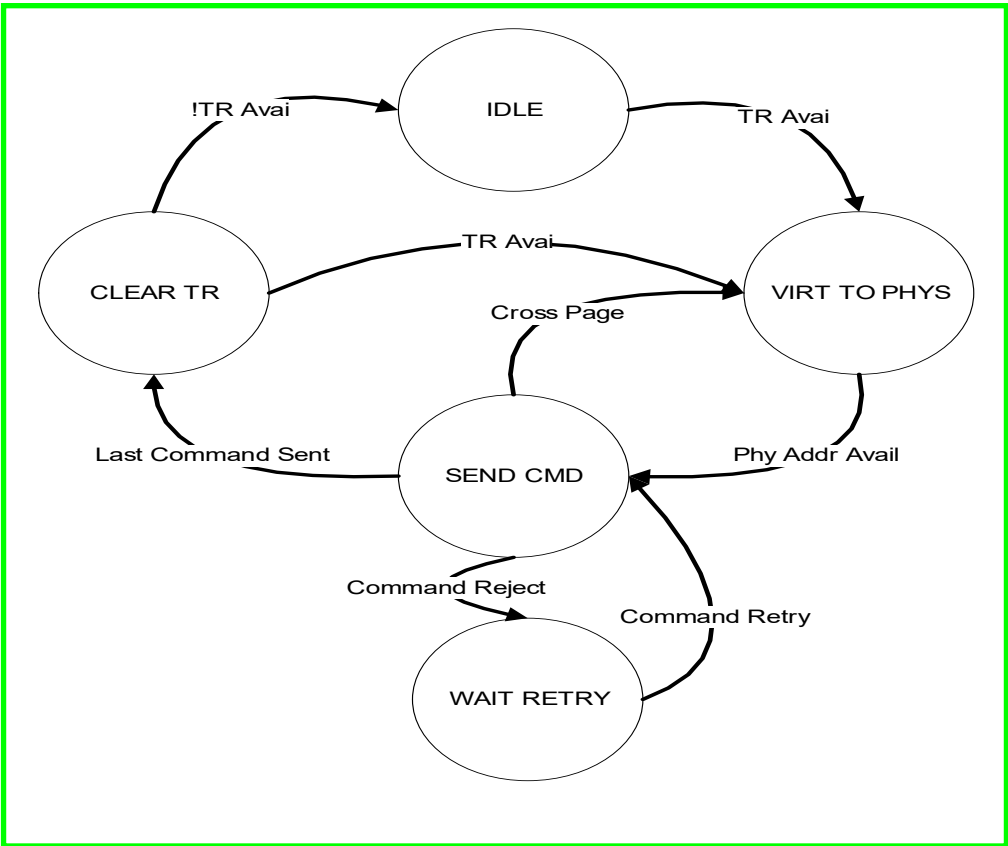
6.13.2.9 Write Data Engine

The Write Data Engine is responsible for sending the write commands for the matching read engine. The write engine has six main functions the TR handling, the address calculation, the size calculation, the virtual to physical translation through the uTLB interface, the data manipulation and the TR completion handling.

6.13.2.9.1 Write Engine TR Handling

The TR Handling in the Write Engine is managed by the simple state machine below.

Figure 6-26 TR Handling State Machine Write Engine



The table below explains the states and conditions of the state machine.

Table 6-21 Write Engine TR Handling State Machine Description

State or Condition	Description
IDLE State	This is the reset state or whenever the Write Engine is not performing any function.
VIRT TO PHY State	This is state when the Write Engine is waiting for the response from the uTLB or it is loading all of the attributes for the command to be sent to the Write Command Interface.
SEND CMD State	This is when the Line Counter and Address Calculation are active and it will be issuing a command to the Write Command Interface block when ever sufficient data is available as determined by the data shifter.
WAIT RETRY	Since the Write Engine does not know what other modules are requesting and what resources are available this state allows the Write Command Interface block to halt commands from the write engine. If conditions change the Write Command Interface block will issue a command retry and commands will be sent again.
CLEAR TR State	This is the state when the TR is removed from the Queue Memory. If another TR is in the Queue the next TR will be loaded.
TR Avail	This is looking at the Total count for the queue and is set if the Total count is not 0.
Phy Addr Avail	This is set if either the uTLB has returned the response to the request or if a new TR is a physical transaction then it will be set the next cycle.
Cross Page	This is if the field of the Next Address do not match the Page Address which is loaded with the uTLB. The size of the page will be provided by the uTLB so the number of bits that are compared will vary based on the page size. This allows for support of page crossing both in a burst or across a line.
Last Command Sent	This is set if the ACNT == Byte Cnt and BCNT == 1 or 0 which signifies that the last line is being transferred
Command Reject	The write command interface has rejected the command. The address and counters should not be issues and the command can be resent after the Command Retry has been sent.
Command Retry	The previous command can be sent again to see if the proper resources are now available.
End of Table 6-21	

6.13.2.9.2 Write Engine Size Calculation

It has two internal counters that it keeps track of the ACNT and BCNT. The ACNT is used to count the number of bytes in a line and then every time it is reset to the original value the BCNT is decremented.

The write engine is optimized for accesses on a 128 byte boundary. As such it will ensure that no command crosses a 128 byte boundary. It uses this requirement in the size calculation. The table below shows the Size Calculation functions.

Table 6-22 Size Calculation Functions

Count Condition	Calculation
New Line	Byte Cnt0 = 128 - Current Address[7:0]
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Byte Cnt1
During Burst	Byte Cnt0 = 128
	Byte Cnt1 = ACNT
	Byte Cnt = Smaller of Byte Cnt0 and Cyte Cnt1
ACNT != ByteCnt	ACNT = ACNT - ByteCnt
ACNT == ByteCnt	ACNT == Original ACNT
	BCNT == BCNT - 1
	Last Command Sent = BCNT == 1 BCNT == 0
End of Table 6-22	

6.13.2.9.3 Write Engine Address Calculation

The write engine has two addresses that it must keep track of the next line address and the next burst address. The address calculations have three states where they have unique calculations. The three states are when a new TR is loaded into the Write Engine, the end of a burst when ACNT > byte cnt, and when ACNT == Byte Count which is the end of the Line. Depending on the current ACNT value in the read engine will select which of the two addresses will be used. The table below shows how Current Address and Next Line Address are calculate for the three conditions.

Table 6-23 Address Calculation Functions

Address Condition	Calculation
New TR	Current Address = TR Address
	Next Line Address = TR Address + TR SRCDIM
End of Burst	Current Address = Current Address + Byte Count
	Next Line Address = Next Line Address
End of Line	Current Address = Next Line Address
	Next Line Address = Next Line Address + SRCDIM
End of Table 6-23	

6.13.2.9.4 Virtual to Physical Translation

The Virtual to Physical translation function in the write engine works with the uTLB to update all of the parameters that can be specified by the MMU and pass them to the Write Command Interface module.

Upon entering the VIRT TO PHY state in the TR Handling state machine the Write Engine will wait until either the uTLB request is returned or it determines that the TR is a physical TR.

If the uTLB returns any type of fault the Write Engine will signal an error and mark the read complete field for the TR to allow the write engine to properly complete any outstanding transactions and clear them from the buffer.

The table below shows the memory type attributes that are received from the uTLB interface and the value of those signals if the TR is a physical TR.

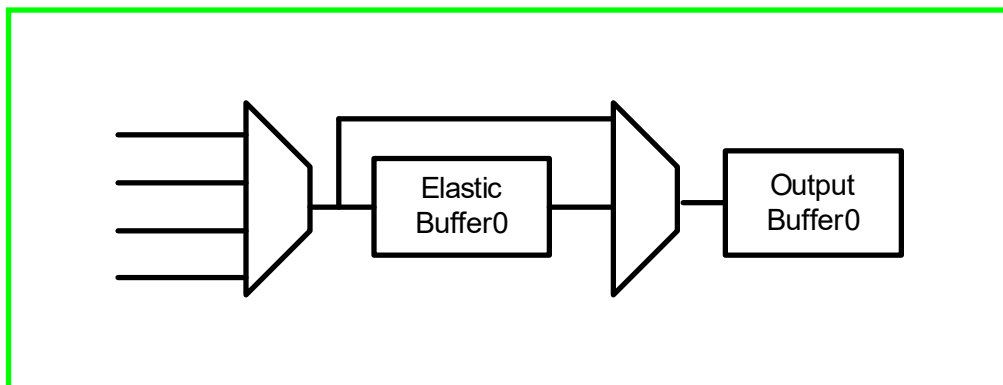
Table 6-24 Memory Type Attributes

Signal	Virtual TR	Physical TR
cmemtype	pg_mem_type[1:0]	0 (Device Memory Type)
ccinner	pg_mem_type[3:2]	0 (Non-allocatable)
ccouter	pg_mem_type[5:4]	0 (Non-allocatable)
csdomain	pg_mem_type[7]	0 (Don't Care since it won't be shareable)
shareable	pg_mem_type[7:6] != 0	0 (Not Shareable)
cpable	pg_mem_type[8]	0 (Non-prefetchable)
End of Table 6-24		

6.13.2.9.5 Write Engine Data Manipulation

The write engine data manipulation looks at the data received from the read requests and reformats it to match the requirements of the destination request. The address calculation logic and the byte count logic will determine the amount and location of the data to send while the data manipulation logic will determine when sufficient data has been received to send the command.

Figure 6-27 Write Data Engine Shifter Structure



The figure above shows the basic structure of the data shifter. There are two buffers equal to the width of the memory. A mux in front of the buffer can select any byte from the data input from the read buffer controller and place it in any byte location of the read. Additionally this word can also be loaded into the output buffer. The output buffer will take either the input to the elastic buffer or the output of the elastic buffer. This allows for storage of data that is not required for the current required output and use it with the next data.

The only function supported by the write data engine is a simple shift based on the address differences between the write address and the read address. Therefore the selection of the muxes is controlled by simply comparing the read address and write address of the current block.

Table 6-25 **Shifting Function**

Term	Function
shift	write_address - read_address. This is a 6bit subtracter for 64 byte data width that rolls over for negative case.
read_address	read address is the lower 6bits of the read address from the TR and is updated every line by adding the lower 6bits of dimension to know the 128 byte alignment of the line
write_address	write address is the lower 6 bits of the write address from the TR and is updated with the lower 6 bits of next_line_address from the address calculation so the adder does not have to be replicated
load	same as shift since the shift will determine which words are in the current word and which are in the next
read_data_avail	128-read_address if new line otherwise it is 128 bytes.
data_avail	data_avail + read_data_avail cleared on a new line
write_cmd_avail	data_avail > Byte Count

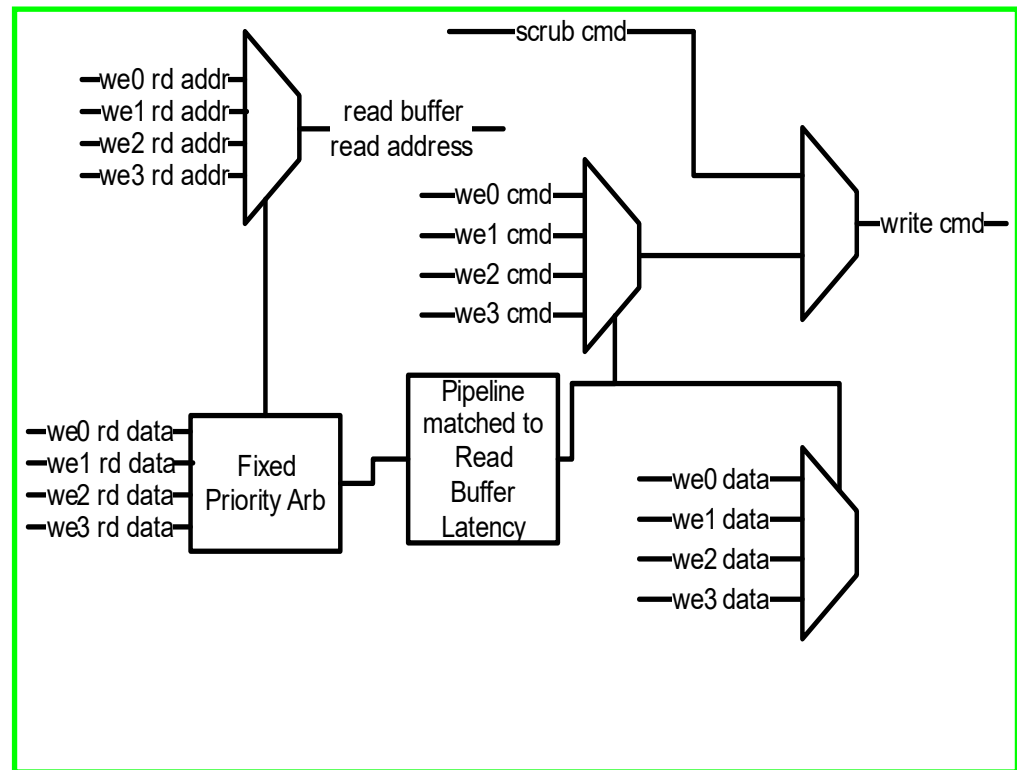
6.13.2.9.6 TR Completion Handling

The Write Engine is also responsible for sending out the completion response. The completion response engine looks at the TR Handling engine and when it sends the last command it waits until the response is received for the specified write command and then will send a command to the MSG Transmit interface to send the completion code for the TR and clear the TR from the queue. The TR completion engine will update the TR status register anytime an error is received. The status registers are stored in the queue controller and will be cleared on a new TR being received and only the first error received will be set. If a read and write error are received at the same time the read error will be stored.

6.13.2.10 Write Arbitration

The write arbitration is actually pre-scheduled on the data fetches to the read buffer control. Once a write engine wins arbitration to the read buffer control it will hold the arbitration until it completes the reads and send out as many commands as required. The selection for the muxes is a pipeline delay of the arbitration to the read buffer control. The write arbitration logic to the read buffer control is a simple fixed arbitration with write engine 0 being highest priority and write engine 1 being lower and so forth. The write arbitration port is shared with the scrubbing engine. If there is a scrubbing command then the scrubbing command will always win arbitration and go out as the next command. The Pre-warm commands go out a direct path into the return path and do not share the port with the write commands. A representation of this scheme can be seen in the figure below.

Figure 6-28 Write Arb Logic



6.13.2.11 Write Controller

The Write Controller is responsible for issuing optimally sized write command. The data buffer state dictates when a write command can be scheduled. In addition, the write controller must wait to issue any new write command until the final data phase of a write in progress.

The Write Controller is also responsible for issuing the scrubbing commands and the messages for cache warming

6.13.2.11.1 Space Occupied Rules

The Write Controller must (conceptually) track the amount of data occupied in the data buffer. The write controller can have a counter initialized to a value of zero. The counter value can be incremented when read return data lands in the data buffer. Every time a write command is issued the counter value is decremented according to the amount of data that has been scheduled to be written. The Write Controller can only issue an optimally sized write command when there is that amount of data available in the buffer. The write command and data should be available at the same time to get the write command going, so DRU will hold a write command until all its data is available.

6.13.2.11.2 Outstanding write commands for a TR

Based on the aforementioned rules there will never be more than 2 outstanding write commands relative to write data. This does not take into account write status. The number of outstanding write commands will be based on the credit availability and also the data availability.

6.13.2.11.3 Write Status Errors

When write status is returned, the wstatus bus is monitored for error conditions (wstat non-zero). If an error occurs, this will cause the DRU error register BUSERR bit to be set and will result in the DRU error register being set to the value detected on the wstatus bus along with the TCC, TCINTEN and TCCHEN settings for that transfer. No further action is taken, if the command is still in progress it continues to be serviced as normal.

6.13.2.12 Write Command Interface

The Write Command Interface handles converting the general write request from the winning write engine to the specific protocol of the interface being used on the external write bus. This is the one of the few modules that should have to change if the external write bus changes. The general write format assumes that read commands and write data are separate and that the access will not cross a 128 byte boundary. Logic can be added in this module for any mapping or control state machine required. The sub-sections below discuss the current implemented Write Command Interface modules.

6.13.2.12.1 MSMC

The MSMC interface is based off of the MBA specification with minor changes required to simplify the coherency information. The MSMC interface is intended to be a direct connect of the signals and introduces no additional cycles to the initial request time.

The table below shows the MBA signals and how they are generated.

Table 6-26 DRU to MBA TAC Mapping

DRU Signal/Logic	MBA
request	cvalid
See credit function	ccredit
Next cid value	cid
WriteNoSnoop	ctype
Tied to DRU cmstid tieoff/parameter	cmstid
Controlled by if the scrubbing engine or write engine is sending a command	cdata
pri	cpriority
address	caddress
cmemtype	cmemtype
cpable	cpable
ccinner	ccinner
ccouter	ccouter

Table 6-26 DRU to MBA TAC Mapping

DRU Signal/Logic	MBA
csdomain	csdomain
bytecnt	cbytecnt
Tied to 0 as this is not an emulation port	cemudbg
secure	csecure
priv	cpriv
End of Table 6-26	

The MSMC write data engine will present the command to the MSMC pre-arb interface and wait for taken to send the second data phase. It will always send 2 data phases no matter the amount of data required in the data transfer. This is based on the MSMC internal arbitration scheme used.

6.13.2.13 Pre-Warm Engine

The Pre-Warm Engine is a unique queue handler in the DRU that generates the Pre-warm requests into the system. It receives a specialized TR that informs it of the cache it wants to warm and the address and size of the warming request. The Pre-Warm will always send a virtual address to the requested cache controller. The DRU will NOT convert the virtual address to a physical address and leave that to the receiving cache.

The Pre-Warm engine will send the number of Pre-warm commands as specified in the size of the TR and increment the address for each subsequent Pre-warm command.

Writer's Comment (6.3): The PreWarm TR still needs to be finalized. This could increase the functionality in the block

The Pre-Warm engine will directly send the command into the MSMC return path since it does not need to go into the Central Arb.

PRELIMINARY SPECIFICATION

This section of the specification is intended to give the required information to the end user of the DRU. The primary intent of this section is to explain the best way to setup the DRU to maximize the performance. It also includes section on how to reset the DRU if required and what needs to be done to ensure proper power down and wake up procedures. An additional section on all the possible error cases that can occur in the system and how the DRU will respond in those cases and how the system is expected to recover from that failure. There is also a section on how to use the DRU in a secure system. The next section will detail how the DRU works with the Emulation and Debugger infrastructure in the system. The final section is on how to debug failing transaction to determine the root cause and what mechanisms reside inside the DRU to aid in the debug effort.

7.1 Performance

7.1.1 Virtual vs. Physical

7.1.2 Multiple Virtual TCs

7.1.3 Multiple Dimensions

7.2 Reset and Power Down

7.3 Errors

The table below describes all the potential error cases that can occur during a TR. In all cases the DRU will report the error to the system and move on to the next TR in the queue.

In all error cases the DRU will ensure that all outstanding commands for the TR have completed. The DRU will then be able to send an optional Error Descriptor to an external Error Buffer. If the ERROR_DETAILS register has not been set then the information in the ERROR_DETAILS register will be set and the ERRORS_RCVD will increment.

7.3.1 Error Descriptor

The DRU supports writing an error descriptor to aid in debugging DRU error cases. The Error Descriptor can be enabled by setting the `ERROR_CAPTURE` field in the `ERROR_DESCRIPTOR_CONTROL` register to a non-zero value for the specific queue. The Error Descriptors will be written into a circular buffer that is defined by the `ERROR_DESCRIPTOR_START` and `ERROR_DESCRIPTOR_END`. The current write pointer of the error descriptor buffer can be seen in the `ERROR_DESCRIPTOR_POINTER` register. The `ERROR_DESCRIPTOR_POINTER` will be equal to the `ERROR_DESCRIPTOR_START` whenever the `ERROR_DESCRIPTOR_START` register is written. Otherwise when an error descriptor is written out the `ERROR_DESCRIPTOR_POINTER` will increment by 96 bytes. If the new pointer equals the `ERROR_DESCRIPTOR_END` then the pointer will become the `ERROR_DESCRIPTOR_START` value. The error descriptor writing will always cease when the `ERROR_RECIEVED` value is equal to the `ERROR_CAPTURE` field in the `ERROR_DESCRIPTOR_CONTROL` register. The example below shows a typical setup and behavior for the error descriptors.

Example 7-1 Error Descriptor Running

```
// Setup by Software
ERROR_DESCRIPTOR_START = 0x1000;
ERROR_DESCRIPTOR_END = 0x1240; // ERROR_DESCRIPTOR_END + 6* 96;
ERROR_CAPTURE = 8; // This enables that 4 Error Descriptor will be sent

// Hardware Recieves First Error
0x1000: First Error Descriptor
ERROR_RCVD = 1;

// Hardware Recieves Second Error
0x1060: Second Error Descriptor
ERROR_RCVD = 2;

// Hardware Recieves Third Error
0x10C0: Third Error Descriptor
ERROR_RCVD = 3;

// Hardware Recieves Fourth Error
0x1120: Fourth Error Descriptor
ERROR_RCVD = 4;

// Hardware Recieves Fifth Error but Error RCVD has not been cleared
ERROR_RCVD = 5;

// SOFTWARE Clears the ERROR_RCVD
ERROR_RCVD_CLR = 1; // This makes ERROR_RCVD = 0;

// Hardware Recieves Sixth Error
0x1180: Sixth Error Descriptor
ERROR_RCVD = 1;

// Hardware Recieves Seventh Error
0x11E0: Seventh Error Descriptor
ERROR_RCVD = 2;

// Hardware Recieves Eighth Error
0x1000: Eighth Error Descriptor
ERROR_RCVD = 3;

// Hardware Recieves Ninth Error
0x1060: Ninth Error Descriptor
ERROR_RCVD = 4;
```

End of Example 7-1

The Figure below shows the format of the descriptor that will be written to the error response buffer. The lower 64 bytes will be the exact TR that was provided to the DRU. The upper 32 bytes will be based on the current counts and pointers that the data format of the write commands was on when it reached the error condition.

Figure 7-1 TR Error Descriptor Format

767	880	879	864	863	832	831	800	799	640		
ERROR_FLAGS			CURR_ICNT3		CURR_ICNT2		CURR_ICNT1		CURR_ICNT0		
639					576	575	520			512	
CURR_DSTADDR					Reserved					CURR_SRC_ADDR	
511			480	479	448		447	416	415	384	
ORIG_SRCDIM3			ORIG_SRCDIM2			ORIG_SRCDIM1			ORIG_DSTDIM1		
383			368	37	352	351	320	319	288	287	256
ORIG_FLAGS			ORIG_ICNT3		ORIG_ICNT2		ORIG_ICNT1			ORIG_ICNT0	
255			224		223	192		191	128		
ORIG_DSTDIM3			ORIG_DSTDIM2			ORIG_COMPADDR					
127					64	63	0				
ORIG_DSTADDR					ORIG_SRCADDR						

7.3.2 Error Cases

The table below goes through the error conditions that could be detected by the DRU. As stated before all error detection is done by the data formatter in the write engine even if the error occurs on the read side. In most cases the read transaction will complete as normal but the responses will be flushed by the write engine as needed. This ensures that both the read and the write will start fresh on a the next TR. In all cases it is expected if an error occurs that the next TR will perform identically as if the prior TR finished normally.

Table 7-1 Error Conditions in DRU

Condition	Error code	Error Classification
"Virtual TR to Physical Only DRU"	5'b00001	Submission Error
"Illegal Completion Address in Direct TR"	5'b00010	Submission Error
"Zero size for ICNT0"	5'b00011	Submission Error
"Queue does not support submission through submission port"	5'b00100	Submission Error
"Virtual TR to Physical Only DRU"	5'b00101	Read Transmission Error Write Transmission Error
"Firewall Rejection of Read Command"	5'b00110	Read Transmission Error
"MMU Read Page Fault"	5'b00111	Read Transmission Error
"Non-successful Status on Read Response"	5'b10STA STA is 3 bit sstatus returned	Read Transmission Error
"Parity Error on Read Response"	5'b01000	Read Transmission Error
"Parity Error when Reading Data from Response Buffer"	5'b01001	Read Transmission Error
"Read Response Does Not Return"	5'b01010	Read Transmission Error
"Firewall Rejection of Write Command"	5'b01011	Write Transmission Error Write Completion Error

Table 7-1 Error Conditions in DRU

Condition	Error code	Error Classification
"MMU Write Page Fault"	5'b01100	Write Transmission Error Write Completion Error
"Non-successful Status on Write Response"	5'b11STA STA is 3 bit sstatus returned	Write Transmission Error Write Completion Error
"Write Response Does Not Return"	5'b01101	Write Transmission Error Write Completion Error
End of Table 7-1		

7.3.3 TR and CR Format/Submission Errors

This class of errors are when a TR is sent to the DRU that the DRU will not be able to process successfully. All of these errors are caught during the submission either through direct TR submission or through the MSG submission. The direct TR submission will always return an Addressing Error status to the write that does the illegal submission. The MSG submission will return a response MSG with the status field containing the error code for the type of submission error as list in Table 7-1 "Error Conditions in DRU".

In all cases the TR will not take any slots in the TR queue or in the response buffer as the error occurs before entering into either buffer.

7.3.3.1 Virtual TR to Physical Only DRU

Since support for Virtual TR is a configuration option on the DRU it is possible to have a DRU that only supports physical TRs. If this is the case and software attempts to submit a virtual TR the DRU will determine that the TR is virtual based on the IOSID field not being zero and reject the submission. This can only occur if the DRU is configured to NOT support Virtual TRs.

7.3.3.2 Illegal Completion Address in Direct TR

The completion address is required to be 64 bit aligned for a direct TR. Therefore, if the lower 8 bits are not zero in a direct TR submission then the MMR will respond with an address error and not write the TR into the response queue. This error can NOT occur through the MSG bus since it does not support the completion write.

7.3.3.3 Zero size for ICNT0

Since ICNT0 is the amount of data in the inner loop a value of zero means that there is no data transfer. Since there is nothing for the DRU to do in this case it is treated as a submission error that the

7.3.3.4 Queue does not support submission through submission port

7.3.4 Read Transmission Errors

7.3.4.1 Parity Error in TR from Queue Memory

If a bit flip occurs in a TR entry in the memory that holds the TR prior to it reaching the processing engine that this will occur. The engine will compare the TR parity with the received parity and if the two do not match then this TR will not run. There are three possible cases for this error. The read and write both see the parity error, the read sees the parity error and the write does not see a parity error and the read does not see the parity error and the write does see the parity error.

7.3.4.1.1 Read and Write Both Receive Parity Error

This is the most typical case when the bit flip in the memory occurs before either engine reads the TR. In this case the read will mark the buffer and move to the next and the write will go to flush and the first buffer will be marked with the last and TR parity error flag and the error completion will be sent out if requested.

7.3.4.1.2 Read Parity Error and Write No Parity Error

This case would occur when there is a bit flip in the memory before the read writes it and a second bit flip occurs after the read and before the write reads it. In this case the read marks its first buffer with a TR error flag and last. The write engine will start as normal and will prepare to send the first data. It will see immediately that the buffer has been received and it is the last buffer of the packet. It will immediately go to the flush state to flush the buffer and then send the completion writi

7.3.4.2 Firewall Rejection of Read Command

7.3.4.3 MMU Read Page Fault

7.3.4.4 Non-successful Status on Read Response

7.3.4.5 Parity Error on Read Response

7.3.4.6 Parity Error when Reading Data from Response Buffer

7.3.4.7 Read Response Does Not Return

7.3.5 Write Transmission Errors and Write Completion Errors

7.3.5.1 Firewall Rejection of Write Command

7.3.5.2 MMU Write Page Fault

7.3.5.3 Non-successful Status on Write Response

7.3.5.4 Write Response Does Not Return

7.4 Security

7.5 Emulation Support

7.6 Debug

PRELIMINARY SPECIFICATION

8.1 Configuration History

8.1.1 DRU Maxwell Configuration

Table 8-1 DRU for Maxwell Configuration

Configuration Option	Value
Bus Type	VBUSM.C
Data Width	256
PSI-L Data Width	128
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Dedicated DMA Port	N
Cache Warm Support	Y
Number of In-flight Requests	64
Number of Priority In Flight Requests	0
Maximum Transfer Size	64 Bytes
Atomic Direct TR Submission	Y
Non Atomic Core Registers	3
Data Reformat Functions	None
Minimum Transpose Size	NA
Clocking Multiplier Pipelining	3
Virtual Address Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Safety	Y
BIST Interface	MCAD
Embedded Firewall	Y
Mem wrapper Architecture	Standard
Event Generation	Encoded
Maximum Trigger Counter Size	1
Circular Buffer Addressing	N
Static TR support	Y
Constant Copy	N
Bit Expansion ELTYPE 8-12 Support	N
Multiple Buffer Offset	N

Table 8-1 DRU for Maxwell Configuration

Configuration Option	Value
Different Source and Desitnation Size Blocks	N
Memory Attribute Table	Y
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
DRU Capabilities Value	0x00000062_000D0520
End of Table 8-1	

8.1.2 DRU J7ES Configuration

Table 8-2 DRU for Jacinto7ES Configuration

Configuration Option	Value
Bus Type	VBUSM.C
Data Width	512
PSI-L Data Width	128
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Dedicated DMA Port	N
Cache Warm Support	Y
Number of In-flight Requests	64
Number of Priority In Flight Requests	0
Maximum Transfer Size	128 Bytes
Atomic Direct TR Submission	Y
Non Atomic Core Registers	3
Data Reformat Functions	Transpose
Minimum Transpose Size	32 bits
Clocking Multiplier Pipelining	3
Virtual Address Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Safety	Y
BIST Interface	MCAD
Embedded Firewall	Y
Mem wrapper Architecture	Standard
Event Generation	Encoded
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static TR support	Y
Constant Copy	N

Table 8-2 DRU for Jacinto7ES Configuration

Configuration Option	Value
Bit Expansion ELTYPE 8-12 Support	Y
Multiple Buffer Offset	Y
Different Source and Destination Size Blocks	Yes with trigger and event size limitation
Memory Attribute Table	N
SRAM Table Entries	0
DDR Table Entries	0
Boot Table Entries	0
DRU Capabilities Value	0x00000561_000D0F20
End of Table 8-2	

8.1.3 DRU UTC DMPAC Configuration for J7ES

Table 8-3 DRU UTC for DMPAC for Jacinto7ES Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	128
MMR Data Width	32
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Dedicated DMA Port	N
Cache Warm Support	Y
Number of In-flight Requests	64
Number of Priority In Flight Requests	0
Maximum Transfer Size	128 Bytes
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Clocking Multiplier Pipelining	2
Virtual Address Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Safety	Y
BIST Interface	GPIO Ports
Embedded Firewall	Y
Mem wrapper Architecture	ECC Memwrap
Event Generation	Bit Vector
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static TR support	Y

Table 8-3 DRU UTC for DMPAC for Jacinto7ES Configuration

Configuration Option	Value
Constant Copy	N
Bit Expansion ELTYPE 8-12 Support	Y
Multiple Buffer Offset	N
Different Source and Desitnation Size Blocks	Yes with trigger and event size limitation
Memory Attribute Table	N
SRAM Table Entries	0
DDR Table Entries	0
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
End of Table 8-3	

8.1.4 DRU UTC VPAC for J7ES

Table 8-4 DRU UTC for VPAC for Jacinto7ES Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	128
MMR Data Width	32
Number of Channels	64
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Dedicated DMA Port	N
Cache Warm Support	Y
Number of In-flight Requests	64
Number of Priority In Flight Requests	0
Maximum Transfer Size	128 Bytes
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Clocking Multiplier Pipelining	2
Virtual Address Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Safety	Y
BIST Interface	GPIO Ports
Embedded Firewall	Y
Mem wrapper Architecture	ECC Memwrap
Event Generation	Bit Vector

Table 8-4 DRU UTC for VPAC for Jacinto7ES Configuration

Configuration Option	Value
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static TR support	Y
Constant Copy	N
Bit Expansion ELTYPE 8-12 Support	Y
Multiple Buffer Offset	N
Different Source and Desitnation Size Blocks	Yes with trigger and event size limitation
Memory Attribute Table	N
SRAM Table Entries	0
DDR Table Entries	0
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
End of Table 8-4	

8.1.5 DRU UTC 32CH for J7Quad

Table 8-5 DRU UTC 32Ch for J7Quad Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	128
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	N
Compression and Decompression Support	N
Video Compression Support	N
Cache Warm Support	N

Table 8-5 DRU UTC 32Ch for J7Quad Configuration

Configuration Option	Value
Cache Warm Unique Opcode Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	720 MHz
Minimum Frequency	0 MHz
Maximum Frequency	720 MHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	Y
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	GP
Embedded Firewall	N
Mem wrapper	ECC
Event Generation	bit_vector
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static	Y
Constant Copy	N
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	4
Write TDC VBUSM.C Credits	8
Read TAC VBUSM.C Credits	4
Write Response VBUSM.C Credits	4
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	Y
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1102
End of Table 8-5	

8.1.6 DRU UTC 64 Ch Configuration for J7Quad

Table 8-6 DRU UTC 64Ch for J7Quad Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	128
MMR Data Width	NA
Number of Channels	64
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	N
Compression and Decompression Support	N
Video Compression Support	N
Cache Warm Support	N
Cache Warm Unique Opcode Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	720 MHz
Minimum Frequency	0 MHz
Maximum Frequency	720 MHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	Y
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	GP
Embedded Firewall	N
Mem wrapper	ECC
Event Generation	bit_vector
Maximum Trigger Counter Size	1

Table 8-6 DRU UTC 64Ch for J7Quad Configuration

Configuration Option	Value
Circular Buffer Addressing	Y
Static	Y
Constant Copy	N
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	4
Write TDC VBUSM.C Credits	8
Read TAC VBUSM.C Credits	4
Write Response VBUSM.C Credits	4
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	Y
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1102
End of Table 8-6	

8.1.7 DRU VC Configuration for J7Quad

Table 8-7 DRU for J7Quad Configuration

Configuration Option	Value
Bus Type	VBUSM.C
Data Width	512
PSI-L Data Width	128
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0

Table 8-7 DRU for J7Quad Configuration

Configuration Option	Value
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	Y
Compression and Decompression Support	Y
Video Compression Support	Y
Cache Warm Support	Y
Cache Warm Unique Opcode Support	Y
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	1 GHz
Minimum Frequency	0 MHz
Maximum Frequency	1 GHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	N
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	MCAD
Embedded Firewall	N
Mem wrapper	Standard
Event Generation	encoded
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static	Y
Constant Copy	Y
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	13
Write TDC VBUSM.C Credits	14
Read TAC VBUSM.C Credits	14
Write Response VBUSM.C Credits	5
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	Y
Boot Table Entries	0

Table 8-7 DRU for J7Quad Configuration

Configuration Option	Value
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1102
End of Table 8-7	

8.1.8 DRU VC R Configuration for AM67

Table 8-8 DRU for AM67 Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	0
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	Y
Compression and Decompression Support	Y
Video Compression Support	Y
Cache Warm Support	N
Cache Warm Unique Opcode Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	1 GHz
Minimum Frequency	0 MHz
Maximum Frequency	1 GHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	N

Table 8-8 DRU for AM67 Configuration

Configuration Option	Value
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	MCAD
Embedded Firewall	N
Mem wrapper	Standard
Event Generation	encoded
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static	Y
Constant Copy	Y
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	13
Write TDC VBUSM.C Credits	14
Read TAC VBUSM.C Credits	14
Write Response VBUSM.C Credits	5
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	N
Ring Support	Y
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1103
End of Table 8-8	

8.1.9 DRU UTC 32CH for J7Quad

Table 8-9 DRU UTC 32Ch R for AM67 Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	0
MMR Data Width	NA
Number of Channels	32
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1

Table 8-9 DRU UTC 32Ch R for AM67 Configuration

Configuration Option	Value
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	N
Compression and Decompression Support	N
Video Compression Support	N
Cache Warm Support	N
Cache Warm Unique Opcode Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	720 MHz
Minimum Frequency	0 MHz
Maximum Frequency	720 MHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	Y
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	GP
Embedded Firewall	N
Mem wrapper	ECC
Event Generation	bit_vector
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static	Y
Constant Copy	N
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128

Table 8-9 DRU UTC 32Ch R for AM67 Configuration

Configuration Option	Value
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	4
Write TDC VBUSM.C Credits	8
Read TAC VBUSM.C Credits	4
Write Response VBUSM.C Credits	4
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	N
Ring Support	Y
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1103
End of Table 8-9	

8.1.10 DRU UTC 64 Ch R Configuration for AM67

Table 8-10 DRU UTC 64Ch R for AM67 Configuration

Configuration Option	Value
Bus Type	VBUSM
Data Width	256
PSI-L Data Width	0
MMR Data Width	NA
Number of Channels	64
Number of TRs per Channel	3
Number of Round Robin Queues	4
Number of Priority Queues	1
Write Status Fifo Size	2
Dedicated DMA Port	N
Cache Warm Support	N
Number of In-flight Requests	64
Priority Mask	Y
Maximum Transfer Size	128 Bytes
CLEC MMR Feed-through Port	N
Maximum Number of Dimensions	4
Atomic Direct TR Submission	N
Non Atomic Core Registers	0
Data Reformat Functions	None
Minimum Transpose Size	NA
Virtual Address Support	N
Compression and Decompression Support	N
Video Compression Support	N

Table 8-10 DRU UTC 64Ch R for AM67 Configuration

Configuration Option	Value
Cache Warm Support	N
Cache Warm Unique Opcode Support	N
PSI Split TR Ingress	N
PSI Split TR Egress	N
PSI Input Buffer Entries	0
Nominal Frequency	720 MHz
Minimum Frequency	0 MHz
Maximum Frequency	720 MHz
PSI Event Input Lanes	1
PSI Event Output Lanes	1
Split Wide Memories for smaller ECC Calculations	Y
IDLE depends on PSI Link being disable	N
Safety	Y
BIST Interface	GP
Embedded Firewall	N
Mem wrapper	ECC
Event Generation	bit_vector
Maximum Trigger Counter Size	1
Circular Buffer Addressing	Y
Static	Y
Constant Copy	N
Bit Expansion	Y
Multiple Buffer Offset	N
Different Source and Destination Size Blocks	Y
Memory Attribute Table	N
SRAM Table Entries	128
DDR Table Entries	2048
Boot Table Entries	128
Write TAC VBUSM.C Credits	4
Write TDC VBUSM.C Credits	8
Read TAC VBUSM.C Credits	4
Write Response VBUSM.C Credits	4
Clocking Multiplier Pipelining	3
Forced Teardown	Y
Secondary PSIL Teardown	N
Ring Support	Y
Boot Table Entries	0
DRU Capabilities Value	0x0000008B_000D0F00
PID Value	0x662C1103
End of Table 8-10	

PRELIMINARY SPECIFICATION

9.1 Requirements Traceability

9.1.1 DRU Maxwell Configuration

Table 9-1 DRU for Maxwell Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-34	shall	Shall have Bus Type VBUSMC	8.1.1
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.1
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.1
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.1
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.1
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.1
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.1
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.1
PROC_IP_DMA-49	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.1
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.1
PROC_IP_DMA-52	shall	Shall have support for Atomic Direct TR Submission	8.1.1
PROC_IP_DMA-53	shall	Shall have support for 3 Number of Non Atomic Cores	8.1.1
PROC_IP_DMA-54	shall not	Shall NOT have support for Transpose Reformat Function	8.1.1
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.1
PROC_IP_DMA-60	shall	Shall have Cache Warm Support	8.1.1
PROC_IP_DMA-63	shall	Shall have Embedded Firewall	8.1.1
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.1
PROC_IP_DMA-72	shall	Firewall for bus master accesses	8.1.1
PROC_IP_DMA-73	shall	Memory ECC	8.1.1
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.1
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.1
PROC_IP_DMA-77	shall	DRU channelized firewall	8.1.1

Table 9-1 DRU for Maxwell Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-78	shall	DRU Region based firewall	8.1.1
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.1
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.1
PROC_IP_DMA-81	shall	ECC of data signals	8.1.1
PROC_IP_DMA-82	shall	Non-burst direct submission	8.1.1
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.1
End of Table 9-1			

9.1.2 DRU J7ES Configuration

Table 9-2 DRU for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-34	shall	Shall have Bus Type VBUSMC	8.1.2
PROC_IP_DMA-36	shall	Shall have Data Width 512 bits	8.1.2
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.2
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.2
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.2
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.2
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.2
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.2
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.2
PROC_IP_DMA-50	shall	Shall have CLEC MMR Feedthrough Port	8.1.2
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.2
PROC_IP_DMA-52	shall	Shall have support for Atomic Direct TR Submission	8.1.2
PROC_IP_DMA-53	shall	Shall have support for 3 Number of Non Atomic Cores	8.1.2
PROC_IP_DMA-55	shall	Shall have support for Transpose Reformat Function	8.1.2
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.2
PROC_IP_DMA-60	shall	Shall have Cache Warm Support	8.1.2
PROC_IP_DMA-61	shall	Shall have Cache Warm Support with Unique Opcode support	8.1.2
PROC_IP_DMA-63	shall	Shall have Embedded Firewall	8.1.2

Table 9-2 DRU for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.2
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.2
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.2
PROC_IP_DMA-69	shall	Shall have support for Secondary PSIL Teardown	8.1.2
PROC_IP_DMA-70	shall	Shall have Multiple Buffer Offset	8.1.2
PROC_IP_DMA-72	shall	Firewall for bus master accesses	8.1.2
PROC_IP_DMA-73	shall	Memory ECC	8.1.2
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.2
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.2
PROC_IP_DMA-77	shall	DRU channelized firewall	8.1.2
PROC_IP_DMA-78	shall	DRU Region based firewall	8.1.2
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.2
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.2
PROC_IP_DMA-81	shall	ECC of data signals	8.1.2
PROC_IP_DMA-82	shall	Non-burst direct submission	8.1.2
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.2
End of Table 9-2			

9.1.3 DRU UTC DMPAC Configuration for J7ES

Table 9-3 DRU UTC for DMPAC for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.3
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.3
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.3
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.3
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.3
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.3
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.3
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.3
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.3

Table 9-3 DRU UTC for DMPAC for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.3
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.3
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.3
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.3
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.3
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.3
PROC_IP_DMA-73	shall	Memory ECC	8.1.3
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.3
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.3
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.3
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.3
PROC_IP_DMA-81	shall	ECC of data signals	8.1.3
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.3
End of Table 9-3			

9.1.4 DRU UTC VPAC for J7ES

Table 9-4 DRU UTC for VPAC for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.4
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.4
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.4
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.4
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.4
PROC_IP_DMA-44	shall	Shall have 64 Number of Channels	8.1.4
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.4
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.4
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.4
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.4

Table 9-4 DRU UTC for VPAC for Jacinto7ES Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.4
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.4
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.4
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.4
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.4
PROC_IP_DMA-73	shall	Memory ECC	8.1.4
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.4
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.4
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.4
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.4
PROC_IP_DMA-81	shall	ECC of data signals	8.1.4
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.4
End of Table 9-4			

9.1.5 DRU UTC 32CH for J7Quad

Table 9-5 DRU UTC 32Ch for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.5
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.5
PROC_IP_DMA-38	shall	Shall have PSI-L Data Width 128 bits	8.1.5
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.5
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.5
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.5
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.5
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.5
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.5
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.5
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.5

Table 9-5 DRU UTC 32Ch for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.5
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.5
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.5
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.5
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.5
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.5
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.5
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.5
PROC_IP_DMA-73	shall	Memory ECC	8.1.5
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.5
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.5
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.5
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.5
PROC_IP_DMA-81	shall	ECC of data signals	8.1.5
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.5
End of Table 9-5			

9.1.6 DRU UTC 64 Ch Configuration for J7Quad

Table 9-6 DRU UTC 64Ch for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.6
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.6
PROC_IP_DMA-38	shall	Shall have PSI-L Data Width 128 bits	8.1.6
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.6
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.6
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.6
PROC_IP_DMA-44	shall	Shall have 64Number of Channels	8.1.6
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.6
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.6

Table 9-6 DRU UTC 64Ch for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.6
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.6
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.6
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.6
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.6
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.6
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.6
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.6
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.6
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.6
PROC_IP_DMA-73	shall	Memory ECC	8.1.6
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.6
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.6
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.6
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.6
PROC_IP_DMA-81	shall	ECC of data signals	8.1.6
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.6
End of Table 9-6			

9.1.7 DRU VC Configuration for J7Quad

Table 9-7 DRU for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-34	shall	Shall have Bus Type VBUSMC	8.1.7
PROC_IP_DMA-36	shall	Shall have Data Width 512 bits	8.1.7
PROC_IP_DMA-39	shall	Shall have 48 bit TR address	8.1.7
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.7
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.7
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.7
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.7
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.7
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.7

Table 9-7 DRU for J7Quad Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.7
PROC_IP_DMA-50	shall	Shall have CLEC MMR Feedthrough Port	8.1.7
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.7
PROC_IP_DMA-52	shall	Shall have support for Atomic Direct TR Submission	8.1.7
PROC_IP_DMA-53	shall	Shall have support for 3 Number of Non Atomic Cores	8.1.7
PROC_IP_DMA-55	shall	Shall have support for Transpose Reformat Function	8.1.7
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.7
PROC_IP_DMA-60	shall	Shall have Cache Warm Support	8.1.7
PROC_IP_DMA-61	shall	Shall have Cache Warm Support with Unique Opcode support	8.1.7
PROC_IP_DMA-63	shall	Shall have Embedded Firewall	8.1.7
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.7
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.7
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.7
PROC_IP_DMA-69	shall	Shall have support for Secondary PSIL Teardown	8.1.7
PROC_IP_DMA-70	shall	Shall have Multiple Buffer Offset	8.1.7
PROC_IP_DMA-72	shall	Firewall for bus master accesses	8.1.7
PROC_IP_DMA-73	shall	Memory ECC	8.1.7
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.7
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.7
PROC_IP_DMA-77	shall	DRU channelized firewall	8.1.7
PROC_IP_DMA-78	shall	DRU Region based firewall	8.1.7
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.7
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.7
PROC_IP_DMA-81	shall	ECC of data signals	8.1.7
PROC_IP_DMA-82	shall	Non-burst direct submission	8.1.7
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.7
End of Table 9-7			

9.1.8 DRU VC R Configuration for AM67

Table 9-8 DRU for AM67 Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.8
PROC_IP_DMA-37	shall	Shall have Data Width 512 bits	8.1.8
PROC_IP_DMA-39	shall	Shall have 48 bit address width support	8.1.8
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.8
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.8
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.8
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.8
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.8
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.8
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.8
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.8
PROC_IP_DMA-50	shall	Shall have CLEC MMR Feedthrough Port	8.1.8
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.8
PROC_IP_DMA-52	shall	Shall have support for Atomic Direct TR Submission	8.1.8
PROC_IP_DMA-53	shall	Shall have support for 3 Number of Non Atomic Cores	8.1.8
PROC_IP_DMA-54	shall not	Shall NOT have support for Transpose Reformat Function	8.1.8
PROC_IP_DMA-56	shall	Shall have Virtual Address Support	8.1.8
PROC_IP_DMA-58	shall	Shall have Analytic Compression and Decompression Support	8.1.8
PROC_IP_DMA-59	shall	Shall have Video CompressionSupport	8.1.8
PROC_IP_DMA-60	shall	Shall have Cache Warm Support	8.1.8
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.8
PROC_IP_DMA-63	shall	Shall have Embedded Firewall	8.1.8
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.8
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.8
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.8
PROC_IP_DMA-68	shall	Shall have Forced Teardown support	8.1.8
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.8

Table 9-8 DRU for AM67 Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-72	shall	Firewall for bus master accesses	8.1.8
PROC_IP_DMA-73	shall	Memory ECC	8.1.8
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.8
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.8
PROC_IP_DMA-77	shall	DRU channelized firewall	8.1.8
PROC_IP_DMA-78	shall	DRU Region based firewall	8.1.8
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.8
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.8
PROC_IP_DMA-81	shall	ECC of data signals	8.1.8
PROC_IP_DMA-82	shall	Non-burst direct submission	8.1.8
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.8
PROC_IP_DMA-341	shall	Shall Support Low Cost DMA Rings	8.1.8
End of Table 9-8			

9.1.9 DRU UTC 32CH R for AM67

Table 9-9 DRU UTC 32Ch R for AM67 Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.9
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.9
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.9
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.9
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.9
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.9
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.9
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.9
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.9
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.9
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.9
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.9

Table 9-9 DRU UTC 32Ch R for AM67 Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.9
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.9
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.9
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.9
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.9
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.9
PROC_IP_DMA-73	shall	Memory ECC	8.1.9
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.9
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.9
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.9
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.9
PROC_IP_DMA-81	shall	ECC of data signals	8.1.9
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.9
PROC_IP_DMA-341	shall	Shall Support Low Cost DMA Rings	8.1.9
End of Table 9-9			

9.1.10 DRU UTC 64 Ch R Configuration for J7Quad

Table 9-10 DRU UTC 64Ch R for AM^& Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.10
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.10
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.10
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.10
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.10
PROC_IP_DMA-44	shall	Shall have 64Number of Channels	8.1.10
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.10
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.10
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.10
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.10

Table 9-10 DRU UTC 64Ch R for AM^& Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.10
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.10
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.10
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.10
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.10
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.10
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.10
PROC_IP_DMA-68	shall	Shall have Forced Teardown support	8.1.10
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.10
PROC_IP_DMA-73	shall	Memory ECC	8.1.10
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.10
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.10
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.10
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.10
PROC_IP_DMA-81	shall	ECC of data signals	8.1.10
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.10
PROC_IP_DMA-341	shall	Shall Support Low Cost DMA Rings	8.1.10
End of Table 9-10			

9.1.11 DRU UTC 32Ch R Configuration for AM62A

Table 9-11 DRU UTC 32Ch R for AM62A Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-35	shall	Shall have Bus Type VBUSM	8.1.10
PROC_IP_DMA-37	shall	Shall have Data Width 256 bits	8.1.10
PROC_IP_DMA-40	shall	Shall have 40 bit Address Width support	8.1.10
PROC_IP_DMA-41	shall	Shall have 4 Round Robin Queues	8.1.10
PROC_IP_DMA-42	shall	Shall have 1 Priority Queue	8.1.10
PROC_IP_DMA-43	shall	Shall have 32 Number of Channels	8.1.10
PROC_IP_DMA-45	shall	Shall have 3 TRs per Channel	8.1.10

Table 9-11 DRU UTC 32Ch R for AM62A Configuration

Jira Requirement ID	Level	Requirement Summary	Specification Reference
PROC_IP_DMA-46	shall	Shall have support for 64 In-flight Requests	8.1.10
PROC_IP_DMA-47	shall	Shall have Priority Mask	8.1.10
PROC_IP_DMA-48	shall	Shall have support for 64 Byte Maximum Transfer Size	8.1.10
PROC_IP_DMA-51	shall	Shall have support for Maximum 4 Number of Dimensions	8.1.10
PROC_IP_DMA-54	shall not	Shall not have support for Transpose Reformat Function	8.1.10
PROC_IP_DMA-57	shall not	Shall NOT have Virtual Address Support	8.1.10
PROC_IP_DMA-62	shall	Shall have ASIL-B Safety	8.1.10
PROC_IP_DMA-64	shall	Shall have Circular Buffer Addressing mode	8.1.10
PROC_IP_DMA-65	shall	Shall have Static TR support	8.1.10
PROC_IP_DMA-67	shall	Shall have Bit Expansion support	8.1.10
PROC_IP_DMA-68	shall	Shall have Forced Teardown support	8.1.10
PROC_IP_DMA-71	shall not	Shall NOT have Multiple Buffer Offset	8.1.10
PROC_IP_DMA-73	shall	Memory ECC	8.1.10
PROC_IP_DMA-74	shall	PBIST check of memory	8.1.10
PROC_IP_DMA-76	shall	DRU Legal TR Check	3.3.1.3,8.1.10
PROC_IP_DMA-79	shall	DRU Active TR event overflow detection	8.1.10
PROC_IP_DMA-80	shall	Parity of control and address signals	3.3.12,8.1.10
PROC_IP_DMA-81	shall	ECC of data signals	8.1.10
PROC_IP_DMA-83	shall	Bit multiplexing in memory array	8.1.10
PROC_IP_DMA-341	shall	Shall Support Low Cost DMA Rings	8.1.10
End of Table 9-11			

PRELIMINARY SPECIFICATION

UTC TR Format Specification

10.1 Overview

This section describes the standard transfer request (TR) format to initiate a DMA transfer as defined in the [KS3 DMA specification](#). The TRs can support both half and full duplex with multiple dimensions as well as cache wars. The UTC (Universal Transfer Controller) does not have to support all types of TRs. Each TR will also generate a TR response. A TR can be sent to the UTC either through a PSI-L interface or an optionally supported Direct TR submission (previously QDMA) which will have Memory Mapped TR submission registers that can be written. The Memory Mapped TR submission registers further be classified by submission type either a single burst (atomic write) or multiple burst (non-atomic). The non-atomic TR will be sent when the submission register with word 0 is written.

Table 10-1 Terms and Definition

Term	Definition
TR	A transfer request to move data.
CR	A cache operation request. A request to send messages to a specified cache controller to prepare the cache for a future operation.
UTC	Universal Transfer Controller the module in the UDMA architecture that handled the actual movement of data requested in the TR .
End of Table 10-1	

10.2 Addressing algorithm

For a basic, TR request the same algorithm is used. The innermost loop always consumes physically contiguous elements from memory. Its implicit dimension is 1 byte, this can be changed if the TR contains the formatting flags field and it is supported by the UTC. The pointer itself moves from fetch to fetch on the maximum byte alignments specified for the UTC, in increasing order. In each level outside the inner loop, the loop moves the pointer to a new location based on the size of that loop level's dimension.

Linear Addressing (Forward)

The following code illustrates the basic algorithm for a 4 level loop nest block move. This model assumes that it transfers data 1 byte at a time and the input and output block are the same size in all dimensions.

Example 10-1 Linear Memory Addressing Block Move Algorithm

```
-----
// sptr is a byte pointer.
// dptr is a byte pointer.
// TR_TRIGX can be selected to come from Global events, Local event, or none.

// Check for trigger of TYPE3
if (TR_TRIG0_TYPE == TYPE3)
    while (!TR_TRIG0);
if (TR_TRIG1_TYPE == TYPE3)
    while (!TR_TRIG1);

for (i3 = 0; i3 < ICNT3; i3++)
{
    sptr3 = sptr; // save current position before entering next level
    dptr3 = dptr; // save current position before entering next level

    // Check for trigger of TYPE2
    if (TR_TRIG0_TYPE == TYPE2)
        while (!TR_TRIG0);
    if (TR_TRIG1_TYPE == TYPE2)
        while (!TR_TRIG1);

    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptr2 = sptr; // save current position before entering next level
        dptr2 = dptr; // save current position before entering next level

        // Check for trigger of TYPE1
        if (TR_TRIG0_TYPE == TYPE1)
            while (!TR_TRIG0);
        if (TR_TRIG1_TYPE == TYPE1)
            while (!TR_TRIG1);

        for (i1 = 0; i1 < ICNT1; i1++)
        {
            sptr1 = sptr; // save current position before entering next level
            dptr1 = dptr; // save current position before entering next level

            // Check for trigger of TYPE0
            if (TR_TRIG0_TYPE == TYPE0)
                while (!TR_TRIG0);
            if (TR_TRIG1_TYPE == TYPE0)
                while (!TR_TRIG1);

            for (i0 = 0; i0 < ICNT0; i0++)
            {
                // UTC can combine these in optimized burst aligned accesses.
                *dptr = *sptr;
                sptr = sptr++;
                dptr = dptr++;
            }
            // Update based on saved pointer for this level
            sptr = sptr1 + SDIM1;
            dptr = dptr1 + DDIM1;
        }
        // Update based on saved pointer for this level
        sptr = sptr2 + SDIM2;
        dptr = dptr1 + DDIM2;
    }
}
```

```
    }  
    // Update based on saved pointer for this level  
    sptr = sptr3 + SDIM3;  
    dptr = dptr1 + DDIM3;  
}
```

This form of addressing allows programs to specify regular paths through memory in a small number of parameters. Additionally, it allows for various stall points through the loop to allow for hardware or software induced pausing of the transfer. The following table defines these parameters more explicitly.

Table 10-2 Addressing Parameters for a Basic Stream

Parameter	Definition
ICNT0	Number of iterations for the innermost loop dimension, loop level 0..That is, DIM0 = BYTES.
ICNT1	Number of iterations for the first level above the innermost loop, loop level 1.
SDIM1	Number of bytes between the starting points for consecutive iterations of loop level 1 for the source.
DDIM1	Number of bytes between the starting points for consecutive iterations of loop level 1 for the destination.
ICNT2	Number of iterations for loop level 2.
SDIM2	Number of bytes between the starting points for consecutive iterations of loop level 2 for the source.
DDIM2	Number of bytes between the starting points for consecutive iterations of loop level 2 for the destination.
ICNT3	Number of iterations for loop level 3.
SDIM3	Number of bytes between starting points for consecutive iterations of loop level 3 for the source.
DDIM3	Number of bytes between the starting points for consecutive iterations of loop level 3 for the destination.
End of Table 10-2	

10.3 Transfer Request Formats

The Transfer Request format builds on the same structure adding fields to the TR to give more configuration

The below structure shows the 512-bit data structure that makes up the maximum sized UTC Transfer request.

Figure 10-1 Transfer Request Template

word 15		word 14		word 13		word 12	
DICNT3	DICNT2	DICNT1	DICNT0	DDIM3		DDIM2	
word 11		word 10		word 9		word 8	
DADDR				DDIM1		FMTFLAGS	
word 7		word 6		word 5		word 4	
DIM3		DIM2		ICNT3	ICNT2	DIM1	
word3		word 2		word 1		word0	
ADDR				ICNT1	ICNT0	FLAGS	

The UTC defines a four-level loop nest for addressing elements within the TR, using the algorithms described in [“Addressing algorithm”](#) on page 10-275. Most of the fields in the TR template map directly to the parameters in those algorithms.

Table 10-3 Transfer Request Fields

Field Name	Description	Size
FLAGS	TR flags that specify type of TR and how the TR should be handled. It also supports the TR triggering and output events.	32 bits
ICNT0	Total loop iteration count for level 0 (innermost)	16 bits
ICNT1	Total loop iteration count for level 1	16 bits
ADDR	Starting address for the source data or destination data if it is a half duplex write.	64 bits
DIM1	Signed dimension for loop level 1 for the source data	32 bits
ICNT2	Total loop iteration count for level 2	16 bits
ICNT3	Total loop iteration count for level 3 (outermost)	16 bits
DIM2	Signed dimension for loop level 2	32 bits
DIM3	Signed dimension for loop level 3	32 bits
FMTFLAGS	Flags that tell how the data is formatted either between the input and the output or if the data should use different addressing schemes or sizes.	32 bits
DDIM1	Signed dimension for loop level 1 for the destination data	32 bits
DADDR	Starting address for the destination of the data	64 bits
DDIM2	Signed dimension for loop level 2 for the destination data	32 bits
DDIM3	Signed dimension for loop level 3 for the destination data	32 bits
DICNT0	Total loop iteration count for level 0 (innermost) used for destination	16 bits
DICNT1	Total loop iteration count for level 1 used for destination	16 bits
DICNT2	Total loop iteration count for level 2 used for destination	16 bits
DICNT3	Total loop iteration count for level 3 used for destination	16 bits
End of Table 10-3		

The TR assumes all iteration counts as unsigned integers, and all dimensions as signed integers representing byte offsets.

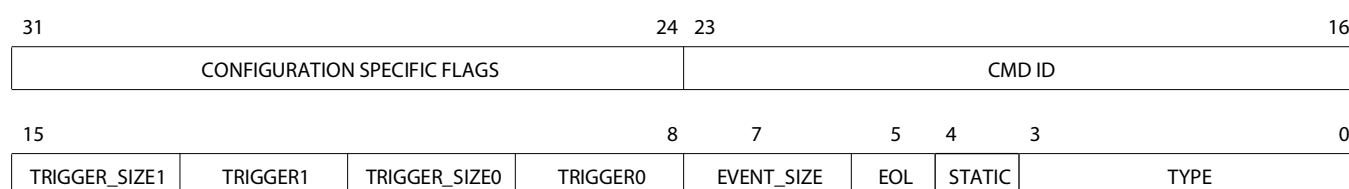
The template above fully specifies the type of elements, length, and dimensions of the transfer.

Since all transfers will not require all the fields the TRs have the type field which allow the number of words required to be sent for the TR to be reduced.

10.4 Flags Field Definition

The next diagram expands the 32-bit FLAGS field. The numbers above each field here indicate bit numbers within the field.

Figure 10-2 Transfer Request Template FLAGS Field



The flags field fills in the remaining details about the stream, as follows:

Table 10-4 FLAGS Field Descriptions (Part 1 of 2)

Bit	Field	Description	Reference
0-3	TYPE	The TYPE of TR that is being sent 0: One dimensional data move 1: Two dimensional data move 2: Three dimensional data move 3: Four dimensional data move 4: Four dimensional data move with data formatting 5: Four dimensional Cache Warm 6-7: Reserved 8: Four Dimensional Block Move 9: Four Dimensional Block Move with Repacking 10: 2 Dimensional Block Move 11: 2 Dimensional Block Move with Repacking 12-15: Reserved	"Type: TR Type Field" on page 10-279
4	STATIC	The TR is static and will continually reload into the channel upon completion of the TR until a channel tear down.	"STATIC: Static Field Definition" on page 10-281
5	EOL	The data on the PSI-L should be presented as a 2D data stream with EOL and SOL markers at the end of every ICNT0. This field is reserved for types that are	"EOL: End of Line Definition" on page 10-281
6-7	EVENT_SIZE	This is how often the TR will generate an output event. 0: Event is only generated with the TR is complete 1: Event is generated when the second inner most loop (ICNT1) is decremented by 1. 2: Event is generated when the third inner most loop (ICNT2) is decremented by 1. 3: Event is generated when the outer most loop (ICNT3) is decremented by 1.	"EVENT_SIZE: Event Generation Definition" on page 10-281

Table 10-4 FLAGS Field Descriptions (Part 2 of 2)

Bit	Field	Description	Reference
8-9	TRIGGER0	This is one of two selectable triggers. The receipt of this trigger will enable the TR to be active for enough data transfer as specified by the TRIGGER0_TYPE Field. 0: No Trigger 1: Global Trigger 0 for the channel 2: Global Trigger 1 for the channel 3: Local Event for the channel	"TRIGGER INFO: TR Triggers" on page 10-282
10-11	TRIGGER0_TYPE	This is the type of data transfer that will be enabled by receiving a trigger. 0: The second inner most loop (ICNT1) will be decremented by 1. 1: The third inner most loop (ICNT2) will be decremented by 1. 2: The outer most loop (ICNT3) will be decremented by 1. 3: The entire TR will be allowed to complete.	"TRIGGER INFO: TR Triggers" on page 10-282 "TRIGGERX_TYPE: Trigger Type" on page 10-282
12-138:9	TRIGGER1	This is one of two selectable triggers. The receipt of this trigger will enable the TR to be active for enough data transfer as specified by the TRIGGER1_TYPE Field. 0: No Trigger 1: Global Trigger 0 for the channel 2: Global Trigger 1 for the channel 3: Local Event for the channel This field is reserved for a Direct TR.	"TRIGGER INFO: TR Triggers" on page 10-282
14-15	TRIGGER1_TYPE	This is the type of data transfer that will be enabled by receiving a trigger. 0: The second inner most loop (ICNT1) will be decremented by 1. 1: The third inner most loop (ICNT2) will be decremented by 1. 2: The outer most loop (ICNT3) will be decremented by 1. 3: The entire TR will be allowed to complete.	"TRIGGER INFO: TR Triggers" on page 10-282 "TRIGGERX_TYPE: Trigger Type" on page 10-282
16-23	CMD ID	The Command ID for the TR. This will be sent back with the response for the Channel controller to identify the specific TR. For a Direct TR this is the event number that will be generated based on the event trigger.	"CMD ID: Command ID Field Definition" on page 10-282
24-31	Configuration Specific Flags	These are flag bits that can be specified by the specific UTC configuration. These bits will remain undefined in the global TR format to allow for customization requirements that might be required in the specific UTC implementation.	"Configuration Specific Flags Definition" on page 10-282
End of Table 10-4			

The following sections expand on each of these fields.

10.4.1 Type: TR Type Field

The TR Type field gives the size of the TR and which fields are expected in the TR. Based on the type the number of words required to be sent to complete the TR can be decreased. The table below shows the minimum size for each TR as well if any fields will be treated as reserved.

Figure 10-3 Transfer Request Minimum Size Type 0 One Dimensional Transfer

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0	ADDR	RESERVED/NOT REQUIRED												

Figure 10-4 Transfer Request Minimum Size Type 1 Two Dimensional Transfer

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	RESERVED/NOT REQUIRED											

Figure 10-5 Transfer Request Minimum Size Type 2 Three Dimensional Transfer

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2	DIM2	RESERVED/NOT REQUIRED									

Figure 10-6 Transfer Request Minimum Size Type 3 Four Dimensional Transfer

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2/ 3	DIM2	DIM3	RESERVED/NOT REQUIRED								

Figure 10-7 Transfer Request Minimum Size Type 4 Four Dimensional Transfer with Formatting

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2/ 3	DIM2	DIM3	FMT FLAGS	RESERVED/NOT REQUIRED							

Figure 10-8 Transfer Request Minimum Size Type 5 Cache Warm

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2/ 3	DIM2	DIM3	CACHE FLAGS	RESERVED/NOT REQUIRED							

Figure 10-9 Transfer Request Minimum Size Type 8 Four Dimensional Block Copy

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2/ 3	DIM2	DIM3	FMT FLAGS	DDIM1	DADDR	DDIM2	DDIM3	RESERVED/NOT REQUIRED			

Figure 10-10 Transfer Request Minimum Size Type 9 Four Dimensional Block Copy with Repacking

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	ICNT2/ 3	DIM2	DIM3	FMT FLAGS	DDIM1	DADDR	DDIM2	DDIM3	DICNT0 /1	DICNT2 /3		

Figure 10-11 Transfer Request Minimum Size Type 10 Two Dimensional Block Copy

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	RESERVED				FMT FLAGS	DDIM1	DADDR	RESERVED/NOT REQUIRED				

Figure 10-12 Transfer Request Minimum Size Type 11 Two Dimensional Block Copy with Repacking

word 0	word 1	word 2	word 3	word 4	word 5	word 6	word 7	word 8	word 9	word 10	word 11	word 12	word 13	word 14	word 15
FLAGS	ICNT0/ 1	ADDR	DIM1	RESERVED				FMT FLAGS	DDIM1	DADDR		RESERVED		DICNT0 /1	RESERV ED/NO T REQUIR ED

10.4.2 STATIC: Static Field Definition

The static field is used to allow for a single TR to be reused repeatedly. If the static bit is set then the UTC upon completing the specified TR will restart the TR with the same values as were in the initial TR. This TR will stay active until the channel is torn down. The initiating of a tear down of the channel will clear the STATIC field in the active TR and it will be removed from the channel FIFO after the TR completion.



Note—If a TR is submitted to a channel FIFO with the STATIC field set then any TRs placed in the FIFO after that will NOT be run until the teardown has been initiated.

10.4.3 EOL: End of Line Definition

The end of line field is used to allow for a TR to present its data on the PSI-L bus as a two dimension construct with the SOL and EOL signals being generated and the data being packed along these boundaries. In this case the SOL and EOL will be generated based on the values of the ICNT0. This is only used for a Split TR type of transaction and is ignored for all other TR Types.

10.4.4 EVENT_SIZE: Event Generation Definition

The UTC allows for an event to be generated at specified intervals for each TR. As the TR passes through the various loops in the addressing algorithm it will generate an event that then can be routed to other event receivers, such as other channels or interrupts to processors.

Table 10-5 Event Size Encoding

Value	Event Type	When it Fires
0	Completion Event	When the TR is complete and all status for the TR has been received.
1	ICNT1 Decrement	Type 0: When the last data transaction is sent for the TR. Type 1-11: When ICNT1 is decremented
2	ICNT2 Decrement	Type 0 - 1,10-11: When the last transaction is sent for the TR. All Other Types: When ICNT2 is decremented
3	ICNT3 Decrement	Type 0 - 2,10-11: When the last transaction is sent for the TR. All Other Types: When ICNT3 is decremented

10.4.5 TRIGGER INFO: TR Triggers

The TR allows for two different triggers to be set to allow for the data transfer to be prevented or halted through the process of transferring the data. The triggers are specified by selecting a type of trigger and the size of the transfer that can be allowed for the receipt of a given trigger. The triggers themselves in the UTC are collected on a per channel basis and for each of three sources. Anytime the trigger event is received the internal counter is incremented. The trigger event will not be cleared until the specified block has started its transfer and it will **only** clear the triggers that are active.



CAUTION—This does allow for one TR in the channel to use global event 0 and the next TR to use global event 1. If while the first TR is running global event 0 can increment and will decrement each time the TR reaches the specified level. At the same time global event 1 will be incremented whenever it is received but it will not decrement until the next TR runs. If the global event 1 counter reaches overflows then an error event will NOT be generated but the event will be lost.

10.4.5.1 TRIGGERX_TYPE: Trigger Type

The trigger type sets the value of TR_TRIGX as shown in “[Addressing algorithm](#)” on page 10-275. The TR_TRIGX value allows for the temporary halting of the TR until the expected event has been received. After the given loop has been entered the selected trigger will be decremented.

10.4.5.2 TRIGGERX: Trigger Selection

The TR is able to select up to 2 of 3 trigger sources for a given TR. The events can come from the dedicated local event on the UTC itself or from two assigned global events that come from the PSI-L interface on the UTC. The trigger counters are always enabled so that events can be received prior to the TR getting loaded. The trigger counters are only decremented when the trigger is active and the TR has scheduled the first transfer of the transaction.

10.4.6 CMD ID: Command ID Field Definition

The command ID is used as a unique identifier for the TR. The value is not used during the data transfer but is only used by the TR Completion Response to allow for hardware and software to identify the specific TR.

10.4.7 Configuration Specific Flags Definition

The configuration specific flags is being intentionally left open to allow for the development of custom DMAs to be created that can still use the standard TR format. The definition of the field will be defined in the specific UTCs specification. If it is not used then the field should be filled with 0's.

10.5 TR Address and Size Attributes

The fields described below all describe the data transfer that needs to be made as described in “[Linear Memory Addressing Block Move Algorithm](#)” on page 10-275

10.5.1 ICNT0

The ICNT0 is the number of elements to transfer in the inner loop of the TR. If the TR type does not contain a FMTFLAGS field then this count is assumed to be the number of bytes to transfer. If the FMTFLAGS field is included in the types then the number of bytes to be transferred will be ICNT0 times the Element size rounded up to the nearest byte.

10.5.2 ICNT1

The ICNT1 field is the loop count for the second innermost loop count as defined in “Addressing algorithm” on page 10-275.

10.5.3 ADDR

The address location is the initial address that will be accessed at the start of the transfer. All of the dimensions will also be based off of this value. This address can either physical or virtual based upon sideband signals that are captured during the TR submission.

10.5.4 DIM1

This is the offset of the address from the initial address for the first access of the second loop. This will be added to the address from the loop before. This is a signed value so that the address can be both above and below the initial address.

10.5.5 ICNT2

This is the count for the third innermost loop in the addressing algorithm.

10.5.6 ICNT3

This is the count for the outermost loop in the addressing.

10.5.7 DIM2

This is the offset of the address from the initial address to the next address in the third innermost loop. This is a signed vales so that the address can be both above and below the previous address.

10.5.8 DIM3

This is the offset of the address from the initial address to the next address in the outermost loop. This is a signed vales so that the address can be both above and below the previous address.

10.5.9 DDIM1

This is the offset of the destination address from the initial destination address for the first access of the second loop. This will be added to the destination address from the loop before. This is a signed value so that the destination address can be both above and below the initial destination address.

10.5.10 DADDR

The destination address location is the initial destination address that will be accessed at the start of the transfer. All of the dimensions will also be based off of this value. This address can either physical or virtual based upon sideband signals that are captured during the TR submission.

10.5.11 DDIM2

This is the offset of the destination address from the initial destination address for the first access of the third loop. This will be added to the destination address from the loop before. This is a signed value so that the destination address can be both above and below the initial destination address.

10.5.12 DDIM3

This is the offset of the destination address from the initial destination address for the first access of the outer loop. This will be added to the destination address from the loop before. This is a signed value so that the destination address can be both above and below the initial destination address.

10.5.13 DICNT0

This is the number of elements to use in the destination if the TR is repacking the data. This number reflects the number of elements to be transferred.

10.5.14 DICNT1

This is the number of times to execute the second loop to use in the destination transfer.

10.5.15 DICNT2

This is the number of times to execute the third loop to use in the destination transfer.

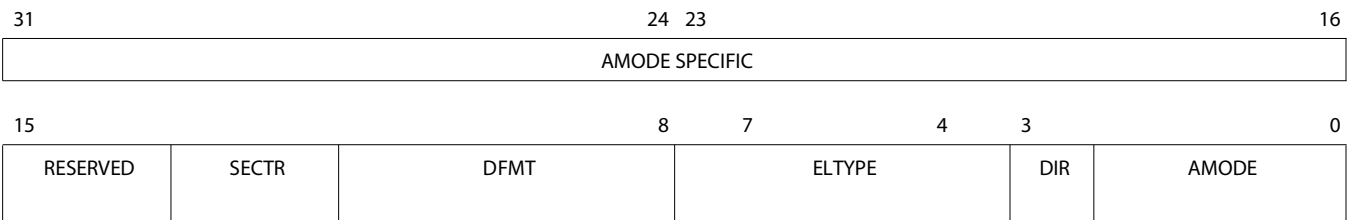
10.5.16 DICNT3

This is the number of times to execute the fourth loop to use in the destination transfer. The value of DICNT0 * DICNT1 * DICNT2 * DICNT3 must equal ICNT0 * ICNT1 * ICNT2 * ICNT3. If any of the values are zero they are NOT included in the multiplication.

10.6 FMTFLAGS

The format flags are used to specify special formatting of the transfer. The figure below shows the definition of the sub-fields in the FMTFLAGS field.

Figure 10-13 Transfer Request Template FMTFLAGS Field



The flags field fills in the remaining details about the stream, as follows:

Table 10-6 FMTFLAGS Field Descriptions (Part 1 of 2)

Bit	Field	Description	Reference
0-2	AMODE	The Addressing Mode of TR that is being sent 0: Linear Addressing 1: Circular Addressing 2-7: Reserved for future use	“AMODE: Addressing Mode Definition” on page 10-285
3	DIR	For a block type transfer does the addressing mode apply for source or destination. The other one will use linear. If it is a non-block type TR then this field is ignored	“DIR: Addressing Mode Direction Definition” on page 10-286
4-7	ELTYPE	The type of element and its size and potential output size for the innermost loop. If this is not specified in the TR then a byte is the assumed element.	“ELTYPE: Element Type Definition” on page 10-286
8-11	DFMT	The data reformatting function to apply in the middle of the TR	“DFMT:Data Formatting Algorithm Definition” on page 10-287

Table 10-6 FMTFLAGS Field Descriptions (Part 2 of 2)

Bit	Field	Description	Reference
12-13	SECTR	A secondary TR is required and the format of the secondary TR.	“SECTR: Secondary Transfer Request Definition” on page 10-287
14-15	Reserved	Reserved for Future use	
16-31	AMODE SPECIFIC	This 16 bit field is defined based on the addressing mode used as selected in the AMODE field.	“AMODE SPECIFIC: Addressing Mode Field” on page 10-291
End of Table 10-6			

10.6.1 AMODE: Addressing Mode Definition

The addressing mode field allows for different algorithms to be used other than the default linear addressing mode described in [“Addressing algorithm”](#) on page 10-275.

10.6.1.1 Linear Addressing

This is the addressing that is used if there is no FMTFLAGS field in the TR.

10.6.1.2 Circular Addressing

Circular addressing modifies how the UTC performs address arithmetic so that addresses remain within a power-of-2 sized window. Circular addressing works by holding upper address bits constant during an address update, while allowing the lower address bits to vary. This contrasts with the default—linear addressing—which allows all of the address bits to vary.

The UTC TR provides address mode selection for each level of loop nest. Each level can select between linear addressing or circular addressing with one of two circular block sizes.

Conceptually, selectable addressing mode support replaces all address arithmetic with a function similar to the following.

Example 10-2 Address Arithmetic with Selectable Linear / Circular Addressing Modes

```

-----
enum addr_mode_t
{
    LINEAR,           // Linear addressing mode
    CIRC0,            // Circular addressing with block size 0
    CIRC1             // Circular addressing with block size 1
};

uint64_t circ_mask_0; // Circular addressing mask for block size 0
uint64_t circ_mask_1; // Circular addressing mask for block size 1

uint64_t address_add( uint64_t base, int32_t offset, addr_mode_t mode )
{
    uint64_t new_addr = base + offset;

    if ( mode == LINEAR )
        return new_addr;

    // Look up address mask based on selected circular buffer size.
    // Mask contains 1s for bits that remain fixed, and 0s elsewhere.
    uint64_t addr_mask = mode == CIRC0 ? circ_mask_0 : circ_mask_1;

    return ( base & addr_mask ) | ( new_addr & ~addr_mask );
}

```

End of Example 10-2

The `address_add` primitive then takes the place of plain addition for all TR address computations in this mode. The following example illustrates `address_add`.

Example 10-3 Linear Stream Addressing Algorithm with Circular Addressing Add

```

for (i3 = 0; i3 < ICNT3; i3++)
{
    ptr3 = ptr; // save current position before entering next level
    for (i2 = 0; i2 < ICNT2; i2++)
    {
        ptr2 = ptr; // save current position before entering next level
        for (i1 = 0; i1 < ICNT1; i1++)
        {
            ptr1 = ptr; // save current position before entering next level
            for (i0 = 0; i0 < ICNT0; i0++)
            {
                fetch( ptr, ELEM_BYTES );
                ptr = address_add( ptr, ELEM_BYTES, addr_mode_0 );
            }
            // Update based on saved pointer for this level
            ptr = address_add( ptr1, DIM1, addr_mode_1 );
        }
        // Update based on saved pointer for this level
        ptr = address_add( ptr2, DIM2, addr_mode_2 );
    }
    // Update based on saved pointer for this level
    ptr = address_add( ptr3, DIM3, addr_mode_3 );
}

```

End of Example 10-3

The values for the circular addressing can be selected by using the 16 bits of the AMODE specific Field as described in [AMODE SPECIFIC: Addressing Mode Field](#).

10.6.2 DIR: Addressing Mode Direction Definition

This field is used if the TR TYPE specifies the TR is for a block move if the addressing mode specified in the AMODE field applies to the source addressing or the destination addressing. A value of 0 means the source addressing will use the method selected in the AMODE field and the destination address will use the default linear addressing. A value of 1 means that the destination addressing will use the method selected in the AMODE field and the source addressing will use the default linear addressing. If the TR TYPE is not a block transfer then this field will be ignored.

10.6.3 ELTYPE: Element Type Definition

The Element Type field allows to specify the basic unit that is used for the innermost loop. If the FMFLAGS field is not present then the UTC will assume a value of 0 which is a byte size element. The element type also allows for the some expansion or contraction of the element size from the source to the destination.

Table 10-7 ELTYPE Encoding

Encoding	Definitino
0	1 Byte per element
1	1.5 Bytes(12 bits) per element
2	2 Bytes per element
3	3 Bytes per element
4	4 Bytes per element
5	8 Bytes per element
6	16 Bytes per element
7	32 Bytes per Element

Table 10-7 ELTYPE Encoding

Encoding	Definitino
8	1 Byte per Input Element 2 Bytes per Output Element
9	1.5 Bytes per Input Element 2 Bytes per Output Element
10	2 Bytes per Input Element 1 Byte per Output Element
11	2 Bytes per Input Element 1.5 Bytes per Output Element
12-15	Reserved for Future Use
End of Table 10-7	

10.6.4 DFMT:Data Formatting Algorithm Definition

The Data Formatting Algorithm is a method to specify a manipulation of the data between how it is read and how it is sent. If the TR Type is not a block operation TR then this field will be ignored. Otherwise the table below specifies what the encodings mean.

Table 10-8 DFMT Encoding

Encoding	Definitino
0	No Change. The input and output block will remain identical
1	Constant Copy. The input block is not an address but the address is up to a 64 bit constant
2	Transpose. The inner and second most inner loops are swapped so that rows become columns and columns become rows.
3	Reverse. The data in the row will be accessed in the reverse of the order that it is read. So the largest address read in a row will be the first address written.
4	Reverse Transpose. The data will be written in the reverse of the order that is read as well as transposed. So the largest address read in a row will be the address in the first line being written out.
5	Compress the input data the output data will be after compression
6	Decompress the input data the output data will be the original data that was read from the compression TR
5-15	Reserved for Future Use
End of Table 10-8	

10.6.5 SECTR: Secondary Transfer Request Definition

The Secondary Transfer Request field allows to specify if the TR requires an additional TR located in memory. If this is used then the address given in the TR is the pointer to the Secondary TR and the secondary TR will have a pointer to the actual final TR.

Table 10-9 Secondary Transfer Request Format

Encoding	Size	Definition
0	0	The TR does not require a secondary TR
1	64 bytes	The TR will fetch a 64 byte Secondary TR prior to the initial read.
2	128 bytes	The TR will fetch a 128 byte Secondary TR prior to the initial read.
3	TBD	Reserved for future use
End of Table 10-9		

10.6.5.1 Secondary TR Formats

The secondary TR format is a TR that is fetched by the UTC when it is executing a TR to fill in additional information that does not fit in the original TR. It is not included in the standard TR as the information required in it is not required until execution time and is kept separate so the Channel FIFO space can be minimized. The first four words of the Secondary TR are always the real start address of the transfer followed by a Secondary TR Flags field. The remaining words in the secondary TR are defined by the Secondary TR TYPE from the Secondary TR Flags field.

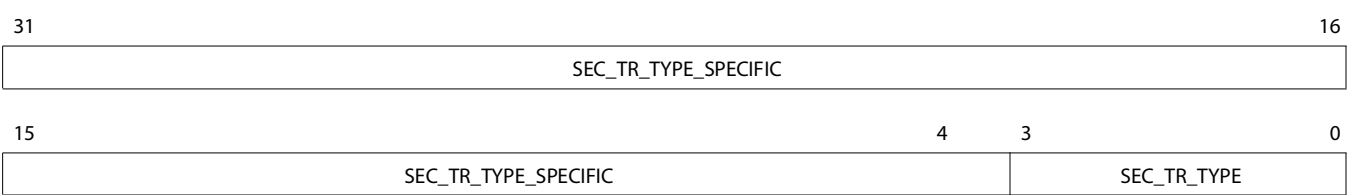
Figure 10-14 Secondary Transfer Request Template for 64 Byte Secondary TR

word 15	word 14	word 13	word 12
Type Specific			
word 11	word 10	word 9	word 8
Type Specific			
word 7	word 6	word 5	word 4
Type Specific			
word3	word 2	word 1	word0
Type Specific	SECONDARY TR FLAGS	ADDR	

10.6.5.2 Secondary TR FLAGS

The next diagram expands the generic 32-bit Secondary TR FLAGS field. The numbers above each field here indicate bit numbers within the field.

Figure 10-15 SECONDARY TR FLAGS Field Definition



The flags field fills in the remaining details about the stream, as follows:

Table 10-10 FLAGS Field Descriptions

Bit	Field	Description	Reference
0-3	SEC_TR_TYPE	The TYPE of SEC_TR 0: Multiple Buffer Interleave 1: Compression/Decompression Table 2-15: Reserved	"Type: TR Type Field" on page 10-279
4-31	SEC_TR_TYPE_SPECIFIC	Reserved for the SEC_TR_TYPE to define based on its requirements	"STATIC: Static Field Definition" on page 10-281
End of Table 10-10			

The following sections expand on each of these fields.

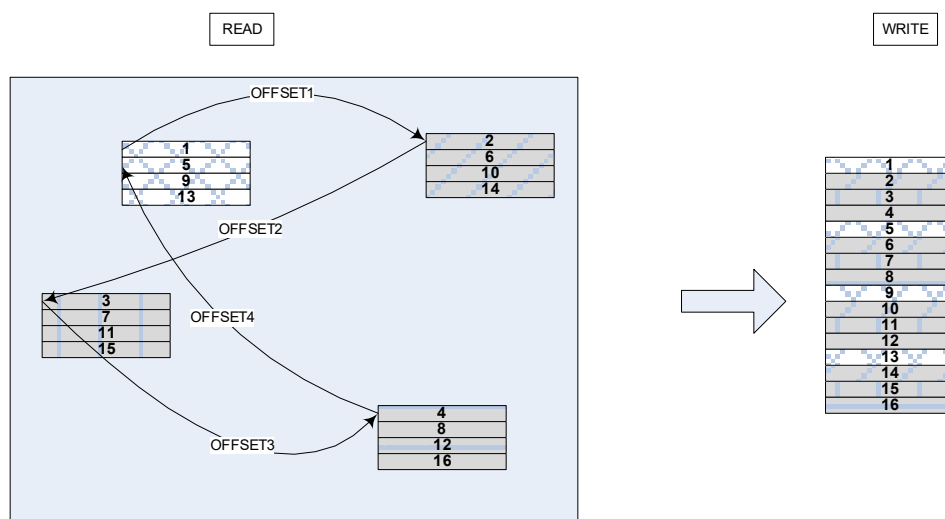
10.6.5.2.1 SEC_TR_TYPE: Secondary TR Type Field

The Secondary TR Type field is used to state the purpose of the secondary TR. This can be defined

10.6.5.2.2 Multiple Buffer Interleave

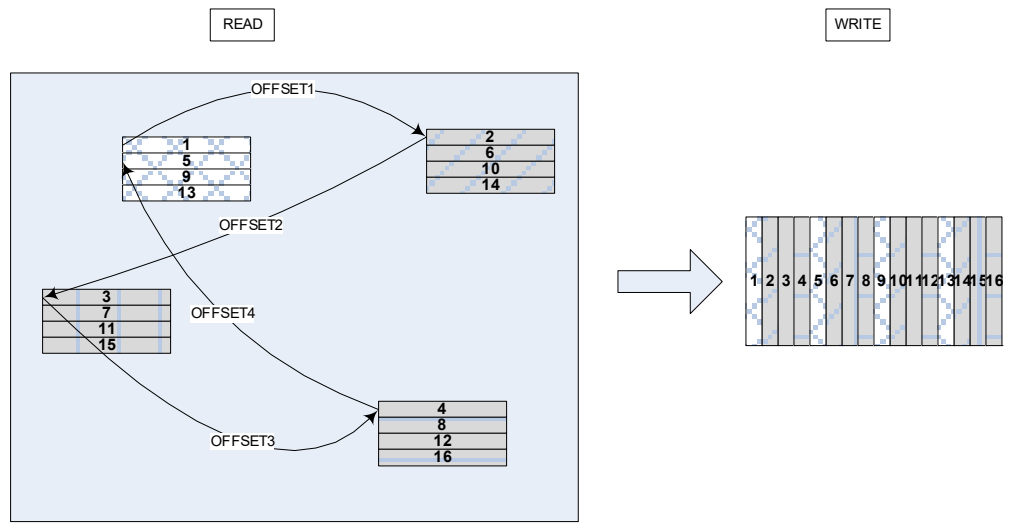
The multiple buffer interleave is used to have a single TR plus Secondary TR that can define a fetch where a single line is selected from multiple buffers and then sent as a single buffer. This can be used along with transpose of the data to present data in slices optimal for processing the same algorithm on different data sets at the same time. An example of this can be seen in the figure below.

Figure 10-16 Example of Multiple Buffer Interleaving Read with 4 Offsets



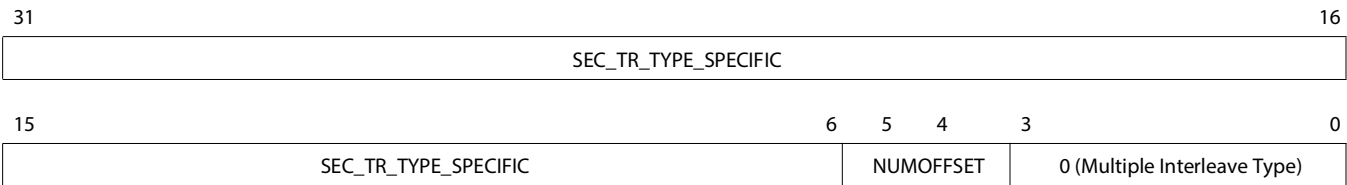
The case above provides the ability to minimize the number of TRs submitted to make the transfer but the interleaved Read main advantage comes when used with the transpose mode enabled. The figure below shows the advantage of the interleaved mode along with the transpose mode to allow for maximum size transfer on both read and write transactions.

Figure 10-17 Example of Interleaved Read with 4 Offsets and Transpose Enabled



As the figure shows the data that is written out now contains one element from each block of memory and can be read in directly to processing units.

Figure 10-18 SECONDARY TR FLAGS Field Definition



The flags field fills in the remaining details about the stream, as follows:

Table 10-11 Multiple Buffer Interleave FLAGS Field Descriptions

Bit	Field	Description	Reference
0-3	SEC_TR_TYPE	0: Multiple Buffer Interleave	“SEC_TR_TYPE: Secondary TR Type Field” on page 10-289
4-5	NUMOFFSET	The encoded value for the number of offsets. 0: 4, 1: 8, 2: 16	
6-31	Reserved	Reserved for Future Use	
End of Table 10-11			

Secondary TR TYPE from the Secondary TR Flags field.

Figure 10-19 Multiple Buffer Interleave Secondary Transfer Request Template 128 Byte Secondary TR

word 31	word 30	word 29	word 28
RESERVED			
word 27	word 26	word 25	word 24
RESERVED			

Figure 10-19 Multiple Buffer Interleave Secondary Transfer Request Template 128 Byte Secondary TR

word 23	word 22	word 21	word 20
RESERVED			
word 19	word 18	word 17	word 16
OFFSET15	OFFSET14	OFFSET13	OFFSET12
word 15	word 14	word 13	word 12
OFFSET11	OFFSET10	OFFSET9	OFFSET8
word 11	word 10	word 9	word 8
OFFSET7	OFFSET6	OFFSET5	OFFSET4
word 7	word 6	word 5	word 4
OFFSET3	OFFSET2	OFFSET1	OFFSET0
word3	word 2	word 1	word0
RESERVED	SECONDARY TR FLAGS	ADDR	

10.6.5.2.2.1 Multiple Buffer Interleave Functionality

Multiple Buffer Interleave will work by setting the SECTR field in the TR presented to the channel to be either 64 or 128 bytes depending on the number of offsets required. An offset number of 4 or 8 will require 64 bytes while 16 offsets will require 128 bytes. When the channel becomes active it will see that the SECTR field is nonzero and fetch the SECTR of the size specified. The first access from the channel will be a read of the secondary TR as specified in the address field of the TR. Upon receiving the Secondary TR the channel will parse the type field and seeing that it is a Multiple Buffer interleave will load the offset index fields for the channel. Additionally it will load the ADDRESS field from the secondary TR into the Address of the actual TR to allow it to process as normal.

The first address fetch will then be to this SEC_TR_ADDRESS. After finishing the inner loop the next address will be SEC_TR_ADDRESS + OFFSET0.

Note—ICNT1 should be equal to the height of the transfer blocks times the number of offsets.

10.6.6 AMODE SPECIFIC: Addressing Mode Field

The AMODE Specific Fields are defined based upon the Addressing Mode being used as specified in [AMODE: Addressing Mode Definition](#).

10.6.6.1 Circular Address Mode Specific Flags

If the circular addressing mode is selected in AMODE the definition of the upper 16 bits of the FMTFLAGS field are used to select the type of addressing for each entry in the loop and to define 2 unique masks that can be used for the circular buffering.

The following example code shows the method used to compute the masks referred to by the `address_add` function described in [“Address Arithmetic with Selectable Linear / Circular Addressing Modes”](#) on page 10-285.

Example 10-4 Circular Address Mask Computation

```
// Circular addressing masks.
// 1 bits in the mask indicate address bits that remain fixed.
// 0 bits in the mask indicate address bits that are allowed to vary.
uint64_t circ_mask_0; // mask for circular addressing block 0
uint64_t circ_mask_1; // mask for circular addressing block 1

void compute_circular_address_masks( int cbk0, int cbk1 )
{
    int block_size_0 = cbk0 + 9; // power-of-2 of block size in bytes
    int block_size_1 = cbk0 + cbk1 + 10; // power-of-2 of block size in bytes

    if ( block_size_1 > 31 ) block_size_1 = 32; // clamp to 4GB maximum size

    circ_mask_0 = (~0ULL) << block_size_0;
    circ_mask_1 = (~0ULL) << block_size_1;
}
```

End of Example 10-4

Figure 10-20 Transfer Request Template FMTFLAGS AMODE SPECIFC Circular Addressing Field

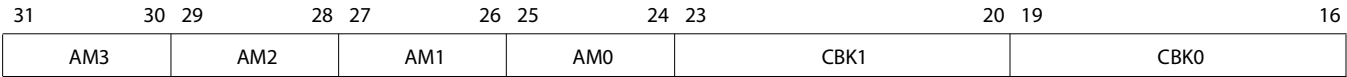


Table 10-12 Circular Address Mode Specific Flags Field Descriptions

Bit	Field	Description	Reference
16-19	CBK0	Specifies the circular block 0 value as used in Circular Address Mask Computation	“CBK0 and CBK1: Circular Block Size Selection” on page 10-292
20-23	CBK1	Specifies the circular block 1value as used in Circular Address Mask Computation	“CBK0 and CBK1: Circular Block Size Selection” on page 10-292
24-25	AM0	The address mode to use with ICNT0 loop	“AMx: Addressing Mode Selection” on page 10-293
26-27	AM1	The address mode to use with ICNT1 loop	“AMx: Addressing Mode Selection” on page 10-293
28-29	AM2	The address mode to use with ICNT2 loop	“AMx: Addressing Mode Selection” on page 10-293
30-31	AM3	The address mode to use with ICNT3 loop	“AMx: Addressing Mode Selection” on page 10-293
End of Table 10-12			

10.6.6.1.1 CBK0 and CBK1: Circular Block Size Selection

The CBK0 and CBK1 fields set the circular block sizes for circular addressing. CBK0 directly determines the circular block size for block 0. CBK1 combines with CBK0 to determine the circular block size for block 1. Specifically, the streaming engine sets block 1’s size according to CBK0 + CBK1 + 1. Therefore, circular block 1 is always larger than circular block 0.

The following table illustrates the resulting valid block sizes for circular block 0 and 1. For circular block 0, the use the value of CBK0 directly. Circular block 0 supports block sizes ranging from 512 bytes to 16M bytes. For circular block 1, use the value of CBK0 + CBK1 + 1. Circular block 1 supports sizes ranging from 1K bytes to 4G bytes.

Table 10-13 Circular Block Size Decoding

Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size	Encoded Block Size	Decoded Block Size
0	512	8	128K	16	32M	24	Reserved
1	1K	9	256K	17	64M	25	Reserved
2	2K	10	512K	18	128M	26	Reserved
3	4K	11	1M	19	1M	27	Reserved
4	8K	12	1M	20	1M	28	Reserved
5	16K	13	1M	21	1M	29	Reserved
6	32K	14	1M	22	1M	30	Reserved
7	64K	15	1M	23	1M	31	Reserved
End of Table 10-13							

10.6.6.1.2 AMx: Addressing Mode Selection

The fields AM0 through AM3 control the addressing modes for each of the 4 dimensions. The UTC supports the following addressing modes:

Table 10-14 AMX Address Mode Selection Encoding

AMX	Meaning
00b	Linear addressing
01b	CKB0 (Circular Buffer 0 is used)
10b	CKB1 (Circular Buffer 1 is used)
11b	Reserved
End of Table 10-14	

Each dimension has its own addressing mode, independent of the other dimensions. This permits any mixture of linear and circular addressing among the 4 dimensions.

When a given dimension selects linear addressing, the UTC uses unmodified 2s complement arithmetic to update addresses for that loop level.

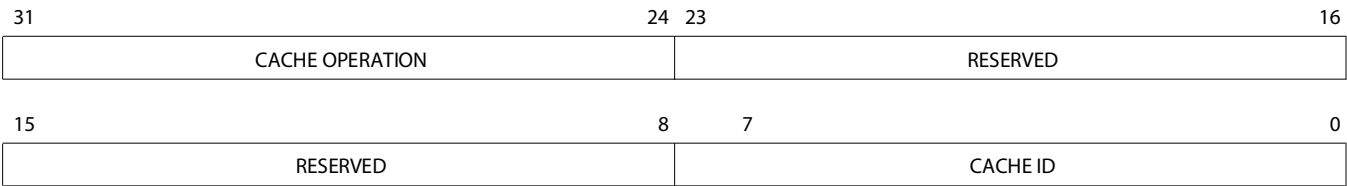
When a given dimension selects circular addressing via CBK0 or CBK1, the UTC uses the circular address arithmetic described above in [“Address Arithmetic with Selectable Linear / Circular Addressing Modes”](#) on page 10-285

10.6.7 Cache Flags

If the TR type is a cache warm the format flags field is replaced with a Cache Flags field.

The figure below shows the breakdown of the 32 bit Cache Flags field.

Figure 10-21 Transfer Request Template CACHEFLAGS Field



The flags field fills in the remaining details about the stream, as follows:

Table 10-15 FMTFLAGS Field Descriptions

Bit	Field	Description
0-6	Cache ID	The Cache number to perform the operation on. 0-127. This field is ignored if the L3 Cache Field is set
7	L3 Cache	The Cache Operation should be applied to the L3 Cache in the system.
8-23	Reserved	Reserved for future use
24-31	Cache Operation	0: Prewarm the Cache Unique 1: Prewarm the Cache Shared 2: Prewarm MMU 3-255: Reserved for future use
End of Table 10-15		

10.7 Compression and Decompression

The Compression and Decompression as selected by the DFMT field in the Format flags use the TR to specify the number of Super Blocks that are going to be compressed or decompressed. The compression and decompression requires a secondary TR to be fetched to give the full information for the AFC of how to compress the set of super blocks defined by the TR.

The super block is itself a 2D structure that is defined inside the secondary TR that has a width defined by SB_ICNT0 and height SB_ICNT1. To avoid having to add shifting logic the SB_ICNT0 value is required to be a 128 bit value and 128 bit aligned at all times.

10.7.1 Compression and Decompression Addressing Description

The following code illustrates the basic algorithm for the implied 6 level loop nest block move as used in compression. This model assumes that it transfers data 1 byte at a time and the input and output block are the same size in all dimensions.

Example 10-5 Linear Memory Addressing Block Move Algorithm for Compression

```

// sptr is a byte pointer.
// dptr is a byte pointer to the compression table.

// cdptr is a byte pointer to the compressed data
// TR_TRIGX can be selected to come from Global events, Local event, or none.
// secondary_TR_address is a byte pointer
// Check for trigger of TYPE3
if (TR_TRIG0_TYPE == TYPE3)
    while (!TR_TRIG0);
if (TR_TRIG1_TYPE == TYPE3)
    while (!TR_TRIG1);

sptr = ADDR;

dptr = DADDR;

secondary_TR_address = *dptr;
cdptr = secondary_TR_address;

compress_offset = 0;

dptr = dptr + 32; // First table write is at the secondary TR pointer in dptr + 32
for (i3 = 0; i3 < ICNT3; i3++)
{
    sptr3 = sptr; // save current position before entering next level
    dptr3 = dptr; // save current position before entering next level

    // Check for trigger of TYPE2
    if (TR_TRIG0_TYPE == TYPE2)
        while (!TR_TRIG0);
    if (TR_TRIG1_TYPE == TYPE2)
        while (!TR_TRIG1);

    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptr2 = sptr; // save current position before entering next level
        dptr2 = dptr; // save current position before entering next level

        // Check for trigger of TYPE1
        if (TR_TRIG0_TYPE == TYPE1)
            while (!TR_TRIG0);
        if (TR_TRIG1_TYPE == TYPE1)
            while (!TR_TRIG1);

        for (i1 = 0; i1 < ICNT1; i1++)
        {
            sptr1 = sptr; // save current position before entering next level
            dptr1 = dptr; // save current position before entering next level

            // Check for trigger of TYPE0
            if (TR_TRIG0_TYPE == TYPE0)
                while (!TR_TRIG0);
            if (TR_TRIG1_TYPE == TYPE0)

```

```

        while(!TR_TRIG1);
    for (i0 = 0; i0 < ICNT0; i0++)
    {
        // Compression Unit size
        sptr0 = sptr; // save current position before entering compression
        dptr0 = dptr; // save current position before entering compression
        compress_size = 0;
        for (sb1 = 0; sb1 < SB_ICNT1; sb1++)
        {
            sb_sptr1 = sptr;
            for (sb0 = 0; sb0 < SB_ICNT0; sb0++) {
                *cdptr = compress_symbol(*sptr);
                sptr = sptr += 16; // SB_ICNT0 is element based not byte
                cdptr += compress_size(*sptr);

                compress_size += compress_size(*sptr);
            }
            sptr = sb_ptr1 + SB_DIM1;
        }
        sptr = sptr1 + SDIM0;
        *dptr = compress_size;
        dptr += 4;
        *dptr = compress_offset;
        // Since CDB is 128 bits ADDRESS OFFSET needs to be multiplied by 16
        compress_offset += compress_size*16;
        dptr = dptr0 + DDIM0;
    }
    // Update based on saved pointer for this level
    sptr = sptr1 + SDIM1;
    dptr = dptr1 + DDIM1;
}
// Update based on saved pointer for this level
sptr = sptr2 + SDIM2;
dptr = dptr1 + DDIM2;
}
// Update based on saved pointer for this level
sptr = sptr3 + SDIM3;
dptr = dptr1 + DDIM3;
}

```

This form of addressing allows programs to specify regular paths through memory in a small number of parameters. Additionally, it allows for various stall points through the loop to allow for hardware or software induced pausing of the transfer.

Example 10-6 Linear Memory Addressing Block Move Algorithm for Decompression

```

// sptr is a byte pointer.
// dptr is a byte pointer to the compression table.

// csptr is a byte pointer to the compressed data
// TR_TRIGX can be selected to come from Global events, Local event, or none.

// Check for trigger of TYPE3

```

```

if(TR_TRIGO_TYPE == TYPE3)
    while(!TR_TRIGO);
if(TR_TRIG1_TYPE == TYPE3)
    while(!TR_TRIG1);

sptr = ADDR;
dptr = DADDR;

secondary_TR_address = *sptr;

sptr += 60; // SB Start Offset is the last 4 bytes of the secondary TR
SB_Start_Offset = *sptr;

csptr = secondary_TR_address;
compress_offset = 0;
compress_size = 0;

decompress_start = sptr + signed(SB_Start_Offset); // First table read is from the
SB_Start_Offset and the sptr

for (i3 = 0; i3 < ICNT3; i3++)
{
    sptr3 = sptr; // save current position before entering next level
    dptr3 = dptr; // save current position before entering next level

    // Check for trigger of TYPE2
    if(TR_TRIGO_TYPE == TYPE2)
        while(!TR_TRIGO);
    if(TR_TRIG1_TYPE == TYPE2)
        while(!TR_TRIG1);

    for (i2 = 0; i2 < ICNT2; i2++)
    {
        sptr2 = sptr; // save current position before entering next level
        dptr2 = dptr; // save current position before entering next level

        // Check for trigger of TYPE1
        if(TR_TRIGO_TYPE == TYPE1)
            while(!TR_TRIGO);
        if(TR_TRIG1_TYPE == TYPE1)
            while(!TR_TRIG1);

        for (i1 = 0; i1 < ICNT1; i1++)
        {
            sptr1 = sptr; // save current position before entering next level
            dptr1 = dptr; // save current position before entering next level

            // Check for trigger of TYPE0
            if(TR_TRIGO_TYPE == TYPE0)
                while(!TR_TRIGO);
            if(TR_TRIG1_TYPE == TYPE0)
                while(!TR_TRIG1);

            for (i0 = 0; i0 < ICNT0; i0++)
            {
                // Compression Unit size

                sptr0 = sptr; // save current position before entering compression
                dptr0 = dptr; // save current position before entering compression
                compress_size = *sptr;

                sptr += 4;

                compress_offset = *sptr;

                csptr = decompress_start + unsigned(uncompress_offset);

                for (sb1 = 0; sb1 < SB_ICNT1; sb1++)
                {

                    sb_sptr1 = dptr;

```

```

        for (sb0 = 0; sb0 < SB_ICNT0; sb0++) {
            *dptr = decompress_symbol(*csptr);
            dptr = dptr += 16; // SB_ICNT0 is element not byte based
            csptr += decompress_size(*csptr);
        }

        dptr = sb_ptr1 + SB_DIM1;
    }

    sptr = sptr1 + DIM0;
    dptr = dptr0 + DDIM0;
}
// Update based on saved pointer for this level
sptr = sptr1 + DIM1;
dptr = dptr1 + DDIM1;
}
// Update based on saved pointer for this level
sptr = sptr2 + DIM2;
dptr = dptr1 + DDIM2;
}
// Update based on saved pointer for this level
sptr = sptr3 + DIM3;
dptr = dptr1 + DDIM3;
}

```

10.7.2 Compression and Decompression Table Formats

10.7.2.1 Compression Table Format

The AFC module packs a byte stream into a set of 128 bit CDB that is defined as a super block. At the end of each super block the compression engine will generate a 8 byte write of the Super Block information block. These Super Block information blocks can then be read back in when decompressing the data to know the size location and decompression information to recreate the original data.

Figure 10-22 Compression Information Block Field Definition

63				32	
OFFSET from Compression Data Block Start					
31	18	17	16	15	0
Reserved for Future Use		k value used		Number of CDBs in the Compressed data stream	

The figures below show the templates used to setup the compression and decompression.

Figure 10-23 Compression Secondary Transfer Request Template 64 Byte Secondary TR

word 15	word 14	word 13	word 12
RESERVED			

Figure 10-23 Compression Secondary Transfer Request Template 64 Byte Secondary TR

word 11	word 10	word 9	word 8	
RESERVED		Unused for Read Super Block Information Entry for first Super Block in compression		
word 7	word 6	word 5	word 4	
Max Offset/CDB Table Start Offset	DDIM0	DIM0	SB ICNT1	SB ICNT0
word3	word 2	word 1	word0	
Compression Flags	SECONDARY TR FLAGS	ADDR of Compressed Data		

Table 10-16 Compression Secondary TR Field Descriptions

Word	Field	Description	Reference
0-1	ADDR of Compressed Data	The Location of where the compressed data starts.	
2	Secondary TR FLAGS	The Secondary TR Flags that are common for all secondary TRs	
3	Compression Flags	The Compression Flags used for the compression	
4	SB_ICNT0 & SB_ICNT1	The size of the super block for the compression. SB_ICNT0 is element based so since eltype must be 6 for compression is for number of 16 byte elements to be compressed.	
5	DDIM0	The dimension to use for DICNT0 in the original TR after each super block	
6	DIM0	The dimension to use for ICNT0 in the original TR after each super block is complete.	
7	Max Offset CDB Table Start Offset	For a compression TR this is the maximum offset value that can be written in the CDB table. This means that compressed data will not be written beyond ADDR + Max Offset For a decompression TR this is the offset from the secondary TR that the first CDB entry should be read from in bytes. It is a signed value.	
8	CDB0 Format	This is the format data for the first CDB table value written after the first super block. It will never be used in an initial fetch.	
9	CDB0 Offset	This is where the CDB data for this entry begins relative to the ADDR of Compressed data and is always a positive number.	
End of Table 10-16			

Figure 10-24 Compression and Decompression SECONDARY TR FLAGS Field Definition

31	4	3	0
SB_DIM0[31:4]			1 (Compression Type)

The flags field fills in the remaining details about the stream, as follows:

Table 10-17 Multiple Buffer Interleave FLAGS Field Descriptions

Bit	Field	Description	Reference
0-3	SEC_TR_TYPE	0: Multiple Buffer Interleave 1: CDB INFO Block and Decompression/Compression Configuration	"SEC_TR_TYPE: Secondary TR Type Field" in KS3 DMA Specification
4-31	SB_DIM1	The dimension to use between lines of the super block. Lower 4 bits are always 0 since minimum element size is 16 byte element.	
End of Table 10-17			

Figure 10-25 Compression FLAGS Field for Non-Differential Algorithms Definition

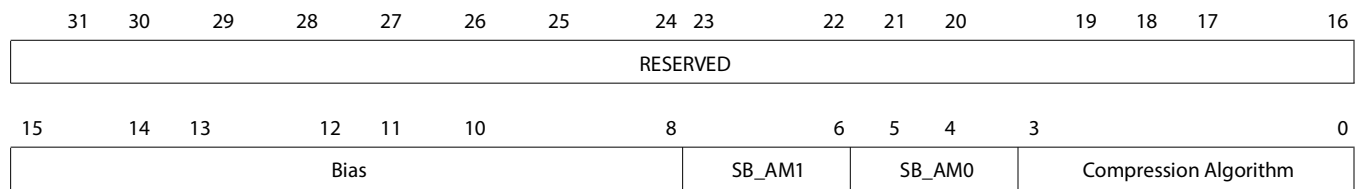


Table 10-18 Compression FLAGS Field for Non-Differential Algorithms Descriptions

Bit	Field	Description	Reference
0-3	Compression Algorithm	0: Zero Based Algorithm 1: Unsigned Golumb 2: Signed Golumb 3-7: Reserved for Future Use 8-15: Differential Type see the Differential Compression Flags Definition	
5-4	SB_AM0	The addressing mode to use for the super block during ICNT0 is decrementing. This is only used if the TR is using circular addressing for the direction of the compression engine.	
6-7	SB_AM1	The addressing mode to use for the super block when SB_ICNT1 is decremented. This is only used if the TR is using circular addressing for the direction of the compression engine.	
8-15	Bias	The constant value subtracted from the symbol before the compression algorithm is applied	
16-31	RSVD	Reserved for future use	
End of Table 10-18			

Figure 10-26 Compression FLAGS Field for Differential Algorithms Definition

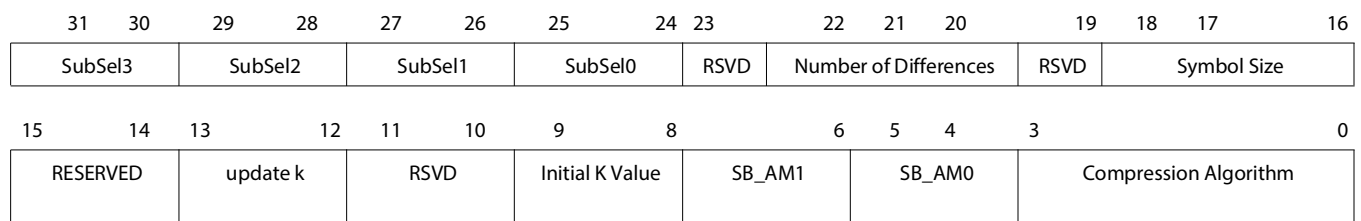


Table 10-19 Compression FLAGS Field for Differential Algorithms Descriptions (Part 1 of 3)

Bit	Field	Description	Reference
0-3	Compression Algorithm	0-7 Are non-differential algorithms see there definition in the Compression flags Field 8: Signed Exponential Golumb with differential logic 9-15: Reserved for Future use	
5-4	SB_AM0	The addressing mode to use for the super block during ICNT0 is decrementing. This is only used if the TR is using circular addressing for the direction of the compression engine.	5-4
6-7	SB_AM1	The addressing mode to use for the super block when SB_ICNT1 is decremented. This is only used if the TR is using circular addressing for the direction of the compression engine.	6-7

Table 10-19 Compression FLAGS Field for Differential Algorithms Descriptions (Part 2 of 3)

Bit	Field	Description	Reference
8-9	Initial k value	The k value to be used for the initial super block. K value for later super blocks will be based upon the optimal k value calculation logic along with the Update K field. This is combined with the symbol size. 8 bit symbol 0: k = 0 1: k = 1 2: k = 2 3: k = 3 12 bit symbol 0: k = 4 1: k = 5 2: k = 6 3 k = 7 16 bit symbol 0: k = 8 1: k = 9 2; k = 10 3: k = 11	
11-10	RSVD	Reserved for future use	
12-13	Update K Value	Describes how the k value changes from one super block to the next. 0: K is constant and uses the Initial K value specified in the Compression Flags Field 1: K value moves by 1 towards the calculated optimal value. 2: K value moves instantly to the calculated optimal value	
14-15	RSVD	Reserved for future use	
16-18	Symbol Size	The size of a signal symbol used for compression 0: 8 bit symbol 1: 12 bit symbol 2: 16 bit symbol 3-7 Reserved for Future Use	
19	RSVD	Reserved for Future Use	
22-20	Number of Differences	The number of elements to use in the differential logic 0: No subtraction 1: all symbols use SubSel0 field to determine which symbols are used 2: SubSel0 is used for even symbols and SubSel1 is used for the odd symbols 3: repeating pattern of 3 using SubSel0, SubSel1, SubSel2 4: repeating pattern of 4 using SubSel0, SubSel1, SubSel2 and SubSel3 5-7: Reserved for future Use	
23	RSVD	Reserved for Future Use	
24-25	SubSel0	Selection for the subtraction module 0: Current symbol uses the adjacent symbol 1: Current symbol uses the symbol 2 spaces away 2: Current symbol uses the symbol 3 spaces away 3: Current symbol uses the symbol 4 spaces away	

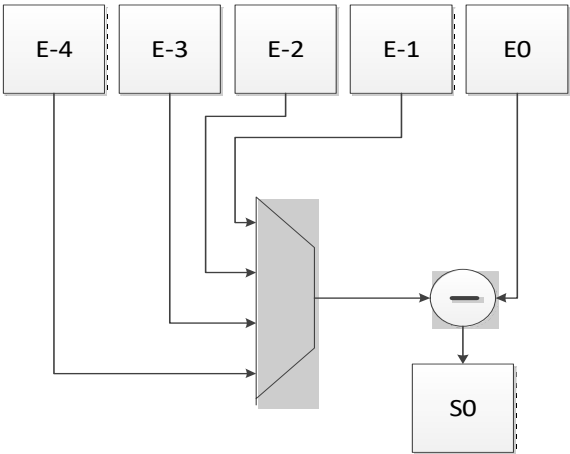
Table 10-19 Compression FLAGS Field for Differential Algorithms Descriptions (Part 3 of 3)

Bit	Field	Description	Reference
26-27	SubSel1	Selection for the subtraction module 0: Current symbol uses the adjacent symbol 1: Current symbol uses the symbol 2 spaces away 2: Current symbol uses the symbol 3 spaces away 3: Current symbol uses the symbol 4 spaces away	
28-29	SubSel2	Selection for the subtraction module 0: Current symbol uses the adjacent symbol 1: Current symbol uses the symbol 2 spaces away 2: Current symbol uses the symbol 3 spaces away 3: Current symbol uses the symbol 4 spaces away	
30-31	SubSel3	Selection for the subtraction module 0: Current symbol uses the adjacent symbol 1: Current symbol uses the symbol 2 spaces away 2: Current symbol uses the symbol 3 spaces away 3: Current symbol uses the symbol 4 spaces away	
End of Table 10-19			

10.7.2.2 Difference Engine

If the compression or decompression is selected to use differential encoding then instead of compressing the original symbol the symbol is subtracted from an adjacent symbol. The figure below shows the structure of how the selectable difference block can be used. By selecting the number of differences it can be selected how often the same subssel is used for an element.

Figure 10-27 AFC Difference Phase Block Diagram



For non-differential data the bias value will be subtracted from the symbol in the Difference phase block. After the Difference phase the symbol unbiased or differential symbol is passed to the compression engine. The compression engine will adjust the symbol based on the k factor to have the specified bits compressed while the bits

specified by the k factor selection will be passed to the packing engine without any change in value or size. The data that is compressed will be passed through the specified compression algorithm and combined with the pass through data to create a final prefix to be packed with the other data. The output prefix of each symbol will then be merged as described in the AFC section. After all the symbols are merged the engine will generate 128 bit CDBs as described in the AFC section.

10.7.3 Compression and Decompression Examples

The compression and decompression work by having the TR refer to a number of super blocks to compress or decompress. The super block is the smallest unit that can be compressed. The following sections will describe the changes to TRs to take an existing TR and convert it to be used for compression and decompression and values for various simple scenarios. All cases will follow the standard addressing algorithm with the addition that within the compression and decompression engine they use an additional 2 dimensional loop based on the super block size as specified in the secondary TR. Since the ICNT0 dimension now refers to an additional looping structure it will now decrement by 1 for each super block processed and additional dimensions were added in the secondary TR for the source and destination to use when ICNT0 is decremented. During compression the DRU writes a compression table the order of this table is what is specified by the DDIMX values in the TR as the actual compression data will be written in sequential order starting at the location specified in the secondary TR. The first CDB table write is always to the location of the secondary TR plus 32 bytes.

For decompression TRs the DIMX values are used for the order to read the table based on the previous read. The starting location of the table read is based on the CDB table start offset value which takes the address of the secondary TR and adds the offset to determine the first entry. So the following formula should be used to determine the CDB table start offset is shown below.

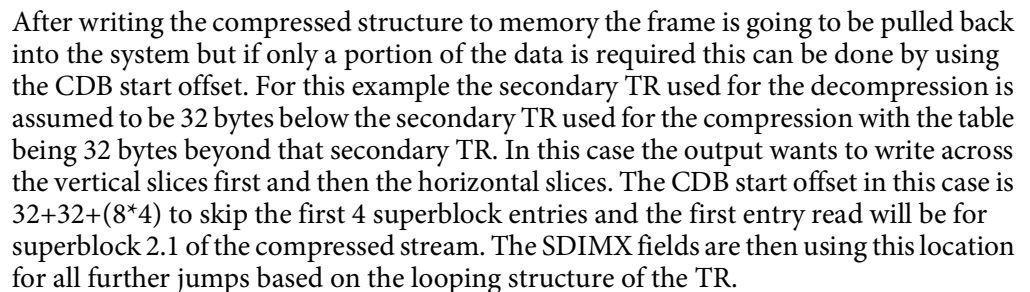
```
CDB_start_offset = ((*Compression Secondary TR + 32) + 8*(Start_SB)) -
*Decompression_Secondary_TR ;
```



Note—The order of the table is fully controlled by the compression TR so software should ensure that the table is stored in a consistent manner.

10.7.3.1 Compression of multiple slices

In this example the TR original TR is moving slices that are 4096 x 32 and there are 4 rows of data and 3 columns. The compression algorithm will break each data slice into 1024 x 16 super block. The compressed data will be 4 x 2 x 4 x 3. As the example illustrates the data is written into the compressed data structure with each superblock written sequentially after the other as they are processed by each loop count. By setting the destination dimensions the final table is written so the superblock table is accessible in raster based format. Software fully controls the how the data table will be written.



PRELIMINARY SPECIFICATION



Table 10-20 Circular Buffering Compression Field usage

Digital Signal Processor DRU Device Specification (Revision: 1.0.11)
TI Confidential—NDA Restrictions

PRELIMINARY SPECIFICATION

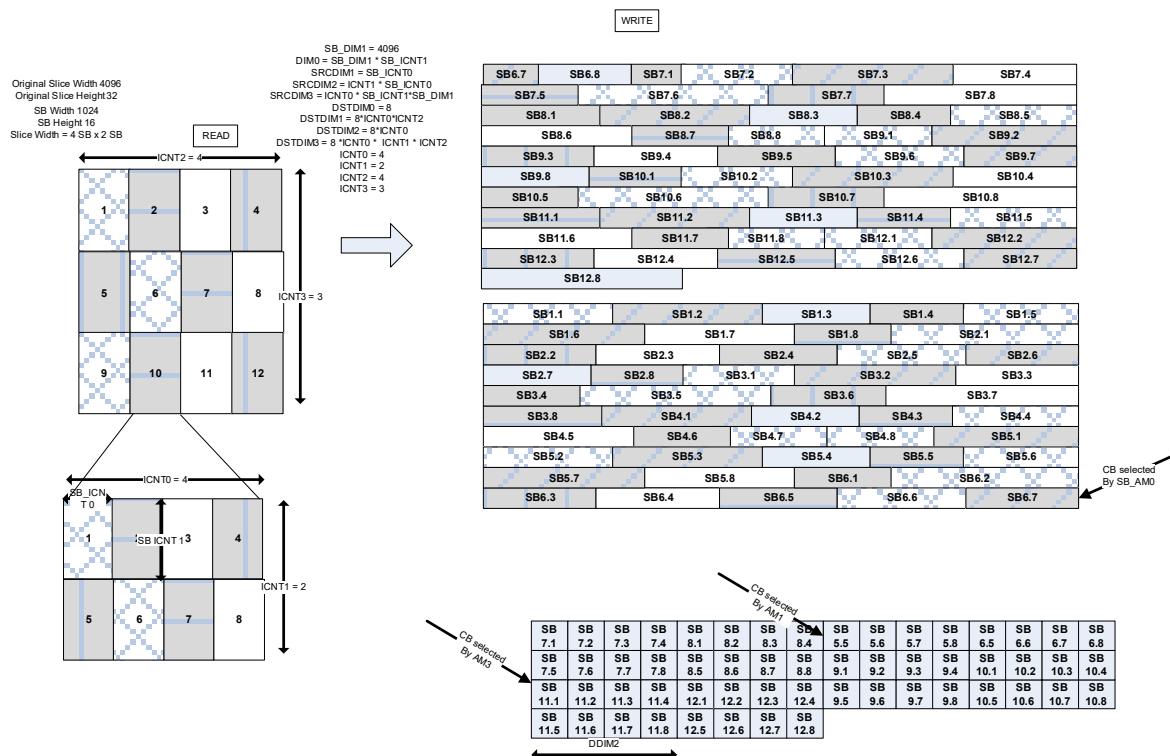
PRELIMINARY SPECIFICATION

PRELIMINARY SPECIFICATION



PRELIMINARY SPECIFICATION

Figure 10-31 Compressed Data flow of multiple slices with circular addressing DIR 0



After writing the compressed structure to memory the frame is going to be pulled back into the system but if only a portion of the data is required this can be done by using the CDB start offset. For this example the secondary TR used for the decompression is assumed to be 32 bytes below the secondary TR used for the compression with the table being 32 bytes beyond that secondary TR. In this case the output wants to write across the vertical slices first and then the horizontal slices. The CDB start offset in this case is $32 + 32 + (8 \times 4)$ to skip the first 4 superblock entries and the first entry read will be for superblock 2.1 of the compressed stream. The SDIMX fields are then using this location for all further jumps based on the looping structure of the TR.

Figure 10-32 Decompressed Data flow for multiple slices with different start location

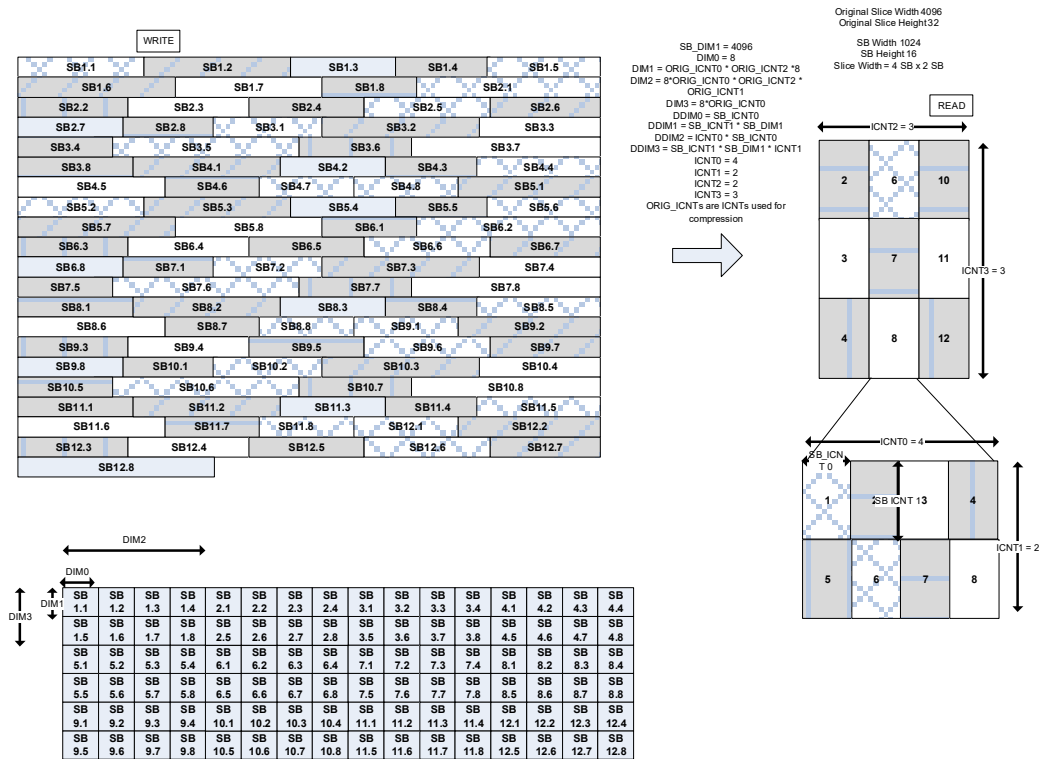
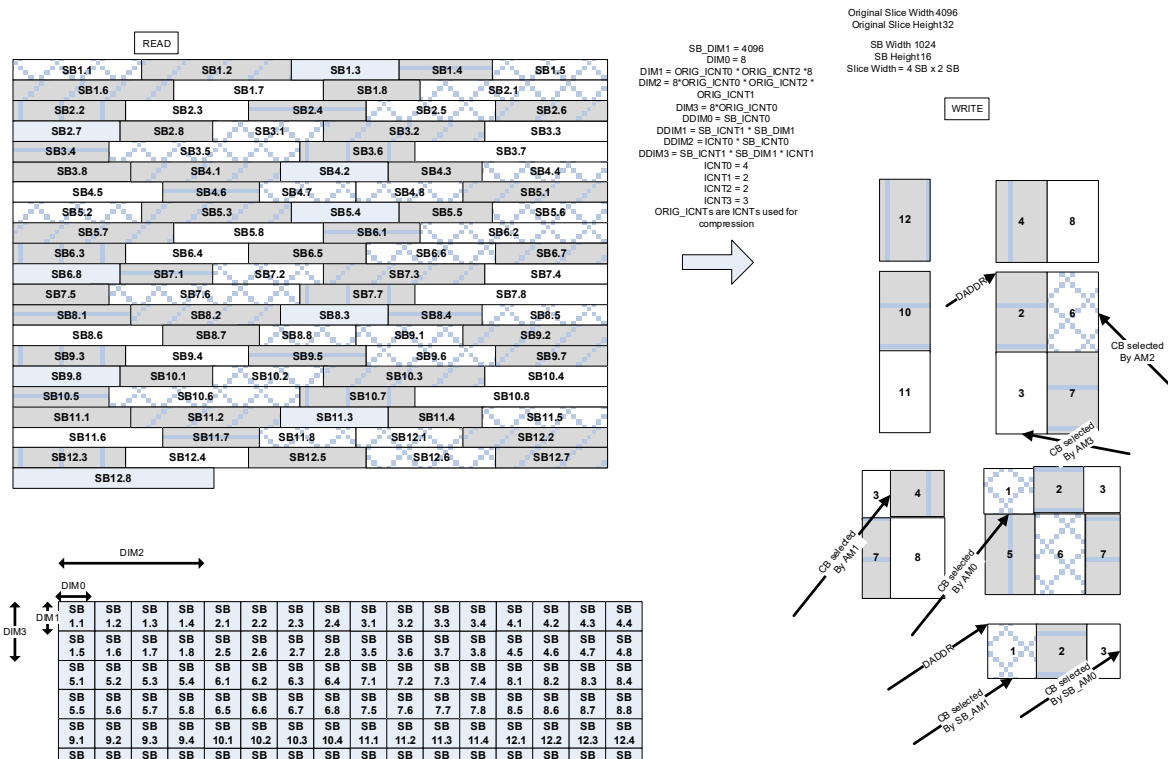
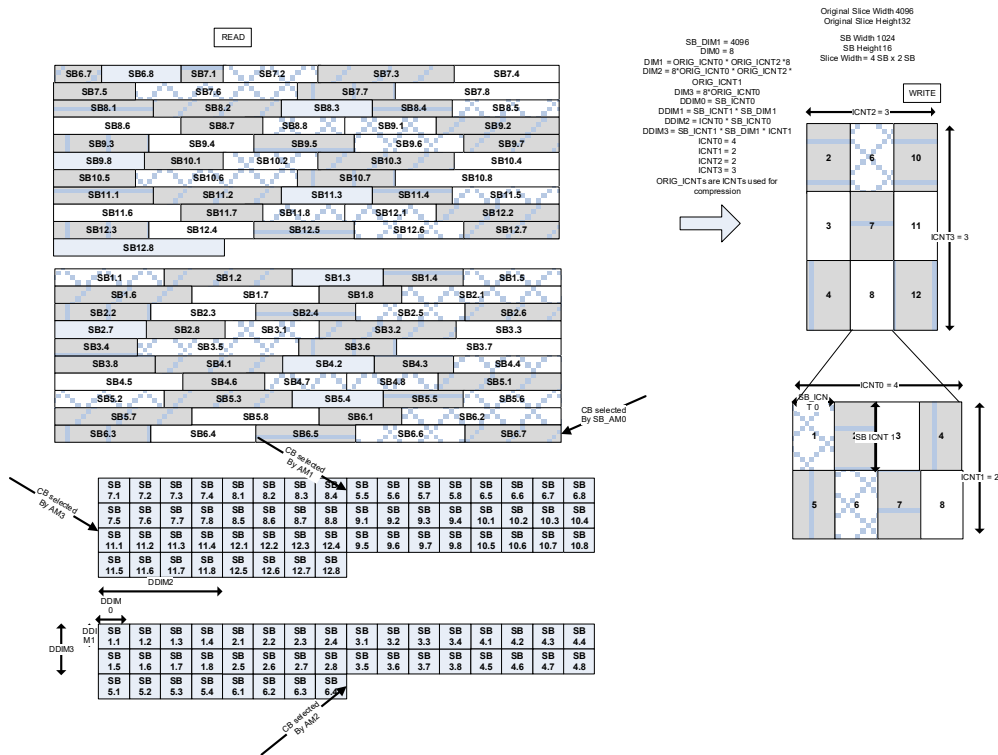


Figure 10-33 Decompressed Data flow for multiple slices with different start location using Circular Buffer DIR 0



After writing the compressed structure to memory the frame is going to be pulled back into the system but if only a portion of the data is required this can be done by using the CDB start offset. For this example the secondary TR used for the decompression is assumed to be 32 bytes below the secondary TR used for the compression with the table being 32 bytes beyond that secondary TR. In this case the output wants to write across the vertical slices first and then the horizontal slices. The CDB start offset in this case is $32 + 32 + (8 \times 4)$ to skip the first 4 superblock entries and the first entry read will be for superblock 2.1 of the compressed stream. The SDIMX fields are then using this location for all further jumps based on the looping structure of the TR.

Figure 10-34 **Decompressed Data flow for multiple slices with different start location and Circular Addressing DIR 1**



10.8 TR Response

Upon completing a TR the UTC will send back a TR Response if it was submitted through a UDMA-C. A directly submitted TR will only return a completion or error event. If an error then the TR response will be stored in a register inside the UTC.

10.8.1 TR Response Format

The UTC TR response format is a single 32 bit word as shown in the figure below.

Figure 10-35 Transfer Request Response Template STATUS FLAGS Field

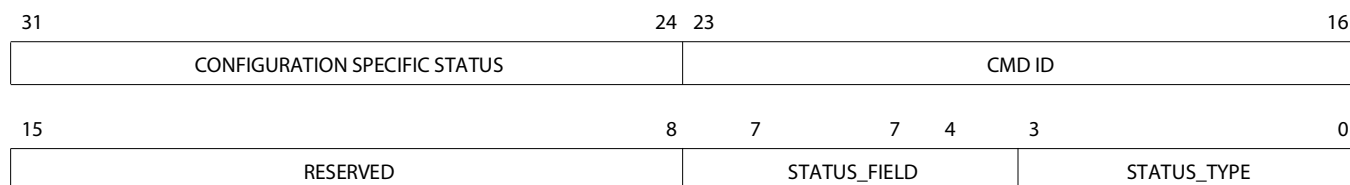


Table 10-21 STATUS FLAGS Field Descriptions

Bit	Field	Description	Reference
0-3	STATUS_TYPE	The completion status of the TR.	"STATUS_TYPE Definition" on page 10-311
4-7	STATUS_INFO	Information that is unique based on the	
8-15	Reserved	Reserved for Future use	
16-23	CMD ID	The Command ID for the TR. This will be sent back with the response for the Channel controller to identify the specific TR.	"CMD ID: Command ID Field Definition" on page 10-282
24-31	Configuration Specific Flags	These are flag bits that can be specified by the specific UTC configuration. These bits will remain undefined in the global TR format to allow for customization requirements that might be required in the specific UTC implementation.	"Configuration Specific Flags Definition" on page 10-282
End of Table 10-21			

10.8.2 STATUS Field definition

The Status field is made up of two sub-fields STATUS_TYPE and STATUS_INFO. A value of 0 means it completed as requested any other value means an error has occurred. The Table below states the legal STATUS_TYPES and the use of the STATUS_INFO in each case.

10.8.2.1 STATUS_TYPE Definition

The STATUS_TYPE field is used to determine what type of status is being returned. Depending on the type of Status it might additionally have information in the STATUS_FIELD to give more details about the specific reason why the status was returned.

Table 10-22 STATUS_TYPE Encoded Values

Value	Error Type	Completion	STATUS FIELD Definition
0	None	Complete	None
1	Transfer Error	None to Partial	CBA Non-Complete Status Received
2	Aborted Error	None to Partial	None

Table 10-22 STATUS_TYPE Encoded Values

Value	Error Type	Completion	STATUS FIELD Definition
3	Submission Error	None	Submission Error Type
4	Unsupported Feature	None	Feature Type
5-15	Reserved	Unknown	Reserved
End of Table 10-22			

10.8.2.1.1 Transfer Error

A transfer ERROR occurs anytime that one of the CBA transactions performed by the UTC returns a status other than complete. The UTC will return back to an IDLE state and send back the TR Response with the STATUS TYPE of Transfer Error. The STATUS FIELD will contain the 3 bit CBA Status field the upper bit specifies if it was a read status or a write status.

10.8.2.1.2 Aborted Error

An aborted error occurs if the PSI-L interface asserts the drop signal prior to the completion of the TR. The UTC will return back to an IDLE state and send back the TR Response with the STATUS TYPE. If the UTC receives a transfer error after receiving the abort the transfer error should be reported instead of the Abort Error.

10.8.2.1.3 Submission Error

A submission error is returned for a TR that is received that can not be run. The STATUS FIELD specifies the type of submission errors.

Table 10-23 Submission Error

Value	Submission Error Type
0	ICNT0 was 0
1	Channel FIFO was full when TR received
2	Channel is not owned by the submitter. Either CC submitting to Direct or Direct submitting to a CC channel
3-15	Reserved
End of Table 10-23	

10.8.2.1.4 Unsupported Feature

An unsupported feature error is returned for a TR that is received that can not be run because it specifies a feature that is optional and not supported by the UTC that received the TR. The STATUS FIELD specifies the feature that was not supported. If it specifies multiple features that were not supported the UTC implementation can determine which type it wants to return

Table 10-24 Submission Error STATUS FIELD Encodings

Value	Submission Error Type
0	TR Type not supported
1	STATIC not supported
2	EOL not supported
3	CONFIGURATION SPECIFIC not supported
4	AMODE not supported

Table 10-24 Submission Error STATUS FIELD Encodings

Value	Submission Error Type
5	ELTYPE not supported
6	DFMT not supported
7	SECTR not supported
8	AMODE SPECIFIC Field not supported
9-15	Reserved
End of Table 10-24	

PRELIMINARY SPECIFICATION

Accelerator Framework Compression Module

The Accelerator Framework Compression (AFC) module is a hardware accelerator that compresses and uncompresses sequences of bytes. While it is designed as standalone pipelines, it is integrated with the Keystone3 compute cluster data routing unit or could be integrated with other data transfer engines.

This specification describes the programmer-visible behavior of AFC and is focused on what AFC does. Two related documents cover how MMA works and how it is constructed (the AFC Microarchitecture Specification) and how to make it do specific tasks (the AFC User's Guide).

Notes on the text:

- The term *encode* may be used as a synonym for compress.
- The term *decode* may be used as a synonym for decompress.

PRELIMINARY SPECIFICATION

12.1 Requirements

The detailed requirements for AFC are located in <https://pds.design.ti.com/cgi-bin/TBD>. Some key requirements that have significant impact on the architecture are described below.

12.1.1 Supported Data Types

Compression and decompression with AFC is byte-granular. Datasets composed of larger structures like 32-bit integers or double precision floating point are processed a byte sequence without regard for data boundaries.

Sideband control information is necessary for correctly compressing and decompressing data. This is described in the Data Structures section.

12.1.2 Required Functions

Compression: AFC compression operates on uncompressed byte sequences and converts them to a specific compressed data structure to be stored in memory using a small set of fixed-dictionary algorithms. Compression is lossless. AFC compression does not directly produce or use metadata used by software to organize and store compressed and uncompressed objects.

Decompression: AFC decompression operates on a data structure containing a compressed byte sequence with a small set of fixed-dictionary algorithms and converts it to uncompressed data. Decompression is lossless.

Emulatability: both compression and decompression are implementable in software that does not rely on special hardware facilities. Systems can be constructed with software compression and hardware decompression or, probably less useful, with hardware compression and software decompression.

Both compression and decompression rely on sideband control information for correct operation. This information is not stored with the compressed representation of a dataset. The sideband control information includes the specific compression algorithm and size information about the compressed and uncompressed objects.

12.1.3 Performance

Performance targets are relative to the CNN reference datasets located at `/db/mcad/CNN_data`.

AFC compression can sustain encoding 16 bytes per clock cycle for well behaved and sufficiently large datasets. The unweighted arithmetic average performance target over all the datasets is 90% of the maximum sustained rate, or 14.4 bytes per clock cycle.

AFC decompression can sustain decoding 16 bytes per clock cycle for well behaved and sufficiently large datasets. The unweighted arithmetic average performance target over all the datasets is 90% of the maximum sustained rate, or 14.4 bytes per clock cycle.

12.1.4 Area

0.4mm² area in 16FF technology with the Synopsys cell library.

12.1.5 Safety

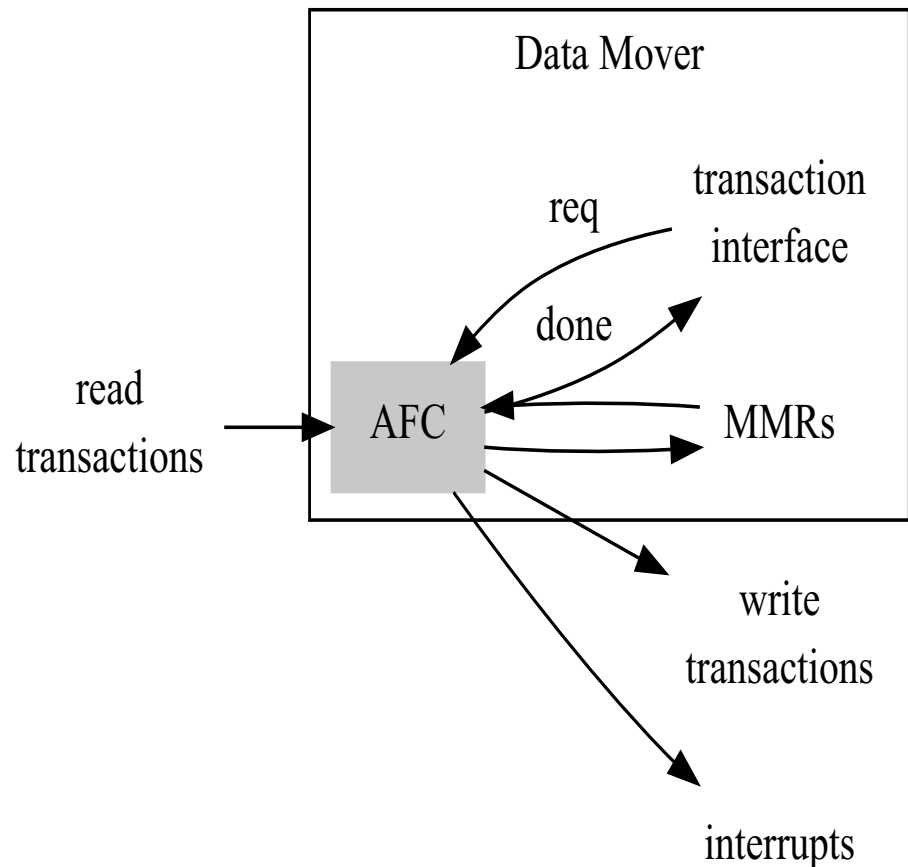
AFC is designed to be integrated with C7x and MMA in SOCs and others ASICs for a variety of end-uses. Automotive markets are very important to us and as a result, AFC is architected to detect many types of soft or hard failures.

AFC includes these safety-related items:

- Self-checking FSMs.
- Compressed data structure corruption detection.
- Parity protection on configuration and status registers.
- Parity error injection on configuration and status registers.

12.2 Theory of Operation

AFC is intended to be embedded in a data movement engine, like a DMA controller or streaming interface. The data movement engine supplies the MMR, transaction enqueueing facilities, data read interface and data write interface. AFC introduces new interrupts that are either concatenated onto the data movement engine's interrupts or are exported directly from AFC.

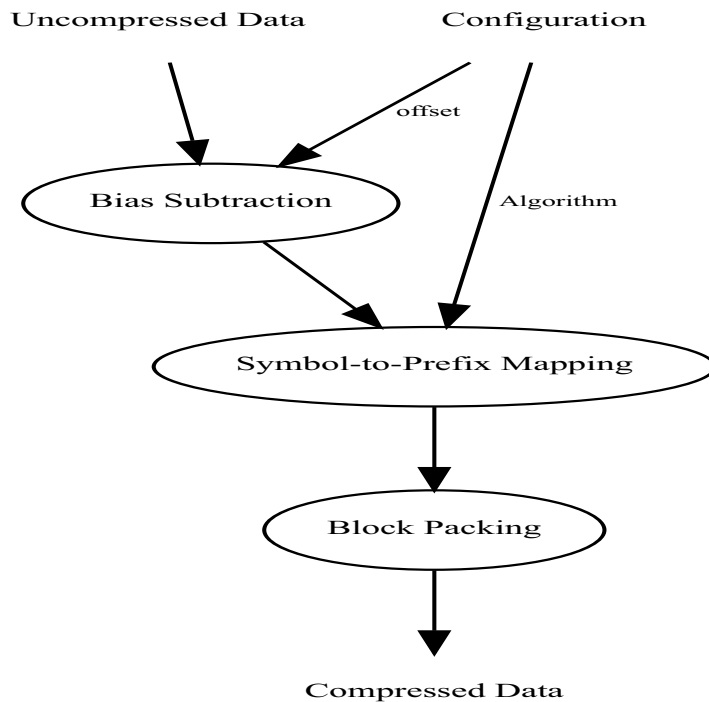


The interfaces between the AFC and data movement engine are defined and described in the AFC microarchitecture specification.

The AFC architecture implements three fixed compression algorithms that are useful for some machine learning datasets. These algorithms are zero compression, unsigned exponential Golomb and signed exponential Golomb. All of the algorithms work within the same framework that provides configurable modulo-256 bias subtraction on compression and addition on decompression.

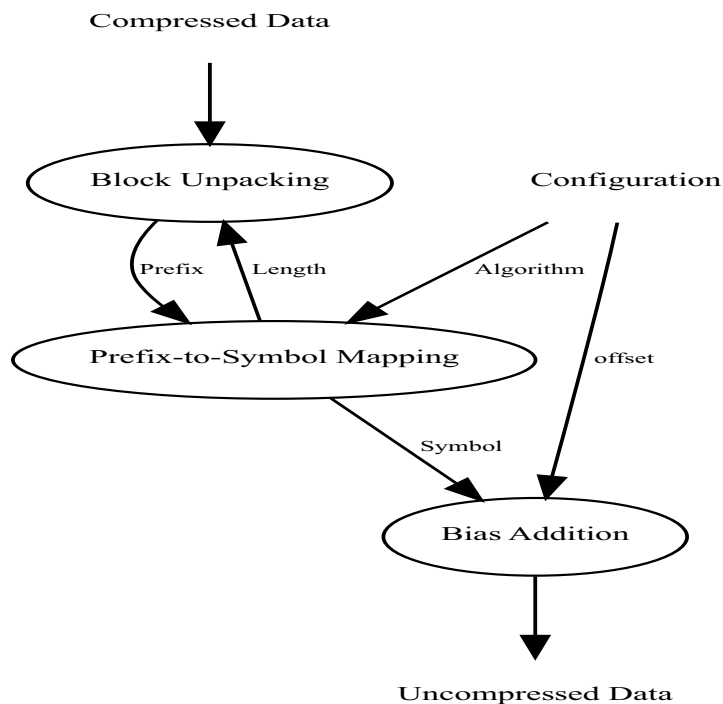
For compression, the offset from a configuration register is subtracted from the uncompressed data bytes. Next, the de-biased data bytes are mapped to an algorithm-specific variable-length value (a prefix in Huffman nomencla-

ture). Finally, the variable length prefix sequences are packed into fixed-length blocks for storage.



For decompression, variable-length prefix bit sequences are extracted by block unpacking logic from the compressed data blocks and presented to the symbol-granular decoding logic. The decoding logic uses longest prefix match (LPM) to associate the compressed data with the de-biased byte values. A feedback path from the decoding logic provides a shift amount for the block unpacking logic to advance to the next prefix. The de-biased byte values from the prefix-to-symbol decoder are added to a pre-configured value in the bias

addition block to complete the transformation from compressed data to uncompressed data.



12.2.1 Zero Compression

Zero compression is a simple encoding algorithm that provides the minimum storage space for zero-valued byte symbols and fixed space for all other symbol values. It provides good compression results when datasets are dominated by a single value.

For de-biased data values of 0, the prefix is 1'b0. For all other values, the prefix is a 9-bit value formed by prepending 1'b1 to the value. Finally, for little-endian packing of the compressed data structure, the prefix is bit-reversed. The following code generates the mapping for all uncompressed symbols.

```

for i in range(0,256) :
    if ( i ) :
        print( "
[\'1"+format(i,\'08b\')[::-1]+\'\',\'"+str(i)+\'\'], " ) ;
    else :
        print( " [\'0\',"+str(i)+\'\'], " ) ;
  
```

The complete uncompressed symbol to prefix mapping is shown below.

Symbol Value	Prefix
8'b00000000	1'b0

Symbol Value	Prefix
8'b00000001	9'b100000001
8'b00000010	9'b010000001
8'b00000011	9'b110000001
8'b00000100	9'b001000001
8'b00000101	9'b101000001
8'b00000110	9'b011000001
8'b00000111	9'b111000001
8'b00001000	9'b000100001
8'b00001001	9'b100100001
8'b00001010	9'b010100001
8'b00001011	9'b110100001
8'b00001100	9'b001100001
8'b00001101	9'b101100001
8'b00001110	9'b011100001
8'b00001111	9'b111100001
8'b00010000	9'b000010001
8'b00010001	9'b100010001
8'b00010010	9'b010010001
8'b00010011	9'b110010001
8'b00010100	9'b001010001
8'b00010101	9'b101010001
8'b00010110	9'b011010001
8'b00010111	9'b111010001
8'b00011000	9'b000110001
8'b00011001	9'b100110001
8'b00011010	9'b010110001
8'b00011011	9'b110110001
8'b00011100	9'b001110001

Symbol Value	Prefix
8'b00011101	9'b101110001
8'b00011110	9'b011110001
8'b00011111	9'b111110001
8'b00100000	9'b000001001
8'b00100001	9'b100001001
8'b00100010	9'b010001001
8'b00100011	9'b110001001
8'b00100100	9'b001001001
8'b00100101	9'b101001001
8'b00100110	9'b011001001
8'b00100111	9'b111001001
8'b00101000	9'b000101001
8'b00101001	9'b100101001
8'b00101010	9'b010101001
8'b00101011	9'b110101001
8'b00101100	9'b001101001
8'b00101101	9'b101101001
8'b00101110	9'b011101001
8'b00101111	9'b111101001
8'b00110000	9'b000011001
8'b00110001	9'b100011001
8'b00110010	9'b010011001
8'b00110011	9'b110011001
8'b00110100	9'b001011001
8'b00110101	9'b101011001
8'b00110110	9'b011011001
8'b00110111	9'b111011001
8'b00111000	9'b000111001

Symbol Value	Prefix
8'b00111001	9'b100111001
8'b00111010	9'b010111001
8'b00111011	9'b110111001
8'b00111100	9'b001111001
8'b00111101	9'b101111001
8'b00111110	9'b011111001
8'b00111111	9'b111111001
8'b01000000	9'b000000101
8'b01000001	9'b100000101
8'b01000010	9'b010000101
8'b01000011	9'b110000101
8'b01000100	9'b001000101
8'b01000101	9'b101000101
8'b01000110	9'b011000101
8'b01000111	9'b111000101
8'b01001000	9'b000100101
8'b01001001	9'b100100101
8'b01001010	9'b010100101
8'b01001011	9'b110100101
8'b01001100	9'b001100101
8'b01001101	9'b101100101
8'b01001110	9'b011100101
8'b01001111	9'b111100101
8'b01010000	9'b000010101
8'b01010001	9'b100010101
8'b01010010	9'b010010101
8'b01010011	9'b110010101
8'b01010100	9'b001010101

Symbol Value	Prefix
8'b01010101	9'b101010101
8'b01010110	9'b011010101
8'b01010111	9'b111010101
8'b01011000	9'b000110101
8'b01011001	9'b100110101
8'b01011010	9'b010110101
8'b01011011	9'b110110101
8'b01011100	9'b001110101
8'b01011101	9'b101110101
8'b01011110	9'b011110101
8'b01011111	9'b111110101
8'b01100000	9'b000001101
8'b01100001	9'b100001101
8'b01100010	9'b010001101
8'b01100011	9'b110001101
8'b01100100	9'b001001101
8'b01100101	9'b101001101
8'b01100110	9'b011001101
8'b01100111	9'b111001101
8'b01101000	9'b000101101
8'b01101001	9'b100101101
8'b01101010	9'b010101101
8'b01101011	9'b110101101
8'b01101100	9'b001101101
8'b01101101	9'b101101101
8'b01101110	9'b011101101
8'b01101111	9'b111101101
8'b01110000	9'b000011101

Symbol Value	Prefix
8'b01110001	9'b100011101
8'b01110010	9'b010011101
8'b01110011	9'b110011101
8'b01110100	9'b001011101
8'b01110101	9'b101011101
8'b01110110	9'b011011101
8'b01110111	9'b111011101
8'b01111000	9'b000111101
8'b01111001	9'b100111101
8'b01111010	9'b010111101
8'b01111011	9'b110111101
8'b01111100	9'b001111101
8'b01111101	9'b101111101
8'b01111110	9'b011111101
8'b01111111	9'b111111101
8'b10000000	9'b000000011
8'b10000001	9'b100000011
8'b10000010	9'b010000011
8'b10000011	9'b110000011
8'b10000100	9'b001000011
8'b10000101	9'b101000011
8'b10000110	9'b011000011
8'b10000111	9'b111000011
8'b10001000	9'b000100011
8'b10001001	9'b100100011
8'b10001010	9'b010100011
8'b10001011	9'b110100011
8'b10001100	9'b001100011

Symbol Value	Prefix
8'b10001101	9'b101100011
8'b10001110	9'b011100011
8'b10001111	9'b111100011
8'b10010000	9'b000010011
8'b10010001	9'b100010011
8'b10010010	9'b010010011
8'b10010011	9'b110010011
8'b10010100	9'b001010011
8'b10010101	9'b101010011
8'b10010110	9'b011010011
8'b10010111	9'b111010011
8'b10011000	9'b000110011
8'b10011001	9'b100110011
8'b10011010	9'b010110011
8'b10011011	9'b110110011
8'b10011100	9'b001110011
8'b10011101	9'b101110011
8'b10011110	9'b011110011
8'b10011111	9'b111110011
8'b10100000	9'b000001011
8'b10100001	9'b100001011
8'b10100010	9'b010001011
8'b10100011	9'b110001011
8'b10100100	9'b001001011
8'b10100101	9'b101001011
8'b10100110	9'b011001011
8'b10100111	9'b111001011
8'b10101000	9'b000101011

Symbol Value	Prefix
8'b10101001	9'b100101011
8'b10101010	9'b010101011
8'b10101011	9'b110101011
8'b10101100	9'b001101011
8'b10101101	9'b101101011
8'b10101110	9'b011101011
8'b10101111	9'b111101011
8'b10110000	9'b000011011
8'b10110001	9'b100011011
8'b10110010	9'b010011011
8'b10110011	9'b110011011
8'b10110100	9'b001011011
8'b10110101	9'b101011011
8'b10110110	9'b011011011
8'b10110111	9'b111011011
8'b10111000	9'b000111011
8'b10111001	9'b100111011
8'b10111010	9'b010111011
8'b10111011	9'b110111011
8'b10111100	9'b001111011
8'b10111101	9'b101111011
8'b10111110	9'b011111011
8'b10111111	9'b111111011
8'b11000000	9'b000000111
8'b11000001	9'b100000111
8'b11000010	9'b010000111
8'b11000011	9'b110000111
8'b11000100	9'b001000111

Symbol Value	Prefix
8'b11000101	9'b101000111
8'b11000110	9'b011000111
8'b11000111	9'b111000111
8'b11001000	9'b000100111
8'b11001001	9'b100100111
8'b11001010	9'b010100111
8'b11001011	9'b110100111
8'b11001100	9'b001100111
8'b11001101	9'b101100111
8'b11001110	9'b011100111
8'b11001111	9'b111100111
8'b11010000	9'b000010111
8'b11010001	9'b100010111
8'b11010010	9'b010010111
8'b11010011	9'b110010111
8'b11010100	9'b001010111
8'b11010101	9'b101010111
8'b11010110	9'b011010111
8'b11010111	9'b111010111
8'b11011000	9'b000110111
8'b11011001	9'b100110111
8'b11011010	9'b010110111
8'b11011011	9'b110110111
8'b11011100	9'b001110111
8'b11011101	9'b101110111
8'b11011110	9'b011110111
8'b11011111	9'b111110111
8'b11100000	9'b000001111

Symbol Value	Prefix
8'b11100001	9'b100001111
8'b11100010	9'b010001111
8'b11100011	9'b110001111
8'b11100100	9'b001001111
8'b11100101	9'b101001111
8'b11100110	9'b011001111
8'b11100111	9'b111001111
8'b11101000	9'b000101111
8'b11101001	9'b100101111
8'b11101010	9'b010101111
8'b11101011	9'b110101111
8'b11101100	9'b001101111
8'b11101101	9'b101101111
8'b11101110	9'b011101111
8'b11101111	9'b111101111
8'b11110000	9'b000011111
8'b11110001	9'b100011111
8'b11110010	9'b010011111
8'b11110011	9'b110011111
8'b11110100	9'b001011111
8'b11110101	9'b101011111
8'b11110110	9'b011011111
8'b11110111	9'b111011111
8'b11111000	9'b000111111
8'b11111001	9'b100111111
8'b11111010	9'b010111111
8'b11111011	9'b110111111
8'b11111100	9'b001111111

Symbol Value	Prefix
8'b11111101	9'b101111111
8'b11111110	9'b011111111
8'b11111111	9'b111111111

12.2.2 Unsigned Exponential Golomb

Unsigned exponential Golomb coding is a fixed compression algorithm provides good compression when unsigned data is distributed with decreasing frequency as values become more positive EG. To encode a symbol which has been de-biased, the value of the symbol is incremented and the width of the minimum binary number needed to encode that incremented value is determined. The prefix is then a sequence of 0 bits equal to the binary width followed by the incremented value. For little-endian packing of the compressed data structure, the prefix is bit-reversed. Example code that generates the mapping of all 8-bit de-biased data to the variable length prefixes is included below.

```
for i in range(0,256) :
    j = i+1 ;
    # find leading 1 position
    z = -1 ;
    for k in range(0,9) :
        if ( (j & (1<<k) ) ) : z = k ;
    if ( z == -1 ) :
        print( 'error!' ) ;
        sys.exit(1) ;
    # pad by leading one position
    pad = '' ;
    for n in range(0,z) : pad += '0' ;
    pad += "{0:b}".format(j) ;
    print( " ['"+pad[::-1]+"', '"+str(i)+"'], " ) ;
```

The complete uncompressed symbol to prefix mapping is shown below.

Symbol Value	Prefix
8'b00000000	1'b1
8'b00000001	3'b010
8'b00000010	3'b110
8'b00000011	5'b00100
8'b00000100	5'b10100

Symbol Value	Prefix
8'b000000101	5'b01100
8'b000000110	5'b11100
8'b000000111	7'b0001000
8'b000001000	7'b1001000
8'b000001001	7'b0101000
8'b000001010	7'b1101000
8'b000001011	7'b0011000
8'b000001100	7'b1011000
8'b000001101	7'b0111000
8'b000001110	7'b1111000
8'b000001111	9'b000010000
8'b000010000	9'b100010000
8'b000010001	9'b010010000
8'b000010010	9'b110010000
8'b000010011	9'b001010000
8'b000010100	9'b101010000
8'b000010101	9'b011010000
8'b000010110	9'b111010000
8'b000010111	9'b000110000
8'b000011000	9'b100110000
8'b000011001	9'b010110000
8'b000011010	9'b110110000
8'b000011011	9'b001110000
8'b000011100	9'b101110000
8'b000011101	9'b011110000
8'b000011110	9'b111110000
8'b000011111	11'b000000100000
8'b000100000	11'b100000100000

Symbol Value	Prefix
8'b00100001	11'b01000100000
8'b00100010	11'b11000100000
8'b00100011	11'b00100100000
8'b00100100	11'b10100100000
8'b00100101	11'b01100100000
8'b00100110	11'b11100100000
8'b00100111	11'b00010100000
8'b00101000	11'b10010100000
8'b00101001	11'b01010100000
8'b00101010	11'b11010100000
8'b00101011	11'b00110100000
8'b00101100	11'b10110100000
8'b00101101	11'b01110100000
8'b00101110	11'b11110100000
8'b00101111	11'b00001100000
8'b00110000	11'b10001100000
8'b00110001	11'b01001100000
8'b00110010	11'b11001100000
8'b00110011	11'b00101100000
8'b00110100	11'b10101100000
8'b00110101	11'b01101100000
8'b00110110	11'b11101100000
8'b00110111	11'b00011100000
8'b00111000	11'b10011100000
8'b00111001	11'b01011100000
8'b00111010	11'b11011100000
8'b00111011	11'b00111100000
8'b00111100	11'b10111100000

Symbol Value	Prefix
8'b00111101	11'b011111100000
8'b00111110	11'b111111100000
8'b00111111	13'b00000001000000
8'b01000000	13'b10000001000000
8'b01000001	13'b01000001000000
8'b01000010	13'b11000001000000
8'b01000011	13'b00100001000000
8'b01000100	13'b10100001000000
8'b01000101	13'b01100001000000
8'b01000110	13'b11100001000000
8'b01000111	13'b00010010000000
8'b01001000	13'b10010010000000
8'b01001001	13'b01010010000000
8'b01001010	13'b11010010000000
8'b01001011	13'b00110010000000
8'b01001100	13'b10110010000000
8'b01001101	13'b01110010000000
8'b01001110	13'b11110010000000
8'b01001111	13'b00001010000000
8'b01010000	13'b10001010000000
8'b01010001	13'b01001010000000
8'b01010010	13'b11001010000000
8'b01010011	13'b00101010000000
8'b01010100	13'b10101010000000
8'b01010101	13'b01101010000000
8'b01010110	13'b11101010000000
8'b01010111	13'b00011010000000
8'b01011000	13'b10011010000000

Symbol Value	Prefix
8'b01011001	13'b0101101000000
8'b01011010	13'b1101101000000
8'b01011011	13'b0011101000000
8'b01011100	13'b1011101000000
8'b01011101	13'b0111101000000
8'b01011110	13'b1111101000000
8'b01011111	13'b0000011000000
8'b01100000	13'b1000011000000
8'b01100001	13'b0100011000000
8'b01100010	13'b1100011000000
8'b01100011	13'b0010011000000
8'b01100100	13'b1010011000000
8'b01100101	13'b0110011000000
8'b01100110	13'b1110011000000
8'b01100111	13'b0001011000000
8'b01101000	13'b1001011000000
8'b01101001	13'b0101011000000
8'b01101010	13'b1101011000000
8'b01101011	13'b0011011000000
8'b01101100	13'b1011011000000
8'b01101101	13'b0111011000000
8'b01101110	13'b1111011000000
8'b01101111	13'b0000111000000
8'b01110000	13'b1000111000000
8'b01110001	13'b0100111000000
8'b01110010	13'b1100111000000
8'b01110011	13'b0010111000000
8'b01110100	13'b1010111000000

Symbol Value	Prefix
8'b01110101	13'b0110111000000
8'b01110110	13'b1110111000000
8'b01110111	13'b0001111000000
8'b01111000	13'b1001111000000
8'b01111001	13'b0101111000000
8'b01111010	13'b1101111000000
8'b01111011	13'b0011111000000
8'b01111100	13'b1011111000000
8'b01111101	13'b0111111000000
8'b01111110	13'b1111111000000
8'b01111111	15'b000000010000000
8'b10000000	15'b100000010000000
8'b10000001	15'b010000010000000
8'b10000010	15'b110000010000000
8'b10000011	15'b001000010000000
8'b10000100	15'b101000010000000
8'b10000101	15'b011000010000000
8'b10000110	15'b111000010000000
8'b10000111	15'b000100010000000
8'b10001000	15'b100100010000000
8'b10001001	15'b010100010000000
8'b10001010	15'b110100010000000
8'b10001011	15'b001100010000000
8'b10001100	15'b101100010000000
8'b10001101	15'b011100010000000
8'b10001110	15'b111100010000000
8'b10001111	15'b000010010000000
8'b10010000	15'b100010010000000

Symbol Value	Prefix
8'b10010001	15'b0100100100000000
8'b10010010	15'b1100100100000000
8'b10010011	15'b0010100100000000
8'b10010100	15'b1010100100000000
8'b10010101	15'b0110100100000000
8'b10010110	15'b1110100100000000
8'b10010111	15'b0001100100000000
8'b10011000	15'b1001100100000000
8'b10011001	15'b0101100100000000
8'b10011010	15'b1101100100000000
8'b10011011	15'b0011100100000000
8'b10011100	15'b1011100100000000
8'b10011101	15'b0111100100000000
8'b10011110	15'b1111100100000000
8'b10011111	15'b0000010100000000
8'b10100000	15'b1000010100000000
8'b10100001	15'b0100010100000000
8'b10100010	15'b1100010100000000
8'b10100011	15'b0010010100000000
8'b10100100	15'b1010010100000000
8'b10100101	15'b0110010100000000
8'b10100110	15'b1110010100000000
8'b10100111	15'b0001010100000000
8'b10101000	15'b1001010100000000
8'b10101001	15'b0101010100000000
8'b10101010	15'b1101010100000000
8'b10101011	15'b0011010100000000
8'b10101100	15'b1011010100000000

Symbol Value	Prefix
8'b10101101	15'b011101010000000
8'b10101110	15'b111101010000000
8'b10101111	15'b000011010000000
8'b10110000	15'b100011010000000
8'b10110001	15'b010011010000000
8'b10110010	15'b110011010000000
8'b10110011	15'b001011010000000
8'b10110100	15'b101011010000000
8'b10110101	15'b011011010000000
8'b10110110	15'b111011010000000
8'b10110111	15'b000111010000000
8'b10111000	15'b100111010000000
8'b10111001	15'b010111010000000
8'b10111010	15'b110111010000000
8'b10111011	15'b001111010000000
8'b10111100	15'b101111010000000
8'b10111101	15'b011111010000000
8'b10111110	15'b111111010000000
8'b10111111	15'b000000110000000
8'b11000000	15'b100000110000000
8'b11000001	15'b010000110000000
8'b11000010	15'b110000110000000
8'b11000011	15'b001000110000000
8'b11000100	15'b101000110000000
8'b11000101	15'b011000110000000
8'b11000110	15'b111000110000000
8'b11000111	15'b000100110000000
8'b11001000	15'b100100110000000

Symbol Value	Prefix
8'b11001001	15'b010100110000000
8'b11001010	15'b110100110000000
8'b11001011	15'b001100110000000
8'b11001100	15'b101100110000000
8'b11001101	15'b011100110000000
8'b11001110	15'b111100110000000
8'b11001111	15'b000010110000000
8'b11010000	15'b100010110000000
8'b11010001	15'b010010110000000
8'b11010010	15'b110010110000000
8'b11010011	15'b001010110000000
8'b11010100	15'b101010110000000
8'b11010101	15'b011010110000000
8'b11010110	15'b111010110000000
8'b11010111	15'b000110110000000
8'b11011000	15'b100110110000000
8'b11011001	15'b010110110000000
8'b11011010	15'b110110110000000
8'b11011011	15'b001110110000000
8'b11011100	15'b101110110000000
8'b11011101	15'b011110110000000
8'b11011110	15'b111110110000000
8'b11011111	15'b000001110000000
8'b11100000	15'b100001110000000
8'b11100001	15'b010001110000000
8'b11100010	15'b110001110000000
8'b11100011	15'b001001110000000
8'b11100100	15'b101001110000000

Symbol Value	Prefix
8'b11100101	15'b011001110000000
8'b11100110	15'b111001110000000
8'b11100111	15'b000101110000000
8'b11101000	15'b100101110000000
8'b11101001	15'b010101110000000
8'b11101010	15'b110101110000000
8'b11101011	15'b001101110000000
8'b11101100	15'b101101110000000
8'b11101101	15'b011101110000000
8'b11101110	15'b111101110000000
8'b11101111	15'b000011110000000
8'b11110000	15'b100011110000000
8'b11110001	15'b010011110000000
8'b11110010	15'b110011110000000
8'b11110011	15'b001011110000000
8'b11110100	15'b101011110000000
8'b11110101	15'b011011110000000
8'b11110110	15'b111011110000000
8'b11110111	15'b000111110000000
8'b11111000	15'b100111110000000
8'b11111001	15'b010111110000000
8'b11111010	15'b110111110000000
8'b11111011	15'b001111110000000
8'b11111100	15'b101111110000000
8'b11111101	15'b011111110000000
8'b11111110	15'b111111110000000
8'b11111111	17'b00000000100000000

12.2.3 Signed Exponential Golomb

Signed exponential Golomb coding is a fixed compression algorithm provides good compression when signed data is distributed with decreasing frequency as values have increasing magnitude (for example, a normal distribution centered at 0). Before encoding, the symbol is debiased. Then, to encode a symbol with a negative value, an intermediate term is computed by multiplying the value by -2. For a positive value, the intermediate term is computed by multiplying by 2 and incrementing. Then, the width of the minimum binary number needed to encode the intermediate term is determined. The prefix value is then a sequence of 0 bits equal to the binary width followed by the intermediate term. For little-endian packing in the compressed data structure the prefix is finally bit-reversed. Example code that generates the mapping of all 8-bit de-biased data to the variable length prefixes is included below.

```
for i in range(-128,128) :
    # compute intermediate term
    if ( i < 0 ) :
        j = -2*i ;
    else :
        j = 2*i+1 ;
    # determine minimum encoding width
    z = -1 ;
    for k in range(0,9) :
        if ( (j & (1<<k) ) ) : z = k ;
    if ( z == -1 ) :
        print( 'error!' ) ;
        sys.exit(1) ;
    # pad with zeros on the left
    pad = '' ;
    for n in range(0,z) : pad += '0' ;
    pad += "{0:b}".format(j) ;
    print( " ['"+pad[::-1]+"', '"+str(i)+"']," ) ;
```

The complete uncompressed symbol to prefix mapping is shown below.

Symbol Value	Prefix
8'b10000000	17'b000000000100000000
8'b10000001	15'b0111111100000000
8'b10000010	15'b0011111100000000
8'b10000011	15'b0101111100000000
8'b10000100	15'b0001111100000000
8'b10000101	15'b0110111100000000

Symbol Value	Prefix
8'b10000110	15'b0010111110000000
8'b10000111	15'b0100111110000000
8'b10001000	15'b0000111110000000
8'b10001001	15'b0111011110000000
8'b10001010	15'b0011011110000000
8'b10001011	15'b0101011110000000
8'b10001100	15'b0001011110000000
8'b10001101	15'b0110011110000000
8'b10001110	15'b0010011110000000
8'b10001111	15'b0100011110000000
8'b10010000	15'b0000011110000000
8'b10010001	15'b0111101110000000
8'b10010010	15'b0011101110000000
8'b10010011	15'b0101101110000000
8'b10010100	15'b0001101110000000
8'b10010101	15'b0110101110000000
8'b10010110	15'b0010101110000000
8'b10010111	15'b0100101110000000
8'b10011000	15'b0000101110000000
8'b10011001	15'b0111001110000000
8'b10011010	15'b0011001110000000
8'b10011011	15'b0101001110000000
8'b10011100	15'b0001001110000000
8'b10011101	15'b0110001110000000
8'b10011110	15'b0010001110000000
8'b10011111	15'b0100001110000000
8'b10100000	15'b0000001110000000
8'b10100001	15'b0111111010000000

Symbol Value	Prefix
8'b10100010	15'b001111010000000
8'b10100011	15'b010111010000000
8'b10100100	15'b000111010000000
8'b10100101	15'b011011010000000
8'b10100110	15'b001011010000000
8'b10100111	15'b010011010000000
8'b10101000	15'b000011010000000
8'b10101001	15'b011101010000000
8'b10101010	15'b001101010000000
8'b10101011	15'b010101010000000
8'b10101100	15'b000101010000000
8'b10101101	15'b011001010000000
8'b10101110	15'b001001010000000
8'b10101111	15'b010001010000000
8'b10110000	15'b000001010000000
8'b10110001	15'b011110010000000
8'b10110010	15'b001110010000000
8'b10110011	15'b010110010000000
8'b10110100	15'b000110010000000
8'b10110101	15'b011010010000000
8'b10110110	15'b001010010000000
8'b10110111	15'b010010010000000
8'b10111000	15'b000010010000000
8'b10111001	15'b011100010000000
8'b10111010	15'b001100010000000
8'b10111011	15'b010100010000000
8'b10111100	15'b000100010000000
8'b10111101	15'b011000010000000

Symbol Value	Prefix
8'b10111110	15'b001000010000000
8'b10111111	15'b010000010000000
8'b11000000	15'b000000010000000
8'b11000001	13'b011111110000000
8'b11000010	13'b001111110000000
8'b11000011	13'b010111110000000
8'b11000100	13'b000111110000000
8'b11000101	13'b011011110000000
8'b11000110	13'b001011110000000
8'b11000111	13'b010011110000000
8'b11001000	13'b000011110000000
8'b11001001	13'b011101110000000
8'b11001010	13'b001101110000000
8'b11001011	13'b010101110000000
8'b11001100	13'b000101110000000
8'b11001101	13'b011001110000000
8'b11001110	13'b001001110000000
8'b11001111	13'b010001110000000
8'b11010000	13'b000001110000000
8'b11010001	13'b011110100000000
8'b11010010	13'b001110100000000
8'b11010011	13'b010110100000000
8'b11010100	13'b000110100000000
8'b11010101	13'b011010100000000
8'b11010110	13'b001010100000000
8'b11010111	13'b010010100000000
8'b11011000	13'b000010100000000
8'b11011001	13'b011100100000000

Symbol Value	Prefix
8'b11011010	13'b0011001000000
8'b11011011	13'b0101001000000
8'b11011100	13'b0001001000000
8'b11011101	13'b0110001000000
8'b11011110	13'b0010001000000
8'b11011111	13'b0100001000000
8'b11100000	13'b0000001000000
8'b11100001	11'b01111100000
8'b11100010	11'b00111100000
8'b11100011	11'b01011100000
8'b11100100	11'b00011100000
8'b11100101	11'b01101100000
8'b11100110	11'b00101100000
8'b11100111	11'b01001100000
8'b11101000	11'b00001100000
8'b11101001	11'b01110100000
8'b11101010	11'b00110100000
8'b11101011	11'b01010100000
8'b11101100	11'b00010100000
8'b11101101	11'b01100100000
8'b11101110	11'b00100100000
8'b11101111	11'b01000100000
8'b11110000	11'b000000100000
8'b11110001	9'b011110000
8'b11110010	9'b001110000
8'b11110011	9'b010110000
8'b11110100	9'b000110000
8'b11110101	9'b011010000

Symbol Value	Prefix
8'b11110110	9'b001010000
8'b11110111	9'b010010000
8'b11111000	9'b000010000
8'b11111001	7'b0111000
8'b11111010	7'b0011000
8'b11111011	7'b0101000
8'b11111100	7'b0001000
8'b11111101	5'b01100
8'b11111110	5'b00100
8'b11111111	3'b010
8'b00000000	1'b1
8'b00000001	3'b110
8'b00000010	5'b10100
8'b00000011	5'b11100
8'b00000100	7'b1001000
8'b00000101	7'b1101000
8'b00000110	7'b1011000
8'b00000111	7'b1111000
8'b00001000	9'b100010000
8'b00001001	9'b110010000
8'b00001010	9'b101010000
8'b00001011	9'b111010000
8'b00001100	9'b100110000
8'b00001101	9'b110110000
8'b00001110	9'b101110000
8'b00001111	9'b111110000
8'b00010000	11'b10000100000
8'b00010001	11'b11000100000

Symbol Value	Prefix
8'b00010010	11'b10100100000
8'b00010011	11'b11100100000
8'b00010100	11'b10010100000
8'b00010101	11'b11010100000
8'b00010110	11'b10110100000
8'b00010111	11'b11110100000
8'b00011000	11'b10001100000
8'b00011001	11'b11001100000
8'b00011010	11'b10101100000
8'b00011011	11'b11101100000
8'b00011100	11'b10011100000
8'b00011101	11'b11011100000
8'b00011110	11'b10111100000
8'b00011111	11'b11111100000
8'b00100000	13'b1000001000000
8'b00100001	13'b1100001000000
8'b00100010	13'b1010001000000
8'b00100011	13'b1110001000000
8'b00100100	13'b1001001000000
8'b00100101	13'b1101001000000
8'b00100110	13'b1011001000000
8'b00100111	13'b1111001000000
8'b00101000	13'b1000101000000
8'b00101001	13'b1100101000000
8'b00101010	13'b1010101000000
8'b00101011	13'b1110101000000
8'b00101100	13'b1001101000000
8'b00101101	13'b1101101000000

Symbol Value	Prefix
8'b00101110	13'b1011101000000
8'b00101111	13'b1111101000000
8'b00110000	13'b1000011000000
8'b00110001	13'b1100011000000
8'b00110010	13'b1010011000000
8'b00110011	13'b1110011000000
8'b00110100	13'b1001011000000
8'b00110101	13'b1101011000000
8'b00110110	13'b1011011000000
8'b00110111	13'b1111011000000
8'b00111000	13'b1000111000000
8'b00111001	13'b1100111000000
8'b00111010	13'b1010111000000
8'b00111011	13'b1110111000000
8'b00111100	13'b1001111000000
8'b00111101	13'b1101111000000
8'b00111110	13'b1011111000000
8'b00111111	13'b1111111000000
8'b01000000	15'b100000010000000
8'b01000001	15'b110000010000000
8'b01000010	15'b101000010000000
8'b01000011	15'b111000010000000
8'b01000100	15'b100100010000000
8'b01000101	15'b110100010000000
8'b01000110	15'b101100010000000
8'b01000111	15'b111100010000000
8'b01001000	15'b100010010000000
8'b01001001	15'b110010010000000

Symbol Value	Prefix
8'b01001010	15'b101010010000000
8'b01001011	15'b111010010000000
8'b01001100	15'b100110010000000
8'b01001101	15'b110110010000000
8'b01001110	15'b101110010000000
8'b01001111	15'b111110010000000
8'b01010000	15'b100001010000000
8'b01010001	15'b110001010000000
8'b01010010	15'b101001010000000
8'b01010011	15'b111001010000000
8'b01010100	15'b100101010000000
8'b01010101	15'b110101010000000
8'b01010110	15'b101101010000000
8'b01010111	15'b111101010000000
8'b01011000	15'b100011010000000
8'b01011001	15'b110011010000000
8'b01011010	15'b101011010000000
8'b01011011	15'b111011010000000
8'b01011100	15'b100111010000000
8'b01011101	15'b110111010000000
8'b01011110	15'b101111010000000
8'b01011111	15'b111111010000000
8'b01100000	15'b100000110000000
8'b01100001	15'b110000110000000
8'b01100010	15'b101000110000000
8'b01100011	15'b111000110000000
8'b01100100	15'b100100110000000
8'b01100101	15'b110100110000000

Symbol Value	Prefix
8'b01100110	15'b101100110000000
8'b01100111	15'b111100110000000
8'b01101000	15'b100010110000000
8'b01101001	15'b110010110000000
8'b01101010	15'b101010110000000
8'b01101011	15'b111010110000000
8'b01101100	15'b100110110000000
8'b01101101	15'b110110110000000
8'b01101110	15'b101110110000000
8'b01101111	15'b111110110000000
8'b01110000	15'b100001110000000
8'b01110001	15'b110001110000000
8'b01110010	15'b101001110000000
8'b01110011	15'b111001110000000
8'b01110100	15'b100101110000000
8'b01110101	15'b110101110000000
8'b01110110	15'b101101110000000
8'b01110111	15'b111101110000000
8'b01111000	15'b100011110000000
8'b01111001	15'b110011110000000
8'b01111010	15'b101011110000000
8'b01111011	15'b111011110000000
8'b01111100	15'b100111110000000
8'b01111101	15'b110111110000000
8'b01111110	15'b101111110000000
8'b01111111	15'b111111110000000

12.2.4 Parallel Encode and Decode

All of the implemented compression algorithms encode the most frequently expected symbols with the smallest bit-length prefix and this results in a variable number of prefix bits per uncompressed symbol. In a packed stream of encoded symbols, it is difficult to decode the prefixes in an interleaved or block-parallel fashion because the prefix boundaries are a function of the lengths of the prior prefixes in the stream.

For AFC, compressed or encoded data is stored in fixed bit-length blocks aligned on integer multiples of the length of the compressed data block. Compressed data blocks are composed of a packed sequence of variable length prefixes, an explicit or implicit End-Of-Block (EOB) marker, and zero or more padding bits. Multiple encode and decode engines use the fixed-length property of the compressed data blocks to operate on blocks in parallel, because the location of the initial prefix in the block is aligned with the start of the block.

For signed exponential Golomb and unsigned exponential Golomb algorithms, the EOB prefix is always present and always has the same length and encoding. For zero compression, the EOB prefix may or may not be present as described in the Data Structures section.

The EOB encodings are:

Name	Value	Description
AFC_ZERO_EOB	9'b000000001	Zero compression explicit EOB maps to what would have been the redundant 0 encoding
AFC_EGU_EOB	9'b000000000	Exponential Golomb Unsigned EOB is a 9-bit leading zero string with no payload
AFC_EGS_EOB	9'b000000000	Exponential Golomb Signed EOB is a 9-bit leading zero string with no payload

PRELIMINARY SPECIFICATION

12.3 Data Structures

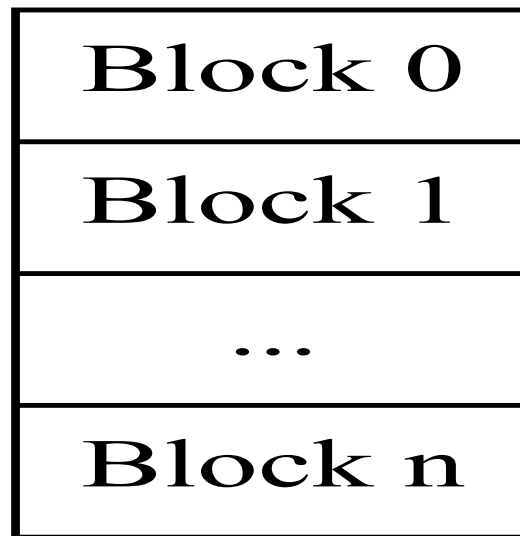
12.3.1 Configuration Constants

The following constants are used in data structures and exported for models and tools.

Name	Value	Description
AFC_CDB_SIZE	128	Compressed data block size in bits
AFC_CDB_SIZE_LOG2	7	Log base 2 of compressed data block size in bits
AFC_READ_SIZE	1024	Memory system data read size
AFC_READ_SIZE_LOG2	10	Log base 2 of memory system data read size
AFC_WRITE_SIZE	1024	Memory system data write size
AFC_WRITE_SIZE_LOG2	10	Log base 2 of memory system data write size
AFC_ENC_ENGINES	16	Number of encoding engines
AFC_DEC_ENGINES	16	Number of decoding engines
AFC_MAXIMUM_PREFIX	17	Maximum prefix length in bit

12.3.1.1 Compressed Data

Compressed or encoded data is stored in a packed array of an arbitrary number of compressed data blocks. No data is stored in the compressed data structure to directly determine number of symbols, the total number of compressed data blocks or the total byte capacity consumed by the compressed data. Software must manage the storage and representation of this metadata.



12.3.1.1.1 Compressed Data Block

Application software normally does not need to read or operate on the Compressed Data Block (CDB) structure to use AFC. However, debug and verification software does need to read, display, and possibly write compressed data in a way that can be processed by the decompression hardware.

The CDB structure is arrayed to form the compressed data in memory. The CDB has a fixed size of 128 bits. CDBs have three field types. Even though the total CDB size is fixed, these fields have variable length.

Two formats using these fields are defined and the specific format for a CDB is a function of the compression algorithm, the length of the uncompressed data and the length of the prefixes in the CDB. The choice of the CDB format is determined on-the-fly during compression, and is detected automatically during decompression.

Regardless of the specific CDB format, during compression prefixes are packed into the CDB from LSB toward MSB and during decompression, symbols are decoded from prefixes serially processed from LSB toward the MSB. This endian behavior is fixed but has consequences for agents that view or manipulate compressed data in different endian modes.

The CDB for Exponential Golomb and zero compression with explicit EOB (Format 1) is:

- A sequence of variable-length prefixes with a length of zero or more bits packed in little endian order starting at byte offset 0.
- Exactly one end-of-block (EOB) prefix. The EOB prefix value is specific to the selected compression algorithm.
- A string of 0-value bits that pad the total size of the compressed data block to 128 bits. There may be zero or more of these bits. For the expo-

nential Golomb algorithm can use the 0 padding to the 128 bit boundary be an implicit EOB.

bits[127:m] 0...0	EOB	prefix[n]	...	prefix[1]	bits[n:0] prefix[0]
----------------------	-----	-----------	-----	-----------	------------------------

The CDB for zero compression with an implicit EOB (Format 2) is:

- A sequence of variable-length prefixes with a length of zero or more bits packed in little endian order starting at byte offset 0.
- A string of 1-value bits that pad the total size of the compressed data block to 128 bits. There may zero or no more than 8 of these bits. The padding bit values must be 1'b1 to avoid decoding the padding as additional 8'b00000000 symbols. A decompression engine scanning the CDB recognizes the implied EOB when the start of the next prefix is 1'b1 but there are less than 9 bits remaining in the CDB.

This format is used when the length of prefix sequence leaves less than the width of the explicit EOB prefix for zero compression, and results in a slight improvement in compression ratio. This format cannot be used by the exponential Golomb algorithms.

The CDB for exponential Golomb compression with an implicit EOB (Format 2) is:

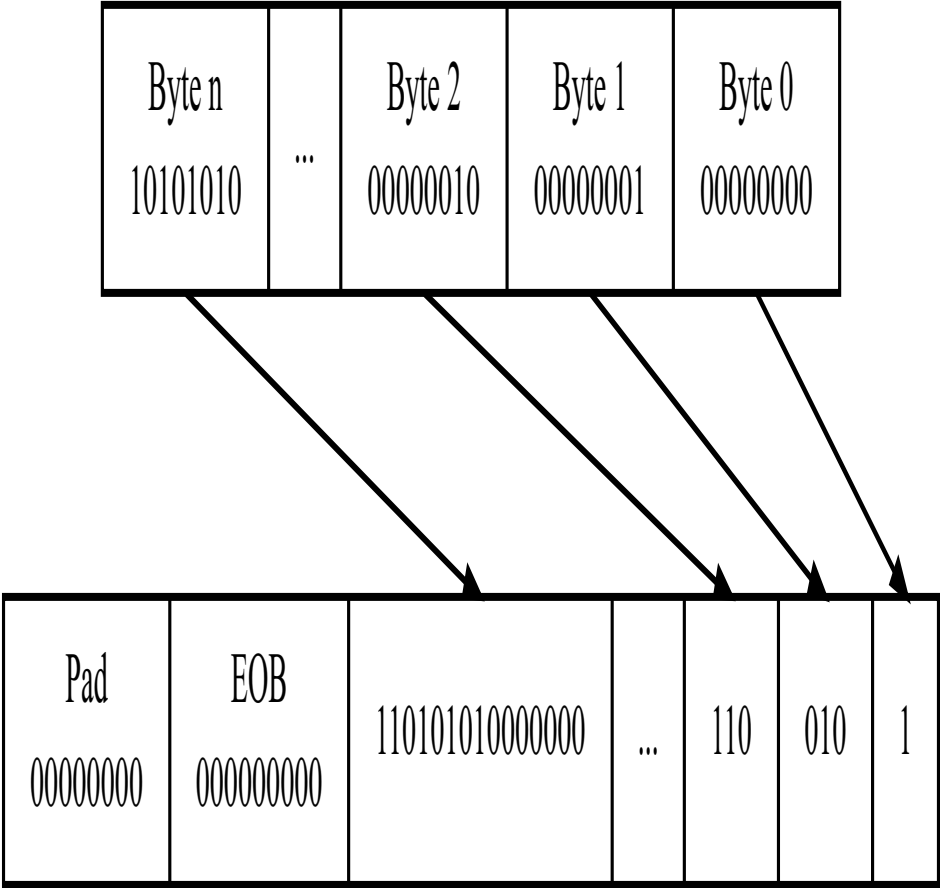
- A sequence of variable-length prefixes with a length of zero or more bits packed in little endian order starting at byte offset 0.
- A string of 0-value bits that pad the total size of the compressed data block to 128 bits. There may zero or no more than the maximum symbol size bits. The padding bit values must be 1'b0 to avoid decoding the padding as additional 8'b00000000 symbols. A decompression engine scanning the CDB recognizes the implied EOB when the start of the next prefix is 1'b1 but there are less than 9 bits remaining in the CDB.

This format is used when the length of prefix sequence leaves less than the width of the explicit EOB prefix for exponential Golomb compression, and results in a slight improvement in compression ratio.

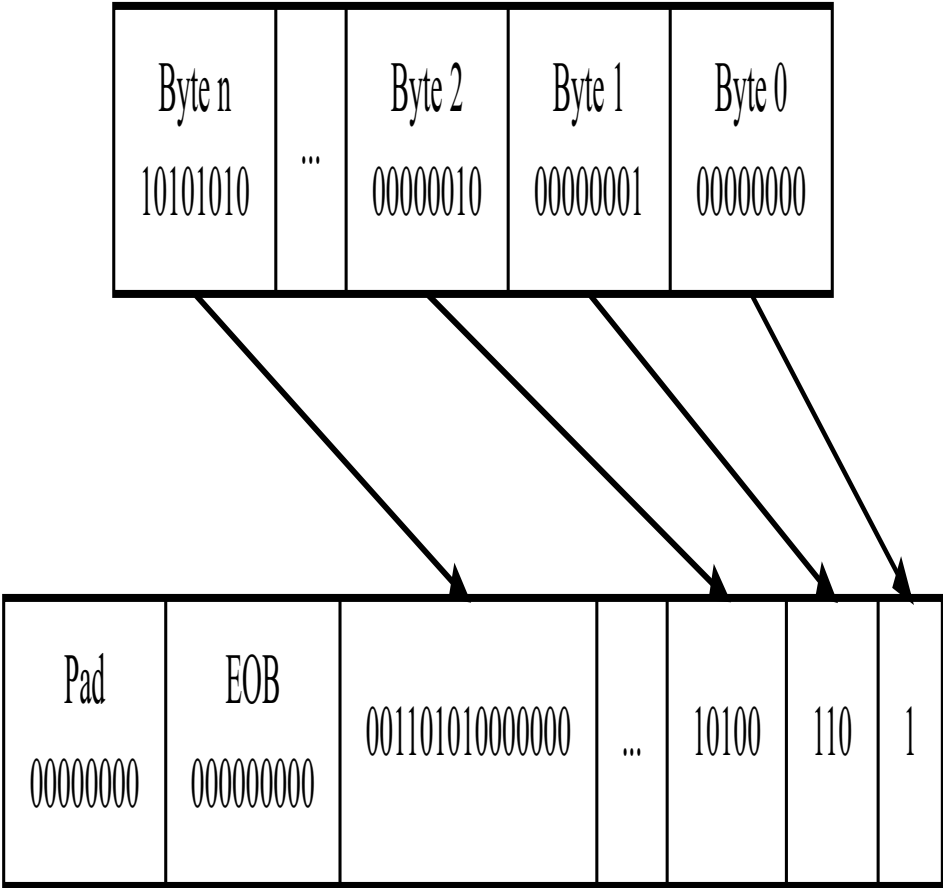
bits[127:m] 0...0	prefix[n]	...	prefix[1]	bits[n:0] prefix[0]
----------------------	-----------	-----	-----------	------------------------

12.3.1.1.2 Compressed Data Block Examples

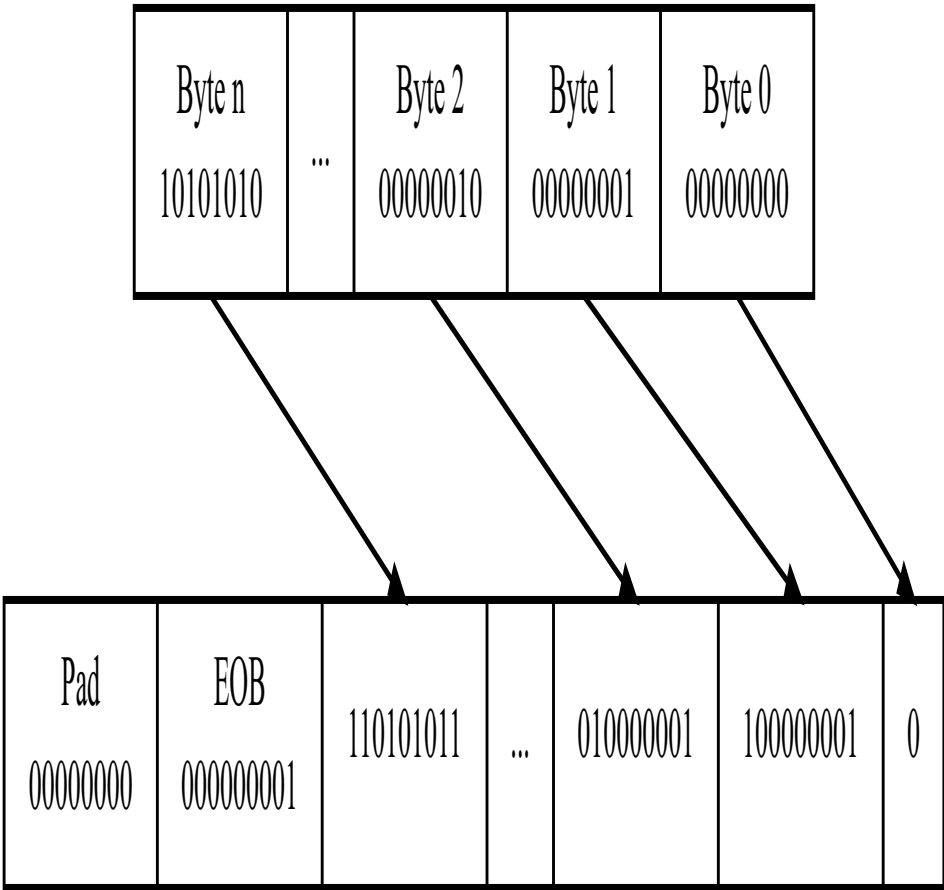
Example CDB FORMAT1 compressed block with unsigned exponential Golomb compression with bias = 0:



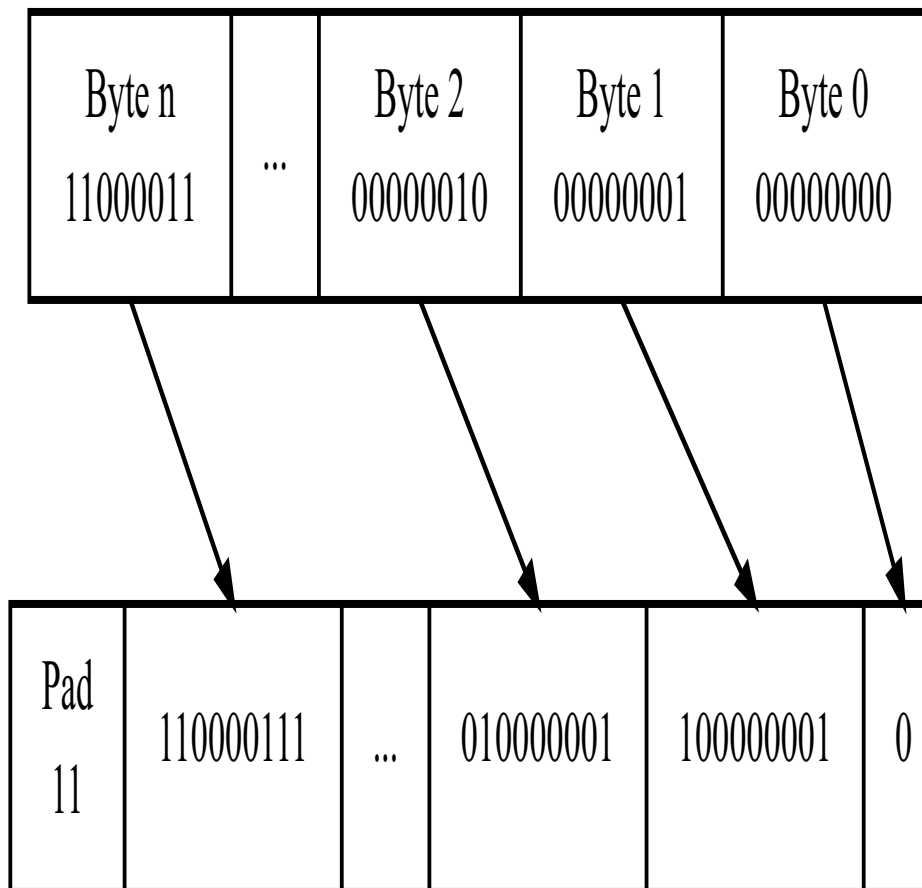
**Example CDB FORMAT1 compressed block with signed exponential Golomb
compression with bias = 0:**



Example CDB FORMAT1 compressed block with zero compression with bias = 0:



Example CDB FORMAT2 compressed block with zero compression with bias = 0:



12.3.1.1.3 Support for Variable k factor

The AFC will support variable k factor by taking the lower bits of the symbol as specified by the selected k value and placing them in the lower bits of the prefix and increase the symbol length by the amount of the k factor. No matter the k factor the maximum size to go through the selected algorithm table is always 8 bits this requires that the minimum k factor for a given input size must always be the input size minus 8. This will minimize the area for the compression table. The packing and unpacking engine will add or remove the extra bits based on the k factor prior to doing any determination for EOB placement or packing.

12.3.1.1.4 Control and Status Registers

Compression and decompression datapaths in AFC are independent control and have separate control registers. The control registers are:

Table 1: Definition of bus AFC_COMPRESSION_CONTROL[203:102]

Field:	C_BYTE_COUNT	C_START	D_BYTE_COUNT
Size:	48	41	13
Range ..	203:156	155:115	114:102

Table 2: Definition of bus AFC_COMPRESSION_CONTROL[101:0]

Field:	D_BYTE_COUNT	D_START	BIAS	ALGORITHM	TID
Size:	35	41	8	8	10
Range ..	101:67	66:26	25:18	17:10	9:0

C_BYTE_COUNT: Compressed byte count**C_START:** Compressed storage start address in 16B units in virtual address space**D_BYTE_COUNT:** Uncompressed byte count**D_START:** Uncompressed storage start address in 16B units in virtual address space**BIAS:** Pre-compression bias to be subtracted from uncompressed bytes**ALGORITHM:** Compression algorithm

Encoding	Name	Description
8'h00	ZERO	Zero compression
8'h01	RESERVED	Reserved encoding. Software should not use this value.
8'h10	EGU	Unsigned exponential Golomb
8'h11	EGS	Signed exponential Golomb

TID: Transaction identifier

Table 3: Definition of bus AFC_DECOMPRESSION_CONTROL[235:118]

Field:	C_BYTE_COUNT	C_START	D_BYTE_COUNT
Size:	48	41	29
Range:	235:188	187:147	146:118

Table 4: Definition of bus AFC_DECOMPRESSION_CONTROL[117:0]

Field:	D_BYTE_COUNT	D_CHECKSUM	D_START	BIAS	ALGORITHM	TID
Size:	19	32	41	8	8	10
Range:	117:99	98:67	66:26	25:18	17:10	9:0

C_BYTE_COUNT: Compressed byte count

C_START: Compressed storage start address in 16B units in virtual address space

D_BYTE_COUNT: Decompressed byte count

D_CHECKSUM: mod 2^{32} checksum of decompressed data

D_START: Decompressed storage start address in 16B units in virtual address space

BIAS: Post-decompression bias to be added to decompressed bytes

ALGORITHM: Compression algorithm

Encoding	Name	Description
8'h00	ZERO	Zero compression
8'h10	EGU	Unsigned exponential Golomb
8'h11	EGS	Signed exponential Golomb

TID: Transaction identifier

Like the control of the compression and decompression datapaths, there are independent completion status registers. The completion status registers are:

Table 5: Definition of bus AFC_COMPRESSION_COMPLETION_STATUS[189:95]

Field:	CYCLES	C_BYTE_COUNT
Size:	48	47
Range:	189:142	141:95

Table 6: Definition of bus AFC_COMPRESSION_COMPLETION_STATUS[94:0]

Field:	C_BYTE_CO UNT	D_CHECKSU M	D_BYTE_COUNT	T I D	C_INCOMPL ETE	D_INCOMP LETE	COMPLET ION
Si ze :	1	32	48	1 0	1	1	2
R a n g e:	94:94	93:62	61:14	1 3 : 4	3:3	2:2	1:0

CYCLES: Elapsed cycle count from transaction start to completion

C_BYTE_COUNT: Actual compressed byte count including CDB overhead.

D_CHECKSUM: mod 2^32 checksum of uncompressed data

D_BYTE_COUNT: Actual uncompressed byte count

TID: Transaction identifier

C_INCOMPLETE: The requested compressed byte count was not satisfied.

D_INCOMPLETE: The requested uncompressed byte count was not satisfied.

COMPLETION: Transaction completion indication

Encoding	Name	Description
2'b00	NOT_DONE	Transaction is active or has not started
2'b01	DONE	Transaction is complete but may not have satisfied compressed or decompressed byte counts.

Encoding	Name	Description
2'b10	DONE_ERROR	Transaction is done but completed early due to an error.
2'b11	RESERVED_3	Reserved status

Table 7: Definition of bus AFC_DECOMPRESSION_COMPLETION_STATUS[157:79]

Field:	CYCLES	C_BYTE_COUNT
Size:	48	31
Range:	157:110	109:79

Table 8: Definition of bus AFC_DECOMPRESSION_COMPLETION_STATUS[78:0]

Field:	C_BYTE_COUNT	D_BYTE_COUNT	TID	C_INCOMPLETE	D_INCOMPLETE	COMPLETION
Size:	17	48	10	1	1	2
Range:	78:62	61:14	13:4	3:3	2:2	1:0

CYCLES: Elapsed cycle count from transaction start to completion

C_BYTE_COUNT: Actual compressed byte count including the CDB overhead. This must always be a multiple of the CDB size.

D_BYTE_COUNT: Actual decompressed byte count

TID: Transaction identifier

C_INCOMPLETE: The requested compressed byte count was not satisfied.

D_INCOMPLETE: The requested decompressed byte count was not satisfied.

COMPLETION: Transaction completion indication

Encoding	Name	Description
2'b00	NOT_DONE	Transaction is active or has not started

Encoding	Name	Description
2'b01	DONE	Transaction is complete but may not have satisfied compressed or decompressed byte counts.
2'b10	DONE_ERROR	Transaction is done but completed early due to an error.
2'b11	RESERVED_3	Reserved status

The status registers are also independent and are defined as follows:

Table 9: Definition of bus AFG_COMPRESSION_STATUS[14:0]

Field:	READY	TID_ACTIVE	STATUS
Size:	1	10	4
Range:	14:14	13:4	3:0

READY: Compression can accept a new transaction

TID_ACTIVE: Identifier for the currently-active transaction

STATUS: Compression controller status

Encoding	Name	Description
4'h0	IDLE	Compression is idle
4'h1	WAIT_R	All engines are blocked waiting for uncompressed read transactions
4'h2	WAIT_W	All engines are blocked waiting for compressed write transactions
4'h3	HALT	Compression is halted, probably due to an error

Table 10: Definition of bus AFG_DECOMPRESSION_STATUS[14:0]

Field:	READY	TID_ACTIVE	STATUS
Size:	1	10	4
Range:	14:14	13:4	3:0

READY: Decompression can accept a new transaction

TID_ACTIVE: Identifier for the currently-active transaction

STATUS: Decompression controller status

Encoding	Name	Description
4'h0	IDLE	Decompression is idle
4'h1	WAIT_R	All engines are blocked waiting for compressed read transactions
4'h2	WAIT_W	All engines are blocked waiting for decompressed write transactions
4'h3	HALT	Decompression is halted, probably due to an error

Issues:

- 1) Does all the address stuff stay in the parent data mover ? Maybe only the counts are handled inside AFC. If addresses really are inside AFC:
 - a) Should the addresses be virtual or intermediate or physical ? **I think virtual is correct** b/c it allows supv or hypv to scatter data in the physical address space.
 - b) Should the control registers take in a chunk of memory attributes for each address ?
- 2) Should the status ready flag be an interrupt ? Or is it even visible to software ?
- 3) I'm resisting the urge to provide per-engine *control*... might be cool but would it really be useful ?
- 4) I'm resisting the urge to provide per-engine *status*... might be cool but would it really be useful ?
- 5) I have not defined any performance monitoring other than elapsed cycle count. Performance monitoring gets hardware-intensive quickly, is diffi-

cult to verify and often goes un-used. Some things that could be reported:

- a) cumulative cycles where all engines are blocked on reads
- b) cumulative cycles where all engines are blocked on writes
- c) average engines active per cycle

12.4 Programming

The following sections describe how to configure and control common transaction types.

12.4.1 Compressing data of known size and bounded compressed size

The byte control controls in the compression control can limit the both the uncompressed and compressed data regions to fixed sizes. These parameters are set up like this:

- set AFC_COMPRESSION_CONTROL.D_BYTE_COUNT field to the uncompressed data size.
- set AFC_COMPRESSION_CONTROL.C_BYTE_COUNT field to the upper bound of the compressed data size.

On completion:

- The AFC_COMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.
- The AFC_COMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 1 in the common case where the actual compressed data is less than the upper bound.
- The AFC_COMPRESSION_COMPLETION_STATUS.D_INCOMPLETE field contains the value 0.
- The AFC_COMPRESSION_COMPLETION_STATUS.C_BYTE_COUNT field contains the actual number of bytes of storage occupied by the compressed data.

12.4.2 Compressing data of known size and unbounded compressed size

The compression parameters are set up like this:

- set AFC_COMPRESSION_CONTROL.D_BYTE_COUNT field to the uncompressed data size.
- set AFC_COMPRESSION_CONTROL.C_BYTE_COUNT field to the maximum field value.

On completion:

- The AFC_COMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.
- The AFC_COMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 1.
- The AFC_COMPRESSION_COMPLETION_STATUS.C_BYTE_COUNT field contains the actual number of bytes of storage occupied by the compressed data.

12.4.3 Compressing data of unknown size and bounded compressed size

The compression parameters are set up like this:

- set AFC_COMPRESSION_CONTROL.D_BYTE_COUNT field to the maximum field value.
- set AFC_COMPRESSION_CONTROL.C_BYTE_COUNT field to the compressed size bound.

On completion:

- The AFC_COMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.
- The AFC_COMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 0.
- The AFC_COMPRESSION_COMPLETION_STATUS.D_INCOMPLETE field contains the value 1.
- The AFC_COMPRESSION_COMPLETION_STATUS.D_BYTE_COUNT field contains the actual number of bytes of uncompressed data processed.

12.4.4 Decompressing data of known compressed size and bounded uncompressed size

The decompression parameters are set up like this:

- set AFC_DECOMPRESSION_CONTROL.C_BYTE_COUNT field to the compressed data size.
- set AFC_DECOMPRESSION_CONTROL.D_BYTE_COUNT field to the upper bound of the uncompressed data size.

On completion:

- The AFC_DECOMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 0.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_INCOMPLETE field contains the value 1 in the common case where the actual decompressed data is less than the upper bound.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_BYTE_COUNT field contains the actual number of bytes of storage occupied by the decompressed data.

12.4.5 Decompressing data of known compressed size and unbounded uncompressed size

The decompression parameters are set up like this:

- set AFC_DECOMPRESSION_CONTROL.C_BYTE_COUNT field to the compressed data size.
- set AFC_DECOMPRESSION_CONTROL.D_BYTE_COUNT field to the field's upper bound.

On completion:

- The AFC_DECOMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.

- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_INCOMPLETE field contains the value 1.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 0.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_BYTE_COUNT field contains the actual number of bytes of storage occupied by the decompressed data.

12.4.6 Decompressing data of unknown compressed size and bounded uncompressed size

The decompression parameters are set up like this:

- set AFC_DECOMPRESSION_CONTROL.C_BYTE_COUNT field to the field's upper bound.
- set AFC_DECOMPRESSION_CONTROL.D_BYTE_COUNT field to the uncompressed data region's upper bound.

On completion:

- The AFC_DECOMPRESSION_COMPLETION_STATUS.COMPLETION field contains the value DONE.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_INCOMPLETE field probably contains the value 1 unless the 0 the exact size of the uncompressed data is known.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.C_INCOMPLETE field contains the value 1.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.D_BYTE_COUNT field contains the actual number of bytes of storage occupied by the decompressed data.
- The AFC_DECOMPRESSION_COMPLETION_STATUS.C_BYTE_COUNT field contains the actual number of bytes of storage processed during decompression.

PRELIMINARY SPECIFICATION

12.5 Errors

Error conditions in AFC that are detected by hardware are uniformly logged in the AFC_FEC and AFC_LEC registers. The AFC_FEC register logs the first error code and related information that is generated after the AFC_FEC register is cleared. It is a sticky register that holds the information about the first error code until it is explicitly cleared by software or AFC is reset. The complement to this register is the AFC_ERROR_LEC register, which logs the last error code and related information. Unlike AFC_FEC, the AFC_LEC register is updated on every error that is detected.

Table 12-1 Definition of bus AFC_ERROR_REGISTER_FORMAT[63:0]

Field:	CDB	OFFSET	COMPONENT	ERRORCODE
Size:	48	7	5	4
Range:	63:16	15:9	8:4	3:0

CDB: For some error conditions, this field holds the Compressed Data Block number

OFFSET: For some error conditions, this field holds the bit offset in Compressed Data Block

COMPONENT: For some error conditions, this field holds the AFC hardware component that generated the error

Encoding	Name	Description
5'h00	ENGINE_0	Decompression or Compression engine 0.
5'h01	ENGINE_1	Decompression or Compression engine 1.
5'h02	ENGINE_2	Decompression or Compression engine 2.
5'h03	ENGINE_3	Decompression or Compression engine 3.
5'h04	ENGINE_4	Decompression or Compression engine 4.
5'h05	ENGINE_5	Decompression or Compression engine 5.

Encoding	Name	Description
5'h06	ENGINE_6	Decompression or Compression engine 6.
5'h07	ENGINE_7	Decompression or Compression engine 7.
5'h08	ENGINE_8	Decompression or Compression engine 8.
5'h09	ENGINE_9	Decompression or Compression engine 9.
5'h0a	ENGINE_10	Decompression or Compression engine 10.
5'h0b	ENGINE_11	Decompression or Compression engine 11.
5'h0c	ENGINE_12	Decompression or Compression engine 12.
5'h0d	ENGINE_13	Decompression or Compression engine 13.
5'h0e	ENGINE_14	Decompression or Compression engine 14.
5'h0f	ENGINE_15	Decompression or Compression engine 15.
5'h10	NON_ENGINE	Hardware outside the decompression or compression engines.

Table 12-2 ERRORCODE: Error identifier

Encoding	Name	Description
4'h0	NO_ERROR	Default value after reset. No error was detected.
4'h1	RESERVED_CODE_1	Hardware does not generate this error code.
4'h2	RESERVED_CODE_2	Hardware does not generate this error code.

Encoding	Name	Description
4'h3	C_RE	Compression read error. See the C_RE section.
4'h4	C_WE	Compression write error. See the C_WE section.
4'h5	RESERVED_CODE_5	Hardware does not generate this error code.
4'h6	RESERVED_CODE_6	Hardware does not generate this error code.
4'h7	C_MCE	Compression machine check error. See the D_MCE section.
4'h8	RESERVED_CODE_8	Hardware does not generate this error code.
4'h9	RESERVED_CODE_9	Hardware does not generate this error code.
4'hA	RESERVED_CODE_A	Hardware does not generate this error code.
4'hB	D_WE	Decompression write error. See the D_WE section.
4'hC	D_RE	Decompression read error. See the D_RE section.
4'hD	D_PRE	Decompression prefix error. See the D_PRE section.
4'hE	D_EOBE	Decompression end of block error. See the D_EOBE section.
4'hF	D_MCE	Decompression machine check error. See the D_MCE section.

Table 12-3 Definition of reg AFC_FEC[63:0]

Field:	AFC_ERROR_REGISTER_FORMAT
Size:	64
Range:	63:0

AFC_ERROR_REGISTER_FORMAT: Sticky First Error Code log. See AFC_ERROR_REGISTER_FORMAT for field definitions. Cleared at reset.

Table 12-4 Definition of reg AFC_LEC[63:0]

Field:	AFC_ERROR_REGISTER_FORMAT
Size:	64
Range:	63:0

AFC_ERROR_REGISTER_FORMAT: Last Error Code log. See AFC_ERROR_REGISTER_FORMAT for field definitions. Cleared at reset.

Table 12-5 Error Condition C_RE

Description	An error occurred while reading uncompressed data during compression. This may be due to soft error but is more likely to be a programming error.
ERRORCODE value	4'h3
Error type	Software
Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE
AFC state	Compression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-6 Error Condition C_WE

Description	An error occurred while writing compressed data during compression. This may be due to soft error but is more likely to be a programming error.
ERRORCODE value	4'h4
Error type	Software
Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE
AFC state	Compression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-7 Error Condition C_MCE

Description	A hardware error inside AFC was detected while compressing data. This may be due to soft error or a design defect.
ERRORCODE value	4'h7
Error type	Software
Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE
AFC state	Compression halted and end-of-transaction is signaled.
AFC resume	Reset

Table 12-8 Error Condition D_RE

Description	An error occurred while reading compressed data during decompression. This may be due to soft error but is more likely to be a programming error.
ERRORCODE value	4'hC
Error type	Software
Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE
AFC state	Decompression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-9 Error Condition D_WE

Description	An error occurred while writing uncompressed data during decompression. This may be due to soft error but is more likely to be a programming error.
ERRORCODE value	4'hB
Error type	Software
Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE

AFC state	Decompression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-10 Error Condition D_PRE

Description	A prefix in a compressed data block could not be associated with a symbol during decompression. This can occur when the compressed data block is corrupted.
ERRORCODE value	4'hD
Error type	Software
Signaling	Interrupt
Fields updated	CDB, OFFSET, ENGINE, ERRORCODE
AFC state	Decompression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-11 Error Condition D_EOBE

Description	A compressed data block (CDB) was processed but the explicit or implicit EOB indication was corrupted. This can occur when the compressed data block is corrupted.
ERRORCODE value	4'hE
Error type	Software
Signaling	Interrupt
Fields updated	CDB, OFFSET, ENGINE, ERRORCODE
AFC state	Decompression halted and end-of-transaction is signaled.
AFC resume	Next operation or reset

Table 12-12 Error Condition D_MCE

Description	A hardware error inside AFC was detected while decompressing data. This may be due to soft error or a design defect.
ERRORCODE value	4'hF
Error type	Software

Signaling	Interrupt
Fields updated	ERRORCODE, ENGINE
AFC state	Decompression halted and end-of-transaction is signaled.
AFC resume	Reset

PRELIMINARY SPECIFICATION

12.6 Performance

Compression and decompression performance varies with memory system performance, parent data mover microarchitecture and the compression ratio of the dataset. See the discussion in the AFC microarchitecture specification for details AFCU.

PRELIMINARY SPECIFICATION

12.7 Appendix - Exporting Tables

To extract data from this specification for design or verification, the following example illustrates how to parse and dump the tables that define the dictionaries for each compression algorithm.

```
import os ;
import sys ;
import xml.etree.ElementTree as ET

xml_file = sys.argv[1] ;
table_id = sys.argv[2] ;

if (not os.path.exists(xml_file)) :
    sys.stderr.write( 'error: cannot locate
'+xml_file+'\n' ) ;
    sys.exit(1) ;

parsed = ET.parse(xml_file) ;
root = parsed.getroot() ;
tables = root.findall('.//simpletable') ; # python
2.6-ism
for table in tables :
    if ( 'id' in table.attrib ) :
        if ( table.attrib['id'] == table_id ) :
            rows = table.findall('.//strow') ;
            for row in rows :
                fields = row.findall('.//stentry') ;
                data = [] ;
                for field in fields :
                    data.append(field.text) ;
                print( ' '.join(data) ) ;
                sys.exit(0) ;

sys.stderr.write( 'error: cannot locate table with id
'+table_id+'\n' ) ;
sys.exit(1) ;
```

Example usage:

```
% compression_table.py theory_of_operation.xml
zero_compression_table
% compression_table.py theory_of_operation.xml
egu_compression_table
% compression_table.py theory_of_operation.xml
egs_compression_table
```

PRELIMINARY SPECIFICATION

12.8 References

[ARFU] "Accelerator Framework Compression Microarchitecture Specification" eventually in PDS.

[DRU] "Data Router Unit (DRU)"
https://pds.design.ti.com/d/3/2145/10817/12133/2/MSMC_DRU.pdf

[EG] "Exponential-Golomb Coding" https://en.wikipedia.org/wiki/Exponential-Golomb_coding

PRELIMINARY SPECIFICATION

AFC Micro-Architecture

The Accelerator Framework Compression (AFC) module is a hardware accelerator that compresses and uncompresses sequences of bytes. While it is designed as standalone pipelines, it is integrated with the Keystone3 compute cluster data routing unit or could be integrated with other data transfer engines.

This specification describes how the AFC works and how it is constructed. The intended audience for this information is designers and verification engineers. Two related documents cover the programmer-visible behavior (the AFC Architecture Specification) and how to make it do specific tasks (the AFC User's Guide).

Notes on the text:

- The term *encode* may be used as a synonym for compress.
- The term *decode* may be used as a synonym for decompress.

PRELIMINARY SPECIFICATION

13.1 Compression

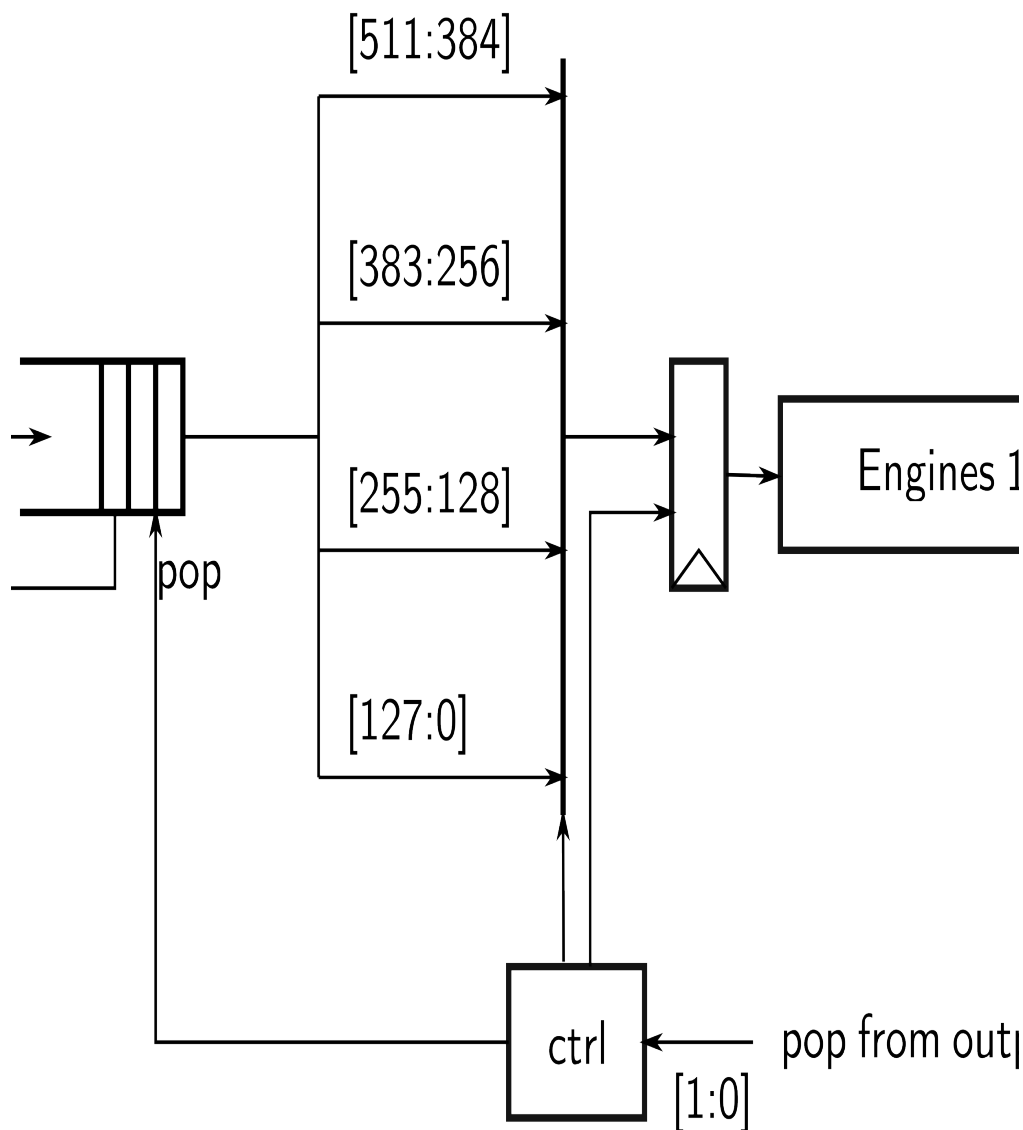
The compression function is divided into 4 subsystems:

- Compression Ingress, which distributes data from DRU to the compression engines.
- Compression Engines that perform the symbol-to-prefix mapping.
- Merge Datapath, which aggregates the results from the compression engines.
- Compression Egress, which constructs the compressed data structure and send data to the DRU.

13.1.1 Compression Ingress

The ingress datapath accepts 512 bits of in-order data per cycle and an end-of-stream marker from the DRU read datapath. A single FIFO decouples flow control events in the compression engine from DRU. The 512-bit payload is divided into 4 128-bit or 16-byte sections and sequentially sent to the 16 compression engines where all 16 bytes are simultaneously processed.

Compression DRU Ingress



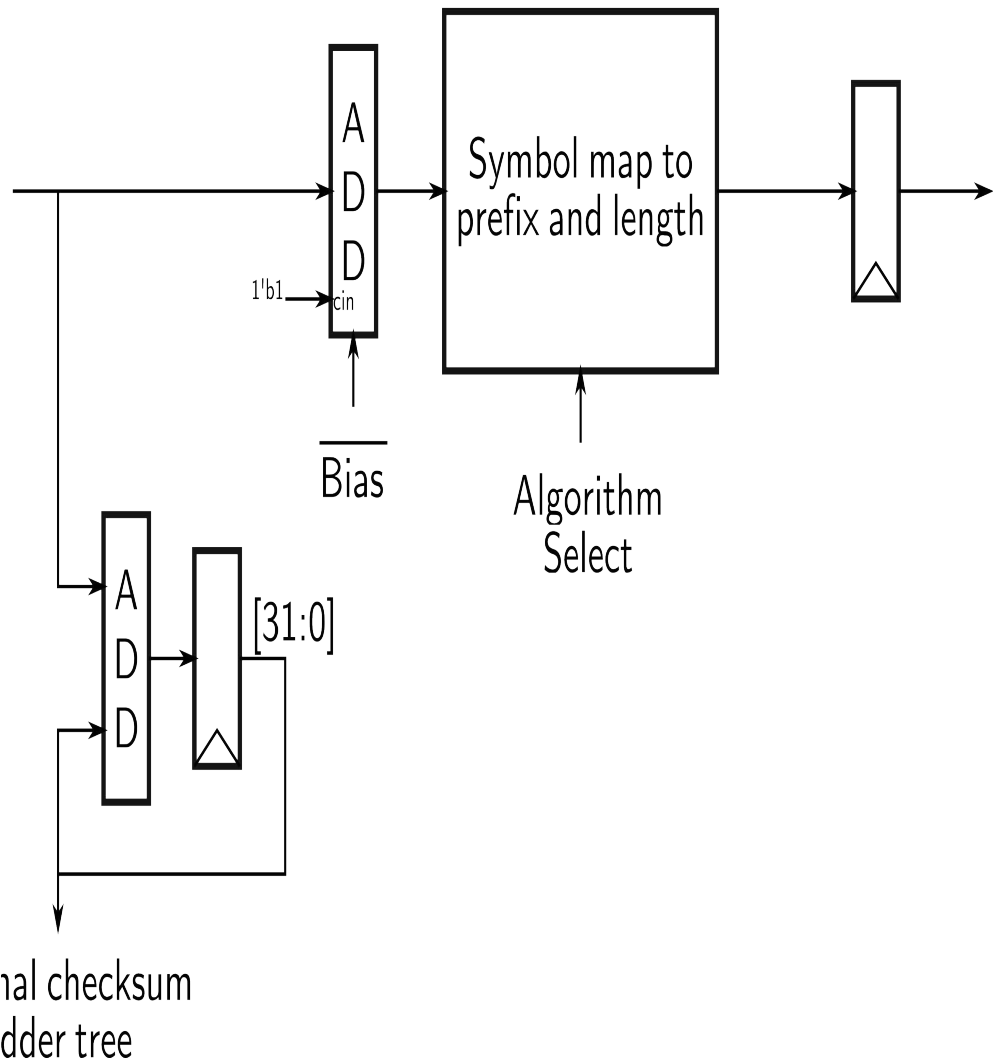
There is one backpressure mechanism inside the AFC compression module. The output buffer in the compression egress block acts as a FIFO and returns credits to the ingress control unit when a block of data is completely used and packed into the compressed data structure to be sent to the DRU write pipeline. The ingress control pops its input FIFO in every cycle where sufficient credits are available to hold the compressed data from at least 128 uncompressed bits.

13.1.2 Compression Engine

Compared to the decompression engines described later, the compression engine is less complicated. The compression engine processes a single 8-bit symbol from the ingress logic with fixed latency. Processing takes several steps. First, a software-specified bias is subtracted from the incoming symbol. Then, the de-biased symbol is mapped to a variable length prefix, according to the specified algorithm. Each compression engine delivers a 3-tuple representing a single prefix to the prefix merge datapath. The 3-tuple contains the actual prefix LSB aligned in a 17-bit bus, a 17-bit one-hot end-of-prefix (EOP) mask, and the binary-encoded length of the prefix. For end-to-end checking, each

valid uncompressed symbol is accumulated into a 32b checksum that is summed across the compression engines at the end of the transaction.

Compression Engine



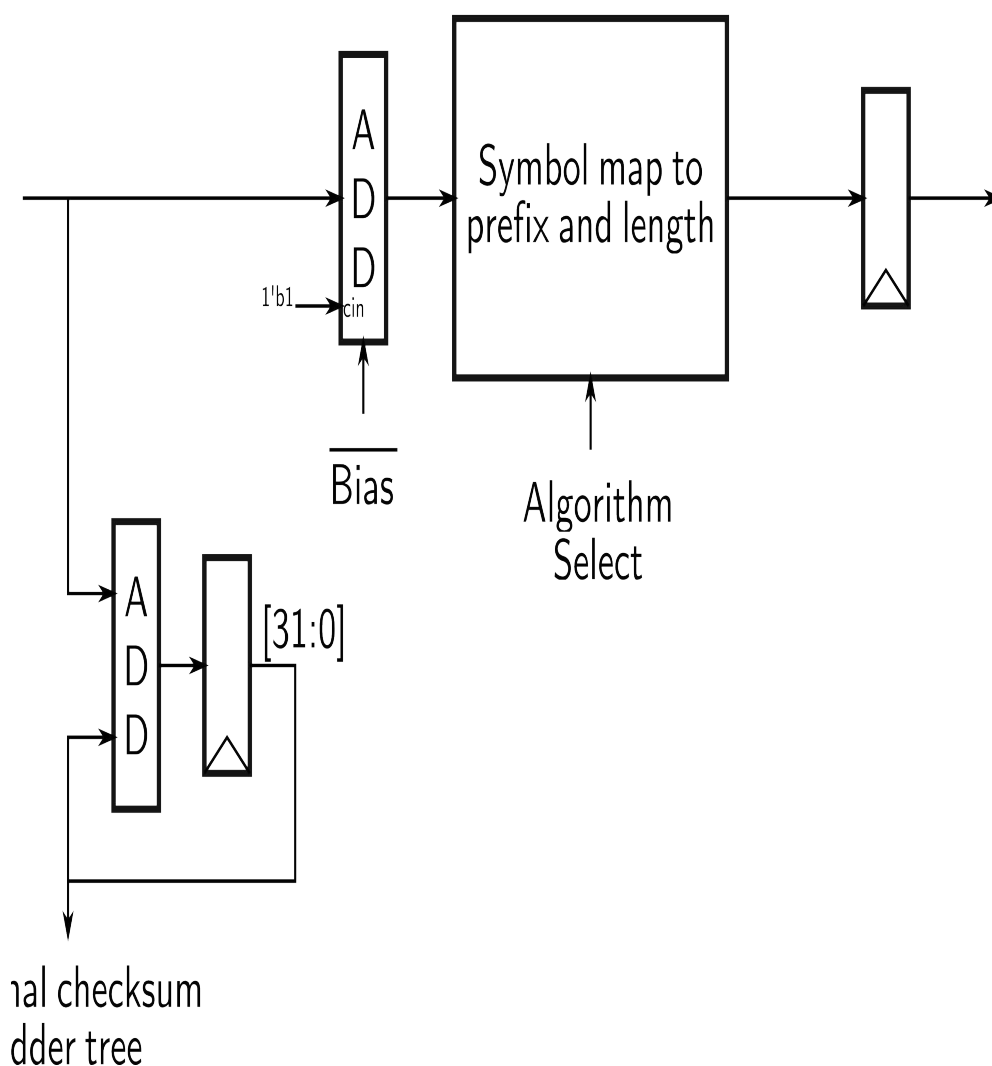
13.1.3 Compression Prefix Merge Datapath

Prefixes from the compression engines correspond to exactly one symbol and occupy a variable-length field in a fixed bit-width bus. The prefix merge datapath packs all the variable length prefixes from the compression engines into a variable length ordered prefix string that contains the 16 symbols processed by the compression engines. The datapath can sustain this prefix merging at a single clock cycle rate. To reduce wiring and component count while maintaining the desired compression rate, the prefix merge datapath is imple-

mented as a pipelined binary tree. Each level in the tree doubles the number of prefixes that are compressed in a single bus. The following picture illustrates the binary tree. Note that it does not explicitly show the EOP strings that mark the most significant bit of each prefix and the prefix string lengths.

For physical design purposes it may be useful to handle two compression engines plus the first-level merge function as a macro in order to minimize wiring congestion.

Compression Engine



The nodes in the binary tree have a similar function regardless of the position in the tree. The prefix and EOP strings from the left operand (more significant

data) are left-shifted by the length of the right operand. The shifted components are then concatenated on the matching components from the right operand. Finally, the new length is the sum of the left and right prefix string lengths. This is depicted in the next figure.

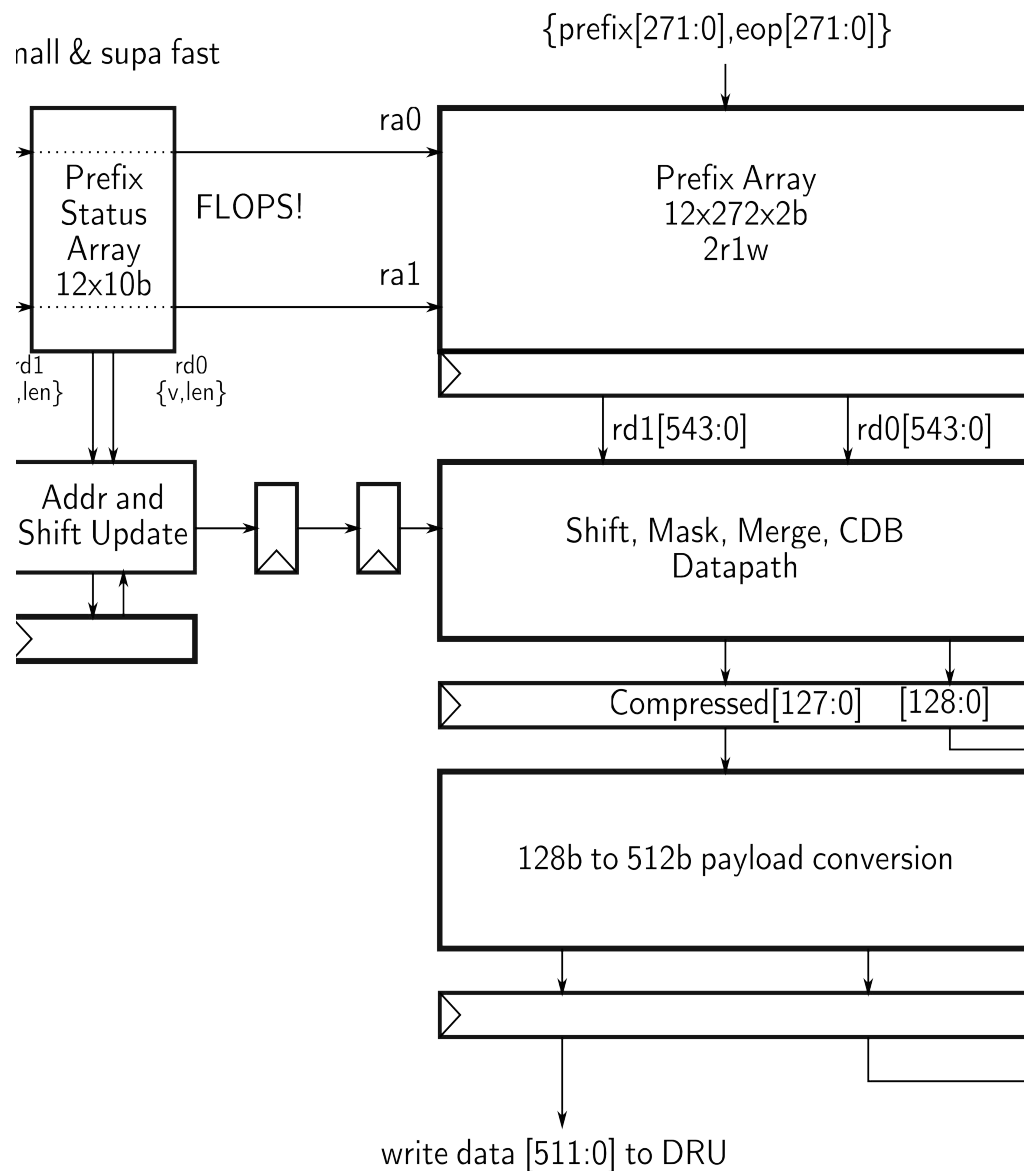
At the root of the tree, all 16 prefixes are packed into a single 272-bit bus (16 symbols that could expand to 17 bits each), a string of the same length containing the end-of-prefix mask, and the number of bits occupied by the packed prefixes. This data is pushed into the output buffer for Compressed Data Block (CDB) packing by the egress logic. The CDB format is defined in AFC Architecture because it is software-visible.

13.1.4 Compression Egress

The output buffer has 12 entries that operate as a 1-write 2-read FIFO. The depth of the FIFO covers:

- The 6 cycle pipeline from the ingress FIFO for in-flight operations.
- Up to 2 entries that may be accessed while constructing a CDB.
- 3 clock cycles to return credits.
- 1 clock cycle for credit calculation and ingress FIFO pop input setup.

Compression Datapath Output Buffer



Once a row in the output buffer is completely processed, a credit is returned to the ingress control logic. In rare cases, two rows can be simultaneously finished so up to 2 credits are returned.

When packing the 128-bit CDBs, three components are merged, similar to the merge datapath described above, but with a 128-bit output width. The three components are any excess prefixes from the prior CDB or prefix strings from one or more rows, and up to two incoming rows from the output buffer. The end-of-prefix mask is used to quickly determine the location of the final complete prefix in the 128-bit field. This is used in algorithm-specific logic to mark the end of the CDB with the appropriate EOB prefix as defined in AFC Architecture. In the special case of the end of the compressed stream, the compression engines encode one or more EOB prefixes that may occupy the middle of the CDB.

Finally, the 128-bit CDBs are packed into 512-bit payloads and pushed in-order into the DRU write pipeline along with end of stream markers.

At the end of processing data there needs to be some way to tell the egress logic that we are done so it can possibly build a partial CDB and do a final push into the DRU pipeline. This probably should be pipelined along with the prefixed/EOP/Len from the engines to the egress block. It might be as simple as having the compression engines generate the algorithm-appropriate EOB prefix to fill the current CDB.

13.2 Decompression

Like the compression function, the decompression hardware has several subsystems. These are:

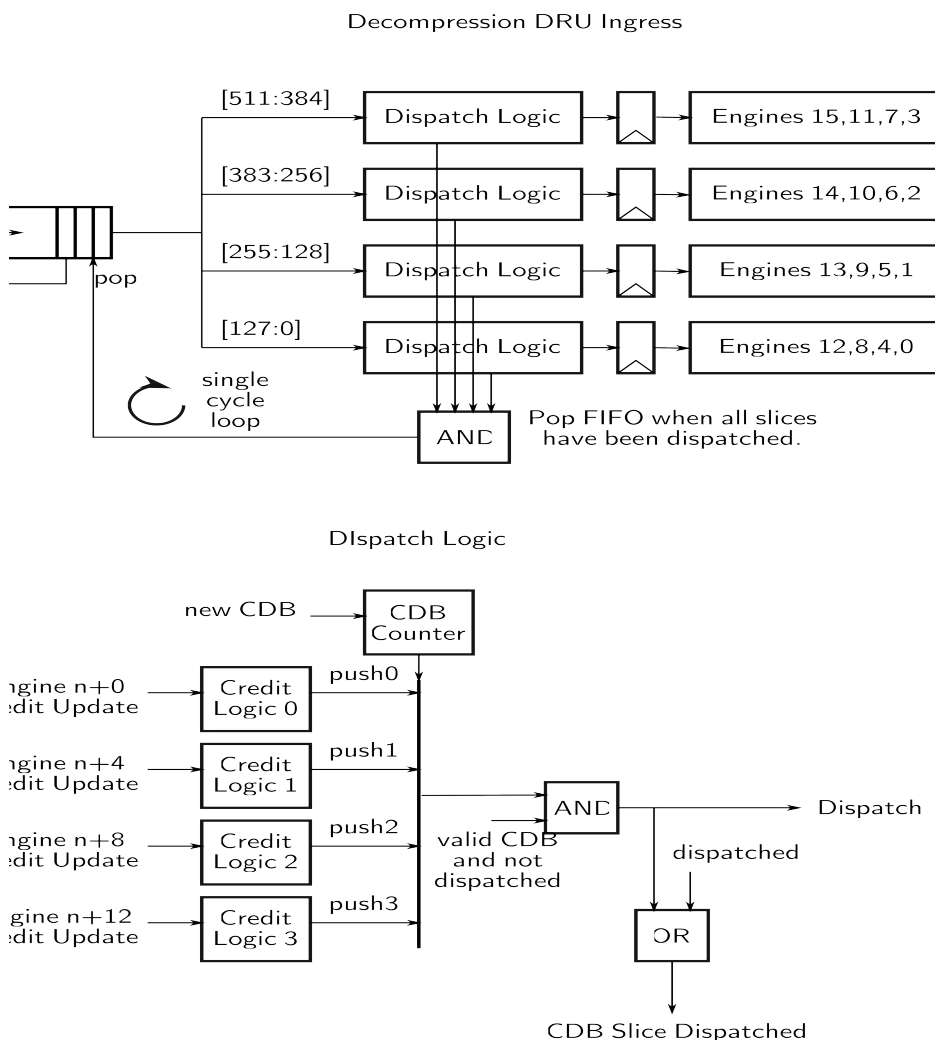
- Decompression Ingress, which accepts data from DRU and distributes 128-bit compressed data blocks to decompression engines.
- Decompression Engines that map compression prefixes to symbols.
- Decompression Egress, which orders and aligns sequences of symbols from the Decompression Engines and pushes data into DRU.

13.2.1 Decompression Ingress

The ingress portion of the decompression unit accepts 512-bit in-order payloads and end of stream markers from DRU and enqueues them in a FIFO. When an ingress FIFO entry is popped, it is distributed in 128-bit units corresponding to CDBs to individual decompression engines that operate on CDBs in parallel. Because of the variable length coding, processing the CDBs takes variable time. Dispatch logic in the ingress block tracks the progress of each decompression engine and once the final CDB is distributed, the ingress FIFO is popped to supply a new CDB in the next clock cycle.

Each ingress dispatch block sends CDBs to a pool of 4 engines such that sequential CDBs packed in the 512-bit DRU payload are assigned to sequentially-numbered engines. Internally, the dispatch block maintains credit logic that receives pop indications from the associated decompression engines and

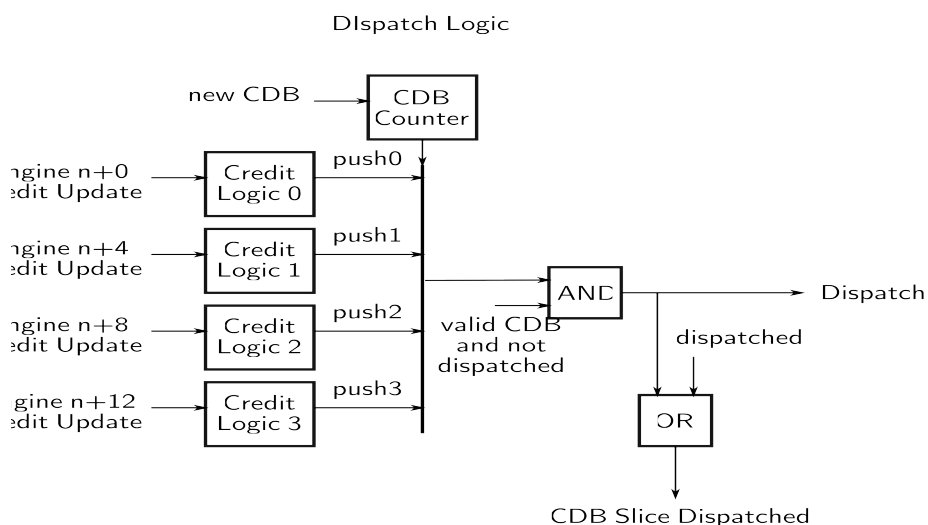
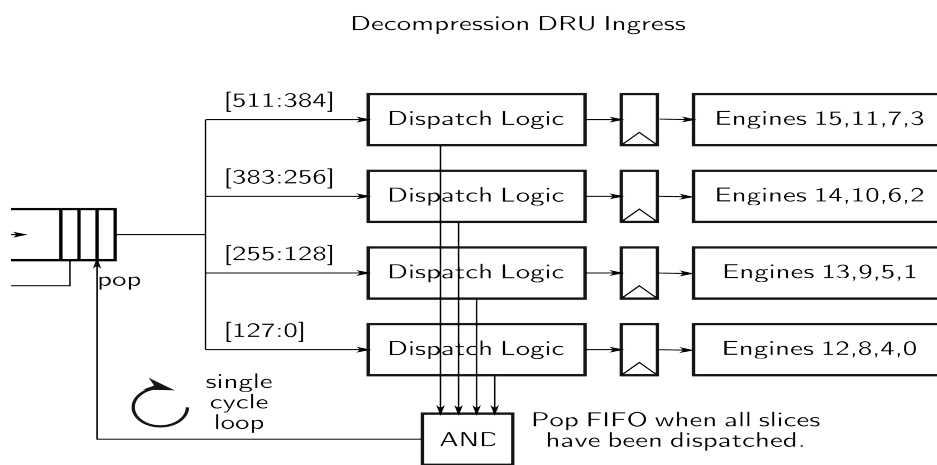
increments the credit counts. Per-engine credits are decremented as CDBs are distributed to specific engines.



13.2.2 Decompression Engine

There is a fundamental difference between the compression engines and decompression engines: the compression engines operate on a sequential stream of 16-byte blocks and process each byte within that block simultaneously. But, the decompression engines operate in parallel on the 128-bit CDBs concurrently.

A decompression engine instance iteratively processes one 128-bit CDB and delivers a variable number of symbols to the decompression order buffer. Each decompression engine has a small input FIFO that allows the decompression process to run slightly out of order with respect to other decompression engines. This is useful because of the wide range of processing times (from 7 cycles to 128 cycles), and potential serializing effects that can occur between highly compressed data and poorly compressed data.



CDBs are dispatched from the ingress logic and enqueued in the decompression engine. In normal operation, a CDB is popped from the input FIFO and

moved to the working buffer. The working buffer is also loadable from a shift network that serially removes decoded symbols.

Two modules decode portions of the working buffer. Logic in the EOB Detect submodule tracks the number of bits consumed and locates EOB prefixes to issue input FIFO pops without dead cycles. In parallel with EOB Detect submodule, a the Prefix Decode submodule operates on the head of the working buffer and maps the leading bits to a symbol, as defined by the specific compression algorithm. As a side-effect, the length of the decoded prefix is used by the shifter, as described earlier, to prepare the working buffer contents for the next clock cycle.

The symbol from the Prefix Decode submodule is added to a software-specified bias to complement the de-bias operation from the compression engines and the result is registered. After the register, each valid 8-bit symbol value is added to a 32-bit accumulator, which is used to compute a final checksum over the dataset for end-to-end checking.

For control and datapath operations, the decompression egress logic needs to track the symbol offset in the CDB for the currently-delivered symbol. This offset is produced by a 7-bit incrementer in each decompression engine. The decompression engine also produces a flag on the final symbol that is used by the egress logic to stop processing an entry.

13.2.2.1 Decompression Egress

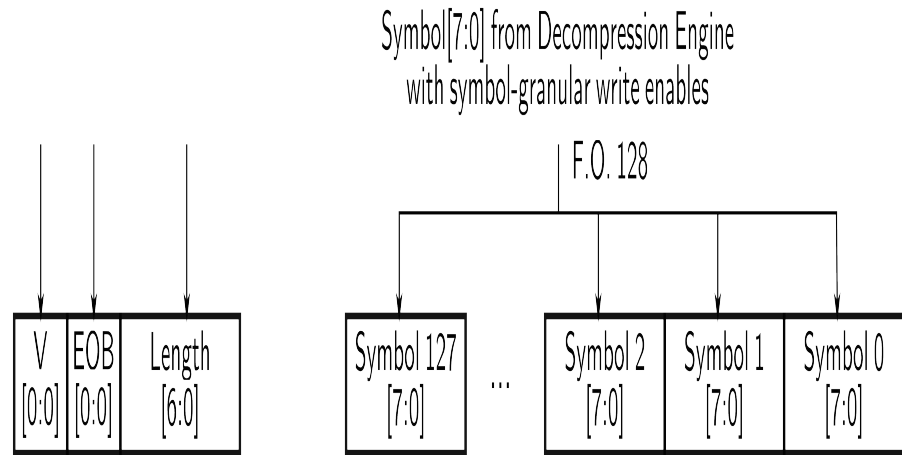
Uncompressed bytes, or symbols, are generated by the decompression engine and buffered in dedicated locations in the Symbol Array, which has 16 write ports and 2 read ports. While the write ports are symbol-granular, the read ports are 1024-bits wide to accommodate the maximum possible number of symbols that could be decoded from a maximally compressed sequence (128 single-bit prefixes expanding to 8-bit symbols).

Incoming symbols are written at the offset specified by the decompression engines, in one of a pair of rows dedicated to the specific engine and managed like a FIFO. 16 simultaneous writes can occur per cycle. Two read addresses are produced per clock cycle and are used to access a small array (Symbol Status Array) containing the end-of-CDB flag and number of decoded symbols. In parallel, the larger Symbol Array is accessed. The Symbol Status Array has an asynchronous read and the results are used to compute the read addresses and shift controls for the Symbol Array data. The larger Symbol Array is pipelined and read data is available a cycle after the Symbol Status Array. Shift and merge logic combines the symbols from up to two rows to form the 128-bit

result. This is subsequently packed into a 512 bus and pushed in-order into the

DRU's write pipeline along with end of stream markers.

Decompression Ordering Buffer Row



I glossed over the flow control here... the decompression engine must stall if both rows assigned the engine are occupied. Since these are probably adjacent maybe simple XON/XOFF ?

PRELIMINARY SPECIFICATION

13.3 Performance and Area

13.3.1 Compression Performance

Bandwidth limitations:

- an input rate of no more than 16 bytes per cycle can be sustained due to the 4:1 physical multiplexing of the 64-byte input bus. The number of compression engines and prefix merge pipeline bandwidth are matched to this rate.
- an output rate of no more than 16 bytes per cycle can be sustained due to the datapath width of the compression egress logic.

Small dataset: A sufficient number of bytes must be available to simultaneously occupy the parallel compression engines for full performance. There is a spin-up period, for the first 16 bytes, where the compression engines are getting their first data and a spin-down period of the last 16 bytes where less than 16 engines are processing uncompressed data. Production of the CDBs is serialized during a portion of these periods. Datasets should be larger than 320 bytes to minimize this overhead. The decompression hardware is more sensitive to dataset size and the guidance in that discussion sets the lower limit for full performance.

In the case of highly compressible data the input bandwidth limitation of 16 bytes per cycle determines the performance of the compression function since highly compressible data will result in less than 16 bytes per cycle leaving the compression function. The steady state occupancy of the egress buffer will be low since multiple egress buffer rows will be consumed per cycle to form an out-bound CDB.

In the case of data with a compression ratio less than or equal to 1.0 with stationary statistics the input bandwidth limitation of 16 bytes per cycle determines the performance of the compression function since compressible data will result in less than 16 or fewer bytes per cycle leaving the compression function.

In the case of poorly compressible data, compressed data with CDB overhead occupies more bytes than the input data, the output rate will limit performance and the ingress pipeline backpressures DRU. In this case, the egress buffer will tend to be full. Using poorly compressible data on AFC is not recommended.

13.3.2 Decompression Performance

Bandwidth limitations:

- an input rate of no more than 16 bytes per cycle of compressed data can be sustained due to the 4:1 physical multiplexing of the 64-byte input bus. The number of decompression engines is matched to this rate.
- an output rate of no more than 16 bytes per cycle can be sustained due to the datapath width of the decompression egress logic.

Small dataset: A sufficient number of CDBs must be available to simultaneously occupy the parallel decompression engines for full performance. There is a spin-up period, for the first 16 CDBs, where the compression engines are getting their first data and a spin-down period of the last 16 CDBs where less than 16 engines are processing compressed data. Production of the uncompressed data is serialized during a portion of these periods because as the number of CDBs approaches 1, the performance of AFC approaches one byte per cycle. For full performance, the time in the spin-up and spin-down periods must be small compared to the steady state when all of the decompression engines are active. On the order of 320 CDBs of compressed data (5120 bytes of compressed data or roughly 8KiB of uncompressed data at typical compression ratios) must be processed for these overheads to become small.

In the case of highly compressible data the output bandwidth determines the performance of the decompression function since the bandwidth entering the decompression function will be less than 16 bytes per cycle but limited to 16 bytes per cycle on the output to DRU.

In the case of data with a compression ratio less than or equal to 1.0 with stationary statistics the output bandwidth determines the performance of the decompression function since the bandwidth entering the decompression function will be less than 16 bytes per cycle but limited to 16 bytes per cycle on the output to DRU.

In the case of poorly compressible data, especially when the compressed data with CDB overhead occupies more bytes than the input data, the input rate limits performance. In the worst case, 7 17-bit prefixes plus one 9-bit EOB prefix occupy the CDBs at a compression ratio of 128bits/56bits = ~2.3. While the ingress side of the decompression function is running at the full 16 bytes per cycle rate in this case, decompressed data is being delivered to the DRU at a disappointing average of 7 bytes per cycle.

13.3.3 Area

total cell area	248446.52424		
total flops	83123		
flop area	90491.02272		
total routed area	354923.606057	um**2	
module	instances	total area	fraction
of total			
adder32	63	5.329981e+03	2.145323e-02
adder4	18	1.903565e+02	7.661869e-04
adder5	8	1.057536e+02	4.256594e-04
adder6	8	1.269043e+02	5.107913e-04
adder7	42	7.772890e+02	3.128597e-03
adder8	50	1.057536e+03	4.256594e-03
adder9	17	4.045075e+02	1.628147e-03
adec12	6	3.856896e+01	1.552405e-04
adec128	16	9.787392e+02	3.939436e-03
adec16	4	3.317760e+01	1.335402e-04
adec32	4	6.386688e+01	2.570649e-04

adec4	36	8.211456e+01	3.305120e-04
checksum_tree	2	3.844454e+03	1.547397e-02
compression_engine	16	6.320125e+03	2.543857e-02
control	2	3.410089e+04	1.372565e-01
decoder	1	1.655060e+05	6.661636e-01
decompression_engine	16	4.796921e+04	
1.930766e-01			
enc_egr_datapath	1	2.516330e+04	1.012826e-01
encoder	1	8.294049e+04	3.338364e-01
flop_array1024	36	5.016453e+04	2.019128e-01
flop_array108	1	1.469664e+02	5.915414e-04
flop_array128	38	6.618931e+03	2.664127e-02
flop_array129	17	2.984234e+03	1.201158e-02
flop_array143	4	7.783776e+02	3.132978e-03
flop_array1752	1	2.384122e+03	9.596116e-03
flop_array2048	2	5.573837e+03	2.243475e-02
flop_array264	1	3.592512e+02	1.445990e-03
flop_array280	2	7.620480e+02	3.067252e-03
flop_array3	36	1.469664e+02	5.915414e-04
flop_array32	65	2.830464e+03	1.139265e-02
flop_array39	16	8.491392e+02	3.417795e-03
flop_array4	17	9.253440e+01	3.724520e-04
flop_array5	10	6.804000e+01	2.738618e-04
flop_array512	22	1.532805e+04	6.169558e-02
flop_array544	3	2.220826e+03	8.938848e-03
flop_array553	1	7.525224e+02	3.028911e-03
flop_array64	4	3.483648e+02	1.402172e-03
flop_array6528	1	8.883302e+03	3.575539e-02
flop_array7	32	3.048192e+02	1.226901e-03
flop_array74	8	8.055936e+02	3.242523e-03
flop_array7556	1	1.028220e+04	4.138599e-02
flop_array9	35	4.286520e+02	1.725329e-03
leading_bit_detect17	16	3.807130e+02	
1.532374e-03			
leading_bit_detect34	1	4.758912e+01	
1.915467e-04			
leading_bit_detect9	16	2.015539e+02	
8.112568e-04			
merge_datapath	1	3.157275e+04	1.270807e-01
merge_node_136_8	1	1.486892e+04	5.984756e-02
merge_node_17_5	8	3.503762e+03	1.410268e-02
merge_node_34_6	4	4.858186e+03	1.955425e-02
merge_node_68_7	2	8.341885e+03	3.357618e-02
ordering_buffer	1	8.236712e+04	3.315286e-01
rf_1r1w1_16x64	2	4.310444e+03	1.734959e-02
rf_1r1w1_4x128	16	2.224682e+04	8.954372e-02
rf_1r1w1_4x512	2	1.104539e+04	4.445783e-02
rf_2r1w1_12x544	1	1.827037e+04	7.353845e-02
rf_2r1w1_12x9	1	3.431549e+02	1.381202e-03
shifter136,136	1	4.697533e+03	1.890762e-02

shifter136,272	1	9.395067e+03	3.781525e-02
shifter15,128	1	4.686336e+02	1.886255e-03
shifter16,128	1	4.686336e+02	1.886255e-03
shifter16,256	2	1.866240e+03	7.511637e-03
shifter16,384	2	2.811802e+03	1.131753e-02
shifter17,128	16	1.199702e+04	4.828814e-02
shifter17,17	40	4.462387e+03	1.796116e-02
shifter17,34	8	1.678787e+03	6.757134e-03
shifter34,34	4	1.364014e+03	5.490172e-03
shifter34,68	4	2.641766e+03	1.063314e-02
shifter68,136	2	4.997376e+03	2.011449e-02
shifter68,68	2	2.540160e+03	1.022417e-02

Revision History

Revision	Author	Date	Comments
0.0.1	David Smith	8/1/2013	Create Initial Draft
0.0.2	David Smith	8/14/2013	Updates based on comments received
0.0.3	David Smith	8/28/2013	Updates based on internal spec review. Add conditions for the micro-architecture so it can be hidden.
0.0.4	David Smith	9/17/2013	Add architectural details of the high level operation of the DRU and more detailed information on the interfaces and what portions of the MBA specification will apply. Also defines the formats for the TR and CR.
0.0.5	David Smith	11/19/2015	Added new configuration options to support UTC alignment. Updated the documentation for the channelized queues and other things required for UTC alignment.
0.0.6	David Smith	4/27/2016	Changed structure to channelized
0.9.0	David Smith	7/6/17	Added detailed flow diagrams as well as the memory attribute table and registers required and safety registers required. Also added safety section and security section on firewall configuration and setup.
0.9.1	David Smith	10/30/17	Added more detailed description of the data formatting functions that are supported by TR formats. Added table for CTSET export.
0.9.2	David Smith	11/16/17	Added the submission error table and added description of the shared hardware resource arbitration. Fixed some errors in register mapping.
1.0	David Smith	8/8/2018	Fixed Cause register description to match RTL. Addressed Open WEBs and added a configuration tables for each of the different configurations.
1.01	David Smith	12/18/2018	Add note in security section to explain how registers are divided as channelized and non-channelized and its effect on the firewall logging.
1.02	David Smith	3/7/2019	Added Virtualization feature by including the CMMU from C7x
1.03	David Smith	4/17/2019	Added the Compression and Decompression description
1.04	David Smith	7/15/2019	Fixed bit expansion definition to show that we add 0's to the upper bits and not the most significant bit. Dec 2022. Added the TR Specification information into the specification so it is in a single document.
1.05	David Smith	12/18/2019	Update the Compression and Decompression descriptions to match the implementation.
1.0.6	David Smith	3/1/2020	Update the Compression and Decompression to handle video compression requirements and support for selectable starting point for decompression and fixed superblock 2 dimensional block to address TIDL concerns.
1.0.7	David Smith	11/15/2020	Updated compression descriptions including adding circular addressing examples. Cleaned up issues from WEBs filed.
1.0.8	David Smith	11/21/2020	Fixed diagrams for circular addressing compression added missing register field descriptions. Added DRU R30 Configurations
1.0.9	David Smith	2/26/2021	Added Ring Support Functionality and added traceability section for all current DRUs/UTCs
1.0.10	David Smith	12/12/2022	Fixed open webs issues and updated traceability
1.0.11	David Smith	7/5/2023	Updated Traceability

PRELIMINARY SPECIFICATION